General Aptitude

GATE CSE 2027 · complete formula, rule, method & trap sheet

73 reference boxes · **8** chapters ·

Verbal Aptitude | Numerical Computation | Data Interpretation | Mensuration, Geometry & Probability | Analytical Aptitude | Spatial Aptitude | Problem → Standard Method Playbook | 🎯 Representative GATE-Style Worked Problems | Last-minute one-liners

1 Verbal Aptitude

Grammar essentials GATE tests repeatedly

REFERENCE TABLE

Area	What to watch for
Subject-verb agreement	Singular subject → singular verb, even when other nouns sit in between ("The *list* of items **is** long", not "are")
Tenses	Consistency across a sentence; signal words like "since", "for", "yet" hint at the correct tense
Prepositions	Fixed idiomatic pairings: "married **to**", "different **from**", "capable **of**"
Articles (a/an/the)	"a" before consonant SOUNDS (a university), "an" before vowel SOUNDS (an hour)

WORKED EXAMPLE

Worked example

Q: Choose the correct sentence.
(a) "Neither of the students *have* submitted their assignment."
(b) "Neither of the students *has* submitted their assignment."
A: (b) is correct. "Neither" is a singular indefinite pronoun — it always takes a singular verb ("has"), regardless of the plural noun "students" that follows "of".

Vocabulary: idioms & words-in-context

NOTES

GATE rarely tests rote word-lists; it tests whether you can pick the word that fits the **context** and tone of a sentence.

WORKED EXAMPLE

Worked example

Q: "Despite the heavy rain, the team's spirit remained _____." Choose the best fit: (a) damp (b) undaunted (c) soaked (d) reluctant
A: (b) undaunted — it means "not discouraged", fitting the contrast set up by "Despite". (a) and (c) are literal plays on "rain" (a common distractor trick), and (d) doesn't logically follow from the sentence's structure.

Reading comprehension strategy

KEY POINT

1. Read the QUESTIONS first (skim), so you know what to look for.
2. Skim the passage once for overall structure/argument, don't over-read details yet.
3. For "which of the following is true/false" questions, go back and verify each option against the specific line in the passage — don't rely on memory or general impression.
4. Watch for **absolute words** in options ("always", "never", "all") — passages rarely support such strong claims, making these options likely wrong.

Narrative sequencing

NOTES

You're given 4-5 jumbled sentences and must arrange them into a coherent paragraph.

WORKED EXAMPLE

Worked example

Q: Arrange into a coherent paragraph:
P: As a result, crop yields in the region fell sharply.
Q: The monsoon arrived nearly a month late this year.
R: Farmers had already delayed sowing in anticipation.
S: This further compounded the losses they faced.
A: Q-R-P-S. Start with the triggering event (Q: late monsoon) → the farmers' reaction (R: delayed sowing) → the consequence (P: fell yields) → the compounding effect (S: further losses). Look for pronouns ("this", "as a result") and cause-effect chains — they're the strongest clues to correct order.

Sentence completion — a systematic method

NOTES

Single- and double-blank sentence completion is one of the most repeated 1-mark verbal patterns. Do not try the options one by one — read the **logical signal** first.

REFERENCE TABLE

Signal type	Typical words	What the blank must do
Contrast	but, although, though, despite, however, yet, while, unlike	Be the **opposite** of the clue
Continuation	and, moreover, furthermore, hence, therefore, so, as a result	Carry the **same direction** as the clue
Cause and effect	because, since, thus, consequently	Be the logical cause or result of the other clause
Definition / example	colon, dash, "that is", "for instance"	Restate the clue in other words

KEY POINT

Method: (1) predict a word BEFORE reading the options; (2) in a two-blank sentence fix the blank you are surer of and eliminate options with it; (3) check the polarity (positive or negative) of both blanks; (4) read the finished sentence back once — it must sound natural, not merely grammatical.

WORKED EXAMPLE

Worked example — contrast signal

Q: "Although the plan looked _____ on paper, it proved perfectly workable in practice." (a) practicable (b) sensible (c) unrealistic (d) reliable
A: (c) unrealistic. "Although … perfectly workable" sets up a contrast, so the blank must be the opposite of "workable". The other three all mean roughly "workable", which would make "although" pointless.

WORKED EXAMPLE

Worked example — two blanks

Q: "The witness's account was so _____ that even the lawyer who called her found it hard to _____ it." (a) bizarre … defend (b) clear … accept (c) reliable … trust (d) simple … follow
A: (a). "So … that even her own lawyer found it hard to …" needs a strongly negative first word and a verb of support: "bizarre … defend". (b) and (c) make the difficulty illogical (a clear or reliable account is easy to accept or trust), and (d) "so simple … hard to follow" contradicts itself.

Critical reasoning — arguments, assumptions, strengthen / weaken

NOTES

An **argument** is premises (evidence) plus a conclusion, joined by an **unstated assumption**. Find the conclusion first (clue words: therefore, hence, thus, so, it follows that), then ask: *what must be true for these premises to lead there?*

REFERENCE TABLE

Question type	What to look for	The correct option…
Assumption	The hidden bridge from premise to conclusion	Is **necessary**: negate it and the argument collapses
Strengthen	Support for the bridge	Rules out an alternative cause or confirms the link
Weaken	A gap in the bridge	Offers an **alternative cause**, or shows the evidence is unrepresentative
Inference	What the passage forces	Must be true using ONLY the passage — not merely plausible

KEY POINT

Assumption example: "Profit rose after the firm adopted a four-day week; therefore the four-day week raised profit." The hidden assumption is that *nothing else that affects profit changed at the same time*. Negate it — "something else did change" — and the conclusion no longer follows, which is exactly the test for a necessary assumption.

WORKED EXAMPLE

Worked example — weaken

Q: "After the city made bus travel free for students, attendance at its schools rose 8%. Hence free bus travel raises attendance." Which statement most weakens the argument?
(a) Free travel cost the city a large sum. (b) In the same year every school began serving free midday meals. (c) Students said they liked the free passes. (d) The attendance figures come from the schools' own registers.
A: (b). It supplies an *alternative cause* for the rise, so the bus scheme can no longer be assumed responsible. (a) is about cost, not effect; (c) mildly supports the argument; (d) is irrelevant to causation.

Verbal deduction — what MUST follow

NOTES

Deduction questions give a short passage and ask which statement **must be true**. Use only the passage; ignore real-world knowledge.

GATE FAVOURITE

GATE trap

"All A are B" does **not** give "All B are A" (the converse). Its valid twin is the **contrapositive**: "if not B then not A". Also watch quantity words — "some" never becomes "all", and "most" never becomes "all".

WORKED EXAMPLE

Worked example

Q: "Everyone who completed the safety course received a badge. Ravi has no badge." Which must be true?
(a) Ravi is not an employee. (b) Ravi did not complete the safety course. (c) Everyone with a badge completed the course. (d) Ravi will never receive a badge.
A: (b). The rule is "course → badge"; its contrapositive is "no badge → no course". (a) is not forced — the passage never mentions employment. (c) is the converse. (d) predicts the future, which the passage never does.

Word groups & odd-one-out

NOTES

Find the property shared by three words and use the fourth as the tie-breaker. Test several lenses: **category**, **function**, **degree or intensity**, **part–whole**, **origin**.

REFERENCE TABLE

Group	Odd one out	Reason
Brass, bronze, steel, copper	Copper	Copper is an element; the other three are alloys
Sprint, dash, dart, stroll	Stroll	The first three are fast movement; a stroll is slow (degree)
Lion, tiger, leopard, wolf	Wolf	Big cats versus a canine (biological family)
Square, rectangle, rhombus, circle	Circle	The others are quadrilaterals

2 Numerical Computation

Ratios & Percentages

WORKED EXAMPLE

Worked numerical

Q: A solution has acid and water in ratio 3:7. If 10 litres of water is added, the new ratio becomes 3:10. Find the original quantity of acid.
A:

```
Let acid = 3x, water = 7x (original ratio 3:7)
After adding 10 L water: acid=3x, water=7x+10, new ratio = 3:10

3x / (7x+10) = 3/10
30x = 3(7x+10) = 21x + 30
9x = 30  →  x = 30/9 = 10/3

Acid = 3x = 3×(10/3) = 10 litres
```

FORMULA / RULE

Worked numerical

Q: A shopkeeper marks an item 40% above cost price, then offers a 25% discount on the marked price. Find the net profit/loss percentage.
A:

```
Let cost price = 100.
Marked price = 100 × 1.40 = 140
Selling price (after 25% discount) = 140 × 0.75 = 105

Net result = 105 - 100 = 5  → 5% PROFIT
```

Shortcut formula: Net % change = 40 - 25 - (40×25/100) = 40-25-10 = **+5%** (successive percentage change formula: a + b + ab/100, taking discount as negative).

Powers, Exponents & Logarithms

WORKED EXAMPLE

Worked numerical

Q: If $\log_{10}2 = 0.301$, find $\log_{10}80$.
A:

```
80 = 8 × 10 = 2³ × 10
log(80) = log(2³) + log(10) = 3×log(2) + 1 = 3×0.301 + 1 = 0.903 + 1 = 1.903
```

Permutations & Combinations

KEY POINT

```
Permutation (ORDER matters):   nPr = n! / (n-r)!
Combination (ORDER doesn't matter): nCr = n! / (r!(n-r)!)
```

WORKED EXAMPLE

Worked numerical

Q: In how many ways can a committee of 3 be chosen from 5 men and 4 women, such that at least 1 woman is included?
A:

```
Total ways to choose any 3 from 9 people = 9C3 = 84
Ways with ZERO women (all 3 men from the 5 men) = 5C3 = 10

At least 1 woman = Total - (zero women) = 84 - 10 = 74
```

"At least 1" is almost always easiest via the complement trick: Total minus the one excluded case.

WORKED EXAMPLE

Worked numerical

Q: How many distinct arrangements of the letters in "BANANA" are possible?
A:

```
BANANA has 6 letters: B(1), A(3), N(2)

Arrangements with repeated letters = 6! / (3!×2!×1!) = 720 / 12 = 60
```

Whenever letters repeat, divide the total factorial arrangements by the factorial of each repeated letter's count.

Series & Number Patterns

WORKED EXAMPLE

Worked numerical

Q: Find the next term: 2, 6, 12, 20, 30, ___
A:

```
Differences: 6-2=4, 12-6=6, 20-12=8, 30-20=10  → differences increase by 2 each time
Next difference = 12  →  next term = 30+12 = 42

(Pattern check: each term = n×(n+1): 1×2=2, 2×3=6, 3×4=12, 4×5=20, 5×6=30, 6×7=42 ✓)
```

GATE FAVOURITE

GATE strategy

For number series, always compute first differences, then second differences if the first aren't constant. If differences form a clean arithmetic/geometric pattern, you've found the rule. Also always sanity-check with a closed-form formula (like n×(n+1) above) when the pattern is recognizable — it's faster and safer than extrapolating blindly.

Number system — divisibility, remainders, HCF & LCM

REFERENCE TABLE

Divisor	Test
2, 5, 10	Last digit even / 0 or 5 / 0
3, 9	Digit sum divisible by 3 / by 9
4, 8	Last 2 / last 3 digits divisible by 4 / 8
6	Divisible by both 2 and 3
7	Drop the last digit d and subtract 2d from the rest; repeat (203 → 20 − 6 = 14 ✔)
11	(sum of digits in odd places) − (sum in even places) is 0 or a multiple of 11

KEY POINT

```
Remainder rules    (a + b) mod m = (a mod m + b mod m) mod m
                   (a × b) mod m = (a mod m × b mod m) mod m
Last-digit cycles  2, 3, 7, 8 repeat every 4 powers;  4, 9 every 2;  0, 1, 5, 6 never change
Fermat             a^(p−1) ≡ 1 (mod p)  for prime p not dividing a
HCF × LCM = product of the two numbers          (TWO numbers only)

Greatest number dividing a, b, c and leaving remainders r1, r2, r3 :  HCF(a−r1, b−r2, c−r3)
Least number leaving the SAME remainder r on division by a, b, c  :  LCM(a, b, c) + r
Least number that is k SHORT of a multiple of each of a, b, c      :  LCM(a, b, c) − k
```

WORKED EXAMPLE

Worked numerical — divisibility

Q: For which digit x is the number 2x74 divisible by 9?
A: Digit sum = 2 + x + 7 + 4 = 13 + x must be a multiple of 9 → x = 5 (2574 = 9 × 286).

WORKED EXAMPLE

Worked numerical — last digit

Q: Last digit of 7^{2026}?
A: Powers of 7 end in 7, 9, 3, 1, then repeat (period 4). 2026 = 4 × 506 + 2, so it matches the 2nd entry → **9**.

WORKED EXAMPLE

Worked numerical — remainder

Q: Remainder when 2^{100} is divided by 7?
A: $2^3 = 8 \equiv 1 \pmod 7$. 100 = 3 × 33 + 1, so $2^{100} = (2^3)^{33} \times 2 \equiv 1 \times 2 =$ **2**.

WORKED EXAMPLE

Worked numerical — HCF and LCM

Q: Find the HCF and LCM of 84 and 126 and verify the product rule.
A: 84 = $2^2 \cdot 3 \cdot 7$ and 126 = $2 \cdot 3^2 \cdot 7$. HCF = $2 \cdot 3 \cdot 7$ = 42; LCM = $2^2 \cdot 3^2 \cdot 7$ = 252. Check: 42 × 252 = 10584 = 84 × 126 ✔

Mixtures & alligation

KEY POINT

```
Alligation (mixing a cheaper item of value c with a dearer one of value d to get a mean m):

      cheaper : dearer  =  (d − m) : (m − c)

Replacement: a vessel holds V litres of pure liquid; x litres are drawn off and replaced
by water, n times in all  →  pure liquid left = V × (1 − x/V)^n
```

WORKED EXAMPLE

Worked numerical — alligation

Q: Wine at ₹60 per litre is mixed with wine at ₹80 per litre to give a mixture worth ₹68 per litre. In what ratio?
A: cheaper : dearer = (80 − 68) : (68 − 60) = 12 : 8 = **3 : 2**.

WORKED EXAMPLE

Worked numerical — replacement

Q: A vessel holds 40 L of milk. 4 L is removed and replaced with water; this is repeated twice more (three times in all). How much milk remains?
A: 40 × $(1 - 4/40)^3$ = 40 × 0.9^3 = 40 × 0.729 = **29.16 L** (so 10.84 L is water).

Linear equations & inequalities

KEY POINT

```
Two equations  a1·x + b1·y = c1  and  a2·x + b2·y = c2:
   a1/a2 ≠ b1/b2            → exactly one solution
   a1/a2 = b1/b2 ≠ c1/c2    → no solution (parallel lines)
   a1/a2 = b1/b2 = c1/c2    → infinitely many solutions

Inequalities: multiplying or dividing by a NEGATIVE number reverses the sign.
   |x − a| < b   ⟺   a − b < x < a + b
   |x − a| > b   ⟺   x < a − b   or   x > a + b
```

WORKED EXAMPLE

Worked numerical — simultaneous equations

Q: Solve 3x + 2y = 16 and x − y = 2.
A: From the second, x = y + 2. Then 3(y + 2) + 2y = 16 → 5y = 10 → y = 2, x = **4**. Check: 12 + 4 = 16 ✔

WORKED EXAMPLE

Worked numerical — modulus inequality

Q: How many integers x satisfy $|2x - 3| \le 7$?
A: $-7 \le 2x - 3 \le 7$ → $-4 \le 2x \le 10$ → $-2 \le x \le 5$. The integers −2, −1, 0, …, 5 number **8**.

Progressions (AP, GP) and means

KEY POINT

```
AP:  a_n = a + (n−1)d        S_n = (n/2)·[2a + (n−1)d] = (n/2)·(first + last)
GP:  a_n = a·r^(n−1)         S_n = a(r^n − 1)/(r − 1)
S_∞ = a/(1 − r)   when |r| < 1
For positive numbers: AM ≥ GM ≥ HM; for two numbers  GM² = AM × HM
```

WORKED EXAMPLE

Worked numerical — AP

Q: Find the sum of all multiples of 7 from 1 to 200.
A: The terms are 7, 14, …, 196 — that is 196/7 = 28 terms. S = (28/2)(7 + 196) = 14 × 203 = **2842**.

WORKED EXAMPLE

Worked numerical — GP

Q: Three numbers are in GP with sum 14 and product 64. Find them.
A: Write them as a/r, a, ar. Product = $a^3 = 64 \rightarrow a = 4$. Sum: $4/r + 4 + 4r = 14 \rightarrow 2r^2 - 5r + 2 = 0 \rightarrow r = 2$ or ½. The numbers are **2, 4, 8** (or 8, 4, 2).

Clocks & calendars

KEY POINT

```
Minute hand 6° per minute, hour hand 0.5° per minute → relative speed 5.5° per minute.
Angle between the hands at H:M  =  |30H − 5.5M|     (use 360° − angle if it exceeds 180°)
In 12 hours the hands coincide 11 times, are opposite 11 times and are at right angles 22 times.

Calendar: ordinary year = 52 weeks + 1 odd day;  leap year = 2 odd days.
Leap year: divisible by 4, but a century year only if divisible by 400.
Odd days in 100, 200, 300, 400 years = 5, 3, 1, 0.
```

WORKED EXAMPLE

Worked numerical — clock angle

Q: Angle between the hands at 3:40?
A: |30 × 3 − 5.5 × 40| = |90 − 220| = **130°**.

WORKED EXAMPLE

Worked numerical — calendar

Q: 1 January 2027 is a Friday. Which day is 15 August 2027?
A: Days from 1 Jan to 1 Aug = 31+28+31+30+31+30+31 = 212; add 14 more to reach 15 Aug → 226 days. 226 = 32 × 7 + 2, so move 2 days past Friday → **Sunday**.

Time, speed & distance — trains, relative speed, boats

KEY POINT

```
speed = distance / time        km/h → m/s : × 5/18
m/s → km/h : × 18/5
Equal distances at speeds a and b:   average speed = 2ab/(a + b)   (NOT the simple mean)
Same time, speeds a and b:           average speed = (a + b)/2
For a fixed distance, time ∝ 1/speed — a 25 % speed increase cuts the time by 20 %
```

REFERENCE TABLE

Situation	Distance covered ÷ speed
Train (length L) passes a **pole / man**	L ÷ train speed
Train passes a **platform / bridge** (length P)	(L + P) ÷ train speed
Two trains, **same direction**	(L_1 + L_2) ÷ **difference** of speeds
Two trains, **opposite directions**	(L_1 + L_2) ÷ **sum** of speeds

KEY POINT

Boats and streams. Boat speed in still water = b, stream = s. **Downstream = b + s**, **upstream = b − s**. From the two speeds: **b = (d + u)/2**, **s = (d − u)/2**. Average speed over a round trip is 2du/(d + u), less than b.

WORKED EXAMPLE

Worked numerical

Q: A 100 m train and a 150 m train run in opposite directions at 36 km/h and 54 km/h. How long do they take to cross each other completely?
A: Relative speed = 36 + 54 = 90 km/h = 90 × 5/18 = 25 m/s. Distance = 100 + 150 = 250 m. Time = 250/25 = **10 s**.

Time & work — rates, pipes and wages

KEY POINT

```
Work rate = 1/(days to finish alone);  rates ADD for people working together
A in a days, B in b days together:     time = ab/(a + b)
Pick total work = LCM of the days (units) → each person's units/day becomes an integer
Pipes:  inlet = +rate, outlet/leak = −rate        Wages are shared in proportion to WORK DONE
M₁D₁H₁/W₁ = M₂D₂H₂/W₂   (men × days × hours per day ÷ work is constant)
```

WORKED EXAMPLE

Worked numerical

Q: A fills a tank in 10 h and B empties it in 15 h. With both open, how long to fill the empty tank?
A: Take the tank = 30 units: A adds 3 units/h, B removes 2 units/h → net 1 unit/h → **30 h**.

Profit & loss, averages and ages

KEY POINT

```
Profit % = (SP − CP)/CP × 100  (always on COST unless told otherwise)     SP = CP × (1 ± p/100)
Marked price MP, discount d%:   SP = MP × (1 − d/100)
Successive changes multiply: (1 + a)(1 + b)
Average = sum / count.   Replacing a member changes the SUM by (new − old); so  new sum = old sum − old + new
Weighted average = Σ(nᵢ·xᵢ) / Σnᵢ        Multiplying every value by k and adding c: new average = k·avg + c
Ages: let the younger age be x; "after t years" adds t to EVERY age — the DIFFERENCE of ages never changes
```

WORKED EXAMPLE

Worked numerical

Q: The average of 10 numbers is 15. Each is doubled and then increased by 3. New average?
A: 2 × 15 + 3 = **33** (no need to know the individual numbers).

Sets & Venn diagrams

KEY POINT

```
|A ∪ B| = |A| + |B| − |A ∩ B|
|A ∪ B ∪ C| = |A| + |B| + |C| − |A∩B| − |B∩C| − |A∩C| + |A∩B∩C|
Fill a Venn diagram from the INSIDE out: start with the triple overlap, then "exactly two" regions
(pairwise overlap − triple), then "only one" regions.
Outside all circles = total − union.
```

WORKED EXAMPLE

Worked numerical

Q: In a group of 70, 45 like tea, 40 like coffee and everyone likes at least one. How many like both?
A: 70 = 45 + 40 − both → both = **15**.

Numerical estimation

KEY POINT

Round to friendly numbers, keep the **order of magnitude**, and compare the answer choices — they are usually far apart. Handy benchmarks: 1/8 = 12.5 %, 1/6 ≈ 16.7 %, 1/3 ≈ 33.3 %, 3/8 = 37.5 %, √2 ≈ 1.414, √3 ≈ 1.732, π ≈ 3.14. Check sums with the **last digit** or the **digit-sum (mod 9)** before computing in full.

```
49.8 × 20.1 / 9.95  ≈  50 × 20 / 10 = 100     (exact: 100.6)
Unit digit of 7^45: the last digits of 7ⁿ cycle 7, 9, 3, 1
(length 4); 45 mod 4 = 1 → unit digit 7
```

3 Data Interpretation

Pie charts — the 360°/100% conversion

KEY POINT

```
Degrees in a pie slice = (category's % of total) × 360° / 100
Percentage from degrees = (slice's degrees / 360°) × 100
```

WORKED EXAMPLE

Worked numerical

Q: A pie chart shows a company's expenses: Salaries 40%, Rent 25%, Marketing 15%, Utilities 10%, Other 10%. Total expenses = ₹2,00,000. (a) Find the angle for "Rent". (b) Find the actual amount spent on "Marketing".

A:

```
(a) Angle for Rent = 25% × 360° = 0.25 × 360 = 90°
(b) Marketing amount = 15% × 2,00,000 = 0.15 × 200000 = ₹30,000
```

Bar graphs & tables — classic multi-step question

WORKED EXAMPLE

Worked numerical

Q: A bar graph shows a company's profit (in ₹ lakh) over 4 years: 2021: 50, 2022: 60, 2023: 45, 2024: 75. Find: (a) the year-on-year % change from 2022 to 2023, (b) the average annual profit over all 4 years.

A:

```
(a) % change (2022→2023) = (45-60)/60 × 100 = -15/60 × 100 = -25%  (a 25% DECREASE)

(b) Average = (50+60+45+75)/4 = 230/4 = ₹57.5 lakh
```

Common DI traps

GATE FAVOURITE

GATE angle

- "% change" ALWAYS divides by the ORIGINAL (older) value, never the newer one.
- When comparing two pie charts with DIFFERENT totals, the same PERCENTAGE can represent very different ACTUAL amounts — always convert to absolute values before comparing across charts.
- Read axis labels and units carefully (lakh vs crore, thousands vs millions) — a classic easy-mark-losing mistake under time pressure.

Line graphs, multi-series data & caselets

KEY POINT

Read the scale and the units first (₹ crore? thousands? percent?). Then:

```
% change = (new − old)/old × 100                  (base = the EARLIER year)
Percentage POINTS (difference of two percentages) ≠ percent change:   20 % → 25 % is +5 points but +25 %
Profit % = (revenue − expenditure)/expenditure × 100  (on cost, unless the question says "on revenue")
Ratio questions: compare fractions, don't subtract.
Highest absolute value ≠ highest percentage.
```

WORKED EXAMPLE

Worked numerical — two series

Q: Revenue (R) and expenditure (E) in ₹ crore: 2019: 45 / 30 · 2020: 50 / 42 · 2021: 45 / 40 · 2022: 60 / 48 · 2023: 75 / 55. In which year is the profit % (on expenditure) highest?

A:

```
2019: 15/30 = 50 %   2020: 8/42 ≈ 19.0 %   2021: 5/40 = 12.5 %   2022: 12/48 = 25 %   2023: 20/55 ≈ 36.4 %
Highest percentage → 2019, even though 2023 has the highest absolute profit (₹20 crore).
```

COMMON TRAP

Trap

If A is 20 % more than B, then B is only 16.67 % less than A — the base changes. Two equal successive percentage steps from 200 to 242 are 10 % each (200 × 1.1² = 242), not 21 %.

4 Mensuration, Geometry & Probability

Mensuration — area & perimeter (2D)

REFERENCE TABLE

Shape	Area	Perimeter
Rectangle	l × b	2(l+b)
Triangle	½ × base × height	sum of 3 sides
Circle	πr^2	$2\pi r$ (circumference)
Trapezium	½ × (sum of parallel sides) × height	sum of 4 sides

Mensuration — volume & surface area (3D)

REFERENCE TABLE

Solid	Volume	Total Surface Area
Cube (side a)	a^3	$6a^2$
Cuboid (l,b,h)	l×b×h	2(lb+bh+hl)
Cylinder (r,h)	$\pi r^2 h$	$2\pi r(r+h)$
Cone (r,h,slant l)	$\frac{1}{3}\pi r^2 h$	$\pi r(r+l)$
Sphere (r)	$\frac{4}{3}\pi r^3$	$4\pi r^2$

WORKED EXAMPLE

Worked numerical

Q: A cylindrical water tank has radius 7 m and height 10 m. Find its volume (use π=22/7).

A:

```
Volume = πr²h = (22/7) × 7² × 10 = (22/7) × 49 × 10 = 22 × 7 × 10 = 1540 m³
```

Geometry basics

MUST REMEMBER

Pythagoras theorem: in a right triangle, `(hypotenuse)² = (base)² + (height)²`.
Similar triangles: corresponding angles equal, corresponding sides in the same ratio.

WORKED EXAMPLE

Worked numerical

Q: A ladder 13 m long leans against a wall, its foot 5 m from the wall's base. How high up the wall does the ladder reach?

A:

```
height² = 13² - 5² = 169 - 25 = 144  →  height = √144 = 12 m
```

Probability

KEY POINT

```
P(event) = (number of favorable outcomes) / (total possible outcomes)
P(A or B) = P(A) + P(B) - P(A and B)      [for any two events]
P(A and B) = P(A) × P(B)                   [ONLY if A and B are independent]
```

WORKED EXAMPLE

Worked numerical

Q: Two dice are rolled. Find the probability that the sum is either 7 or 11.

A:

```
Total outcomes = 6×6 = 36
Sum=7: (1,6)(2,5)(3,4)(4,3)(5,2)(6,1) → 6 outcomes
Sum=11: (5,6)(6,5) → 2 outcomes
These are mutually exclusive (can't both happen) → P(7 or 11) = (6+2)/36 = 8/36 = 2/9
```

Elementary Statistics

KEY POINT

```
Mean = (sum of all values) / (number of values)
Median = the middle value when data is sorted (average of 2 middle values if even count)
Mode = the most frequently occurring value
```

WORKED EXAMPLE

Worked numerical

Q: Find the mean, median, and mode of: 4, 8, 6, 5, 8, 9, 8.

A:

```
Mean = (4+8+6+5+8+9+8)/7 = 48/7 ≈ 6.86

Sorted: 4,5,6,8,8,8,9  (7 values, middle = 4th value)
Median = 8

Mode = 8 (appears 3 times, more than any other value)
```

5 Analytical Aptitude

Deduction vs Induction

KEY POINT

Deduction
Reasoning from general rules to a specific, GUARANTEED conclusion. "All birds have feathers. A sparrow is a bird. ∴ A sparrow has feathers." — if the premises are true, the conclusion MUST be true.

KEY POINT

Induction
Reasoning from specific observations to a general (probable, not guaranteed) pattern. "Every sparrow I've seen has feathers. ∴ probably all sparrows have feathers." — likely, but not logically certain.

Syllogisms — the classic GATE pattern

WORKED EXAMPLE

Worked numerical

Q: Statements: "All engineers are logical. Some logical people are creative." Conclusion: "Some engineers are creative." Is this conclusion valid?
A: NOT valid. "All engineers are logical" only tells you engineers are a SUBSET of logical people. "Some logical people are creative" only guarantees overlap SOMEWHERE in the logical-people group — but that overlap might fall entirely outside the engineers subset. Draw a Venn diagram mentally: Engineers ⊂ Logical, and Creative overlaps Logical somewhere — but nothing forces that overlap to touch Engineers specifically.

GATE FAVOURITE

GATE strategy
Always test syllogism conclusions by trying to draw a Venn diagram (or a counter-example scenario) where the premises hold true but the conclusion is FALSE. If you can construct even one such scenario, the conclusion is invalid — no matter how "intuitively likely" it feels.

Analogy

NOTES

Find the relationship between the first pair, then apply the SAME relationship to find the missing term.

WORKED EXAMPLE

Worked numerical

Q: Doctor : Hospital :: Teacher : ?
A: School. The relationship is "person : primary workplace". A doctor's primary workplace is a hospital; a teacher's primary workplace is a school.

WORKED EXAMPLE

Worked numerical

Q: 3 : 9 :: 5 : ? :: 7 : 49
A: 25. The relationship is "n : n²" (3²=9, 5²=25, 7²=49) — always verify your rule against BOTH given pairs before applying it to the unknown one.

Numerical relations & reasoning

WORKED EXAMPLE

Worked numerical — coding

Q: If CODE is written as DPEF (each letter shifted +1), how is WORD written?
A: Shift each letter forward by 1: W→X, O→P, R→S, D→E → **XPSE**.

WORKED EXAMPLE

Worked numerical — blood relations

Q: "A is B's father. B is C's sister. D is C's mother." How is A related to D?
A: B and C are siblings (B is C's sister, so B is female, C's sibling). A is their father, D is their mother. So **A is D's husband** (both are parents of the same children, B and C).

Seating arrangements — method

KEY POINT

Steps: (1) draw the skeleton — a row or a circle, and note which way people **face**; (2) fix the most constrained person first; (3) place all definite clues before any conditional ones; (4) branch only when forced; (5) re-check every clue against the final arrangement.

Counting: n people in a row → n! arrangements; around a circle → (n − 1)! (rotations are the same).

GATE FAVOURITE

GATE trap

In a circle, people facing the **centre** have their *left* neighbour clockwise from them; if they face **outward**, left and right swap. A row facing north has left = west and right = east. Always read which way the people face before using "left" or "right".

WORKED EXAMPLE

Worked example — row

Q: P, Q, R and S sit in a row facing north. S is at the extreme right. Q sits immediately to the left of S. R is not at either end. Who is at the extreme left, and what is the full order (left to right)?
A: S is in seat 4 and Q in seat 3. R is not at an end, so R is in seat 2 (seat 3 is taken). P takes seat 1. Order: **P, R, Q, S**; P is at the extreme left.

Rankings & ordering

KEY POINT

```
Total in a row or class = (rank from one end) + (rank from the other end) − 1
People between two positions = |difference of ranks| − 1
If two people swap places, only the ranks of those two people change.
```

WORKED EXAMPLE

Worked numerical — rank from both ends

Q: Meera is 12th from the top and 25th from the bottom of her class. How many students are there?
A: 12 + 25 − 1 = **36**.

WORKED EXAMPLE

Worked numerical — exchange of positions

Q: In a row, Arun is 7th from the left and Bala is 12th from the right. After they exchange seats, Arun is 22nd from the left. How many are in the row?
A: Arun now sits in Bala's old seat, so Bala's old seat is 22nd from the left and 12th from the right. Total = 22 + 12 − 1 = **33**.

Truth-teller / liar puzzles

MUST REMEMBER

Method: each person is a truth-teller (T) or a liar (L). Assume one person's type, follow what their statement forces on the others, and discard the case at the first contradiction. With 3 people there are only $2^3 = 8$ cases, so a small table is safe. Remember: a liar's statement must be **false as a whole** — the negation of "A and B are both liars" is "at least one of A, B is truthful".

WORKED EXAMPLE

Worked example — exactly one liar

Q: One of P, Q, R, S stole a watch. P: "I did not steal it." Q: "R stole it." R: "S stole it." S: "I did not steal it." Exactly one of the four is lying. Who stole it?
A: Test each suspect and count the false statements:

```
thief P → P, Q, R lie  (3 liars)     thief Q → Q, R lie  (2 liars)
thief R → only R lies  (1 liar) ✔    thief S → Q, S lie  (2 liars)
```

Only "R stole it" gives exactly one liar, so **R** is the thief.

Direction sense & instruction following

KEY POINT

Track position on a grid: north = +y, east = +x. A **right** turn moves the facing clockwise (N → E → S → W), a **left** turn anticlockwise. Distance from the start = $\sqrt{(\Delta x^2 + \Delta y^2)}$; direction = the direction of the displacement vector. At sunrise your shadow points west.

WORKED EXAMPLE

Worked numerical — walk with turns

Q: A walks 5 km north, turns right and walks 12 km, turns right and walks 5 km, then turns left and walks 3 km. How far, and in which direction, is A from the start?
A:

```
start (0,0)  → 5 km N  → (0,5)
turn right (facing E) → 12 km → (12,5)
turn right (facing S) →  5 km → (12,0)
turn left  (facing E) →  3 km → (15,0)
```

A is **15 km due east** of the start.

6 Spatial Aptitude

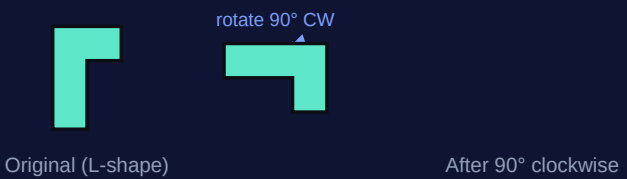
The five core transformations

REFERENCE TABLE

Transformation	What changes
Translation	Shape slides to a new position — size, orientation, and shape stay identical
Rotation	Shape turns around a fixed point by some angle (90°, 180°, etc.) — size stays the same, orientation changes
Scaling	Shape grows/shrinks uniformly — orientation stays the same, size changes
Mirroring (reflection)	Shape is flipped across a line — becomes a mirror image (left-right or up-down reversed)
Assembling/ Grouping	Multiple pieces combine to form a larger recognizable shape (jigsaw-style)

Rotation example

DIAGRAM



NOTES

Notice the L-shape's "foot" that pointed DOWN-RIGHT now points DOWN-LEFT after a 90° clockwise turn — track which specific corner/edge moves where, rather than trying to visualize the whole shape at once.

Mirror images

COMMON TRAP

Key rule

A **mirror image** (reflected left-right, like a vertical mirror) flips horizontal orientation but keeps vertical orientation unchanged. A **water image** (reflected top-bottom, like a reflection in a pond) does the reverse — flips vertical but keeps horizontal unchanged. Letters like **A, H, I, M, O, T, U, V, W, X, Y** look identical in a vertical mirror (they're symmetric left-right); letters like **B, C, D, E, H, I, O, X** look identical in a water/horizontal reflection (symmetric top-bottom).

Paper folding & cutting

WORKED EXAMPLE

Worked numerical

Q: A square paper is folded in half twice (first vertically, then horizontally), and a small triangle is cut from the folded corner (the corner where all folds meet). How many holes appear when fully unfolded?
A: Each fold doubles the number of layers at that corner. 2 folds → 2×2 = **4 layers** at the folded corner → cutting through all layers at once produces **4 holes** when unfolded, arranged symmetrically (one in each quadrant of the original square).

Cube/net folding

NOTES

A flattened "net" of 6 squares folds into a cube. The key skill: identify which faces become OPPOSITE each other (opposite faces never share an edge in the net) versus ADJACENT. In a standard cross-shaped net, the four squares in the middle row all become adjacent side faces, while the top and bottom squares fold to become the cube's top and bottom — opposite each other.

Counting figures & embedded figures

KEY POINT

Count **by size**, never by eye: label the vertices, count all figures of the smallest size, then the next size up, and add. For an embedded (hidden) figure, rotate or reflect the target mentally and check it fits within the larger figure *without altering any line*.

REFERENCE TABLE

n × n grid	Squares = n(n+1)(2n+1)/6	Rectangles (all) = [n(n+1)/2]²
1	1	1
2	5	9
3	14	36
4	30	100
5	55	225

FORMULA / RULE

Triangles: if the base of a triangle is divided into n equal parts and every division point is joined to the opposite apex, the number of triangles is n(n+1)/2 (n = 5 gives 15). Non-square rectangles = all rectangles − squares.

WORKED EXAMPLE

Worked numerical — 4 × 4 grid

Q: How many squares and how many rectangles (squares included) are in a 4 × 4 grid?
A: Squares: 16 (1×1) + 9 (2×2) + 4 (3×3) + 1 (4×4) = **30**. Rectangles: choose 2 of the 5 vertical lines and 2 of the 5 horizontal lines → C(5,2) × C(5,2) = 10 × 10 = **100**, of which 70 are non-square.

Dice & cubes

KEY POINT

A cube has 6 faces, 8 corners and 12 edges, and **three faces meet at every corner**. So faces seen together in one view are always **adjacent — never opposite**. Ordinary dice have opposite faces summing to 7, but a GATE puzzle does not assume that unless it says so.

Method: (1) from each view list the neighbours of every face; (2) a face with four known neighbours has the one remaining face as its opposite; (3) remove numbers already paired and repeat; (4) use the clockwise order around a corner only if adjacency alone does not settle it.

WORKED EXAMPLE

Worked example — two views

Q: One view of a cube shows faces 1, 2 and 5 at a corner; another shows 1, 6 and 3 at a corner. Which face is opposite 1?
A: The views make 2, 5, 6 and 3 all neighbours of face 1. That is four of the five other faces, so the remaining face, **4**, is opposite 1.

Figure series, patterns & abstract reasoning

MUST REMEMBER

For a series of figures, **track one feature at a time**: (1) **rotation** per step (angle and direction), (2) **count** change (dots, lines, sides), (3) **shading / colour** toggling, (4) **position** moving around a cycle. Different features usually have **different periods**; the whole figure repeats after the LCM of the periods.

```
Rotation by θ per step repeats the direction when kθ is a
multiple of 360°  → k = 360/gcd(θ, 360)
Direction after k steps = (k × θ) mod 360 :  clockwise from
the start.  Up 0° · Right 90° · Down 180° · Left 270°
Odd-one-out figure: find the property that all but one share
(symmetry, number of sides, shaded area, closed/open).
```

WORKED EXAMPLE

Worked numerical

Q: An arrow starts pointing east and rotates 90° anticlockwise each step. Which way does it point after 7 steps?
A: 7 × 90 = 630; 630 mod 360 = 270° anticlockwise from east → east → north (90°) → west (180°) → **south** (270°).

7 Problem → Standard Method Playbook

METHOD

1. Percentage / ratio wording

Translate the statement into a base quantity → ratio/equation. For successive percentage changes, use multiplication of factors: `new = old × (1±r/100)`. For “more than / less than”, keep the denominator explicit.

METHOD

2. Remainders / divisibility

Reduce numbers modulo the divisor. For powers, look for cycles; for divisibility, factor the divisor and apply the relevant tests. Do not assume the remainder distributes over multiplication without taking modulo.

METHOD

3. Mixture / alligation

Set quantities around the mean concentration/price. For two components, `quantity of cheaper : quantity of dearer = (dearer−mean) : (mean−cheaper)`.

METHOD

4. Critical reasoning

Identify conclusion → premises → hidden assumption. For a necessary assumption, negate the option: if the argument collapses, it is necessary. For strengthen/weaken, ask whether the option changes the causal or logical link rather than merely repeating a premise.

METHOD

5. Verbal deduction / syllogism

Translate “all/no/some” into set inclusion/exclusion. Test whether the conclusion follows in *every* model satisfying the premises. “Some A are B” does not imply “some B are all A”.

METHOD

6. Seating / ranking

Draw slots. Fix absolute positions first, then relative constraints. For circular seating, remove rotational symmetry by fixing one person. For rankings, convert “from left/right” into a common coordinate before combining.

METHOD

7. Spatial / cube / dice

Track adjacency and opposites. Faces seen together are adjacent. For cube rotations, preserve opposite pairs; for counting figures, count by size systematically rather than visually guessing.

METHOD

8. Speed / time / work

Convert to one unit system (km/h ↔ m/s: ×5/18). For trains add the lengths and use relative speed; for work questions take total work = LCM of the days and compute units per day. Equal-distance round trips use 2ab/(a+b).

METHOD

9. Sets, DI and estimation

Venn: fill from the triple intersection outward. DI: write the formula for the quantity first (profit % is on cost) and separate percent from percentage points. Estimation: round, check the order of magnitude, then eliminate options.

METHOD

10. Figure series

Treat each feature (rotation, count, shading, position) as its own sequence and find its period; the combined period is the LCM.

8 🎯 Representative GATE-Style Worked Problems

NOTE

Number system

Q: Find the remainder when `7^100` is divided by 13.
A: Powers of 7 modulo 13 cycle: `7, 10, 5, 9, 11, 12, 6, 3, 8, 4, 2, 1`, period 12. Since 100 ≡ 4 (mod 12), remainder = **9**.

NOTE

Mixture

Q: Rice costing ₹40/kg and ₹60/kg is mixed to obtain a mixture costing ₹45/kg. Find the ratio of quantities.
A: By alligation, cheaper : dearer = (60−45):(45−40) = 15:5 = **3:1**.

NOTE

Critical reasoning

Q: “The new study method improved scores because students who used it scored higher.” Which observation most weakens the conclusion?
A: If students who chose the method were already higher-performing before using it, the observed difference has an alternative explanation. That directly weakens the causal claim.

NOTE

Ranking

Q: A is 12th from the top and 19th from the bottom. How many people are there?
A: Total = 12 + 19 − 1 = **30**. Subtract 1 because A is counted from both ends.

NOTE

Calendar

Q: If 1 January is Monday in a non-leap year, what day is 1 March?
A: January + February = 31 + 28 = 59 days. 59 mod 7 = 3, so move three weekdays from Monday → **Thursday**.

NOTE

Spatial counting

Q: How many squares are in a 4×4 grid?
A: Sum `1²+2²+3²+4² = 30`. Equivalently `n(n+1)(2n+1)/6` for n=4 → **30**.

RULES & FACTS

1. 60 km/h; 13.5 s
2. 54 km/h; 12 s
3. 54 km/h; 13.5 s
4. 54 km/h; 10.8 s

FORMULA / WORKING

```
Distance while crossing the platform = 180 + 270 = 450 m in 30 s → speed = 15 m/s = 15 × 18/5 = 54 km/h
Man walking the same way: relative speed = 54 − 6 = 48 km/h = 48 × 5/18 = 40/3 m/s
Time to pass him = train length / relative speed = 180 ÷ (40/3) = 13.5 s
(12 s ignores the man's motion; 10.8 s wrongly adds the speeds, as if he were walking towards the train.)
```

NOTES

Correct: Option 3 — **54 km/h; 13.5 s**.

RULES & FACTS

1. 9 km/h; 9 h
2. 12 km/h; 8 h 20 min
3. 10 km/h; 7 h 30 min
4. 10 km/h; 8 h 20 min

FORMULA / WORKING

```
Downstream d = 24/2 = 12 km/h;  upstream u = 24/3 = 8 km/h
Still water b = (d + u)/2 = 10 km/h;  stream s = (d − u)/2 = 2 km/h
Time = 40/12 + 40/8 = 3 h 20 min + 5 h = 8 h 20 min
```

NOTES

Correct: Option 4 — **10 km/h** and **8 h 20 min**.

RULES & FACTS

1. 9 days; ₹1,500
2. 9 days; ₹1,800
3. 8 days; ₹1,500
4. 10 days; ₹1,500

FORMULA / WORKING

```
Let the job take T days. B works all T days; A works (T − 3) days.
Take the job = 36 units → A does 3 units/day, B does 2 units/day.
3(T − 3) + 2T = 36  →  5T = 45  →  T = 9 days
A's work = 3 × 6 = 18 units, B's work = 2 × 9 = 18 units → equal shares of the wages: A gets ₹1,500.
```

NOTES

Correct: Option 1 — **9 days**; A's share **₹1,500**.

RULES & FACTS

1. 42 years
2. 48 years
3. 36 years
4. 30 years

FORMULA / WORKING

```
Son = x, father = 3x.  After 12 years:  3x + 12 = 2(x + 12)  →  3x + 12 = 2x + 24  →  x = 12
Father = 3 × 12 = 36.   Check: in 12 years 48 and 24 → 48 = 2 × 24 ✓
```

NOTES

Correct: Option 3 — **36 years**.

RULES & FACTS

1. 60 exactly one; 0 none
2. 70 exactly one; 10 none
3. 60 exactly one; 10 none
4. 50 exactly one; 20 none

FORMULA / WORKING

```
Only M∩P = 25 − 10 = 15;  only P∩C = 20 − 10 = 10;  only M∩C = 15 − 10 = 5;  all three = 10
Only M = 60 − (15 + 5 + 10) = 30;  only P = 50 − (15 + 10 + 10) = 15;  only C = 40 − (5 + 10 + 10) = 15
Exactly one = 30 + 15 + 15 = 60
Union = 60 + 50 + 40 − 25 − 20 − 15 + 10 = 100  →  none = 110 − 100 = 10
```

NOTES

Correct: Option 3 — **60** study exactly one subject; **10** study none.

RULES & FACTS

1. 20 %; ₹100
2. 13.4 %; ₹67
3. 14 %; ₹70
4. 12 %; ₹60

FORMULA / WORKING

```
Marked price = 500 × 1.40 = 700
After the discounts: SP = 700 × 0.9 × 0.9 = 567   (one 19 % discount, not 20 %)
Profit = 567 − 500 = ₹67  →  67/500 = 13.4 %
```

NOTES

Correct: Option 2 — **13.4 %**, profit **₹67**.

RULES & FACTS

1. 2023; 40 %
2. 2019; 66.7 %
3. 2019; 40 %
4. 2023; 66.7 %

FORMULA / WORKING

```
Profit %: 2019 = 15/30 = 50 %;  2020 = 8/42 ≈ 19.0 %;  2021 = 5/40 = 12.5 %;  2022 = 12/48 = 25 %;  2023 = 20/55 ≈ 36.4 %
Highest percentage: 2019 (although 2023 has the largest absolute profit, ₹20 crore)
Revenue growth 2021 → 2023 = (75 − 45)/45 = 30/45 = 66.7 %   (the base is the earlier year, 2021)
```

NOTES

Correct: Option 2 — **2019** and **66.7 %**.

RULES & FACTS

1. Down; step 8
2. Left; step 4
3. Left; step 8
4. Right; step 8

FORMULA / WORKING

```
After 10 steps: 10 × 135 = 1350°;  1350 mod 360 = 270° clockwise from up → up 0°, right 90°, down 180°, left 270° → LEFT
First return to up: 135k must be a multiple of 360 → k = 360/gcd(135, 360) = 360/45 = 8   (8 × 135 = 1080 = 3 × 360)
```

NOTES

Correct: Option 3 — **Left**, and the arrow first points up again at **step 8**.

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- Ranking total = rank from top + rank from bottom − 1; the same person is counted from both ends.
- Alligation ratio = (dearer−mean):(mean−cheaper).
- Squares in an n×n grid = n(n+1)(2n+1)/6; rectangles = [n(n+1)/2]².
- Percentage change = (new-old)/old × 100%; successive percentage changes are multiplicative, not simply additive.
- **For speed, distance, and time:** distance = speed × time; use consistent units before calculation.
- Simple interest = PRT/100; compound interest depends on compounding frequency and period.

LAST-MINUTE · MUST REMEMBER

- **Trains:** pole = L/v; platform = (L+P)/v; two trains = (L_1+L_2)/relative speed (difference if same direction, sum if opposite).
- **Venn (3 sets):** fill from the triple overlap outward; $|A \cup B \cup C|$ = Σ singles − Σ pairs + triple.
- In aptitude, translate the wording into a mathematical model before calculating; avoid pattern-matching without checking constraints.
- For ratio questions, multiply all ratio parts by the same positive factor; ratios do not by themselves give absolute quantities.
- In probability, define the sample space carefully; mutually exclusive and independent are different concepts.
- For data interpretation, estimate first when options are widely separated; then calculate only as precisely as necessary.
- Geometry questions should begin by identifying the relevant invariant/standard formula rather than expanding algebra blindly.
- **Critical reasoning:** a weakener offers an alternative cause, a strengthener rules one out, and a necessary assumption collapses the argument when negated.

LAST-MINUTE · COMMON TRAPS

- In seating/cube questions, preserve relative constraints; do not infer adjacency merely from visual placement.
- Average of averages is not generally the average of the group averages unless group sizes are equal or weighted appropriately.
- $P(A \cup B)=P(A)+P(B)-P(A \cap B)$; do not omit the intersection unless events are mutually exclusive.
- 'All A are B' does not imply 'All B are A'; only the contrapositive (not B → not A) is guaranteed.

LAST-MINUTE · SHORTCUTS

- **Estimation:** round to friendly numbers, check the order of magnitude, eliminate options. Unit digits cycle (7: 7,9,3,1).
- Use approximation, ratio comparison, cancellation, and option elimination whenever the question permits it.

LAST-MINUTE · GATE FAVOURITES

- **Figure series:** track rotation, count, shading and position separately; combined period = LCM of the periods.
- **Number-system questions:** reduce powers modulo the divisor and look for a cycle before calculating large exponents.

Engineering Mathematics

GATE CSE 2027 · complete formula, rule, method & trap sheet

36 reference boxes · **5** chapters ·

Linear Algebra | Calculus | Probability & Statistics | Problem → Standard Method Playbook | Last-minute one-liners

1 Linear Algebra

Determinant properties (n × n matrix) Core

FORMULA / RULE

```
det(AB) = det(A)·det(B)
det(Aᵀ) = det(A)
det(kA) = kⁿ·det(A)
det(A⁻¹) = 1/det(A)
Swapping two rows → sign changes
Two equal rows or a zero row → det = 0
Row op "add k·row i to row j" → unchanged
Triangular matrix: product of diagonal
```

FORMULA / RULE

Invertibility — all equivalent:

```
A is invertible
det(A) ≠ 0
rank(A) = n
Ax = 0 has only x = 0
columns are linearly independent
no eigenvalue is 0
```

WORKED EXAMPLE

Worked numerical

If A is 3×3 with det(A) = 4, then det(2A) = $2^3 \cdot 4$ = **32**; $\det(A^{-1}) = 1/4$; $\det(A^2) = 16$.

Rank and systems of linear equations Core

RULES & FACTS

- **Rank** = number of non-zero rows in row-echelon form = dimension of the column space. rank(A) ≤ min(m, n).
- **Rank–nullity**: rank(A) + nullity(A) = n (number of columns). A 5×7 matrix of rank 3 has nullity **4**.
- Ax = b is **consistent** iff rank(A) = rank([A | b]). Unique solution iff the common rank = n (number of unknowns); infinitely many iff rank < n; none if the ranks differ.
- **Homogeneous** Ax = 0 always has the trivial solution; a square system has a non-trivial one iff **det(A) = 0**. If m < n there is always a non-trivial solution.

WORKED EXAMPLE

Worked numerical — solving a 3×3 system

```
x +  y +  z =  6
     2y + 5z = −4
2x + 5y −  z = 27
R3 ← R3 − 2R1:   3y − 3z = 15  → y − z = 5
From 2y + 5z = −4 and y = z + 5:  2z + 10 + 5z = −4 → z =
−2, y = 3, x = 6 − 3 + 2 = 5
```

Solution (x, y, z) = **(5, 3, −2)**; the coefficient matrix has rank 3, so it is unique.

WORKED EXAMPLE

Worked numerical — rank

`[[1,2,3],[4,5,6],[7,8,9]]` : R2 − 4R1 = (0,−3,−6), R3 − 7R1 = (0,−6,−12) = 2·(R2 − 4R1) → only 2 independent rows → **rank 2**, det = 0.

Eigenvalues and eigenvectors Core

NOTES

λ is an eigenvalue of A if Ax = λx for some x ≠ 0, i.e. det(A − λI) = 0 (the characteristic equation).

RULES & FACTS

- **Sum of eigenvalues = trace** (sum of diagonal); **product of eigenvalues = det(A)**.
- Triangular (and diagonal) matrices: eigenvalues are the diagonal entries.
- Eigenvalues of A^k are λ^k, of A^{-1} are $1/\lambda$, of A + cI are λ + c, of A^T are the same as A.
- **Cayley–Hamilton**: every matrix satisfies its own characteristic equation.
- Symmetric real matrices: all eigenvalues real, eigenvectors for distinct eigenvalues orthogonal. Eigenvectors for **distinct** eigenvalues are linearly independent; n distinct eigenvalues ⇒ diagonalisable.
- A singular matrix has 0 as an eigenvalue.

WORKED EXAMPLE

Worked numerical — 2×2

A = [[4,1],[2,3]]: trace 7, det 10 → $\lambda^2 - 7\lambda + 10 = 0$ → **λ = 5 and 2**. Check: 5 + 2 = 7 ✓, 5 × 2 = 10 ✓.

WORKED EXAMPLE

Worked numerical — function of a matrix

A = [[2,0,0],[1,3,0],[4,5,6]] is lower triangular, so λ = 2, 3, 6. Then $A^2 + I$ has eigenvalues 4+1, 9+1, 36+1 = 5, 10, 37, so `det(A² + I) = 5·10·37 = 1850`, and `trace(A²) = 4 + 9 + 36 = 49`.

LU decomposition Core

NOTES

A square matrix (with non-zero pivots) factors as A = LU, with L lower-triangular (1s on the diagonal) and U upper-triangular. Solve Ax = b by forward substitution (Ly = b) then back substitution (Ux = y). Cost: about **$(2/3)n^3$** operations versus much more for computing the inverse.

2 Calculus

Standard limits Core

KEY POINT

```
lim (x→0) sin x / x = 1
lim (x→0) tan x / x = 1
lim (x→0) (1 − cos x)/x² = 1/2
lim (x→0) (eˣ − 1)/x = 1
lim (x→0) ln(1+x)/x = 1
```

KEY POINT

```
lim (x→∞) (1 + 1/x)ˣ = e
lim (x→∞) (1 + a/x)ˣ = eᵃ
lim (x→0) (1 + x)^(1/x) = e
lim (x→∞) xⁿ / eˣ = 0
lim (x→∞) ln x / x = 0
```

NOTES

L'Hôpital's rule: if f/g gives 0/0 or ∞/∞, then lim f/g = lim f'/g' (when the latter exists). Other indeterminate forms ($0\cdot\infty$, $\infty-\infty$, 1^∞, 0^0, ∞^0) are first rewritten into a quotient. Taylor expansion also works: $e^x = 1 + x + x^2/2! + \dots$, $\sin x = x - x^3/3! + \dots$, $\cos x = 1 - x^2/2! + \dots$.

WORKED EXAMPLE

Worked numerical

$\lim_{x\to 0} (e^x - 1 - x)/x^2$: $0/0 \to (e^x - 1)/(2x) \to 0/0 \to e^x/2 \to \mathbf{1/2}$

$\lim_{x\to\infty} (1 + 2/x)^x = \mathbf{e^2}$

Continuity and differentiability Core

RULES & FACTS

- f is continuous at a if $\lim_{x\to a} f(x) = f(a)$. **Differentiable ⇒ continuous**; the converse is false.
- f(x) = |x| is continuous everywhere but **not differentiable at 0** (a corner); $f(x) = x^{1/3}$ is continuous but has an infinite slope at 0.
- **Intermediate value theorem**: a continuous f on [a,b] takes every value between f(a) and f(b). **Extreme value theorem**: a continuous f on a closed interval attains a maximum and a minimum.

Mean value theorems Core

RULES & FACTS

- **Rolle**: f continuous on [a,b], differentiable on (a,b), f(a) = f(b) ⇒ some c with f'(c) = 0.
- **Lagrange (MVT)**: some $c \in (a,b)$ with $f'(c) = (f(b) - f(a))/(b - a)$.

WORKED EXAMPLE

Worked numerical

f(x) = x² on [1, 3]: slope of the chord = (9 − 1)/2 = 4, and f'(c) = 2c = 4 → **c = 2**.

Maxima and minima Core

NOTES

Critical points: f'(x) = 0 (or undefined). Second-derivative test: f'' > 0 → local min, f'' < 0 → local max, f'' = 0 → inconclusive. For the **absolute** max/min on a closed interval, compare critical values **and the endpoint values**.

WORKED EXAMPLE

Worked numerical

Q: f(x) = 2x³ − 9x² + 12x + 5 on [0, 3]. Find the local and absolute extrema.
A: f'(x) = 6x² − 18x + 12 = 6(x−1)(x−2) → x = 1, 2. f''(x) = 12x − 18: f''(1) = −6 (local max), f''(2) = 6 (local min).

```
f(0) = 5     f(1) = 10 (local max)     f(2) = 9 (local min)
f(3) = 14
Absolute maximum on [0,3] = 14 (at the endpoint x = 3)
Absolute minimum = 5 (at x = 0)
```

Integration Core

RULES & FACTS

- Fundamental theorem: $\int_a^b f = F(b) - F(a)$. By parts: $\int u\,dv = uv - \int v\,du$.
- Odd function over [−a, a]: integral = 0. Even function: $2\int_0^a$.
- Wallis: $\int_0^{\pi/2} \sin^n x\,dx$ for odd n = (n−1)(n−3)…2 / (n(n−2)…3).
- Improper: $\int_1^\infty 1/x^p\,dx$ converges iff p > 1; $\int_0^1 1/x^p\,dx$ converges iff p < 1. $\int_0^\infty e^{-x}\,dx = 1$.

WORKED EXAMPLE

Worked numerical

$\int_0^1 x\,e^x\,dx = [x\,e^x - e^x]_0^1 = (e - e) - (0 - 1) = \mathbf{1}$

$\int_0^{\pi/2} \sin^3 x\,dx = 2/3$ $\qquad$ $\int_0^{\pi} \sin x\,dx = 2$

Series Extension

RULES & FACTS

- **Geometric**: $\Sigma\, a\cdot r^n$ converges iff |r| < 1, to a/(1−r). $\Sigma_{n\ge 0}\,(1/2)^n = \mathbf{2}$.
- **p-series**: $\Sigma\, 1/n^p$ converges iff p > 1 (so the harmonic series $\Sigma\, 1/n$ diverges).
- Ratio test: $L = \lim |a_{n+1}/a_n|$; converges if L < 1, diverges if L > 1, inconclusive at 1. A convergent series must have terms → 0 (converse false).
- Alternating series with decreasing terms → 0 converges (Leibniz).

L'Hôpital's rule and Taylor series High-yield

RULES & FACTS

- **L'Hôpital:** if f/g gives 0/0 or ∞/∞ at a point, then lim f/g = lim f'/g' provided the latter limit exists; repeat if the form persists. Rewrite $0\cdot\infty$ as a quotient and $\infty - \infty$ as a single fraction first.
- **Maclaurin series:** $e^x = 1 + x + x^2/2! + x^3/3! + \dots$ · $\sin x = x - x^3/3! + x^5/5! - \dots$ · $\cos x = 1 - x^2/2! + x^4/4! - \dots$ · $\ln(1 + x) = x - x^2/2 + x^3/3 - \dots$ $(|x| < 1)$ · $1/(1 - x) = 1 + x + x^2 + \dots$ $(|x| < 1)$.

WORKED EXAMPLE

Worked numerical

Q: Evaluate $\lim_{x\to 0} (\sin x - x)/x^3$ and $\lim_{x\to 0} (e^x - 1 - x)/x^2$.
A: Using series: $\sin x - x = -x^3/6 + \dots$, so the first limit is **−1/6**. Also $e^x - 1 - x = x^2/2 + \dots$, so the second is **1/2**. (L'Hôpital agrees: three differentiations give $(-\cos x)/6 \to -1/6$.) Also $\lim x\cdot\ln x = \mathbf{0}$ as $x \to 0^+$, because a power beats a logarithm.

Improper integrals, Gamma and Beta Extension

RULES & FACTS

- **p-test:** $\int_1^\infty x^{-p}\,dx$ converges iff **p > 1**; $\int_0^1 x^{-p}\,dx$ converges iff **p < 1**. $\int_0^\infty e^{-x}\,dx = 1$.
- **Gamma:** $\Gamma(n) = \int_0^\infty x^{n-1} e^{-x}\,dx$, with $\Gamma(n + 1) = n\cdot\Gamma(n)$, so $\Gamma(n + 1) = n!$ for integers, and $\Gamma(\tfrac{1}{2}) = \sqrt{\pi}$.
- **Beta:** $B(m, n) = \int_0^1 x^{m-1}(1 - x)^{n-1}\,dx = \Gamma(m)\Gamma(n)/\Gamma(m + n)$. Example: B(2, 3) = (1!·2!)/4! = **1/12**.

3 Probability & Statistics

Foundations Core

RULES & FACTS

- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$. $P(A^c) = 1 - P(A)$.
- **Conditional**: $P(A|B) = P(A \cap B)/P(B)$. **Independent** iff $P(A \cap B) = P(A)P(B)$. **Mutually exclusive** events with non-zero probability are **not** independent.
- **Total probability**: $P(B) = \Sigma\, P(B|A_i)P(A_i)$ over a partition. **Bayes**: $P(A_i|B) = P(B|A_i)P(A_i) / P(B)$.

WORKED EXAMPLE

Worked numerical — Bayes

Q: A disease affects 1% of people. A test is positive for 99% of sick people and for 5% of healthy people. If someone tests positive, what is the probability they are sick?
A:

```
P(+) = 0.01·0.99 + 0.99·0.05 = 0.0099 + 0.0495 = 0.0594
P(sick | +) = 0.0099 / 0.0594 = 1/6 ≈ 0.167
```

Even with a very accurate test, a rare condition gives mostly false positives.

WORKED EXAMPLE

Worked numerical — two dice

P(sum = 7) = 6/36 = 1/6. Given that **at least one die shows 6** (11 outcomes), P(sum ≥ 10) = favourable {(4,6),(6,4),(5,6),(6,5),(6,6)} = **5/11**. Expected maximum of two dice = **161/36**.

Random variables Core

RULES & FACTS

- **E[X]** = Σ x·p(x) (or ∫ x f(x) dx). Linearity always holds: E[X + Y] = E[X] + E[Y], even for dependent X, Y.
- **Var(X)** = E[X²] − (E[X])². Var(aX + b) = a²Var(X). For **independent** X, Y: Var(X + Y) = Var(X) + Var(Y).
- Cov(X,Y) = E[XY] − E[X]E[Y]; independent ⇒ Cov = 0 (converse false). Correlation ρ ∈ [−1, 1].
- **Chebyshev**: P(|X − μ| ≥ kσ) ≤ 1/k². **Markov**: P(X ≥ a) ≤ E[X]/a for X ≥ 0.
- Expected number of fixed points of a random permutation = **1** (for any n). Expected tosses of a fair coin until the first head = **2**.

Standard distributions Core

REFERENCE TABLE

Distribution	Model	Mean	Variance
Bernoulli(p)	one trial	p	p(1−p)
Binomial(n, p)	successes in n trials; P(k) = C(n,k)$p^k(1-p)^{n-k}$	np	np(1−p)
Geometric(p)	trials until first success	1/p	(1−p)/p²
Poisson(λ)	events in a fixed interval; P(k) = $e^{-\lambda}\lambda^k/k!$	λ	λ
Uniform(a, b)	continuous, equally likely	(a+b)/2	(b−a)²/12
Exponential(λ)	waiting time; f = $\lambda e^{-\lambda x}$	1/λ	1/λ²
Normal(μ, σ²)	bell curve	μ	σ²

RULES & FACTS

- Exponential is **memoryless**: P(X > s + t | X > s) = P(X > t) = $e^{-\lambda t}$. Geometric is the discrete memoryless one.
- Normal: about 68%, 95%, 99.7% of the mass lies within 1, 2, 3 standard deviations. The sum of independent normals is normal. **Central limit theorem**: the mean of many i.i.d. variables is approximately normal.
- Binomial(n, p) → Poisson(np) for large n, small p.

WORKED EXAMPLE

Worked numerical

```
Binomial n=10, p=0.3:  mean = 3,  variance = 10·0.3·0.7 = 2.1
Binomial n=5,  p=0.5:  P(X = 3) = C(5,3)/32 = 10/32 = 0.3125
Poisson λ = 2:         P(X = 0) = e⁻² ≈ 0.1353
Uniform(0, 6):         mean 3, variance 36/12 = 3
Birthday problem:      with 23 people, P(shared birthday) ≈ 0.507
```

Descriptive statistics Core

WORKED EXAMPLE

Worked numerical — data {2, 3, 5, 7, 8}

```
mean = 25/5 = 5      median = 5 (middle value)
deviations: −3, −2, 0, 2, 3  →  squares sum = 26
population variance = 26/5 = 5.2        sample variance = 26/4 = 6.5
```

NOTES

For a right-skewed distribution mean > median > mode; for a symmetric one they coincide. Sample variance divides by n − 1 (unbiased); population variance divides by n.

PMF, PDF and CDF Core

REFERENCE TABLE

	Discrete (pmf p)	Continuous (pdf f)
Valid when	p(x) ≥ 0 and Σ p(x) = 1	f(x) ≥ 0 and ∫ f(x) dx = 1
Probability of an event	P(X ∈ A) = $\Sigma_{x\in A}$ p(x)	P(a ≤ X ≤ b) = $\int_a^b$ f(x) dx; **P(X = x) = 0** for every single point
CDF F(x) = P(X ≤ x)	Step function	Continuous, with f = dF/dx

NOTES

Every CDF is non-decreasing, runs from 0 to 1, and P(a < X ≤ b) = F(b) − F(a). A density can exceed 1 (only its integral must equal 1); a probability cannot.

WORKED EXAMPLE

Worked numerical

Q: X has density f(x) = kx on [0, 2] and 0 elsewhere. Find k, the CDF, P(X > 1), E[X] and Var(X).
A: $\int_0^2$ kx dx = 2k = 1, so **k = 1/2**. CDF: F(x) = x²/4 on [0, 2]. P(X > 1) = 1 − F(1) = 1 − 1/4 = **3/4**. E[X] = $\int_0^2$ x·(x/2) dx = 8/6 = **4/3**. E[X²] = $\int_0^2$ x²·(x/2) dx = 2, so Var(X) = 2 − 16/9 = **2/9**.

Joint and marginal distributions High-yield

RULES & FACTS

- The **marginal** distribution comes from summing (or integrating) the joint over the other variable: $p_X(x) = \Sigma_y$ p(x, y).
- X and Y are **independent** iff p(x, y) = $p_X(x)\cdot p_Y(y)$ for **every** pair. One failing cell is enough to show dependence.

WORKED EXAMPLE

Worked numerical

Q: P(X,Y): (0,0) = 0.1, (0,1) = 0.3, (1,0) = 0.2, (1,1) = 0.4. Find the marginals, test independence and compute Cov(X, Y).
A: P(X=0) = 0.4, P(X=1) = 0.6; P(Y=0) = 0.3, P(Y=1) = 0.7. Independence would need p(0,0) = 0.4 × 0.3 = 0.12, but p(0,0) = 0.1, so they are **dependent**. E[XY] = P(X=1, Y=1) = 0.4; E[X]E[Y] = 0.6 × 0.7 = 0.42; Cov = 0.4 − 0.42 = **−0.02**.

Normal distribution and z-scores Core

FORMULA / RULE

Standardise: if X ~ N(μ, σ²) then $Z = (X - \mu)/\sigma$ ~ N(0, 1). Use Φ(z) = P(Z ≤ z) with the symmetry $\Phi(-z) = 1 - \Phi(z)$. Useful values: Φ(0) = 0.5, Φ(1) = 0.8413, Φ(1.5) = 0.9332, Φ(1.96) = 0.9750, Φ(2) = 0.9772. The central 95% lies within ±1.96σ.

WORKED EXAMPLE

Worked numerical

Q: X ~ N(50, 100). Find P(X > 60) and P(40 < X < 65).
A: σ = 10. P(X > 60): z = 1, so 1 − Φ(1) = 1 − 0.8413 = **0.1587**. P(40 < X < 65): z from −1 to 1.5, so Φ(1.5) − Φ(−1) = 0.9332 − 0.1587 = **0.7745**.

Hypergeometric distribution Extension

NOTES

Drawing n items **without replacement** from N items of which K are "successes": `P(X = k) = C(K, k)·C(N − K, n − k) / C(N, n)`, with mean `n·K/N`. Compare the binomial, which assumes **with** replacement (independent trials) and has the same mean n·p but a larger variance.

WORKED EXAMPLE

Worked numerical

Q: A box has 20 parts, 5 of them defective. Four are drawn without replacement. Find P(exactly 1 defective) and the expected number of defectives.

A: P = C(5,1)·C(15,3)/C(20,4) = 5 × 455 / 4845 ≈ **0.4696**. Mean = 4 × 5/20 = **1**.

4 Problem → Standard Method Playbook

METHOD

Logic, Relations, Counting, Recurrences & Graphs

The problem → method tables for all discrete topics are kept in one place: Discrete Mathematics → Problem → Standard Method Guide.

METHOD

Linear Algebra Methods

Problem	Standard method	Shortcut / trap
System Ax=b	Row-reduce augmented matrix; compare ranks.	Unique iff rank(A)=rank([A|b])=number of variables.
Find eigenvalues	det(A−λI)=0.	For triangular matrices, eigenvalues are diagonal entries.
Find eigenvectors	Solve (A−λI)x=0 for each λ.	Multiplicity of eigenvalue ≠ automatically number of eigenvectors.
Use Cayley–Hamilton	Find characteristic polynomial p(λ), then substitute A: p(A)=0.	Useful for reducing high powers of A.
Determine invertibility	det(A)≠0 ⇔ A invertible ⇔ rank(A)=n.	For square A.

METHOD

Calculus

Pattern	Method	GATE cue
Maximum/minimum	Find critical points, test endpoints/boundary and compare values.	Don't stop after f′=0.
Improper/infinite series convergence	Choose comparison, ratio, root or integral test based on term structure.	Ratio/root are especially useful for factorials/exponentials.

METHOD

Probability & Statistics Methods

Pattern	Standard method	Formula / trap
Conditional probability	P(A|B)=P(A∩B)/P(B).	Condition changes the sample space.
Reverse conditional / diagnosis	Bayes theorem.	P(A|B)=P(B|A)P(A)/P(B).
Discrete expectation	E[X]=Σx·P(X=x).	E[g(X)]=Σg(x)P(X=x), not generally g(E[X]).
Variance	Var(X)=E[X²]−E[X]².	Var(aX+b)=a²Var(X).
Binomial	n independent Bernoulli trials, constant p; use C(n,k)p^k(1−p)^{n−k}.	E=np, Var=np(1−p).
Poisson	Count events over fixed interval with rate λ; P(X=k)=e^{−λ}λ^k/k!.	E=Var=λ.
Exponential	Continuous waiting time; f(x)=λe^{−λx}, x≥0.	Memoryless property.
Normal	Standardize Z=(X−μ)/σ.	Use symmetry around 0.
Joint → marginal	Sum/integrate joint distribution over the other variable.	Check support before integrating.
Independence	Discrete: p(x,y)=pX(x)pY(y); continuous: fXY=fXfY.	Zero covariance does not generally imply independence.

METHOD

🧮 Representative GATE-style Worked Problems

Problem 1 · Probability

Let X~Binomial(10,0.2). Find E[X] and Var(X).

Method: E=np and Var=np(1−p).

Answer: E[X]=2, Var(X)=1.6.

Problem 2 · Linear Algebra

If A is a 3×3 matrix with characteristic polynomial λ³−6λ²+11λ−6, find A³−6A²+11A.

Method: Apply Cayley–Hamilton directly: p(A)=0.

Answer: 6I.

Problem 3 · Recurrence

T(n)=2T(n/2)+n. Determine the asymptotic order.

Method: Master theorem: a=2,b=2,n^{log₂2}=n and f(n)=n.

Answer: Θ(n log n).

Problem 4 · Graph

An undirected graph has 10 vertices and 9 edges and is connected. What can you conclude?

Method: A connected graph with n−1 edges is a tree.

Answer: It is a tree; hence every edge is a bridge.

Problem 5 · Probability

X is exponential with rate 2. Find P(X>3).

Method: Survival function P(X>t)=e^{−λt}.

Answer: e^{-6}.

METHOD

⚠ Scope note

Keep the main revision path aligned to the GATE CSE syllabus. Any enrichment outside the syllabus should be explicitly labelled optional rather than mixed into the core revision path.

5 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- For finite sets, $|A \cup B| = |A| + |B| - |A \cap B|$.
- For recurrence $T(n) = aT(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b a}$ before applying Master theorem.
- For a square matrix, $\det(AB) = \det(A)\det(B)$; if $\det(A) \neq 0$, A is invertible.
- For an exponential rate λ distribution, $P(X > t) = e^{-\lambda t}$. Standardize normal variables with $Z = (X - \mu)/\sigma$.
- Standardise $Z = (X - \mu)/\sigma$, with $\Phi(-z) = 1 - \Phi(z)$; $\Phi(1) = 0.8413$, $\Phi(1.96) = 0.975$. Hypergeometric mean = nK/N.

LAST-MINUTE · MUST REMEMBER

- A proposition is either true or false; compound propositions are evaluated using logical connectives.
- Implication $P \rightarrow Q$ is false only when P is true and Q is false.
- A relation can be reflexive, symmetric, antisymmetric, and transitive independently; check each property from its definition.
- A function is injective if distinct inputs have distinct outputs; surjective if every codomain element is hit; bijective if both.
- A graph is bipartite iff it contains no odd cycle.
- For continuous random variables, probabilities are areas under the density; $P(X = x) = 0$ for a continuous distribution.
- CNF can be built as a conjunction of clauses; DNF as a disjunction of terms. Use truth-table rows when algebra becomes error-prone.
- **Cayley–Hamilton:** every square matrix satisfies its own characteristic polynomial; use it to reduce high powers of A.
- Convergence-test choice is pattern-driven: ratio/root for factorials and powers; comparison/integral for rational-like terms.
- **Generating functions:** independent choices multiply; the answer is a coefficient. A variable limited to 0…m contributes $1 + x + \ldots + x^m$.
- For a continuous variable $P(X = x) = 0$ and $f = dF/dx$; a density can exceed 1 but its integral is 1. Independence needs $p(x,y) = p_X(x) \cdot p_Y(y)$ in every cell.

LAST-MINUTE · COMMON TRAPS

- $P \rightarrow Q$ is logically equivalent to $\neg P \vee Q$, not $P \vee Q$.
- Mean, median, and mode are different statistics; do not infer one from another without additional distribution assumptions.
- Zero covariance does not generally imply independence. For independence, factor the joint distribution into marginals.
- **Free vs bound:** a quantifier covers only the formula right after it, so the same letter can be bound in one place and free in another. A formula with no free variables is a sentence.
- Equal degree sequences do not prove graph isomorphism (C6 vs two triangles). Self-complementary graphs have $n(n-1)/4$ edges, so $n \equiv 0$ or $1 \pmod 4$.

LAST-MINUTE · GATE FAVOURITES

- Equivalence relations are reflexive, symmetric, and transitive; they partition a set into equivalence classes.
- Know conditional probability, Bayes theorem, expectation, variance, and standard distributions used in GATE-style numericals.
- For a finite set, equivalence relations correspond exactly to partitions of the set.
- **For binomial X~Bin(n,p):** $E = np$ and $\text{Var} = np(1-p)$. For Poisson(λ): $E = \text{Var} = \lambda$.
- Use complement for 'at least one', inclusion–exclusion for overlapping cases, and pigeonhole for guaranteed repetition.

∑ Discrete Mathematics

GATE CSE 2027 · complete formula, rule, method & trap sheet

97 reference boxes · **9** chapters ·

Logic | Sets, Relations & Functions | Partial Orders & Lattices | Monoids & Groups | Combinatorics | Recurrence Relations & Generating Functions | Graph Theory | Problem → Standard Method Guide | High-Yield Extensions & Standard Methods

1 Logic

MUST REMEMBER

Implication

`P → Q ≡ ¬P ∨ Q`. It is false only when P is true and Q is false.

FORMULA / RULE

Equivalences

`P ↔ Q ≡ (P→Q) ∧ (Q→P)` ; De Morgan: `¬(P∧Q)=¬P∨¬Q` , `¬(P∨Q)=¬P∧¬Q` .

COMMON TRAP

Converse vs contrapositive

Contrapositive of `P→Q` is `¬Q→¬P` and is equivalent. Converse `Q→P` is generally not.

GATE FAVOURITE

Quantifiers

Negation flips quantifier: `¬∀x P(x) ≡ ∃x ¬P(x)` , `¬∃x P(x) ≡ ∀x ¬P(x)` .

FORMULA / RULE

Rules of inference (valid argument forms)

Rule	Form
Modus ponens	`P, P→Q ⊢ Q`
Modus tollens	`¬Q, P→Q ⊢ ¬P`
Hypothetical syllogism	`P→Q, Q→R ⊢ P→R`
Disjunctive syllogism	`P∨Q, ¬P ⊢ Q`
Resolution	`P∨Q, ¬P∨R ⊢ Q∨R`
Simplification / Addition	`P∧Q ⊢ P` · `P ⊢ P∨Q`

An argument is **valid** iff the premises true can never coexist with a false conclusion, i.e. (premises ∧ ¬conclusion) is unsatisfiable.

COMMON TRAP

Two fallacies that look like valid rules

Affirming the conclusion: `P→Q, Q ⊢ P` is **invalid**. **Denying the hypothesis:** `P→Q, ¬P ⊢ ¬Q` is **invalid**. Counter-example for both: P = F, Q = T makes the premises true and the conclusion false.

MUST REMEMBER

Normal forms & counting

CNF = AND of ORs of literals; **DNF** = OR of ANDs. A formula in CNF is **valid** iff every clause contains some `x` and `¬x` ; a formula in DNF is **satisfiable** iff some term has no clashing literals. n variables → `2ⁿ` truth-table rows and `2^(2ⁿ)` different Boolean functions (n = 2 gives 16).

GATE FAVOURITE

Translating English into FOL; free vs bound variables

All S are L: `∀x (S(x)→L(x))` . **Some S are L:** `∃x (S(x)∧L(x))` . Using ∧ with ∀ (or → with ∃) is the classic wrong answer. **Order matters:** `∀x∃y Loves(x,y)` = “everybody loves somebody”, while `∃y∀x Loves(x,y)` = “somebody is loved by everybody” (much stronger). A variable is **bound** if a quantifier covers it, otherwise **free**; a formula with no free variables is a **sentence** and has a truth value.

KEY POINT

Is the argument valid?

Premises: if it rains the match is cancelled; the match is not cancelled.
Conclusion: it did not rain.

Let R = rain, C = cancelled: `R→C, ¬C ⊢ ¬R` . This is **modus tollens**, so the argument is valid. Had the second premise been “the match is cancelled”, the argument would commit the fallacy of affirming the conclusion and be invalid.

FORMULA / RULE

Functional completeness

A set of connectives is **functionally complete** if every Boolean function can be built from it.

Set	Status	Why
{∧, ¬}, {∨, ¬}, {→, ¬}	Complete	De Morgan gives the missing one: p∨q = ¬(¬p∧¬q); p∨q = ¬p→q.
{↑} (NAND) alone, {↓} (NOR) alone	Complete	¬p = p↑p; p∧q = ¬(p↑q). NAND and NOR are the only single binary connectives that are complete.
{∧, ∨}	Not complete	Only monotone functions — no way to produce ¬p.
{→} alone	Not complete	Always 1 on the all-true row, so ¬p (false on that row) is unreachable.
{⊕, ∧}, {⊕, ¬}, {↔, ¬}	Not complete	{⊕,∧} keeps 0→0 so it never makes ¬p or the constant 1; {⊕,¬} gives only affine functions. {⊕, ∧, constant 1} *is* complete.

Post's test: a set is complete iff it is *not* contained in any one of the classes 0-preserving, 1-preserving, monotone, self-dual, affine. With n variables there are 2^{2^n} Boolean functions and $2^{2^{n-1}}$ self-dual ones (16 for n = 3).

GATE FAVOURITE

Quantifier distribution — which direction is valid?

∀ distributes over ∧ and ∃ over ∨ **exactly**: ∀x(P∧Q) ≡ ∀xP ∧ ∀xQ, ∃x(P∨Q) ≡ ∃xP ∨ ∃xQ. For the other pairings only **one direction** holds:

Formula	Status	Reason
∀xP ∨ ∀xQ → ∀x(P∨Q)	Valid	Converse is false
∃x(P∧Q) → ∃xP ∧ ∃xQ	Valid	Converse is false
(∀xP → ∀xQ) → ∀x(P→Q)	Not valid	Make ∀xP false, and P true / Q false at some x
∃y∀x R(x,y) → ∀x∃y R(x,y)	Valid	Converse is false (swap of quantifiers changes meaning)

Counter-model for the invalid ones: domain {1, 2}, P true only at 1, Q true only at 2 — then ∀x(P∨Q) is true but ∀xP ∨ ∀xQ is false.

MUST REMEMBER

Valid, satisfiable, unsatisfiable

Valid (tautology) = true under every interpretation. **Satisfiable** = true under at least one. **Unsatisfiable** (contradiction) = true under none. A is valid ⇔ ¬A is unsatisfiable. A formula can be satisfiable yet not valid (e.g. P → Q). Propositional SAT is NP-complete; validity in first-order logic is undecidable (but semi-decidable).

MUST REMEMBER

Free and bound variables

An occurrence of x is **bound** if it lies inside the scope of a quantifier ∀x or ∃x; otherwise it is **free**. A quantifier's scope is the formula that immediately follows it. A formula with **no free variables** is a **sentence** (closed formula) and is simply true or false in a given domain; one with free variables is true or false only after the variables are given values. The same letter can be free in one place and bound in another, and a quantified variable can be renamed (`∃x Q(x) ≡ ∃z Q(z)`) as long as the new name does not clash with a free variable.

Worked example. In `∀x (P(x, y) → ∃z Q(x, z)) ∧ R(x)` , the ∀x covers only `P(x, y) → ∃z Q(x, z)` , so x is bound there. The final `R(x)` lies outside that scope, so that x is **free**; y is free (nothing quantifies it); z is bound by ∃z. Free variables: **y and the x in R(x)**, so the formula is not a sentence.

2 Sets, Relations & Functions

MUST REMEMBER

Relation properties

Reflexive: (a,a) for every a. Symmetric: aRb ⇒ bRa. Antisymmetric: aRb and bRa ⇒ a=b. Transitive: aRb and bRc ⇒ aRc.

GATE FAVOURITE

Equivalence relation

Exactly: reflexive + symmetric + transitive. Equivalence classes form a partition of the set.

FORMULA / RULE

Function composition

If `f:A→B` and `g:B→C` , then `g∘f:A→C` . Composition order matters.

COMMON TRAP

Injective/surjective

Injective concerns distinct inputs having distinct outputs. Surjective means every codomain element is hit. Neither implies the other for finite sets of unequal size.

FORMULA / RULE

Counting relations and functions (A has n elements; A→B uses |A|=m, |B|=n)

Object	Count
All relations on A	`2^(n²)`
Reflexive / irreflexive	`2^(n²−n)` each
Symmetric	`2^(n(n+1)/2)`
Antisymmetric	`2ⁿ · 3^(n(n−1)/2)`
Asymmetric	`3^(n(n−1)/2)`
Equivalence relations	Bell number: 1, 2, 5, 15, 52, 203 for n = 1…6
Functions A→B / injective / bijective	`nᵐ` / `P(n,m)` / `m!` (when m = n)
Onto functions A→B	`Σ (−1)ᵏ C(n,k)(n−k)ᵐ = n!·S(m,n)`

Check with n = 3: antisymmetric = $2^3 \cdot 3^3 = 216$; asymmetric = 27; equivalence = 5.

MUST REMEMBER

Closures of a relation

Reflexive closure = R ∪ Δ. **Symmetric closure** = R ∪ R^{-1}. **Transitive closure** R^+ = smallest transitive relation containing R (Warshall's algorithm, $O(n^3)$). Equivalence closure = take reflexive, symmetric and transitive closure; apply the transitive step **last**. Example: R = {(1,2),(2,3),(3,4)} has transitive closure with **6** pairs: the three originals plus (1,3), (2,4), (1,4).

GATE FAVOURITE

Properties of functions and composition

For finite sets of equal size, injective ⇔ surjective ⇔ bijective. If `g∘f` is injective then **f** is injective; if `g∘f` is surjective then **g** is surjective. A function has an inverse iff it is bijective. `|P(A)| = 2ⁿ` , `|A×B| = |A||B|` .

KEY POINT

How many equivalence relations on a 4-element set have exactly 2 classes?

Count partitions into 2 blocks: Stirling number `S(4,2) = 7` (four of type 3+1 and three of type 2+2). Adding the other block counts: `S(4,1)=1` , `S(4,3)=6` , `S(4,4)=1` totals the Bell number **15**.

MUST REMEMBER

Cardinality and countability

Countably infinite (same size as ℕ): ℤ, ℚ, ℕ×ℕ, the set of all finite strings over a finite alphabet, any finite union or product of countable sets, and any subset of a countable set. **Uncountable**: ℝ, the power set $\mathcal{P}(\mathbb{N})$, the set of all infinite binary strings, the set of all functions ℕ→{0,1} (Cantor's diagonal argument). For any set A, $|\mathcal{P}(A)| = 2^{|A|} > |A|$. Programs (or Turing machines) are finite strings, hence countable, while languages are uncountable — so most languages are not recognised by any machine.

FORMULA / RULE

Composition, inverse and extra counts for relations

Composition (apply R first, then S): `S∘R = {(a,c) : ∃b, aRb and bSc}` ; `(S∘R)⁻¹ = R⁻¹∘S⁻¹` . With Boolean matrices $M_{S\circ R} = M_R \cdot M_S$ (Boolean product). R is transitive iff $R \circ R \subseteq R$; the transitive closure is $R \cup R^2 \cup \ldots \cup R^n$. On an n-element set: reflexive **and** antisymmetric = $3^{n(n-1)/2}$ (27 for n = 3); symmetric **and** antisymmetric = 2^n (subsets of the diagonal); partial orders on 3 labelled elements = 19; transitive relations on 3 elements = 171. Idempotent functions ($f \circ f = f$) on n points: $\Sigma_k\, C(n,k) \cdot k^{n-k}$ (n = 3 gives 3 + 6 + 1 = 10).

COMMON TRAP

Unions and intersections of equivalence relations

The **intersection** of equivalence relations is an equivalence relation, but the **union** need not be: {1~2} ∪ {2~3} lacks (1,3), so transitivity fails. The smallest equivalence relation containing both is the transitive closure of the union.

3 Partial Orders & Lattices

MUST REMEMBER

Partial order

A relation that is reflexive, antisymmetric and transitive. A poset is `(P,≤)`

GATE FAVOURITE

Hasse diagram

Represent the cover relation only; omit reflexive and transitive edges. Read minimal/maximal elements from the diagram carefully.

COMMON TRAP

Minimal vs least

A least element is ≤ every element and is unique if it exists. A poset may have multiple minimal elements without having a least element.

SHORTCUT

Finite lattice check

Every pair must have both a unique least upper bound (join) and a unique greatest lower bound (meet).

MUST REMEMBER

Bounds, chains and antichains

An **upper bound** of a subset S is an element ≥ every member of S; the **least upper bound** (join) is the smallest such bound, and the **greatest lower bound** (meet) is defined dually. A **chain** is a set of pairwise comparable elements and an **antichain** a set of pairwise incomparable ones. **Dilworth:** the minimum number of chains needed to cover a finite poset equals the size of its largest antichain.

FORMULA / RULE

Facts about the divisor lattice D_n ($n = p_1^{a_1} \cdot p_2^{a_2} \cdot \ldots$)

Elements: $\prod(a_i+1)$ ($360 = 2^3 \cdot 3^2 \cdot 5$ has 24 divisors). Meet = gcd, join = lcm. D_n is always **distributive**; it is a **Boolean algebra iff n is square-free**, and then it has 2^k elements for k primes. **Hasse-diagram edges** = $\Sigma_i\, a_i \cdot \prod_{j \neq i}(a_j+1)$: D12 gives 2·2 + 1·3 = 7, D36 gives 2·3 + 2·3 = 12, D30 gives 12 (a cube).

COMMON TRAP

Lattice types

Bounded = has top and bottom (every finite lattice is). **Complemented** = every element has a complement ($x \wedge y = \bot$ and $x \vee y = \top$). **Distributive** = $x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$. A lattice is distributive iff it contains no copy of the diamond M_3 or pentagon N_5. A **Boolean algebra** is a complemented distributive lattice, so it has 2^k elements. A complemented lattice need not be distributive.

KEY POINT

Is the poset ({1,2,3,6,12,18}, divisibility) a lattice?

Check pairs that might fail: lcm(12,18) = 36, which is not in the set, so 12 and 18 have **no upper bound at all**. Hence **not a lattice**. A common mistake is to test only gcd; both meet and join must exist for every pair.

FORMULA / RULE

Chains, antichains and Dilworth in divisor and subset posets

Dilworth: minimum number of chains covering a poset = size of the largest antichain; **Mirsky** (dual): minimum number of antichains = length of the longest chain. In D_n with $n = p_1^{a_1} \cdot p_2^{a_2} \cdot \ldots$: longest chain has $\Sigma a_i + 1$ elements; the number of **maximal chains** = $(\Sigma a_i)! / \prod a_i!$ (D_{36}: 4!/(2!2!) = 6; D_{360}: 6!/(3!2!1!) = 60). In the subset lattice $\mathcal{P}(n)$: n! maximal chains and largest antichain $C(n, \lfloor n/2 \rfloor)$ (Sperner). For divisibility on {1,…,12} the largest antichain is {7,…,12} (size 6), so 6 chains suffice and are needed.

COMMON TRAP

Distributive, complemented and the forbidden sublattices

A lattice is distributive iff it has no sublattice isomorphic to the diamond **M_3** or the pentagon **N_5** (N_5: $0 < a < c < 1$ and $0 < b < 1$; there $c \wedge (a \vee b) = c$ but $(c \wedge a) \vee (c \wedge b) = a$). Both are complemented lattices that are **not** distributive; in N_5 the element b has two complements (a and c). In a distributive lattice complements are unique, so Boolean algebras have unique complements. A chain with more than two elements is distributive but not complemented.

4 Monoids & Groups

MUST REMEMBER

Hierarchy

Semigroup = associative. Monoid = associative + identity. Group = monoid + inverse for every element.

GATE FAVOURITE

Group facts

Identity is unique; inverse of each element is unique. In a finite group, cancellation holds.

COMMON TRAP

Closure is separate

Associativity alone does not make a structure a semigroup. Verify closure under the operation too.

FORMULA / RULE

Algebraic structures at a glance

Structure	Needs	Examples
Semigroup	closure + associativity	$(N^+, +)$, $(Z, \times)$
Monoid	semigroup + identity	$(N, +)$ with 0, $(Z, \times)$
Group	monoid + inverses	$(Z, +)$, $(Q \setminus \{0\}, \times)$, Z_n under + mod n
Abelian group	group + commutative	$(Z, +)$
Ring / Field	two operations, distributive / every nonzero element has a ×-inverse	Z_n is a field iff n is prime

$(Z, -)$ is **not** a semigroup because subtraction is not associative.

MUST REMEMBER

Cyclic groups (order n, generator g)

The number of generators is $\varphi(n)$. There is exactly **one subgroup per divisor** of n. The number of elements of order d (for d | n) is $\varphi(d)$. In Z_{12}: 4 generators (1, 5, 7, 11); the elements of order 4 are 3 and 9 ($\varphi(4) = 2$). The order of a in Z_n is $n / \gcd(a, n)$.

GATE FAVOURITE

Lagrange and its consequences

|H| divides |G| for every subgroup H, so the order of every element divides |G|; a group of **prime order is cyclic**; the converse of Lagrange is false (A_4 has order 12 but no subgroup of order 6). If $a^2 = e$ for all a then G is abelian; every group of even order has an element of order 2. Groups of order 4: Z_4 and the Klein four-group; order 6: Z_6 and S_3.

SHORTCUT

Subgroup test

A non-empty $H \subseteq G$ is a subgroup iff $a, b \in H \Rightarrow a \cdot b^{-1} \in H$. For a **finite** H it is enough that H is closed under the operation.

MUST REMEMBER

Permutation groups

S_n has n! elements and is non-abelian for $n \geq 3$. Write permutations as disjoint cycles: the **order** is the lcm of the cycle lengths, and f^k only depends on k mod order (for (1 2 3)(4 5), $f^{100} = f^4 = (1\ 2\ 3)$). A k-cycle is even iff k is odd; parity is multiplicative; A_n (even permutations) has n!/2 elements and is normal in S_n. Elements with cycle type having m_k cycles of length k: $n! / \prod(k^{m_k} \cdot m_k!)$. In S_5 the orders are 1,2,3,4,5,6 with max 6 (type 3+2, 20 elements); in S_4 there are 8 elements of order 3; S_7 has max order 12 (type 4+3). A_4 has order 12 but no subgroup of order 6.

FORMULA / RULE

Normal subgroups, homomorphisms and direct products

H is **normal** if $gHg^{-1} = H$ for all g; every subgroup of index 2 is normal and every kernel is normal. **First isomorphism theorem:** $G/\ker f \cong \text{im } f$, so $|\text{im } f| = |G|/|\ker f|$. Number of homomorphisms $Z_m \to Z_n = \gcd(m, n)$ (f(1) must have order dividing m). $Z_m \times Z_n$ is cyclic iff $\gcd(m,n) = 1$; the order of (a,b) in a product is lcm(|a|,|b|). Groups of order 4: Z_4, Klein V_4; order 6: Z_6, S_3; order 8: five groups.

FORMULA / RULE

The group of units Z_n^*

$Z_n^* = \{a : \gcd(a,n) = 1\}$ under multiplication mod n has φ(n) elements. It is **cyclic iff n = 1, 2, 4, p^k or $2p^k$** (p an odd prime). So Z_9^*, Z_{10}^*, Z_{18}^* are cyclic, while Z_8^* and Z_{12}^* are the Klein four-group and $Z_{15}^* \cong Z_2 \times Z_4$ (order 8, no element of order 8) are not.

5 Combinatorics

FORMULA / RULE

Permutations & combinations

`nPr = n!/(n-r)!`, `nCr = n!/[r!(n-r)!]`, and `nCr=nC(n-r)`.

GATE FAVOURITE

Inclusion–Exclusion

For two sets: `|AUB|=|A|+|B|-|A∩B|`. For many sets, alternate signs across intersections.

SHORTCUT

At least one

Often easier as **total − none**. For independent repeated choices, compute the complement first when the event is “at least one”.

COMMON TRAP

Pigeonhole

If n objects are placed into k boxes and n>k, at least one box has ≥2 objects. General form: some box has at least `ceil(n/k)` objects.

FORMULA / RULE

Stars and bars (distributing identical objects)

Problem	Count
$x_1+\ldots+x_k = n$, all $x_i \geq 0$	`C(n+k−1, k−1)`
All $x_i \geq 1$	`C(n−1, k−1)`
$x_i \geq a_i$ (lower bounds)	substitute $y_i = x_i - a_i$, then use the first row with $n - \Sigma a_i$
Upper bounds $x_i \leq u$	inclusion–exclusion on the variables that break the bound

Example: $x_1+x_2+x_3+x_4 = 20$ with every $x_i \geq 2$ → shift to sum 12 → `C(15,3) = 455`

FORMULA / RULE

Binomial theorem and identities

`(x+y)^n = Σ C(n,k) x^(n−k) y^k`. Coefficient of x^3 in $(2+x)^5$ = `C(5,3)·2² = 40`. `ΣC(n,k) = 2^n`; alternating sum = 0; Pascal: `C(n,k) = C(n−1,k−1)+C(n−1,k)`; Vandermonde: `Σ C(m,i)C(n,r−i) = C(m+n,r)` (e.g. 35 = C(7,3)); hockey stick: `Σ_{i=r}^{n} C(i,r) = C(n+1,r+1)`. Multinomial: arrangements = `n!/(n₁!n₂!…)`.

MUST REMEMBER

Special counting sequences

Sequence	Values	Counts
Derangements D_n	0, 1, 2, 9, 44, 265 (n = 1…6)	permutations with no fixed point; `D_n = (n−1)(D_{n−1}+D_{n−2})`, ≈ n!/e
Catalan C_n = C(2n,n)/(n+1)	1, 1, 2, 5, 14, 42, 132	binary search trees on n keys, balanced bracket strings, full binary trees with n+1 leaves
Stirling S(n,k)	S(4,2)=7, S(5,3)=25	partitions of n objects into k non-empty blocks
Bell B_n	1, 2, 5, 15, 52	all partitions of an n-set = Σ_k S(n,k)

KEY POINT

5 men and 3 women in a row, no two women adjacent

Seat the men first: 5! = 120. They create 6 gaps; choose 3 for the women: C(6,3) = 20; arrange the women: 3! = 6. Total = 120 × 20 × 6 = **14400**.

FORMULA / RULE

Circular arrangements and restricted arrangements

n distinct objects in a circle: (n − 1)!. Two particular objects adjacent: 2·(n − 2)!; not adjacent: (n − 1)! − 2·(n − 2)!. m men and m women alternating: (m − 1)! ·m!. Bracelet (flip also allowed): (n − 1)!/2. In a line, use the **block method** (glue items that must be together) or the **gap method**: arrange the unrestricted items first, then choose gaps for the restricted ones — r items that may not touch go into C(k + 1, r) of the k + 1 gaps around k other items.

Example: MISSISSIPPI with no two S adjacent: arrange M,I,I,I,I,P,P in 7!/(4!2!) = 105 ways, then pick 4 of the 8 gaps: 105 × C(8,4) = 105 × 70 = **7350**.

FORMULA / RULE

Distribution problems at a glance

Situation	Count
n distinct objects → k distinct boxes	k^n
n identical objects → k distinct boxes (empty allowed / none empty)	C(n+k−1, k−1) / C(n−1, k−1)
n distinct objects → k identical boxes, none empty	Stirling S(n, k)
n identical objects → identical boxes (integer partitions)	p(n): p(6) = 11, p(8) = 22
Lattice paths in an m × n grid	C(m+n, m); not crossing the diagonal: Catalan C_n

COMMON TRAP

Pigeonhole variants

To guarantee r objects in one of k boxes you need (r − 1)·k + 1 objects (37 people ⇒ 4 share a birth month). Among any n + 1 integers two leave the same remainder mod n. **Divisibility form:** from {1,…,2n} any n + 1 numbers contain two with one dividing the other — group the numbers by their odd part (there are n groups), because two numbers with the same odd part divide each other. The bound is tight: {n+1,…,2n} has no such pair.

6 Recurrence Relations & Generating Functions

MUST REMEMBER

Linear homogeneous recurrence

For constant coefficients, form the characteristic equation. Repeated roots require polynomial factors in n.

FORMULA / RULE

Repeated root

Root r of multiplicity k contributes `(C₀+C₁n+...+C_{k−1}n^(k−1))r^n`

GATE FAVOURITE

Particular solution

For non-homogeneous recurrences, choose an appropriate particular form; if it overlaps a homogeneous root, multiply the trial by the required power of n.

SHORTCUT

Generating functions

Translate sequence terms into a power series, manipulate algebraically, then extract the coefficient. Keep indexing consistent.

KEY POINT

Non-homogeneous with an overlapping root

`a_n = 5a_{n−1} − 6a_{n−2} + 2^n`, $a_0 = 0$, $a_1 = 1$. Homogeneous roots are 2 and 3. The forcing term 2^n overlaps root 2, so the trial particular solution is `C·n·2^n` (multiply by n). Substituting gives C = −2. General solution: `A·2^n + B·3^n − 2n·2^n`; a_0 and a_1 give A = −5, B = 5, so $a_n = 5\cdot3^n - 5\cdot2^n - 2n\cdot2^n$ (check: $a_2 = 45 - 20 - 16 = 9 = 5\cdot1 - 0 + 4$).

FORMULA / RULE

Standard generating functions

Sequence	Generating function
1, 1, 1, …	`1/(1−x)`
1, a, a², …	`1/(1−ax)`
C(n+k−1, k−1) (k-fold repetition)	`1/(1−x)^k`
C(n, r)	`(1+x)^n`
1/r!	`e^x`

Rules: a product of generating functions is the convolution of the sequences; multiplying by `1/(1−x)` gives partial sums; shifting by x^m moves the index.

KEY POINT

Counting with a generating function

Ways to pay n using 1-coins and 2-coins: coefficient of x^n in `1/((1−x)(1−x²))` = ⌊n/2⌋ + 1, so n = 10 gives **6**. Another standard model: tilings of a 2×n board with dominoes follow Fibonacci (1, 2, 3, 5, 8 …), so a 2×5 board has **8** tilings.

FORMULA / RULE

Trial form for the particular solution

Forcing term	Trial solution
constant c	A (multiply by n if 1 is a characteristic root)
polynomial of degree d in n	polynomial of degree d (× n^m if 1 is a root of multiplicity m)
$c \cdot r^n$	$A \cdot r^n$ (× n^m if r is a root of multiplicity m)
(polynomial)$\cdot r^n$	(polynomial of the same degree)$\cdot r^n$

Then add the homogeneous solution and fit the initial conditions. Check by iterating the first few terms.

FORMULA / RULE

Standard results worth memorising

`a_n = a_{n-1} + n, a_0 = 0` ⇒ n(n+1)/2. `a_n = 2a_{n-1} + n, a_0 = 1` ⇒ $3 \cdot 2^n - n - 2$ ($a_4 = 42$). `T(n) = 2T(n/2) + n, T(1) = 1` ⇒ $n \cdot \log_2 n + n$ (T(16) = 80). Climbing stairs with 1, 2 or 3 steps: tribonacci, 1, 2, 4, 7, 13, 24, 44, 81, 149, **274** for 10 steps. Ternary strings with no two adjacent 0s: $a_n = 2a_{n-1} + 2a_{n-2}$ (1, 3, 8, 22, 60, 164, 448, 1224). Regions made by n lines in general position: 1 + n(n+1)/2.

SHORTCUT

Generating functions: coins, bounds and parity

Coins of 1, 2, 5: `1/((1−x)(1−x²)(1−x⁵))` — the coefficient of x^n is the number of ways to pay n (n = 15 gives 18). A variable with $0 \le x \le u$ contributes $(1 - x^{u+1})/(1 - x)$; an even count contributes $1/(1 - x^2)$ and an odd count $x/(1 - x^2)$. Partitions of n into parts of size at most k: $\prod_{i \le k} 1/(1 - x^i)$. Without a generating function, a quick hand-count: fix the number of 5-coins, then count (1,2)-ways as ⌊r/2⌋ + 1.

7 Graph Theory

MUST REMEMBER

Handshake lemma

`Σ deg(v)=2|E|` for undirected graphs. Therefore the number of odd-degree vertices is even.

GATE FAVOURITE

Connectivity

For an undirected graph, connected means every pair of vertices has a path. A tree is connected and acyclic.

FORMULA / RULE

Tree invariant

A tree with n vertices has exactly `n−1` edges. Adding one edge creates exactly one cycle; deleting any edge disconnects the tree.

COMMON TRAP

Euler vs Hamilton

Euler concerns using every **edge**; Hamilton concerns visiting every **vertex**. Do not interchange their conditions.

FORMULA / RULE

Euler and Hamilton: the exact tests

Euler circuit: connected and every degree even (directed: in-degree = out-degree). **Euler path:** connected with 0 or 2 odd-degree vertices. K_n has an Euler circuit iff n is odd. **Hamilton:** no simple iff-test; **Dirac** ($n \ge 3$, every degree ≥ n/2) and **Ore** (deg u + deg v ≥ n for non-adjacent pairs) are sufficient only. K_n is Hamiltonian for $n \ge 3$; $K_{m,n}$ is Hamiltonian iff $m = n \ge 2$.

FORMULA / RULE

Planarity and Euler's formula

Connected planar graph: `V − E + F = 2`. For $V \ge 3$: `E ≤ 3V − 6`; if there is no triangle (e.g. bipartite): `E ≤ 2V − 4`. Sum of face degrees = 2E. K_5 (10 edges > 9) and $K_{3,3}$ (9 edges > 8) are non-planar, and by Kuratowski a graph is planar iff it has no subdivision of either. Example: a connected planar cubic graph on 8 vertices has E = 12 and F = 2 − 8 + 12 = **6**.

FORMULA / RULE

Spanning trees

Cayley: K_n has `n^(n−2)` spanning trees ($K_4 \to 16$). $K_{m,n}$ has `m^(n−1)·n^(m−1)` ($K_{2,3} \to 12$; $K_{3,3} \to 81$). The cycle C_n has n. In general use the matrix-tree theorem (any cofactor of the Laplacian). A spanning tree has n−1 edges; if all edge weights are distinct the minimum spanning tree is unique (Kruskal/Prim); cut property and cycle property justify both algorithms.

MUST REMEMBER

Connectivity and special graph counts

Vertex connectivity κ ≤ edge connectivity λ ≤ minimum degree δ. In a tree every edge is a bridge and every non-leaf is a cut vertex. A forest with n vertices and c components has n − c edges. K_n has n(n−1)/2 edges; $K_{m,n}$ has mn. There are `2^(n(n−1)/2)` labelled simple graphs on n vertices (64 for n = 4) but only 11 non-isomorphic ones. A graph is bipartite iff it has no odd cycle.

MUST REMEMBER

Directed graphs

Σ in-degree = Σ out-degree = |E|. The (i,j) entry of A^k counts walks of length k from i to j. A digraph is **strongly connected** if every vertex reaches every other; a **DAG** has a topological order (a vertex with in-degree 0 always exists).

MUST REMEMBER

Matching, vertex cover, independent set, edge cover

For a graph on n vertices: **α + τ = n** (independent set + vertex cover) and **ν + ρ = n** (matching + edge cover, no isolated vertices — Gallai). **König (bipartite graphs):** ν = τ, hence α = n − ν; Hall's condition characterises perfect matchings. Perfect matchings: $K_{n,n}$ has n!, K_{2n} has (2n − 1)!! (K_6: 15, K_8: 105); a tree has at most one; every regular bipartite graph has one. The Petersen graph has a perfect matching and independence number 4.

MUST REMEMBER

Graph colouring

$\chi(G) \geq \omega(G)$ (clique number) but not always equal: the Grötzsch graph is triangle-free with $\chi = 4$, and C_5 has $\omega = 2$, $\chi = 3$. Brooks: $\chi \leq \Delta$ unless G is complete or an odd cycle; $\chi \leq \Delta + 1$ always; planar $\Rightarrow \chi \leq 4$; bipartite $\Leftrightarrow \chi \leq 2$; the Petersen graph has $\chi = 3$. **Chromatic polynomial** P(G,k) (colourings with k colours): tree $k(k-1)^{n-1}$; K_n $k(k-1)\ldots(k-n+1)$; cycle C_n $(k-1)^n + (-1)^n(k-1)$; deletion–contraction $P(G) = P(G - e) - P(G / e)$; disconnected graphs multiply. **Edge colouring:** $\Delta \leq \chi' \leq \Delta + 1$ (Vizing); bipartite $\chi' = \Delta$; $\chi'(K_n) = n - 1$ for even n and n for odd n.

FORMULA / RULE

Degree sequences, complements and extremal counts

Havel–Hakimi: sort descending, delete the largest d_1 and subtract 1 from the next d_1 entries; repeat. The sequence is graphic iff you reach all zeros without a negative entry. An even sum is necessary, not sufficient. Complement: $|E(G)| + |E(\bar{G})| = C(n,2)$; a self-complementary graph has $n \equiv 0$ or $1 \pmod 4$ (P_4, C_5). Mantel: a triangle-free graph has at most $\lfloor n^2/4 \rfloor$ edges (attained by $K_{\lfloor n/2 \rfloor, \lceil n/2 \rceil}$). Unlabelled trees on n = 1…8 vertices: 1, 1, 1, 2, 3, 6, 11, 23; non-isomorphic simple graphs on n = 1…6: 1, 2, 4, 11, 34, 156; connected 3-regular graphs on 6 vertices: 2 ($K_{3,3}$ and the prism).

FORMULA / RULE

Rooted trees and m-ary trees

A full m-ary tree with i internal vertices has $n = m \cdot i + 1$ vertices and $(m - 1) \cdot i + 1$ leaves (binary: i + 1 leaves, 2i + 1 vertices). A binary tree of height h (edges) has at most $2^{(h+1)} - 1$ vertices and at most 2^h leaves; with n_2 vertices of degree 2 it has $n_2 + 1$ leaves. Binary search trees on n keys: Catalan number C_n. A tree has one or two centre vertices.

FORMULA / RULE

Connectivity, planarity bounds, Euler and Hamilton extras

A **cut vertex** increases the number of components when removed; a **bridge** is an edge with that property; **blocks** are the maximal 2-connected pieces, and consecutive blocks meet at cut vertices. Planar with girth g: $E \leq g(V - 2)/(g - 2)$: triangle-free ($g = 4$) gives $E \leq 2V - 4$ ($K_{2,8}$ reaches 16 for V = 10), and the Petersen graph ($g = 5$, $E = 15 > 5 \cdot 8/3$) is non-planar. $K_{m,n}$ has an Euler circuit iff m and n are both even; it has an Euler trail but no circuit when exactly two vertices have odd degree ($K_{2,3}$). The Petersen graph has a Hamilton path but **no** Hamilton cycle.

8 Problem → Standard Method Guide

METHOD

Logic & Proof Patterns

Truth/equivalence question → use truth table or standard logical equivalences. Check only the relevant rows/identities.

Implication validity → rewrite `P→Q` as `¬PvQ` ; look for the single false case.

Quantifier negation → push negation inward while flipping $\forall \leftrightarrow \exists$. Do not negate the predicate incorrectly.

Argument validity → translate premises/conclusion into logic; test whether premises true can coexist with conclusion false.

METHOD

Relations, Functions, Posets & Lattices

Classify a relation → systematically test reflexive, symmetric, antisymmetric, transitive using counterexamples where possible.

Equivalence classes → verify equivalence relation, then group elements related to a chosen representative.

Partial-order question → check R, AS, T. For Hasse diagrams, remove loops and transitive edges.

Least/greatest/minimal/maximal → compare using the order relation, not visual position alone.

Lattice question → for each relevant pair, find both meet (GLB) and join (LUB).

Function classification → injective: compare inputs/outputs; surjective: check every codomain value has a preimage; bijective: both.

METHOD

Combinatorics

"At least one" → complement counting: total − none.

Overlapping cases → Inclusion–Exclusion; watch alternating signs.

Repeated objects / identical items → divide by factorials of repetition counts when appropriate.

Restricted arrangements → count unrestricted arrangements and subtract invalid cases, or use block/gap methods depending on the restriction.

Guaranteed repetition → Pigeonhole principle; formulate the boxes and objects explicitly before calculating.

Choose r from n → combinations when order does not matter; permutations when it does.

METHOD

Recurrences & Generating Functions

Constant-coefficient linear recurrence → characteristic equation → roots → homogeneous solution → particular solution → initial conditions.

Repeated characteristic root → multiply successive terms by powers of n.

Non-homogeneous recurrence → solve homogeneous part first, then construct a particular solution.

Generating-function problem → define the series, transform the recurrence/sequence algebraically, then extract coefficients.

METHOD

Extended GATE Calculation Patterns

Problem signal	Standard method	Key check / shortcut
CNF/DNF conversion	Use logical equivalences; for canonical forms enumerate assignments if needed.	CNF is AND of OR clauses; DNF is OR of AND terms.
Relation represented as matrix	Use rows/columns to test reflexivity, symmetry, antisymmetry; Boolean matrix multiplication can test composition/transitivity.	Transitivity requires $R \circ R \subseteq R$.
Counting onto functions	Inclusion–Exclusion over missing codomain values.	For $n \to m$, onto is possible only when $n \geq m$.
Arrangements with identical items	Multiset permutation formula.	Divide by factorials of repeated multiplicities.
Restricted circular arrangement	Fix a reference position when rotations are considered identical.	Do not divide by n!; circular permutations usually use (n−1)! for distinct objects.
Recurrence order / characteristic roots	Write characteristic polynomial, factor it, apply multiplicity rule.	Initial conditions determine constants, not the characteristic roots.
Recurrence with polynomial/exponential forcing	Guess a particular form matching the forcing term.	If the guess overlaps the homogeneous solution, multiply by n until independent.
Generating-function coefficient	Convert to known series or partial fractions, then extract coefficient.	Index shifts carefully; write first few terms to verify.
Maximum/minimum degree bound	Handshake lemma + graph constraints.	Odd-degree vertices must occur in an even count.
Connected graph edge bounds	Use tree as minimum-edge connected graph and complete graph as maximum simple graph.	Connected simple graph: $V-1 \leq E \leq V(V-1)/2$.
Euler circuit/trail	Use degree parity + connectivity of non-isolated vertices.	All even → Euler circuit; exactly two odd → Euler trail but not circuit.
Bipartite graph	2-colouring via BFS/DFS or identify an odd cycle.	No odd cycle ⇔ bipartite.
Matching question	Look for augmenting paths; use structural constraints/bipartite matching reasoning.	A matching never shares a vertex between selected edges.
Graph colouring lower bound	Find a clique or odd-cycle obstruction; then construct a colouring for an upper bound.	$\chi(G) \geq \omega(G)$; bipartite nonempty graph has $\chi=2$.
Group identity/inverse	Test closure, associativity, identity and inverses under the given operation.	Identity and inverse are unique in a group.
Subgroup test	For a nonempty subset H, verify closure under a^{-1}b for all a,b∈H.	This single test replaces separate closure + inverse checks.
Finite group order	Use divisibility properties of subgroup/order when applicable.	Order of an element divides the order of a finite group.
Which connective sets are complete?	Show each of ¬, ∧ (or ∨) is expressible; otherwise use a class that is closed (monotone, 0- or 1-preserving, affine, self-dual).	$\{\wedge, \vee\}$ and $\{\to\}$ alone are not complete; NAND and NOR alone are.
Is a quantified implication valid?	Try a counter-model on a 2-element domain: P true at one point, Q true at the other.	∃ with ∧ and ∀ with ∨ only give one direction.
Closure / composition of a relation	Boolean-matrix product or Warshall; count pairs of every reachability component (a cycle of length k contributes k^2 pairs).	Compose in the stated order; closure on a cycle is complete.
Minimum chains / maximum antichain	Dilworth: find the largest set of pairwise incomparable elements.	For divisibility on {1..2n}: {n+1..2n} is an antichain of size n.
Order of a permutation / power of it	Disjoint cycles, then lcm of cycle lengths; reduce the exponent modulo the order.	Cycle type 3+2 has order 6; a k-cycle has parity k − 1.
Is Z_n^* cyclic?	Cyclic iff n = 1, 2, 4, p^k, $2p^k$.	Z_8^*, Z_{12}^*, Z_{15}^* are not cyclic.
Homomorphisms $Z_m \to Z_n$	f(1) = a with m·a ≡ 0 (mod n); count = gcd(m, n).	|image| = |G| / |kernel|.
Circular / forbidden-adjacency counting	Fix one person; treat adjacent pairs as a block; use the gap method for non-adjacent.	Not adjacent = (n−1)! − 2(n−2)!.
Pigeonhole with divisibility or sums	Choose boxes that force the property (odd part, remainder class).	Guarantee r in one box needs (r−1)k + 1 objects.
Non-homogeneous recurrence with overlap	Trial = $(n^m)\cdot$(forcing-term form); substitute to find the constant.	Verify by iterating three or four terms.
Coin-change / restricted-sum counting	Product of generating functions, or fix the largest coin and count the rest.	Bounded variable: $(1 - x^{u+1})/(1 - x)$.
Is a degree sequence graphic?	Havel–Hakimi reduction; an odd sum fails immediately.	Even sum is necessary, not sufficient.
Max matching / min vertex cover / max independent set	Find a maximum matching; in bipartite graphs τ = ν (König) and α = n − ν.	A Hall violator shows no perfect matching.
Number of proper colourings	Deletion–contraction, or use the formula for trees, cycles and complete graphs.	C_n: $(k-1)^n + (-1)^n(k-1)$.
Cut vertices and bridges	Remove each vertex or edge and count components, or use low-link values.	A leaf is never a cut vertex; every tree edge is a bridge.
Edge bound for planar graphs	Triangle-free: $E \leq 2V - 4$; otherwise $E \leq 3V - 6$; girth g: $E \leq g(V-2)/(g-2)$.	Faces from Euler: F = 2 − V + E.

METHOD

Graph Theory

Degree-count problem → handshake lemma `Σdeg=2E`.

Tree problem → immediately use `E=V−1`, connected + acyclic, and unique simple path properties.

Bipartite test → 2-color via BFS/DFS; equivalently, no odd cycle.

Euler trail/circuit → reason from vertex degrees and connectivity of the non-isolated portion.

Matching → identify maximum-cardinality constraints and augmenting-path logic where applicable.

Graph colouring → use lower bounds (cliques/odd cycles), upper bounds, and structural properties rather than blindly trying colours.

9 High-Yield Extensions & Standard Methods

MUST REMEMBER

Graph isomorphism

An isomorphism is a bijection preserving adjacency. Before attempting a mapping, compare invariants: number of vertices/edges, degree multiset, connected components, cycle structure and counts of special vertices.

GATE FAVOURITE

Cosets

For subgroup H of finite group G, every left coset has |H| elements and cosets partition G. By Lagrange, |H| divides |G| and the number of distinct cosets is |G|/|H|.

FORMULA / RULE

Homomorphism

A map preserves the operation: `f(a*b)=f(a)*f(b)` . For group homomorphisms, the kernel is a subgroup and the image is a subgroup; injective iff the kernel is the identity-only set.

COMMON TRAP

Homomorphism vs isomorphism

A homomorphism only has to preserve the operation, so it need not be injective or onto; an isomorphism is a **bijective** homomorphism. The kernel measures failure of injectivity: for finite groups `|image| = |G| / |kernel|` .

MUST REMEMBER

Hall's condition

A bipartite graph has a matching saturating the left side iff for every subset S of left vertices, `|N(S)| ≥ |S|`

FORMULA / RULE

Colouring bounds

`χ(G) ≥ ω(G)` , where ω is clique number. A nonempty bipartite graph has chromatic number 2. For a complete graph K_n, χ=n.

MUST REMEMBER

Generating-function workflow

Translate the recurrence/counting condition into coefficient extraction. Use geometric-series identities for unrestricted repetition and multiply generating functions when independent choices combine.

SHORTCUT

Graph invariant filter

If two graphs have different degree multisets, component counts, or vertex/edge counts, they are immediately non-isomorphic; do not attempt a full mapping.

Representative worked problems

METHOD

Q1 · Cosets and subgroup order

Given: |G|=24 and H is a subgroup of order 6. **How many distinct left cosets of H exist?**

Method: Use the coset partition theorem: number of cosets = |G|/|H|.

`24/6 = 4` . Every coset contains exactly 6 elements and the four cosets partition G.

METHOD

Q2 · Graph isomorphism filter

Given: G has degree sequence (3,2,2,1,1,1) while H has (3,3,1,1,1,1). **Can they be isomorphic?**

Method: Isomorphism preserves the degree of every vertex, hence preserves the sorted degree multiset.

The sequences differ, so the graphs are **not isomorphic**. No vertex mapping needs to be attempted.

METHOD

Q3 · Hall's theorem

Given: Left vertices {a,b,c} have neighborhoods N(a)={1,2}, N(b)={2,3}, N(c)={1,2,3}. **Does a matching saturating the left side exist?**

Method: Check Hall's condition for every relevant subset. Singletons have at least one neighbor; {a,b} has {1,2,3}; {a,c} and {b,c} also have 3; all three have 3.

Thus `|N(S)|≥|S|` for every S, so Hall's theorem guarantees a matching saturating {a,b,c}. One such matching is a→1, b→2, c→3.

METHOD

Q4 · Generating function coefficient

Problem: Find the number of solutions to x+y+z=5 with x,y,z≥0.

Method: Each variable contributes `1/(1-t)` ; multiply and extract $[t^5]$.

`(1-t)^-3 = Σ C(n+2,2)t^n` , so the coefficient of t^5 is `C(7,2)=21` .

Digital Logic

GATE CSE 2027 · complete formula, rule, method & trap sheet

90 reference boxes · **11** chapters ·

Number Systems & Computer Arithmetic | Boolean Algebra & Logic Gates | Minimisation: K-Maps & Quine–McCluskey | Combinational Circuits | Latches & Flip-Flops | Counters & Shift Registers | Finite State Machines & Synchronous Design | Memory & Programmable Logic | Arithmetic Circuits & Error-Control Codes | High-Yield Sequential & Boolean Methods | Last-minute one-liners

1 Number Systems & Computer Arithmetic

Positional systems & conversion

NOTES

A number in base *r* is a sum of digit × $r^{position}$. Common bases: binary (2), octal (8, groups of **3 bits**), hexadecimal (16, groups of **4 bits**). To convert integer → base r, divide repeatedly by r and read remainders **bottom-up**; for the fractional part multiply repeatedly by r and read the integer parts **top-down**.

WORKED EXAMPLE

Worked numerical — conversions

```
(1101.101)₂ = 8 + 4 + 0 + 1 + 1/2 + 0 + 1/8 = 13.625
156 ÷ 2 …  = (10011100)₂ = (234)₈ = (9C)₁₆        [group bits: 010 011 100 → 2 3 4 ; 1001 1100 → 9 C]
(0.1)₁₀ in binary = 0.000110011001…  (non-terminating — why 0.1 is inexact in floating point)
```

Digits needed for the unsigned value N in base r: $\lceil \log_r(N+1) \rceil$. n bits hold 0 … 2^n–1.

Signed number representations (n bits)

REFERENCE TABLE

Scheme	How negatives are formed	Range	Zeros
Sign-magnitude	MSB = sign, rest = |x|	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	2 (+0, −0)
1's complement	invert every bit	$-(2^{n-1}-1)$ … $+(2^{n-1}-1)$	2
2's complement	invert every bit, add 1 (= $2^n - x$)	**-2^{n-1} … $+(2^{n-1}-1)$**	1

FORMULA / RULE

Worked numerical — representing −45 in 8 bits

```
+45 = 0010 1101
1's complement of 45 : 1101 0010   (= −45 in 1's complement)
2's complement of 45 : 1101 0011   (= −45 in 2's complement)
Check: 256 − 45 = 211 = 1101 0011 ✔
```

8-bit 2's complement range: **−128 … +127**. Subtraction is done as addition: `A − B = A + (2's complement of B)`.

```
13 − 6 in 5 bits:   6 = 00110 → 1's complement 11001 → 2's complement 11010
  01101 + 11010 = 1 00111 → discard the carry-out → 00111 = 7 ✔
```

GATE FAVOURITE

GATE angle

2's complement has the **asymmetric range** (one more negative than positive), and sign extension (copy the MSB leftwards) preserves the value. Only 2's complement has a **unique zero**.

WORKED EXAMPLE

Worked numerical — 8-bit overflow test

```
  0111 1010   (+122)                  1111 0110   (−10)
+ 0001 1101   (+29)                 + 0000 1010   (+10)
  ---------                           ---------
  1001 0111   (shows −105)            1 0000 0000   (0)
carry into MSB = 1, carry out = 0     carry into MSB = 1, carry out = 1
different → OVERFLOW (151 > 127)      same → no overflow, result 0 ✔
```

Binary codes

REFERENCE TABLE

Code	Idea	Example
BCD (8421)	each decimal digit → 4 bits; 1010–1111 invalid	47 → 0100 0111
Excess-3	BCD + 3 (self-complementing)	5 → 1000
Gray	adjacent values differ in **exactly one bit**	binary 1011 → gray 1110
ASCII	7-bit character code	'A' = 65 = 1000001

NOTES

Binary → Gray: $G_{n-1} = B_{n-1}$, $G_i = B_{i+1} \oplus B_i$ (`1011 → 1, 1⊕0=1, 0⊕1=1, 1⊕1=0 → 1110`). **Gray → binary:** $B_{n-1} = G_{n-1}$, $B_i = B_{i+1} \oplus G_i$. **BCD addition:** if a digit sum exceeds 9, add 6 (0110) to that digit.

Overflow in signed addition

NOTES

Overflow happens when the true sum is outside the representable range. Detect it as: **carry into the MSB ≠ carry out of the MSB**, or equivalently both operands have the same sign but the result has the opposite sign. Adding numbers of **opposite sign never overflows**. Carry-out alone is the overflow flag for **unsigned** arithmetic.

IEEE 754 floating point

REFERENCE TABLE

Format	Sign	Exponent	Mantissa (fraction)	Bias
Single (32-bit)	1	8	23	127
Double (64-bit)	1	11	52	1023

NOTES

Normalised value = $(-1)^s \times 1.m \times 2^{e-bias}$ (a hidden leading 1). Exponent field all 0s → zero / denormal; all 1s → ∞ or NaN.

WORKED EXAMPLE

Worked numerical — encode 13.625 (single precision)

```
13.625 = 1101.101₂ = 1.101101 × 2³
sign = 0
exponent = 3 + 127 = 130 = 1000 0010
mantissa = 1011 0100 0000 0000 0000 000   (drop the leading 1)

bits: 0 10000010 10110100000000000000000  = 0100 0001 0101 1010 0000 … = 0x415A0000
```

COMMON TRAP

Pitfall

Floating-point addition needs the exponents **aligned first**; because mantissas have finite length, `(a + b) + c` may differ from `a + (b + c)` — floating-point addition is **not associative**.

More on IEEE 754 — limits, denormals, rounding

REFERENCE TABLE

Quantity (single precision)	Value
Significand bits (with hidden 1)	24
Largest finite number	$(2 - 2^{-23}) \times 2^{127} \approx 3.4028 \times 10^{38}$ (exponent field 254)
Smallest positive normalised	$2^{-126} \approx 1.1755 \times 10^{-38}$ (exponent field 1)
Smallest positive denormal	$2^{-149} \approx 1.4 \times 10^{-45}$ (exponent field 0, no hidden 1)
Machine epsilon (gap above 1.0)	$2^{-23} \approx 1.19 \times 10^{-7}$ (about 7 decimal digits)
Floats between 1 and 2	2^{23} (evenly spaced)

RULES & FACTS

- **Denormals** (exponent field 0, fraction ≠ 0): value = $(-1)^S \times 0.m \times 2^{-126}$. They fill the gap between 0 and the smallest normalised number (gradual underflow).
- Exponent field 255: fraction 0 → ±∞; fraction ≠ 0 → NaN. Exponent and fraction both 0 → ±0.
- Default rounding: **round to nearest, ties to even**. Spacing between floats doubles each time the exponent increases by 1, so large numbers lose absolute precision.

WORKED EXAMPLE

Worked numerical — adding floats

Q: Add 1.5 and 0.25 in binary floating point.

A: $1.5 = 1.1 \times 2^0$ and $0.25 = 1.0 \times 2^{-2}$. Align the smaller exponent to the larger one by shifting the mantissa right 2 places, add, then renormalise:

```
  1.100  × 2⁰
+ 0.010  × 2⁰      (1.0 × 2⁻² shifted right by 2)
= 1.110  × 2⁰  = 1.75      (already normalised; stored as 0x3FE00000 in double precision)
```

WORKED EXAMPLE

Worked numerical — precision loss

Single precision has 24 significant bits, so integers above $2^{24} = 16\,777\,216$ are no longer all representable: `16777216 + 1` rounds back to **16777216**. In double precision `0.1 + 0.2 = 0.30000000000000004 ≠ 0.3`, because neither 0.1 nor 0.2 has a finite binary expansion. Never test floating-point results with `==`.

Fixed point and range summary

RULES & FACTS

- **Fixed point**: a fixed number of integer and fraction bits. An unsigned Qm.n format with m integer bits and n fraction bits has resolution 2^{-n} and range 0 … $2^m - 2^{-n}$.
- Multiplying two n-bit unsigned numbers needs up to **2n bits** for the product; adding two n-bit numbers needs up to **n + 1** bits.
- Sign extension: to widen a 2's complement number, copy the sign bit into the new high bits (e.g. 1101 → 1111 1101 keeps −3).

2 Boolean Algebra & Logic Gates

Basic gates

REFERENCE TABLE

Gate	Expression	Output is 1 when…
AND	A·B	all inputs are 1
OR	A + B	at least one input is 1
NOT	A'	input is 0
NAND	(A·B)'	at least one input is 0
NOR	(A + B)'	all inputs are 0
XOR	A ⊕ B = A'B + AB'	inputs differ (odd number of 1s)
XNOR	(A ⊕ B)'	inputs are equal

Laws you must know cold

KEY POINT

Identities

```
A + 0 = A      A · 1 = A
A + 1 = 1      A · 0 = 0
A + A = A      A · A = A
A + A' = 1     A · A' = 0
(A')' = A
```

KEY POINT

Absorption / redundancy

```
A + AB = A
A(A + B) = A
A + A'B = A + B
A(A' + B) = AB
```

KEY POINT

De Morgan

```
(A + B)' = A'·B'
(A · B)' = A' + B'
(general): complement of f =
 swap + ↔ ·  and complement every literal
```

KEY POINT

Consensus

```
AB + A'C + BC = AB + A'C
(A+B)(A'+C)(B+C) = (A+B)(A'+C)
```

The consensus term BC is redundant — but it is exactly the term that **removes hazards** (Section 4).

NOTES

Duality: swap AND↔OR and 0↔1 (literals unchanged) to get the dual of an identity — the dual of a true identity is also true. The dual of `A + BC'` is `A·(B + C')`. A function with $f^d = f$ is **self-dual**.

XOR properties

FORMULA / WORKING

```
A ⊕ 0 = A     A ⊕ 1 = A'     A ⊕ A = 0     A ⊕ A' = 1
Commutative and associative.   A ⊕ B ⊕ C = 1  iff an ODD number of inputs is 1  (parity)
The complement of an n-input XOR chain is the even-parity function (1 iff an even number of inputs is 1).
```

Canonical forms & counting

- **Minterm**: product containing every variable once (true or complemented); n variables → 2^n minterms. f = Σm(…) (sum of minterms, canonical SOP).
- **Maxterm**: sum containing every variable once. f = ΠM(…) lists the rows where f = **0**. The minterm numbers of f and the maxterm numbers of f are **complementary sets**.
- Number of distinct Boolean functions of n variables = $\mathbf{2^{2^n}}$ (n = 2: 16, n = 3: 256).
- Number of **self-dual** functions of n variables = $2^{2^{n-1}}$ (n = 3: **16**).

Worked numerical — simplify with consensus

Q: Simplify (A + B)(A' + C)(B + C).

A: This is the dual of the consensus pattern: the term (B + C) is redundant, so the result is (A + B)(A' + C). Expanding: AC + A'B (the BC cross term is absorbed).

Universal gates

NAND alone (and NOR alone) can build every other gate, so each is **functionally complete**. {AND, NOT} and {OR, NOT} are also complete; **{AND, OR} is not** (no way to invert).

```
NOT :  A' = (A·A)'                    (tie the inputs together)
AND :  A·B = ((A·B)')'                (NAND followed by NAND-as-inverter)
OR  :  A + B = (A'·B')'               (NAND of the inverted inputs)
XOR :  4 NAND gates:  N1 = (AB)'; N2 = (A·N1)'; N3 = (B·N1)'; A⊕B = (N2·N3)'
```

Bubble pushing

Any AND-OR (SOP) network can be redrawn as **all-NAND two-level logic** directly: NAND–NAND = AND–OR. Similarly an OR-AND (POS) network becomes NOR–NOR.

3 Minimisation: K-Maps & Quine–McCluskey

K-map rules

RULES & FACTS

- Rows and columns are labelled in **Gray-code order** (00, 01, 11, 10) so neighbouring cells differ in one variable.
- Group 1s (for SOP) in rectangles of size **1, 2, 4, 8, 16** — always a power of 2. A group of 2^k cells removes **k** variables.
- Groups may **wrap around** edges (the map is a torus), and may overlap. Use the **fewest, largest** groups that cover every 1.
- For **POS**, group the 0s to get f' as SOP, then complement (or read the groups as sum terms with literals inverted).
- Every cell in an n-variable map has exactly **n neighbours** (4 in a 4-variable map).

REFERENCE TABLE

Term	Meaning
Implicant	any product term that implies f (a group of 1s)
Prime implicant (PI)	an implicant that cannot be enlarged — a **maximal** group
Essential PI (EPI)	a PI covering at least one 1 that **no other PI** covers — it must be in every minimal cover
Don't care (d / X)	input combination that never occurs or whose output is irrelevant; treat as 1 only if it enlarges a group, never need to cover it

WORKED EXAMPLE

Worked numerical — 4-variable map

Q: f(A,B,C,D) = Σm(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15). Find all prime implicants, the essential ones, and a minimal SOP.

AB \ CD	00	01	11	10
00	1_0	0_1	1_3	1_2
01	0_4	1_5	1_7	0_6
11	0_{12}	1_{13}	1_{15}	0_{14}
10	1_8	1_9	1_{11}	1_{10}

```
Prime implicants (6):  B'D' {0,2,8,10}   BD {5,7,13,15}   CD {3,7,11,15}
                       B'C {2,3,10,11}    AD {9,11,13,15}   AB' {8,9,10,11}
Essential (2):  B'D'  (only PI covering m0)   and   BD (only PI covering m5)
Left over after the EPIs: m3, m9, m11
   → pick one PI for m3/m11 (CD or B'C) and one for m9 (AD or AB')

Minimal SOP:  f = B'D' + BD + CD + AD        → 4 terms, 8 literals
(several equal-cost minimal forms exist — e.g. replacing CD by B'C)
```

WORKED EXAMPLE

Worked numerical — a map with NO essential prime implicants

Q: f(A,B,C) = Σm(0, 1, 2, 5, 6, 7).

A \ BC	00	01	11	10
0	1_0	1_1	0_3	1_2
1	0_4	1_5	1_7	1_6

```
Every 1 is covered by TWO pairs → six PIs, none essential (a "cyclic" map):
  A'B' {0,1}  A'C' {0,2}  BC' {2,6}  AB {6,7}  AC {5,7}  B'C {1,5}
Minimal covers use 3 pairs, e.g.  f = A'C' + AB + B'C   or
f = A'B' + BC' + AC   (3 terms, 6 literals)
```

WORKED EXAMPLE

Worked numerical — don't cares

Q: f(A,B,C,D) = Σm(1, 4, 6, 7, 8, 9, 10, 11, 15) + d(0, 5). Find the minimal SOP.

AB \ CD	00	01	11	10
00	X_0	1_1	0_3	0_2
01	1_4	X_5	1_7	1_6
11	0_{12}	0_{13}	1_{15}	0_{14}
10	1_8	1_9	1_{11}	1_{10}

```
Prime implicants (6):  B'C' {0,1,8,9}   A'C' {0,1,4,5}   A'B {4,5,6,7}
                       AB' {8,9,10,11}   BCD {7,15}   ACD {11,15}
Essential: A'B (only PI covering m6) and AB' (only PI covering m10).
Remaining: m1 → B'C' (or A'C'),  m15 → BCD (or ACD)

Minimal SOP:  f = A'B + AB' + B'C' + BCD      → 4 terms, 9 literals
(here A'B + AB' is the XOR of A and B)
```

The don't cares d0 and d5 were used as 1s to enlarge groups; they did **not** have to be covered.

POS from a map

WORKED EXAMPLE

Worked numerical

For the first function above, the 0s are m1, 4, 6, 12, 14. Grouping them: `BD'` {4,6,12,14} and `A'B'C'D` {1}, so `f' = BD' + A'B'C'D`. Complementing with De Morgan:

```
f = (B' + D)(A + B + C + D')
```

Quine–McCluskey (tabular) method

RULES & FACTS

1. List all minterms (and don't cares) in binary, grouped by the **number of 1s**.
2. Repeatedly combine pairs from **adjacent groups** that differ in exactly one bit, replacing that bit by **–**; mark combined terms as used.
3. Terms that can no longer combine are the **prime implicants**.
4. Build a PI chart (rows = PIs, columns = minterms, **excluding don't cares**); pick essential PIs first, then cover the rest with the fewest/cheapest PIs (Petrick's method for cyclic charts).

GATE FAVOURITE

GATE angle

Typical questions: "number of prime implicants", "number of essential prime implicants", "minimum number of literals/gates/NAND gates". Remember: minimum SOP ≠ unique; PIs count is unique; and the answer changes if you must also realise the complement or use POS.

4 Combinational Circuits

Adders and subtractors

REFERENCE TABLE

Circuit	Outputs
Half adder	Sum = A ⊕ B, Carry = A·B
Full adder	Sum = A ⊕ B ⊕ Cin, Cout = AB + Cin(A ⊕ B) = AB + BCin + ACin (majority)
Half subtractor	Diff = A ⊕ B, Borrow = A'B
Full subtractor	Diff = A ⊕ B ⊕ Bin, Bout = A'B + (A ⊕ B)'Bin

NOTES

A **full adder = 2 half adders + 1 OR gate**. An **n-bit adder/subtractor** uses n full adders, with each B input XORed with a control line M and Cin = M: M = 0 adds, M = 1 computes A + B' + 1 = A − B.

Ripple-carry vs carry-lookahead

RULES & FACTS

- **Ripple-carry**: each stage waits for the previous carry, so delay grows **linearly** with n.
- **Carry-lookahead (CLA)**: define **generate** $G_i = A_iB_i$ and **propagate** $P_i = A_i \oplus B_i$. Then $C_{i+1} = G_i + P_iC_i$, and unrolling gives every carry directly from the inputs and C_0 in two gate levels:

```
C1 = G0 + P0·C0
C2 = G1 + P1·G0 + P1·P0·C0
C3 = G2 + P2·G1 + P2·P1·G0 + P2·P1·P0·C0
Sum_i = P_i ⊕ C_i
```

WORKED EXAMPLE

Worked numerical — 4-bit CLA

Q: A = 1011, B = 0110, C0 = 0. Find P, G, all carries and the sum.
A: (bit 0 is the rightmost)

```
bit :   3 2 1 0
A   :   1 0 1 1
B   :   0 1 1 0
P=A⊕B:  1 1 0 1      G=A·B:  0 0 1 0
C1 = G0 + P0C0 = 0 + 1·0 = 0
C2 = G1 + P1C1 = 1 + 0·0 = 1
C3 = G2 + P2C2 = 0 + 1·1 = 1
C4 = G3 + P3C3 = 0 + 1·1 = 1
Sum_i = P_i ⊕ C_i :  S0 = 1⊕0 = 1,  S1 = 0⊕0 = 0,  S2 = 1⊕1
= 0,  S3 = 1⊕1 = 0
Result = C4 S3 S2 S1 S0 = 1 0001 = 17 = 11 + 6 ✔
```

Multiplexer (MUX), demultiplexer, decoder, encoder

REFERENCE TABLE

Block	Inputs → Outputs	Key fact
2^n:1 MUX	2^n data + n select → 1	output = Σ (minterm of selects · data input); a 2^n:1 mux needs $\mathbf{2^n - 1}$ 2:1 muxes
1:2^n DEMUX	1 data + n select → 2^n	a decoder with enable used as data input
n-to-2^n decoder	n → 2^n (one-hot)	generates all minterms; any function = OR of its minterm outputs
2^n-to-n encoder	one-hot → n-bit code	assumes exactly one input active; a **priority encoder** resolves several active inputs
Comparator	A, B → A>B, A=B, A<B	bit equality is XNOR; A = B iff all bit-XNORs are 1

WORKED EXAMPLE

Worked numerical — function with a 4:1 MUX

Q: Implement f(A,B,C) = Σm(1,2,4,7) using a 4:1 MUX with A and B as select lines.
A: For each value of AB, look at f for C = 0 and C = 1:

```
AB = 00  → m0, m1 : f = 0, 1  → f = C       I0 = C
AB = 01  → m2, m3 : f = 1, 0  → f = C'      I1 = C'
AB = 10  → m4, m5 : f = 1, 0  → f = C'      I2 = C'
AB = 11  → m6, m7 : f = 0, 1  → f = C       I3 = C
```

So I0 = C, I1 = C', I2 = C', I3 = C. In general an **n-variable** function needs only a $\mathbf{2^{n-1}}$**:1 MUX** plus at most one inverter. (This f is the 3-input XOR.)

WORKED EXAMPLE

Worked numerical — building bigger decoders

Q: How many 2-to-4 decoders (with enable) build a 4-to-16 decoder? And how many 3-to-8 decoders plus one 2-to-4 for a 5-to-32?
A: 16 outputs / 4 = 4 decoders in the second level, plus 1 decoder in the first level to enable one of them → **5**. For 5-to-32: 32 / 8 = 4 decoders (3-to-8) at the second level + 1 (2-to-4) at the first → **5 decoders** in total.

Hazards (glitches)

NOTES

A **hazard** is a momentary wrong output caused by unequal gate delays when inputs change.

RULES & FACTS

- **Static-1 hazard**: output should stay 1 but briefly drops to 0 — occurs in SOP when two adjacent 1-cells are covered by **different** product terms with no common term.
- **Static-0 hazard**: output should stay 0 but pulses 1 (in POS circuits).
- **Dynamic hazard**: output changes 3 or more times when it should change once.

WORKED EXAMPLE

Worked numerical — removing a static-1 hazard

`f = AB + A'C`. With B = C = 1 and A switching 1 → 0, f should stay 1, but AB falls before A'C rises (the inverter delay) → glitch. Add the **consensus term** BC, which stays 1 throughout: `f = AB + A'C + BC`. On the K-map this is the extra group straddling the two adjacent cells.

COMMON TRAP

Remember

Hazard-free covers need **every pair of adjacent 1s covered by some single product term** — that can mean using non-essential PIs, so the hazard-free circuit is not the minimal one.

5 Latches & Flip-Flops

SR latch

NOTES

Two cross-coupled NOR (or NAND) gates. For the NOR version: S = 1 sets Q = 1, R = 1 resets Q = 0, S = R = 0 holds, and **S = R = 1 is forbidden** (both outputs go 0, and the next state is unpredictable if both return to 0 together). The NAND version is active-low (S' and R'), with S' = R' = 0 forbidden. Adding a clock/enable gives a **gated SR latch**; connecting S = D, R = D' removes the forbidden state and gives the **D latch**.

Characteristic equations and tables

REFERENCE TABLE

Type	Characteristic equation	Behaviour
SR	$Q^+ = S + R'Q$ (SR = 0)	00 hold, 01 reset, 10 set, 11 invalid
D	$Q^+ = D$	copies D on the clock edge
JK	$Q^+ = JQ' + K'Q$	00 hold, 01 reset, 10 set, 11 **toggle**
T	$Q^+ = T \oplus Q = TQ' + T'Q$	0 hold, 1 toggle

NOTES

Excitation tables (what inputs are needed for a wanted Q → Q^+ transition; X = don't care):

REFERENCE TABLE

Q → Q^+	S	R	J	K	D	T
0 → 0	0	X	0	X	0	0
0 → 1	1	0	1	X	1	1
1 → 0	0	1	X	1	0	1
1 → 1	X	0	X	0	1	0

COMMON TRAP

Memory hook

JK with J = K = 1 **toggles**, so it acts as a T flip-flop; JK with K = J' acts as a D flip-flop. JK is the most flexible, T is the building block of counters, D is the building block of registers.

Flip-flop conversions

REFERENCE TABLE

Build…	From…	Connection
T	JK	J = K = T
D	JK	J = D, K = D'
D	SR	S = D, R = D'
T	D	D = T ⊕ Q
JK	D	D = JQ' + K'Q

Triggering and the race-around condition

RULES & FACTS

- **Level-triggered** (latch): output follows input for the whole time the clock is high.
- **Edge-triggered**: samples inputs only at the rising (or falling) edge.
- **Race-around**: a level-triggered JK with **J = K = 1** toggles repeatedly while the clock stays high, if the clock pulse is **longer than the propagation delay**. Cures: **master–slave** or **edge-triggered** design, or a pulse shorter than t_{pd} (impractical).
- **Master–slave**: two latches on opposite clock phases — the master samples while CLK = 1, the slave copies at the falling edge, so the output changes once per cycle.

Timing parameters

RULES & FACTS

- **Setup time** t_{su}: data must be stable **before** the active edge. **Hold time** t_h: data must stay stable **after** the edge. **Clock-to-Q** t_{cq}: delay from the edge to the new output.
- Minimum clock period between two flip-flops: $T \geq t_{cq} + t_{comb(max)} + t_{su}$ (plus clock skew). Hold condition: $t_{cq(min)} + t_{comb(min)} \geq t_h$. Violating setup is cured by a slower clock; violating hold is **not**.

WORKED EXAMPLE

Worked numerical — maximum clock frequency

Q: t_{cq} = 2 ns, longest combinational delay = 5 ns, t_{su} = 1 ns (no skew). Find the maximum clock frequency.
A: T_{min} = 2 + 5 + 1 = 8 ns → f_{max} = 1 / 8 ns = **125 MHz**.

6 Counters & Shift Registers

Asynchronous (ripple) counters

NOTES

Each flip-flop (toggle mode, J = K = 1 or T = 1) is clocked by the **output of the previous one**, so the clock ripples through. An n-bit ripple counter counts 0 … $2^n - 1$ (modulus 2^n); each stage **divides frequency by 2**, so the n-th output has frequency $f/2^n$.

RULES & FACTS

- **Up counter**: clock from Q (falling-edge FFs) or from Q' (rising-edge FFs). **Down counter**: the opposite connection.
- **Drawback**: delays add up. For an n-bit ripple counter the output settles after **n × t_{pd}**, so $f_{max} \approx 1 / (n \cdot t_{pd})$ and decoding glitches appear while it ripples.

WORKED EXAMPLE

Worked numerical

Q: A 4-bit ripple counter uses flip-flops with t_{pd} = 20 ns. Maximum clock frequency?
A: Settling time = 4 × 20 = 80 ns → f_{max} = 1 / 80 ns = **12.5 MHz**. A synchronous 4-bit counter needs only one t_{pd} plus the AND-gate delay, so it runs much faster.

Synchronous counters

NOTES

All flip-flops share **one clock**, so every output changes together after a single t_{pd}. The T-inputs come from combinational logic; for an up counter $T_i = Q_0 \cdot Q_1 \cdot \ldots \cdot Q_{i-1}$. They are faster but need extra gates.

Modulo-N counters

RULES & FACTS

- Flip-flops needed: $\lceil \log_2 N \rceil$ (mod-10 → 4, mod-60 → 6).
- With an **asynchronous reset**: decode state N with a NAND gate and feed it to the clear inputs. The invalid state N exists only for a few nanoseconds (a glitch).
- Output frequency of a mod-N counter = f_{clk} / N (so 1.6 MHz into a mod-16 gives 100 kHz at its MSB).

WORKED EXAMPLE

Worked numerical — mod-6 from a 3-bit ripple counter

```
Count: 000 → 001 → 010 → 011 → 100 → 101 → (110 appears for an instant) → 000
```

Decode **Q2 Q1 Q0 = 110** (the first state ≥ 6; Q2 and Q1 are enough to detect it) with a NAND and drive all CLEAR lines. Counts 0–5 are stable, so the modulus is 6.

Shift registers

REFERENCE TABLE

Type	Load	Read	Clocks for n bits
SISO	serial	serial	n to load, n more to read out
SIPO	serial	parallel	n to load, then read at once
PISO	parallel	serial	1 to load, n to read out
PIPO	parallel	parallel	1

NOTES

A serial-to-parallel conversion of an n-bit word needs **n clock pulses**. Shifting left by one position doubles an unsigned number.

Ring and Johnson (twisted-ring) counters

KEY POINT

Ring counter
Q of the last FF fed to D of the first. One circulating 1.
n flip-flops → n states

```
1000 → 0100 → 0010 → 0001 → 1000
```

Needs initial preset; self-starting only with extra logic.

KEY POINT

Johnson counter
Q' of the last FF fed to the first.
n flip-flops → 2n states

```
0000 → 1000 → 1100 → 1110 → 1111
→ 0111 → 0011 → 0001 → 0000
```

Adjacent states differ in **one bit**, so decoding is glitch-free; each state decodes with a 2-input AND.

GATE FAVOURITE

GATE angle

Mod comparison: n flip-flops give 2^n states as a binary counter, 2n states as a Johnson counter, and only n states as a ring counter. The unused states of Johnson/ring counters form **lock-out** cycles unless corrected.

7 Finite State Machines & Synchronous Design

Mealy vs Moore

REFERENCE TABLE

	Moore	Mealy
Output depends on	**present state only**	**present state AND present input**
Output labelled on	states	transitions (input / output)
Typical state count	more states	fewer states
Output timing	changes after clock edge (synchronous, glitch-free)	can change as soon as input changes (reacts a cycle earlier, may glitch)

RULES & FACTS

- Moore → Mealy: always possible with the **same number of states**.
- Mealy → Moore: may need up to **(states × distinct outputs)** states.
- Flip-flops needed for N states: $\lceil \log_2 N \rceil$ (5 states → 3 flip-flops).

FORMULA / WORKING

```
Mealy machine  (next state / output)
State    input 0     input 1
S0       S0 / 0      S1 / 0
S1       S2 / 0      S1 / 0
S2       S0 / 0      S3 / 0
S3       S2 / 0      S1 / 1      ← the final 1 of 1011 completes the pattern; suffix "1" stays → S1
```

WORKED EXAMPLE

Worked numerical — states, flip-flops and a trace

Mealy machine: **4 states** → **2 flip-flops**. Moore machine: needs an extra state S4 ("1011", output 1) → **5 states** → **3 flip-flops**.

```
input :  1 0 1 1 0 1 1 0 1 0 1 1 1
output:  0 0 0 1 0 0 1 0 0 0 0 1 0        → three detections
(overlaps at the 4th and 7th bits count)
```

WORKED EXAMPLE

Worked numerical — 101 detector, overlapping

Three Mealy states (none / "1" / "10"); on state "10" with input 1 the output is 1 and the next state is "1". For input `1101011` the output is `0001010`.

State minimisation

NOTES

Two states are **equivalent** if, for every input sequence, they produce the same outputs (and move to equivalent states). Procedure: partition states by output behaviour, then keep splitting while their next states fall in different blocks (partition refinement — the same idea as DFA minimisation). Equivalent states merge, reducing the number of flip-flops and logic.

GATE FAVOURITE

GATE angle

"How many states / flip-flops for this detector?" and "what is the output sequence for this input?" are the two commonest questions. Always check whether the machine is **overlapping or non-overlapping**, and Mealy or Moore, before counting.

Synchronous design procedure

RULES & FACTS

1. Draw the state diagram / state table from the specification.
2. **Minimise states** (merge equivalent states).
3. **Assign binary codes** to states (number of flip-flops = $\lceil \log_2 N \rceil$).
4. Use the flip-flop **excitation table** to derive flip-flop input equations and output equations (K-maps; unused states are don't cares).
5. Draw the circuit; check that unused states eventually return to a valid state (self-starting).

Sequence detector: overlapping 1011

NOTES

State names track the longest suffix of the input that is a prefix of 1011: S0 = nothing, S1 = "1", S2 = "10", S3 = "101".

Worked design — a synchronous mod-5 counter with T flip-flops

NOTES

Design a counter that follows `000 → 001 → 010 → 011 → 100 → 000`. The states 101, 110, 111 never occur, so they are **don't cares**. For a T flip-flop the excitation is $T = Q \oplus Q^+$ (T = 1 exactly when that bit must change).

FORMULA / WORKING

Present Q2Q1Q0	Next Q2Q1Q0	T2 T1 T0
000	001	0 0 1
001	010	0 1 1
010	011	0 0 1
011	100	1 1 1
100	000	1 0 0
101,110,111	X X X	X X X

NOTES

K-map each T over the 3 present-state variables, using the don't cares to enlarge groups:

FORMULA / WORKING

```
T0 = Σm(0,1,2,3) + d(5,6,7)   →  T0 = Q2'
T1 = Σm(1,3)     + d(5,6,7)   →  T1 = Q0
T2 = Σm(3,4)     + d(5,6,7)   →  T2 = Q2 + Q1·Q0
```

NOTE

Self-start check (always do this)

Feed the unused states through the final equations:

```
101 → T2T1T0 = 1 1 0 → 011        110 → T = 1 0 0 → 010
111 → T = 1 1 0 → 001
```

Every unused state falls into the main cycle within one clock, so the counter is **self-starting**. Counter modulus = 5 → $\lceil \log_2 5 \rceil$ = 3 flip-flops.

8 Memory & Programmable Logic

Memory sizing

RULES & FACTS

- A memory with **k address lines** holds 2^k words; with w bits per word, capacity = $2^k \times w$ bits. Address lines needed for N words = $\lceil \log_2 N \rceil$.
- 12 address lines × 8-bit words → 4096 × 8 = 32 768 bits = **4 KB**.
- Building a bigger memory from smaller chips: chips needed = (total words ÷ words per chip) × (total width ÷ chip width). A 16K × 16 memory from 4K × 8 chips: (16K/4K) × (16/8) = 4 × 2 = **8 chips**. The extra high-order address lines (2 here) drive a **decoder** that selects the chip row.
- **RAM** is volatile read/write (SRAM: flip-flop cells, fast; DRAM: capacitor cells, dense, needs refresh). **ROM** is non-volatile (mask ROM, PROM, EPROM, EEPROM, Flash).

ROM as a combinational circuit

NOTES

A ROM with n address lines and m data lines is an **n-to-2^n decoder followed by m OR gates**: the decoder produces every minterm and each output is the OR of the minterms stored as 1. Hence a $2^n \times m$ ROM can implement **any m Boolean functions of n variables**; its size is $2^n \times m$ bits. A ROM with 4 inputs and 3 outputs holds 16 × 3 = **48 bits**. It is simple but wasteful when only a few minterms are needed, since the table doubles with each extra input.

PLA, PAL, CPLD, FPGA

REFERENCE TABLE

Device	AND array	OR array	Note
ROM	fixed (full decoder, all minterms)	programmable	every minterm available
PLA	**programmable**	**programmable**	only the needed product terms are generated and can be shared between outputs
PAL	**programmable**	fixed	cheaper and faster, but each output has a limited number of product terms
CPLD	several PAL-like blocks with programmable interconnect		non-volatile, fast fixed timing
FPGA	array of **look-up tables (LUTs)**, flip-flops and routing		a k-input LUT is a $2^k \times 1$ memory and can realise **any** function of k variables (4-input LUT = 16 bits)

WORKED EXAMPLE

Worked numerical — size of a PLA vs a ROM

Q: A circuit has 5 inputs and 4 outputs. Compare a ROM with a PLA that needs 20 product terms.

A: Assume each input is available true and complemented, so the AND array has 2 × 5 = 10 input lines.

```
ROM : 2⁵ × 4                  = 128 programmable bits
PLA : AND array 10 × 20       = 200 links
      OR  array 20 × 4        =  80 links        total
280 links
```

For this function the ROM is smaller. A PLA wins when the number of product terms is far below 2^n (for example 5 inputs but only 6 product terms: 60 + 24 = 84 links).

Other useful building blocks

RULES & FACTS

- **Tri-state buffer**: output is 0, 1 or high-impedance (Z). Several tri-state drivers can share one **bus** if exactly one is enabled at a time.
- **Open-collector / open-drain** outputs allow a **wired-AND** on a shared line with a pull-up resistor.
- **Fan-in** = number of inputs a gate accepts; **fan-out** = number of gate inputs one output can drive. A wide AND with limited fan-in is built as a tree of smaller gates, adding delay of about $\lceil \log_k n \rceil$ levels.
- **Propagation delay**: critical path = the longest delay from any input to any output; it sets the clock-period limit of a synchronous design.

9 Arithmetic Circuits & Error-Control Codes

BCD adder

NOTES

Add two BCD digits with a 4-bit binary adder. If the binary sum is greater than 9 (that is, the carry-out is 1, or the sum is 1010–1111), **add 6 (0110)** to correct it and propagate a decimal carry. The correction condition is $K = C_{out} + S_3S_2 + S_3S_1$.

WORKED EXAMPLE

Worked numerical

8 + 5 = 13: 1000 + 0101 = 1101 (13 > 9) → add 0110: 1101 + 0110 = 1 0011 → carry 1, digit 0011 → BCD **0001 0011** = 13 ✔.

Multipliers

RULES & FACTS

- **Shift-and-add**: for each multiplier bit that is 1, add the shifted multiplicand — n cycles for n-bit operands, 2n-bit product.
- **Array multiplier** (n × n): n^2 AND gates generate the partial products; they are added by n(n−1) adders in total (n(n−2) full adders and n half adders). A 4 × 4 array multiplier therefore uses **16 AND gates, 8 full adders and 4 half adders**. It is fast (delay grows linearly with n) but large.
- **Booth's algorithm** handles signed 2's complement operands by recoding runs of 1s in the multiplier (the pair Q_0Q_{-1} = 10 → subtract, 01 → add, 00/11 → shift only).
- Adders recap: ripple-carry delay ∝ n; carry-lookahead is about constant to log n; **carry-select** and **carry-skip** adders sit in between.

Parity and Hamming distance

RULES & FACTS

- **Even parity**: add a bit so the total number of 1s is even (odd parity: odd). A single parity bit detects any **odd** number of bit errors but cannot locate them and misses even-numbered errors.
- **Hamming distance** d(x, y) = number of positions where x and y differ. The minimum distance d_{min} of a code decides its power:

```
detect up to  d_min − 1  errors
correct up to ⌊(d_min − 1)/2⌋ errors
detect e and correct t simultaneously if d_min ≥ e + t + 1  (e ≥ t)
```

- A code with d_{min} = 5 detects up to 4 errors or corrects up to 2. A code with d_{min} = 3 corrects 1 error; adding an overall parity bit makes d_{min} = 4 (**SECDED**: single-error-correct, double-error-detect).

WORKED EXAMPLE

Worked numerical

d(10110, 11011): XOR = 01101 → three 1s → distance **3**. Even parity bit for 1011011 (five 1s) = **1**.

Hamming (7,4) single-error-correcting code

NOTES

For m data bits you need r check bits with $2^r \geq m + r + 1$: m = 4 → r = 3 (7-bit code word), m = 8 → r = 4 (12-bit word). Check bits sit at the **power-of-two positions** 1, 2, 4, …; check bit p_i covers every position whose binary index contains bit i.

FORMULA / WORKING

```
Position:   1   2   3   4   5   6   7
Role:       p1  p2  d1  p4  d2  d3  d4
p1 covers positions 1,3,5,7     p2 covers 2,3,6,7     p4 covers 4,5,6,7  (even parity)
```

WORKED EXAMPLE

Worked numerical — encode and correct

Encode data 1011 (d1 d2 d3 d4 = 1,0,1,1):

```
p1 = d1⊕d2⊕d4 = 1⊕0⊕1 = 0
p2 = d1⊕d3⊕d4 = 1⊕1⊕1 = 1
p4 = d2⊕d3⊕d4 = 0⊕1⊕1 = 0
code word (positions 1…7) = 0 1 1 0 0 1 1  =  0110011
```

Receive 0110111 (bit 5 flipped). Recompute the three checks over their groups; the failed checks form the **syndrome**:

```
c1 (1,3,5,7) = 0⊕1⊕1⊕1 = 1       c2 (2,3,6,7) = 1⊕1⊕1⊕1 = 0
c4 (4,5,6,7) = 0⊕1⊕1⊕1 = 1
syndrome c4c2c1 = 101 = 5  →  the error is in position 5:
flip it back → 0110011 → data bits (3,5,6,7) = 1011
```

The syndrome, read as a binary number, **is the position of the error** (0 means no error). All 16 × 7 single-bit-error cases were checked by program.

CRC (brief)

NOTES

A **cyclic redundancy check** divides the message (shifted left by the degree of the generator polynomial) by the generator using XOR (mod-2) division; the remainder is appended. A receiver recomputes it; a non-zero remainder signals an error. A CRC with a degree-r generator detects all burst errors of length ≤ r and all single-bit errors if the generator has at least two terms.

10 High-Yield Sequential & Boolean Methods

Asynchronous sequential circuits

MUST REMEMBER

Fundamental-mode assumption

For fundamental-mode analysis, only one input changes at a time and the circuit is allowed to reach a stable state before the next input change. Do not apply synchronous edge-triggered assumptions.

GATE FAVOURITE

Stable-state analysis

Write the next-state equation, identify states satisfying $Q_{next}=Q$ for the current input, then check whether transitions under the permitted input changes lead to the intended stable state.

COMMON TRAP

Race vs hazard

A **race** concerns multiple state variables changing and potentially reaching different final states. A **hazard** is an unwanted transient caused by unequal propagation delays in a combinational path.

MUST REMEMBER

Critical race

A race is critical when different possible orders of state-variable changes can lead to different stable states. State assignment should avoid assigning adjacent transitions to codes requiring multiple simultaneous bit changes.

GATE FAVOURITE

Essential hazard

Occurs in asynchronous circuits because a feedback path has unequal delays. Unlike a static/dynamic logic hazard, it cannot generally be removed by merely adding a redundant combinational consensus term; the feedback path must be designed with appropriate delays/structure.

FORMULA / RULE

Static-1 hazard

For SOP logic, if adjacent 1-minterms are covered by separate product terms but not a common implicant, unequal delays can create a temporary 0. Add the consensus/overlap implicant when appropriate.

Timing-diagram workflow

METHOD

How to solve a flip-flop timing question

1. Identify the active clock edge/level.
2. Evaluate J/K, D or T immediately before the active edge.
3. Apply the characteristic equation: D → Q_{next}=D; T → Q_{next}=Q⊕T; JK → 00 hold, 01 reset, 10 set, 11 toggle.
4. Apply preset/clear first if asynchronous controls are asserted.
5. Only then draw Q and account for clock-to-Q delay if the question asks for waveform timing.

State assignment

SHORTCUT

Adjacent transitions

Prefer state codes that differ in one bit for transitions that occur frequently. This reduces race risk in asynchronous implementations and often simplifies excitation equations.

COMMON TRAP

Don't confuse minimisation with assignment

State minimisation merges equivalent states; state assignment maps the remaining states to binary codes. They are different design stages.

LFSR

MUST REMEMBER

Linear feedback

An n-bit maximal-length LFSR can generate at most 2^n-1 nonzero states when its feedback polynomial is primitive. The all-zero state is a lock-up state for a standard XOR-feedback LFSR.

FORMULA / RULE

State transition method

Write the current register, compute the feedback XOR from the specified taps, shift in the feedback bit, and repeat until a state repeats. The number of distinct nonzero states is the period.

Shannon expansion

FORMULA / RULE

Shannon expansion

For variable x: $F = x \cdot F|_{x=1} + x' \cdot F|_{x=0}$. It is the algebraic basis of MUX implementation: use x as the select and the two cofactors as the data inputs.

Representative worked problems

METHOD

Q1 · LFSR period

Problem: An n=4 maximal-length XOR LFSR is initialized to any nonzero state. What is its maximum period?

Method: A primitive 4-bit LFSR visits every nonzero state exactly once.

$2^4-1 = 15$. Maximum period = **15 clocks**. The zero state must be avoided.

METHOD

Q2 · Shannon expansion / MUX

Problem: Express $F(A,B,C)=\Sigma m(1,2,5,7)$ using A as the MUX select.

Method: Compute cofactors. For A=0, minterms 1,2 correspond to BC=01,10, giving $F_0=B \oplus C$. For A=1, minterms 5,7 correspond to BC=01,11, giving $F_1=C$. Therefore $F=A' \cdot (B \oplus C)+A \cdot C$. A 2:1 MUX uses select A, D0=B⊕C and D1=C.

METHOD

Q3 · Static-1 hazard recognition

Problem: A two-level SOP changes from one input combination where F=1 to an adjacent combination where F=1, but the two points are covered by different product terms with no common implicant. What failure should you suspect?

Method: Adjacent 1-cells should have an overlapping implicant for a hazard-free static-1 transition.

Suspect a **static-1 hazard**. Add the appropriate consensus/overlap term if it is logically redundant and permitted by the implementation.

METHOD

Q4 · Flip-flop timing

Problem: A positive-edge D flip-flop sees D=1 immediately before one rising edge and D=0 immediately before the next rising edge. Ignore asynchronous inputs. What are the sampled Q values?

Method: Sample D only at the active edge; between edges Q retains its stored value.

After the first rising edge Q becomes **1**; after the second rising edge Q becomes **0**. Changes in D between those edges do not immediately change Q.

11 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- For n variables, a complete truth table has 2^n rows.
- For a k-bit binary counter, the number of states is 2^k.
- **Shannon:** F = x·F|x=1 + x'·F|x=0; the two cofactors can directly feed a 2:1 MUX.

LAST-MINUTE · MUST REMEMBER

- **De Morgan:** (AB)' = A'+B' and (A+B)' = A'B'.
- Use don't-care terms when they are permitted to simplify the implementation; they may be treated as 0 or 1 as convenient.
- MUX can implement Boolean functions; a 2^n:1 MUX can directly realise functions of n select variables plus suitable data inputs.
- Latch is level-sensitive; flip-flop is edge-triggered in the standard synchronous design model.
- Setup and hold time violations can cause metastability; clock-to-Q is a propagation characteristic, not a setup requirement.
- Synchronous counters share a common clock; ripple/asynchronous counters propagate state changes stage by stage.
- Combinational circuits have no memory; sequential circuits depend on present inputs and stored state.
- **Asynchronous circuits:** assume one input change at a time and allow the circuit to settle before the next change.
- A maximal-length n-bit XOR LFSR has period 2^n-1 and must avoid the all-zero lock-up state.

LAST-MINUTE · COMMON TRAPS

- K-map groups must contain 1,2,4,8,... cells and be rectangular; wrapping across edges is allowed.
- Race is a state-transition problem; hazard is a propagation-delay transient.

LAST-MINUTE · GATE FAVOURITES

- For FSMs, distinguish Moore output dependence on state from Mealy output dependence on state and input.

Computer Organization & Architecture

GATE CSE 2027 · complete formula, rule, method & trap sheet

98 reference boxes · **9** chapters ·

Number Systems & Computer Arithmetic | Instruction Set & Addressing Modes | Design of the Arithmetic Logic Unit (ALU) | Design of the Control Unit | Memory Hierarchy & Cache Mapping | I/O Interface: Interrupts & DMA | Instruction Pipelining & Hazards | Problem → Standard Method Playbook | Last-minute one-liners

1 Number Systems & Computer Arithmetic

Signed number representation Core

NOTES

A computer only has 0s and 1s. To represent negative numbers, we use one of three schemes on an **n-bit** word. GATE almost only cares about the third one.

REFERENCE TABLE

Scheme	How -5 (4-bit) looks	Problem
Sign-magnitude	`1101` (sign bit + magnitude 101)	two zeros (+0, -0), messy arithmetic
1's complement	`1010` (flip all bits of 5=0101)	still two zeros, needs "end-around carry"
2's complement	`1011` (flip bits, +1)	none — this is what real CPUs use

FORMULA / RULE

Rule of thumb

To get -x in 2's complement: flip every bit of x, then add 1. To go back from a negative 2's complement number to decimal: flip every bit, add 1, put a minus sign in front.

Range you must memorize

KEY POINT

For an **n-bit** signed 2's complement number:

```
Range = -2^(n-1)  to  +2^(n-1) - 1
```

So for n = 8: **-128 to +127**. Notice it's asymmetric — one extra negative number, because there's only one zero.

FORMULA / RULE

Worked numerical (GATE style)

Q: What is the range of integers representable in 12-bit 2's complement form?
A: n=12 → from -2^{11} to $+2^{11}-1$ = **−2048 to +2047**.

Addition, subtraction & overflow

NOTES

2's complement lets you **subtract by adding the negative**. Just add the bit patterns normally and discard any carry-out of the leftmost bit.

WORKED EXAMPLE

Worked numerical

Q: Using 5-bit 2's complement, compute 7 − 3.
A: 7 = `00111`, −3 = `11101` (flip 3=00011 → 11100, +1 → 11101).
Add: `00111 + 11101 = 100100` → discard the leading carry → `00100` = **4**. Correct!

COMMON TRAP

Overflow detection rule

Overflow happens only when adding two numbers of the **same sign** and the result has the **opposite sign**. Formally: overflow = carry into MSB XOR carry out of MSB. GATE loves asking "does this addition overflow?" — check the sign bits first, don't compute the whole thing if you don't have to.

WORKED EXAMPLE

Worked numerical

Q: In 4-bit 2's complement, does 5 + 4 overflow?
A: 5= `0101`, 4= `0100`. Sum = `1001` = −7 in 4-bit. Both operands positive, result negative → **overflow** (the true sum 9 is above the 4-bit signed maximum of +7, so it cannot be represented).

Floating point: IEEE 754 Core

NOTES

Fixed-point (2's complement) can't represent very large numbers or fractions efficiently. IEEE 754 splits a number into **sign, exponent, mantissa** — like scientific notation in binary.

Single precision (32-bit) layout

LAYOUT

31 S 30 E … E 23 M … M 0 M

NOTES

Sign — 1 bitExponent — 8 bitsMantissa — 23 bits

KEY POINT

Value = `(-1)^S × 1.M × 2^(E - 127)`
127 is the **bias** — it's added to the true exponent so the stored exponent is always non-negative. For double precision: 1 sign + 11 exponent (bias 1023) + 52 mantissa = 64 bits.

WORKED EXAMPLE

Worked numerical

Q: Represent 10.25 in IEEE-754 single precision.
A:
1. Binary: 10.25 = `1010.01`
2. Normalize: `1.01001 × 2³`
3. Sign = 0 (positive)
4. Exponent = 3 + 127 = 130 = `10000010`
5. Mantissa (drop leading 1, pad to 23 bits) = `01001000000000000000000`
Final: `0 10000010 01001000000000000000000`

COMMON TRAP

Special values to remember

E=0, M=0 → zero | E=0, M≠0 → denormal (subnormal) number | E=255 (all 1s), M=0 → ±infinity | E=255, M≠0 → NaN. GATE frequently asks "what does this bit pattern represent" — check E first.

Carry vs overflow, sign extension, and other codes High-yield

REFERENCE TABLE

Flag	Meaning	Test
Carry (C)	Unsigned result does not fit	Carry **out** of the MSB
Overflow (V)	Signed result does not fit	Carry **into** the MSB XOR carry **out of** the MSB

WORKED EXAMPLE

Worked example

Q: In 4 bits, add 1111 and 0001. Report carry and overflow.
A: 1111 + 0001 = 1 0000, so the result is 0000 with **carry = 1**. Unsigned, that is 15 + 1 = 16, which does not fit. Signed, it is (−1) + 1 = 0, which is correct: carry into the MSB = 1 and carry out = 1, so XOR = 0 and **overflow = 0**. The same bit pattern can set one flag and not the other.

KEY POINT

Sign extension (widening a 2's-complement number) copies the sign bit into the new high bits: 1101 (−3) in 8 bits is `11111101`, still −3. Unsigned numbers are **zero-extended** instead.

Other codes. BCD: each decimal digit in 4 bits (59 → 0101 1001). **Excess-3**: BCD digit + 3. **Gray code**: consecutive values differ in exactly one bit; for 3 bits it runs 000, 001, 011, 010, 110, 111, 101, 100, and Gray = binary XOR (binary >> 1).

2 Instruction Set & Addressing Modes

Instruction formats: how many "addresses"? Core

NOTES

An instruction = opcode (what to do) + operands (who to do it to). Older CPUs are classified by how many memory addresses an instruction can reference:

REFERENCE TABLE

Type	Example (for C = A + B)	Used in
3-address	`ADD C, A, B`	RISC-like / register machines
2-address	`MOV C, A` then `ADD C, B`	x86-style CISC
1-address (accumulator)	`LOAD A; ADD B; STORE C`	simple/old machines
0-address (stack)	`PUSH A; PUSH B; ADD; POP C`	stack machines, JVM

GATE FAVOURITE

GATE trap

Fewer addresses per instruction ⇒ more instructions needed to do the same job, but each instruction is shorter. GATE often asks you to count total instructions or total bits for a given expression across these formats.

Addressing modes — where is the actual operand? Core

NOTES

The instruction doesn't always contain the value directly. The **addressing mode** tells the CPU how to compute the **Effective Address (EA)** — where in memory/registers the real data sits.

REFERENCE TABLE

Mode	Instruction says	Effective Address / operand
Immediate	`ADD R1, #5`	operand IS 5 (no memory access)
Direct	`ADD R1, [200]`	EA = 200
Indirect	`ADD R1, [[200]]`	EA = contents of address 200 (memory holds a pointer)
Register	`ADD R1, R2`	operand is inside R2, no memory trip
Register indirect	`ADD R1, [R2]`	EA = value inside R2
Indexed	`ADD R1, [R2+X]`	EA = R2 + index register X (array access)
Base register	`ADD R1, [Base+disp]`	EA = base reg + displacement (relocatable code)
Relative	`BR [PC+disp]`	EA = PC + displacement (branches)
Auto-increment/decrement	`ADD R1, [R2]+`	EA = R2, then R2 = R2+size (array traversal)

FLOW

Indirect: Instruction → points to Address 200 → Mem[200] = Address 350 → Mem[350] = the real operand

WORKED EXAMPLE

Worked numerical

Q: An instruction uses indirect addressing with the address field = 300. Mem[300] = 500, Mem[500] = 42. What is the effective operand?
A: Indirect ⇒ EA = Mem[300] = 500. Operand = Mem[500] = **42**. (Indirect addressing costs one extra memory reference compared to direct — this is exactly why GATE loves timing questions: "if memory access = 100ns and indirect mode used, extra time = 100ns per instruction".)

WORKED EXAMPLE

Worked numerical

Q: Register R1 = 2000. Instruction: `LOAD R2, 20[R1]` using indexed addressing, and Mem[2020] = 77. Find R2 after execution.
A: EA = R1 + 20 = 2000 + 20 = 2020. R2 ← Mem[2020] = **77**.

RISC vs CISC (quick contrast, often 1 mark)

KEY POINT

RISC
Few, simple, fixed-length instructions. Mostly register-to-register. Simple addressing modes. More instructions needed, but each is fast (1 cycle typical) — pipelines well.

KEY POINT

CISC
Many complex, variable-length instructions. Instructions can directly operate on memory. Fewer instructions per program, but each may take several cycles.

Expanding opcodes Core

NOTES

A fixed-length instruction can hold more instruction types by **reusing unused opcode patterns as a prefix** for a longer opcode. Each time the opcode grows, the address fields shrink to make room.

WORKED EXAMPLE

Worked numerical

Q: A 16-bit instruction has 4-bit address fields. The machine needs 14 three-address instructions, 30 two-address instructions, and as many one-address instructions as possible. How many one-address instructions fit?
A:

```
Level 1 (3 addresses): 4-bit opcode, 3 × 4-bit addresses   →
16 patterns, use 14, 2 left
Level 2 (2 addresses): 8-bit opcode, 2 × 4-bit addresses   →
2 × 16 = 32 patterns, use 30, 2 left
Level 3 (1 address)  : 12-bit opcode, 1 × 4-bit address    →
2 × 16 = 32 patterns
```

So up to **32** one-address instructions. If only 31 were used, 1 pattern would remain: 1 × 16 = **16 zero-address** instructions with a 16-bit opcode.

Byte ordering (endianness) High-yield

NOTES

For the 32-bit value `0x12345678` stored at address 100: **big-endian** keeps the most significant byte first (100: 12, 101: 34, 102: 56, 103: 78); **little-endian** keeps the least significant byte first (100: 78, 101: 56, 102: 34, 103: 12). It matters when data is shared between machines or read byte by byte.

3 Design of the Arithmetic Logic Unit (ALU)

Adders: the building block Core

KEY POINT

Ripple Carry Adder (RCA)
n 1-bit full adders chained; carry "ripples" from LSB to MSB. Simple, but slow — the MSB must wait for every earlier carry.
Delay = $n \times \Delta$ (Δ = delay of 1 full adder).

KEY POINT

Carry Look-Ahead Adder (CLA)
Computes all carries in parallel using extra gates (generate/propagate logic), so it doesn't wait for ripples. Faster but more hardware.

WORKED EXAMPLE

Worked numerical (classic GATE pattern)
Q: A 16-bit adder is built from 4-bit carry-look-ahead adder (CLA) blocks connected by ripple carry between blocks. Each 4-bit CLA block has delay 10 ns to produce its sum and its carry-out. Find the total worst-case delay.
A: There are 16/4 = 4 blocks. Block 1 finishes in 10 ns and passes its carry-out to block 2, which then takes another 10 ns, and so on. Total delay = 4 × 10 = **40 ns**. (Contrast: if all 4 blocks used full look-ahead across blocks too, delay would be much closer to a single block's 10 ns.)

Multiplication: Booth's Algorithm Core

NOTES

Booth's algorithm multiplies signed 2's-complement numbers by scanning pairs of bits of the multiplier and deciding to **add**, **subtract**, or **do nothing** to a running partial product, then arithmetic-shift right.

KEY POINT

Rule, looking at multiplier bit Q_0 and the bit just shifted out Q_{-1}:

Q_0 Q_{-1}	Action
0 0	no operation, just shift
0 1	Add multiplicand, then shift
1 0	Subtract multiplicand, then shift
1 1	no operation, just shift

WORKED EXAMPLE

Worked numerical
Q: Multiply 3 × (−4) using Booth's algorithm (4-bit).
A: Multiplicand M = 3 = `0011`, Multiplier Q = −4 = `1100`, Q_{-1} = 0, AC = `0000`. Each step: check Q_0Q_{-1}, do the add/sub, then arithmetic-shift AC:Q:Q_{-1} right by 1.

```
Step  Q0Q-1  Action     AC after action   AC:Q:Q-1 after shift
1       0 0   none       0000              AC=0000 Q=0110 Q-1=0
2       0 0   none       0000              AC=0000 Q=0011 Q-1=0
3       1 0   AC=AC-M    1101              AC=1110 Q=1001 Q-1=1
4       1 1   none       1110              AC=1111 Q=0100 Q-1=1
```

Final product = AC:Q = `1111 0100` (8-bit 2's complement) = **−12** — matching 3 × (−4). ✔

COMMON TRAP

Why Booth's matters
It handles negative multipliers automatically and can skip strings of 0s or 1s, making multiplication faster than naive shift-and-add. GATE frequently gives you a partial trace and asks you to fill in the next AC or Q value — memorize the 4-row table above cold.

Division High-yield

NOTES

Two standard hardware algorithms, both repeat "shift, subtract divisor, check sign, restore if needed":

KEY POINT

Restoring division
Subtract divisor; if remainder goes negative, add divisor back (restore) and quotient bit = 0. Else quotient bit = 1. Simple but the extra "restore" step costs time.

KEY POINT

Non-restoring division
Never restores — instead alternates add/subtract based on the sign of the previous remainder. Same number of steps, no wasted restore cycle, so it's faster.

Carry look-ahead equations Core

NOTES

For each bit position define **generate** $G_i = A_i \cdot B_i$ (this bit makes a carry by itself) and **propagate** $P_i = A_i \oplus B_i$ (this bit passes an incoming carry on). Then $C_{i+1} = G_i + P_i \cdot C_i$ and $Sum_i = P_i \oplus C_i$. Expanding the recurrence removes the wait for ripples: $C_1 = G_0 + P_0C_0$, $C_2 = G_1 + P_1G_0 + P_1P_0C_0$, and so on. Each carry is a two-level AND-OR of G, P and C_0, so its delay does not grow with the bit position (but the gate fan-in does, which is why large adders use 4-bit blocks).

4 Design of the Control Unit

The instruction cycle in micro-operations Core

NOTES

Every instruction is really a short sequence of **register transfers**. GATE loves writing these out; you should be able to read/write them fluently.

KEY POINT

Example — the Fetch cycle (same for every instruction):

```
MAR ← PC
MBR ← Memory[MAR] , PC ← PC + 1
IR  ← MBR
```

Then Decode reads the opcode in IR, and Execute runs instruction-specific micro-ops, e.g. for `ADD R1, [500]`:

```
MAR ← 500
MBR ← Memory[MAR]
R1  ← R1 + MBR
```

Two ways to build the control unit Core

REFERENCE TABLE

	Hardwired	Microprogrammed
How signals are generated	Fixed combinational logic / FSM built from gates	A tiny stored "control program" (microprogram) in a Control Store (ROM)
Speed	Fast — pure logic delay	Slower — an extra memory read per micro-instruction
Flexibility	Hard to change — needs a re-design/re-wiring	Easy to change — just rewrite the control store contents
Best for	RISC, simple instruction sets, speed-critical CPUs	CISC, complex/large instruction sets
Cost / complexity	Complex to design, cheap to run	Simple, systematic design; needs control-store memory

Microinstruction format

NOTES

Each word in the control store is a **microinstruction**: a row of control-signal bits plus the address of the next microinstruction.

LAYOUT

Control field which signals to assert (e.g. MAR-load, ALU-add, Mem-read)
Address field address of next micro-instruction

KEY POINT

Horizontal microprogramming
One bit per control signal — wide word, no decoding needed, but uses lots of control-store space. Faster (parallel signals).

KEY POINT

Vertical microprogramming
Signals are encoded (like a mini opcode) — narrow word, needs a decoder, saves space but slightly slower.

WORKED EXAMPLE

Worked numerical

Q: A control store must hold 256 microinstructions, each needing to specify one of 8 possible micro-operations per field, using vertical microprogramming with 3 independent fields. How many bits per microinstruction (ignoring the next-address field)?
A: Each field encodes 8 choices → needs $\log_2 8 = 3$ bits. With 3 independent fields → $3 \times 3 = $ **9 bits** for control, plus $\log_2 256 = 8$ bits for the next-address field → total 17 bits/microinstruction.

GATE FAVOURITE

GATE angle to remember
Questions often give you total control store size and ask for the number of address bits ($\log_2$ of total microinstructions), or give you the number of control signals and ask you to compare horizontal (1 bit/signal) vs vertical (encoded, fewer bits) word width.

Data path: single-bus vs three-bus Core

NOTES

In a **single-bus** data path only one register can drive the bus at a time, so the ALU needs a holding register **Y** for one operand and an output register **Z** for the result. In a **three-bus** data path two buses feed the ALU and the third carries the result back, so an ALU operation takes one step.

WORKED EXAMPLE

Worked example — R3 ← R1 + R2

```
Single bus (3 control steps):
  1. R1out, Yin
  2. R2out, Add (ALU uses Y and the bus), Zin
  3. Zout, R3in
Three bus (1 step): R1 → bus A, R2 → bus B, ALU result → bus C → R3
```

The instruction fetch on a single bus also takes 3 steps: (1) PCout, MARin, Read, Select4, Add, Zin; (2) Zout, PCin, Yin, wait for memory; (3) MDRout, IRin. More buses mean fewer steps per instruction but more hardware.

5 Memory Hierarchy & Cache Mapping

The hierarchy Core

FLOW

Registers (fastest, smallest) → Cache (L1/L2/L3) → Main Memory (RAM) → Secondary storage (slowest, biggest)

NOTES

The idea: keep frequently-used data in the fast, small, expensive memory close to the CPU (locality of reference), and fall back to slower memory only on a miss.

Average Memory Access Time (AMAT) Core

KEY POINT

```
AMAT = Hit time + Miss rate × Miss penalty
```

Or in terms of hit ratio h: `Effective Access Time = h × Tc + (1 − h) × (Tc + Tm)` , where Tc = cache access time, Tm = main memory access time.

WORKED EXAMPLE

Worked numerical

Q: Cache access time = 10 ns, main memory access time = 100 ns, hit ratio = 0.9. Find effective access time.
A: EAT = 0.9×10 + 0.1×(10+100) = 9 + 11 = **20 ns**.

How an address maps into cache — 3 schemes Core

NOTES

Every physical address is split into three fields: **Tag | Set/Line index | Block offset**. The scheme decides how many places one memory block can go.

LAYOUT

Tag Set / Line index Block offset

REFERENCE TABLE

Mapping	Rule	Index field
Direct mapped	Each memory block → exactly ONE cache line	present — determines the single line
Fully associative	A block can go in ANY cache line	none — compare tag against all lines
Set-associative (n-way)	A block maps to ONE set, but can sit in any of the n lines in that set	present — picks the set, then search n lines

WORKED EXAMPLE

Worked numerical (very common GATE pattern)

Q: Main memory = 4 GB (32-bit address), cache = 64 KB, block size = 32 bytes, direct-mapped. Find tag, index, and offset bit widths.
A:
Offset bits = $\log_2$(block size) = $\log_2 32 =$ **5 bits**
Number of cache lines = cache size / block size = 65536/32 = 2048 lines → index bits = $\log_2 2048 =$ **11 bits**
Tag bits = total address bits − index − offset = 32 − 11 − 5 = **16 bits**

WORKED EXAMPLE

Worked numerical — 4-way set associative

Q: Same cache (64 KB, 32-byte blocks) but organized as 4-way set-associative. Find the index bits.
A: Number of sets = lines / ways = 2048 / 4 = 512 sets → index bits = $\log_2 512$ = **9 bits**. Tag bits become 32 − 9 − 5 = **18 bits** (bigger tag because fewer sets to search).

GATE FAVOURITE

Replacement & write policy — quick recall

When a set is full, a block must be evicted: **LRU** (least recently used) is the GATE favorite, also FIFO / random. Write policy: **write-through** (update memory immediately, simple, slower) vs **write-back** (update memory only on eviction, needs a "dirty bit", faster).

Write misses and cache miss types Core

REFERENCE TABLE

Policy pair	On a write hit	On a write miss
Write-through + **no-write-allocate**	Update cache and memory	Write memory only; the block is not brought into the cache
Write-back + **write-allocate**	Update the cache and set the dirty bit	Fetch the block into the cache, then write it there

NOTES

A write-back cache writes memory only when a **dirty** block is evicted, which saves bus traffic, but a replacement may then cost an extra memory write. **Miss types (the 3 Cs): compulsory** (first access to a block), **capacity** (the working set exceeds the cache) and **conflict** (too many blocks map to the same set; a fully associative cache has none).

Multi-level cache access time Core

NOTES

$AMAT = t_1 + m_1 \cdot (t_2 + m_2 \cdot t_mem)$ where m_i is the miss rate of level i (local miss rate) and t_i its access time.

WORKED EXAMPLE

Worked numerical

Q: L1: 1 ns, miss rate 10%. L2: 10 ns, local miss rate 20%. Memory: 100 ns. Find the average access time.

A: AMAT = 1 + 0.1 × (10 + 0.2 × 100) = 1 + 0.1 × 30 = **4 ns**.

Memory interleaving and memory types High-yield

NOTES

With **m-way low-order interleaving**, consecutive addresses fall in different banks (bank = address mod m), so m accesses can overlap. **Example:** 4 banks, 100 ns per bank access: fetching 8 consecutive words takes 2 × 100 = **200 ns** (two rounds of 4 parallel accesses), against 800 ns from a single bank, assuming no bank conflicts.

REFERENCE TABLE

	SRAM	DRAM
Cell	Flip-flop (about 6 transistors)	1 transistor + 1 capacitor
Refresh	Not needed	**Needed** (charge leaks)
Speed / cost / density	Faster, costlier, less dense	Slower, cheaper, denser
Typical use	Cache	Main memory

Memory interfacing — chips, decoding & expansion Core

1. A memory chip, as the CPU sees it

NOTES

A chip of `N × w` (N words, w bits per word) has **$\log_2$N address lines**, **w data lines**, a **chip select (CS)** and read/write controls (RD/WR or OE/WE). CS is usually **active-low**: with CS inactive the chip ignores the address and its data lines float (tri-state), so many chips can share one data bus.

REFERENCE TABLE

Chip	Address lines	Data lines	Capacity
1K × 8	10	8	1 KB
16K × 1	14	1	2 KB
4M × 8	22	8	4 MB

2. Address decoding & chip select

FLOW

CPU address A(k−1) … A(m) (high bits) → Decoder → CS of one chip / chip group

FLOW

CPU address A(m−1) … A0 (low bits) → Address pins of every chip in parallel

KEY POINT

Rule. Low bits (as many as the chip has address lines) go to **all** chips and pick the word *inside* a chip. High bits go to a **decoder** (or logic) that raises exactly one CS and picks *which* chip. The decoder is enabled only for the address bits that must have a fixed value.

COMMON TRAP

Trap — partial decoding

If some high address bits are **not** used in the decoder, they become don't-cares and the same chip answers at several address ranges (aliasing / "mirroring"). Full decoding uses every high bit and gives a unique address for every location.

3. Expansion — two independent directions

REFERENCE TABLE

Expansion	What grows	How the chips connect
Word-size (width)	Bits per word	Chips side by side: **same address, same CS**; data lines are concatenated (e.g. two 8-bit chips → 16-bit word)
Capacity (depth)	Number of words	Chips stacked: low bits to all, **high bits → decoder → separate CS**; data lines are shared on the bus

FORMULA / RULE

```
Chips needed = (N × W) / (n × w)          memory N × W built from chips n × w
Columns (word-size) = W / w               Rows (capacity) = N / n
Decoder inputs = log₂(Rows)               Rows = number of CS lines
```

4. Representative problems

NOTE

Problem 1 — expansion in both directions

Q: Build a **4K × 16** memory from **1K × 4** chips. How many chips, how are they arranged, and how are the address bits used?

A: Chips = (4096 × 16)/(1024 × 4) = **16**. Columns = 16/4 = 4 chips side by side give the 16-bit word; rows = 4096/1024 = 4 groups give the depth. The 12-bit address splits as: **A9–A0** (10 bits) to every chip; **A11–A10** (2 bits) to a **2-to-4 decoder** whose four outputs are the CS of the four rows.

FORMULA / RULE

Problem 2 — address ranges from a decoder

Q: A CPU has a 16-bit address bus. Four **2K × 8** chips are placed contiguously from 0000H, with A15–A13 required to be 000. Give each chip's range.

A: Each chip has 11 address lines (A10–A0) and covers 800H bytes. A15–A13 = 000 enables a 2-to-4 decoder on **A12, A11**:

```
Chip 0 (A12A11 = 00): 0000H – 07FFH
Chip 1 (A12A11 = 01): 0800H – 0FFFH
Chip 2 (A12A11 = 10): 1000H – 17FFH
Chip 3 (A12A11 = 11): 1800H – 1FFFH       total 8 KB
```

Because A15–A13 are decoded, the chips do not alias elsewhere in the 64 KB space. If A15–A13 were ignored, the same 8 KB would repeat 8 times (partial decoding).

NOTE

Problem 3 — decoder sizing

Q: A 16-bit address bus drives **64 KB** of memory made of **8K × 8** chips. How many chips and which decoder?

A: Chips = 64K/8K = **8** (all in one column, as the word is already 8 bits). Each chip needs 13 address lines (A12–A0); the remaining 3 bits **A15–A13** feed a **3-to-8 decoder** whose 8 outputs are the CS lines. Chip i occupies i × 2000H to i × 2000H + 1FFFH.

COMMON TRAP

Byte-addressable memory on a wider bus

On a byte-addressable machine with a 32-bit data bus, the memory is built as 4 byte-wide banks. The 2 lowest address bits (A1–A0) choose the byte within the word and are **not** sent to the chips; the chips see A(k−1)–A2. Read the question to see whether the "word" is the byte or the bus width.

Virtual memory and the TLB High-yield

NOTES

A virtual address splits into **page number | offset**. The page table maps page → frame. Page-table size = (number of virtual pages) × (entry size): a 32-bit virtual address with 4 KB pages has 2^{20} pages, so with 4-byte entries the table is 2^{20} × 4 B = **4 MB**. A **TLB** caches recent translations so most accesses skip the page-table lookup in memory. With a single-level page table:

WORKED EXAMPLE

Worked numerical

Q: TLB access 10 ns, memory access 100 ns, TLB hit ratio 90%. Find the effective access time.

A: A hit costs 10 + 100 = 110 ns. A miss costs 10 + 100 (page-table read) + 100 (data) = 210 ns. EMAT = 0.9 × 110 + 0.1 × 210 = 99 + 21 = **120 ns**.

Secondary storage — disk access time Extension

NOTES

Access time = seek time + rotational latency + transfer time. Average rotational latency is **half a rotation**; transfer time = (bytes wanted / bytes per track) × rotation time.

WORKED EXAMPLE

Worked numerical

Q: Seek time 4 ms, 7200 rpm, 400 sectors per track. Find the time to read one sector.

A: One rotation = 60/7200 s = 8.33 ms, so the average latency is 4.17 ms and one sector passes under the head in 8.33/400 = 0.021 ms. Total = 4 + 4.17 + 0.02 ≈ **8.19 ms**.

6 I/O Interface: Interrupts & DMA

KEY POINT

Programmed I/O

CPU repeatedly polls the device's status flag in a busy-wait loop. Simple, but wastes CPU cycles.

KEY POINT

Interrupt-driven I/O

CPU carries on with other work; the device raises an interrupt signal only when it's ready. CPU saves state, runs the ISR (interrupt service routine), then resumes. Saves CPU time vs polling, but each transfer still passes through the CPU register-by-register.

KEY POINT

DMA (Direct Memory Access)

A separate DMA controller moves an entire block directly between device and memory, without the CPU touching each word. CPU just sets up (starting address, count, direction) then is interrupted once at the very end. Best for bulk transfers (disk, network).

Cycle stealing

NOTES

While a DMA transfer is in progress, the DMA controller "steals" memory bus cycles from the CPU one at a time, so both can (almost) work concurrently — this is why DMA is much more efficient than interrupt-driven I/O for large transfers.

WORKED EXAMPLE

Worked numerical

Q: A device transfers data at 1 MB/s. Each interrupt-driven transfer moves 4 bytes and costs the CPU 100 cycles of overhead (context switch + ISR) at 1 GHz. Compare CPU overhead per second against a DMA scheme that interrupts only once per 4 KB block.

A:

Interrupt-driven: transfers/sec = 1,000,000/4 = 250,000 interrupts/sec → overhead = 250,000 × 100 cycles = 25,000,000 cycles/sec = **2.5% of a 1 GHz CPU** just on overhead.

DMA: interrupts/sec = 1,000,000/4096 ≈ 244 → overhead ≈ 24,400 cycles/sec — **roughly 1000× less CPU overhead.**

GATE FAVOURITE

GATE angle

Typical questions: given interrupt latency & instruction execution time, compute total time lost to interrupts; or compare polling vs interrupt vs DMA overhead like above. Also know: **vectored** interrupts (device ID tells CPU where its ISR is) vs **non-vectored** (fixed single ISR address, must poll to find the source).

Interrupt priority: daisy chain and nesting Core

NOTES

When several devices share one interrupt line, the CPU needs a priority scheme. **Daisy chain:** the interrupt-acknowledge signal passes serially from device to device, so the device **nearest the CPU has the highest priority**. **Parallel priority** uses a separate request line per device, a mask register and a priority encoder; it is faster but needs more hardware. With **nested interrupts** a higher-priority request can interrupt a running ISR while lower-priority ones stay masked. The CPU finishes the current instruction before it responds to an interrupt.

Memory-mapped vs isolated I/O Core

REFERENCE TABLE

	Memory-mapped I/O	Isolated (port-mapped) I/O
Address space	Device registers share the memory address space	Separate I/O address space
Instructions	Ordinary load/store	Special IN / OUT
Cost	Uses up part of the memory address range	Full memory range kept; needs an extra control signal

DMA transfer modes Core

REFERENCE TABLE

Mode	How it uses the bus	CPU effect
Burst (block)	Controller keeps the bus for the whole block	CPU is blocked from memory for the entire burst
Cycle stealing	Takes the bus for one word at a time	CPU slowed slightly
Transparent	Uses the bus only when the CPU is not using it	No slowdown, but the transfer may be slower

WORKED EXAMPLE

Worked numerical

Q: A device transfers 4 MB/s in 4-byte words using cycle stealing. Each stolen memory cycle lasts 100 ns. What fraction of the memory bus time does DMA use?

A: 4 MB/s ÷ 4 B = 1 million words per second. Each steals 100 ns: 10^6 × 100 ns = 0.1 s per second = **10%** of the bus time, so the CPU keeps the other 90%.

7 Instruction Pipelining & Hazards

The classic 5-stage pipeline Core

KEY POINT

`IF` Instruction Fetch → `ID` Instruction Decode → `EX` Execute → `MEM` Memory access → `WB` Write back

REFERENCE TABLE

Cycle →	1	2	3	4	5	6
Instr 1	IF	ID	EX	MEM	WB	
Instr 2		IF	ID	EX	MEM	WB

NOTES

Without pipelining each instruction takes 5 cycles start-to-finish; with pipelining, once the pipe is full, one instruction **completes every cycle**.

Speedup formula

KEY POINT

```
Time (non-pipelined) = n × k
Time (pipelined)     = k + (n − 1)
Speedup = (n × k) / (k + n − 1)
```

n = number of instructions, k = number of pipeline stages. As n→∞, speedup → k.

WORKED EXAMPLE

Worked numerical

Q: A 5-stage pipeline, clock cycle = 2 ns, runs 100 instructions with no stalls. Find total time and speedup vs non-pipelined execution.
A: Pipelined cycles = k + n − 1 = 5 + 99 = 104 → time = 104 × 2 = **208 ns**.
Non-pipelined time = n × k × cycle = 100 × 5 × 2 = 1000 ns.
Speedup = 1000 / 208 ≈ **4.8×** (close to the ideal max of 5).

Hazards — the reason pipelines stall Core

REFERENCE TABLE

Hazard	Cause	Fix
Structural	Two instructions need the same hardware resource (e.g. one memory port for both instruction fetch and data access) at once	Duplicate hardware (separate I-cache/D-cache), or stall
Data	An instruction needs a result that a previous, still-unfinished instruction hasn't produced yet	Forwarding/bypassing (feed result directly from EX/MEM stage), or stall
Control (branch)	The next instruction to fetch isn't known until a branch is resolved (usually in EX/MEM stage)	Branch prediction, delayed branch, or stall until resolved

WORKED EXAMPLE

Worked numerical — data hazard with stalls

Q: `ADD R1, R2, R3` is immediately followed by `SUB R4, R1, R5` in a 5-stage pipeline with no forwarding. R1 is written in WB (cycle 5 of instr 1) but needed by SUB's EX stage. How many stall (bubble) cycles are needed?
A: Without forwarding, SUB's ID stage (where operands are read) must happen no earlier than the cycle right after instr 1's WB. Instr 1: IF ID EX MEM WB (cycles 1-5). Instr 2 naturally would be ID at cycle 3 — but it needs the value only ready after cycle 5. So ID must be delayed to cycle 6 → **3 stall cycles** inserted. (With full forwarding from EX/MEM to the next instruction's EX, this typically drops to 0–1 stalls.)

GATE FAVOURITE

GATE angle

Questions typically give a sequence of dependent instructions and a pipeline diagram, then ask: total cycles with stalls, or how many bubbles a specific forwarding path removes, or the effect of a branch misprediction (stall cycles = number of stages before branch resolves).

Data hazard types: RAW, WAR, WAW Core

REFERENCE TABLE

Hazard	Meaning	Occurs in a simple in-order 5-stage pipeline?
RAW (true dependency)	A later instruction reads a register before an earlier one has written it	**Yes** — the main cause of stalls
WAR (anti-dependency)	A later instruction writes a register before an earlier one has read it	No — reads happen early, writes late
WAW (output dependency)	Two instructions write the same register; the writes could complete out of order	No — writes stay in order

NOTES

WAR and WAW appear with out-of-order execution or instructions of different latency, and are removed by **register renaming**. With **forwarding**, an ALU result flows from EX/MEM straight into the next instruction's EX stage, removing the stall for back-to-back ALU instructions. A **load followed by a use of the loaded value** still costs **1 stall cycle** even with forwarding, because the data is only available after MEM.

Branch penalty and prediction Core

NOTES

CPI = 1 + (branch frequency) × (fraction mispredicted or stalled) × (penalty in cycles). With 20% branches and a 3-cycle penalty, always stalling gives CPI = 1 + 0.2 × 3 = **1.6**; with a predictor that is wrong 40% of the time, CPI = 1 + 0.2 × 0.4 × 3 = **1.24**. Techniques: **static** prediction (always not-taken, or backward-taken/forward-not-taken), **delayed branch** (the instruction in the delay slot always executes), and **dynamic** prediction with a branch history table. A **2-bit saturating counter** needs two wrong predictions in a row to change its mind, so a loop branch mispredicts only once per loop exit.

Pipeline with unequal stages Core

NOTES

The clock must fit the **slowest stage plus the latch delay**. A non-pipelined machine takes the **sum** of the stage delays per instruction.

WORKED EXAMPLE

Worked numerical

Q: Four stages take 5, 10, 8 and 7 ns, and each pipeline latch adds 1 ns. Find the speedup for 100 instructions.
A: Clock = 10 + 1 = 11 ns. Pipelined time = (4 + 99) × 11 = 1133 ns. Non-pipelined time = (5 + 10 + 8 + 7) × 100 = 3000 ns. Speedup = 3000 / 1133 ≈ **2.65**, far below 4 because the slow stage sets the clock for everyone.

CPU performance: CPI, MIPS and Amdahl's law High-yield

NOTES

CPU time = instruction count × CPI × clock cycle time; **MIPS = clock rate / (CPI × 10^6)**. For a mix, CPI is the frequency-weighted average of the per-class cycle counts. **Amdahl's law:** if a fraction f of the time is sped up by a factor s, `Speedup = 1 / ((1 − f) + f/s)`, and the limit as s → ∞ is 1/(1 − f).

WORKED EXAMPLE

Worked numerical

Q: A 1 GHz CPU runs a mix of 50% ALU (1 cycle), 20% loads (2), 10% stores (2) and 20% branches (3). Find CPI, MIPS and the time for 10^9 instructions. Separately, if 80% of a program is sped up 4×, what is the overall speedup?
A: CPI = 0.5·1 + 0.2·2 + 0.1·2 + 0.2·3 = 0.5 + 0.4 + 0.2 + 0.6 = **1.7**. MIPS = 1000 / 1.7 ≈ **588**. Time = 10^9 × 1.7 × 1 ns = **1.7 s**. Amdahl: 1 / (0.2 + 0.8/4) = 1 / 0.4 = **2.5**, and even an infinite speedup of that part could give at most 1/0.2 = 5.

Flynn's taxonomy and instruction-level parallelism Extension

REFERENCE TABLE

Class	Instruction streams	Data streams	Example
SISD	1	1	Classic uniprocessor
SIMD	1	many	Vector/GPU-style units
MISD	many	1	Rare (fault-tolerant designs)
MIMD	many	many	Multicore, clusters

NOTES

A **superscalar** CPU issues several instructions per cycle (CPI can fall below 1); a **VLIW** machine has the compiler pack independent operations into one long instruction word.

8 Problem → Standard Method Playbook

METHOD

Instruction Encoding & Datapath

Problem pattern	Standard method	Key check
Instruction length / opcode fields	Count total encoding bits and required operand/register/address bits; remainder is opcode/function space.	Check whether the question uses fixed or variable/expanding opcode.
Expanding opcode	Allocate short opcodes first; each longer opcode consumes a prefix branch of the code space.	Never count all bit patterns independently when prefixes are constrained.
Single-bus datapath	Trace one transfer per bus cycle; identify source register → bus → destination and ALU control.	Two registers cannot normally drive the same single bus simultaneously.
Multi-bus datapath	Trace simultaneous source reads and destination write-back using separate buses.	More buses can reduce cycles but add hardware.
Endianness	Split a multi-byte value into bytes and place the least/most significant byte at the lowest address according to little/big endian.	Endianness changes byte order, not bit order within a byte.

METHOD

Cache & Memory

Pattern	Standard method	Formula / trap
Cache address split	Offset bits=$\log_2$(block size); number of lines/sets determines index bits; remaining bits are tag.	For S-way set associative cache, sets=lines/S.
AMAT	AMAT=hit time + miss rate×miss penalty.	For multi-level cache, expand recursively with conditional probabilities.
Write policy	Distinguish write-through/write-back and write-allocate/no-write-allocate.	Write-back updates memory on eviction; write-through updates memory on write.
Memory interleaving	Distribute consecutive words across banks; determine bank number from address modulo number of banks.	Check whether the question assumes low-order or high-order interleaving.
Secondary storage	Use block/sector geometry and seek/rotational/transfer components as specified.	Don't invent a seek component if the problem gives only transfer time.

METHOD

Pipeline & Performance

Pattern	Method	Trap
Pipeline speed up	Compare non-pipelined time with pipeline fill + steady-state cycles.	For n instructions and k stages, ideal cycles=n+k−1.
Hazard counting	Identify RAW/WAR/WAW and structural/control hazards; then apply forwarding/stalls/branch penalty stated in the question.	Forwarding does not remove every RAW hazard.
CPI/performance	CPU time = Instruction Count × CPI × Clock Cycle Time.	Weighted CPI if instruction classes have different CPIs.
Amdahl's law	Speedup=1/[(1−f)+f/s], where f is fraction improved and s is speedup of that fraction.	Speedup applies to execution-time fraction, not necessarily instruction fraction.

METHOD

I/O & Interrupts

Pattern	Standard method	Key distinction
Programmed I/O	CPU polls device status and transfers data.	CPU remains involved in waiting.
Interrupt-driven I/O	Device interrupts CPU when service is needed; CPU runs ISR.	Know interrupt latency and priority assumptions.
DMA	DMA controller transfers blocks between device and memory with limited CPU intervention.	Distinguish DMA setup time from CPU transfer time.
Priority / daisy chain	Apply the stated priority ordering to simultaneous requests; daisy chaining gives priority according to physical chain position.	Priority is an ordering, not a bandwidth calculation.
Memory-mapped vs isolated I/O	Check whether I/O registers share the memory address space or use a separate I/O address space.	This affects instruction/address-space assumptions.

METHOD

🧮 Representative GATE-style Worked Problems

Problem 1 · Expanding opcode

An ISA has 8-bit instructions. A 4-bit opcode identifies 8 instructions using a prefix scheme. How do you reason about additional opcode space?

Method: Draw the prefix tree: each unused prefix branch can be expanded only if it does not collide with an existing complete opcode.

Takeaway: Count available prefix leaves, not raw 2^n patterns.

Problem 2 · Datapath

A single-bus datapath must execute R3 ← R1+R2. What transfers are required?

Method: Place R1 on the bus → latch into ALU input A; place R2 on bus → ALU adds; place result on bus → load R3.

Takeaway: A single bus generally serializes source transfers.

Problem 3 · Amdahl

40% of execution time is improved by a factor of 4. Find overall speedup.

Method: $S=1/[(1-0.4)+0.4/4]$.

Answer: $1/0.7 = 10/7 \approx 1.43\times$.

Problem 4 · Cache

A cache has 64 lines and is 4-way set associative. How many sets?

Method: Sets=64/4.

Answer: 16 sets; therefore index bits=$\log_2 16=4$.

Problem 5 · Interleaving

Memory has 4 banks with low-order interleaving. Which bank receives word address A?

Method: bank=A mod 4.

Trap: If high-order interleaving is specified, the mapping is different.

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **Memory interfacing:** chips needed = (N×W)/(n×w). Word-size expansion = chips in parallel (same address and CS, data lines concatenated); capacity expansion = chips stacked (high address bits → decoder → CS).
- Effective address questions require careful attention to base/index/offset semantics and word vs byte addressing.
- Address bits = tag + set/index bits + block-offset bits; offset bits = $\log_2$(block size in bytes).
- Pipeline speedup is limited by stage imbalance, register overhead, stalls, and pipeline fill/drain effects.
- **Expanding opcode:** each level multiplies leftover patterns by $2^{\text{address bits freed}}$. Carry = carry out of the MSB (unsigned); overflow = carry into MSB XOR carry out (signed).
- Multi-level AMAT = t1 + m1·(t2 + m2·t_mem). TLB EMAT (single level) = h·(t_tlb + t_mem) + (1−h)·(t_tlb + 2·t_mem). Disk access = seek + half-rotation + transfer.
- Pipeline clock = slowest stage + latch delay. CPI = 1 + branch freq × mispredict rate × penalty. CPU time = IC × CPI × cycle time. Amdahl: 1/((1−f)+f/s).

LAST-MINUTE · MUST REMEMBER

- Low address bits go to every chip (one chip's worth: $\log_2 n$ bits); the remaining high bits feed a decoder whose outputs are the chip selects. CS is usually active-low.
- Two's complement represents signed integers; the most significant bit acts as the sign bit.
- Addressing modes determine how an instruction obtains its operand/effective address; distinguish immediate, direct, indirect, register, and indexed forms.
- **Cache mapping types:** direct-mapped, fully associative, and set-associative.
- DMA transfers blocks between I/O and memory with limited CPU involvement; interrupts notify the CPU of events.
- **Pipeline hazards:** structural, data, and control hazards.
- Know branch penalties, forwarding, stalling, and branch prediction at the level required for pipeline traces.
- CNF can be built as a conjunction of clauses; DNF as a disjunction of terms. Use truth-table rows when algebra becomes error-prone.
- **Cayley–Hamilton:** every square matrix satisfies its own characteristic polynomial; use it to reduce high powers of A.
- For a connected undirected graph with n−1 edges, the graph is a tree; every tree edge is a bridge.
- Convergence-test choice is pattern-driven: ratio/root for factorials and powers; comparison/integral for rational-like terms.
- Instruction encoding problems are bit-budget problems: total instruction bits = opcode/function bits + operand/address bits.
- For an S-way set-associative cache, number of sets = total cache lines / S.
- Single-bus datapaths serialize bus transfers; multi-bus datapaths can read multiple sources concurrently.
- Low-order interleaving maps word address A to bank A mod number_of_banks; high-order interleaving uses higher address bits.
- Memory-mapped I/O shares the memory address space; isolated I/O uses a separate I/O address space.
- Write-back + write-allocate vs write-through + no-write-allocate. Miss types: compulsory, capacity, conflict (none in a fully associative cache).
- **Daisy chain:** nearest device has highest priority. DMA modes: burst blocks the CPU, cycle stealing takes one word at a time, transparent uses idle bus cycles. Memory-mapped I/O uses ordinary load/store.
- Single-bus data path needs Y and Z registers (3 steps for R3 ← R1 + R2); three-bus does it in 1 step. Endianness: big-endian stores the most significant byte first.

LAST-MINUTE · COMMON TRAPS

- Partial address decoding leaves unused high bits as don't-cares, so one chip answers at several address ranges (aliasing). Full decoding gives each location a unique address.
- Overflow in signed fixed-width arithmetic is distinct from carry out of the most significant bit.
- AMAT depends on hit time and miss penalty; do not treat cache hit rate as the only performance factor.
- Zero covariance does not generally imply independence. For independence, factor the joint distribution into marginals.
- Endianness changes byte order across addresses; it does not reverse the bit order inside each byte.
- Forwarding does not eliminate every pipeline stall; a load-use RAW hazard may still require a bubble depending on the pipeline.
- Only RAW hazards stall a simple in-order pipeline; WAR and WAW need out-of-order execution. A load-use pair still costs 1 stall even with forwarding.

LAST-MINUTE · GATE FAVOURITES

- Data hazards include RAW, WAR, and WAW; in a simple in-order pipeline, RAW is the central true dependency hazard.
- For a finite set, equivalence relations correspond exactly to partitions of the set.
- **For binomial X~Bin(n,p):** E=np and Var=np(1−p). For Poisson(λ): E=Var=λ.
- Use complement for 'at least one', inclusion–exclusion for overlapping cases, and pigeonhole for guaranteed repetition.
- Expanding opcode questions are best solved with a prefix-tree/code-space view, not by blindly counting all bit patterns.
- Amdahl speedup = 1/[(1−f)+f/s], where f is the original execution-time fraction improved by factor s.
- DMA transfers blocks between an I/O device and memory with much less CPU involvement than programmed I/O.

Programming & Data Structures

GATE CSE 2027 · complete formula, rule, method & trap sheet

63 reference boxes · **9** chapters ·

C Programming Fundamentals | Pointers, Arrays, Strings & Memory | Functions & Recursion | Arrays, Stacks & Queues | Linked Lists | Trees, BST & AVL | Heaps, Hashing & Graph Representation | Problem → Standard Method Playbook | Last-minute one-liners

1 C Programming Fundamentals

COMMON TRAP

Scope of this assumption

These sizes are a convention for the examples and practice questions in this hub, **not a property of C**. The C standard only guarantees `sizeof(char) == 1`, minimum value ranges (`int` ≥ 16 bits, `long` ≥ 32 bits, `long long` ≥ 64 bits) and the ordering `short ≤ int ≤ long ≤ long long`. In a real GATE question, **use the machine assumptions the question gives** (e.g. "4-byte int", "8-byte pointers") and do not carry the numbers below into other problems.

Data types and sizes

REFERENCE TABLE

Type	Size assumed in this section	Notes
char	1 byte	sizeof(char) is always 1; may be signed or unsigned (implementation-defined)
short / int	2 / 4 bytes	int is the "natural" word size
long / long long	8 (Linux 64-bit) / 8 bytes	long is 4 bytes on some platforms — read the question
float / double	4 / 8 bytes	IEEE 754 single / double
pointer	8 bytes (64-bit)	size does not depend on the type pointed to

NOTES

Unsigned arithmetic wraps modulo 2^n. **Signed** overflow is **undefined behaviour**. In mixed signed/unsigned expressions the signed operand is converted to unsigned.

Operators, precedence and evaluation

RULES & FACTS

- **Integer division truncates toward zero** (C99): `7/2 = 3`, `-7/2 = -3`, `7/-2 = -3`. The remainder takes the sign of the dividend: `-7 % 2 = -1`. Always `(a/b)*b + a%b == a`.
- Precedence (high → low, simplified): `() [] -> .` → unary `! ~ ++ -- * & sizeof` → `* / %` → `+ -` → `<< >>` → `< <= > >=` → `== !=` → `&` → `^` → `|` → `&&` → `||` → `?:` → assignment → `,`.
- Relational operators are left-associative and yield 0 or 1, so `5 > 3 > 1` is `(5 > 3) > 1` = `1 > 1` = **0**.
- **Short-circuit**: in `a && b` b is evaluated only if a is true; in `a || b` only if a is false. So `if (i++ && j++)` with i = 0, j = 0 leaves **i = 1, j = 0**.
- Prefix `++i` increments then yields the new value; postfix `i++` yields the old value then increments.
- **Undefined behaviour**

  Modifying a variable twice between sequence points — `i = i++ + ++i`, `a[i] = i++`, `printf("%d %d", i++, i)` — is undefined or unspecified. GATE usually avoids it; if an option says "undefined/compiler dependent", it is often right.

WORKED EXAMPLE

Worked numerical — integer and unsigned semantics

```
unsigned char c = 200;
c = c + 100;                    // 300 mod 256 = 44
→ prints 44
printf("%d", -1 < 1u);          // -1 converted to unsigned
= 4294967295 → comparison false → 0
printf("%d", -7/2 + -7%2);      // -3 + (-1) = -4
```

Output of the three prints: **44, 0, −4** (verified by compiling and running).

WORKED EXAMPLE

Worked numerical — switch fall-through and loops

```
int k = 3;
switch (k) { case 1: printf("one ");  case 3: printf("three
");  case 4: printf("four "); break;  default: printf("def
"); }
// prints:  three four       (no break after case 3, so
execution falls into case 4)

for (i = 0; i < 5; i++) { if (i == 2) continue; if (i == 4)
break; s += i; }
// s = 0 + 1 + 3 = 4, and i ends at 4
```

Bitwise operations

KEY POINT

Handy identities

```
x & (x-1)   clears the lowest set bit
x & -x       isolates the lowest set bit
x << k       = x · 2^k     (no overflow)
x >> k       = ⌊x / 2^k⌋   (unsigned)
x ^ x = 0,   x ^ 0 = x
a ^= b; b ^= a; a ^= b;   swaps without temp
```

KEY POINT

Counting set bits (Kernighan)

```
while (x) { x &= x - 1; count++; }
// 0x2D = 101101  → 4 iterations
// 40 & 39 = 32  (101000 → 100000)
```

Storage classes and scope

REFERENCE TABLE

Class	Lifetime / scope
auto (default for locals)	created on entry to the block, destroyed on exit; stack; uninitialised = garbage
register	hint to keep in a register; you cannot take its address
static (local)	lives for the whole program, initialised **once** (to 0 by default), keeps its value between calls; scope is still the block
static (file level)	internal linkage — visible only inside that file
extern	declares a global defined elsewhere; globals default to 0

WORKED EXAMPLE

Worked numerical — static variable

```
int cnt() { static int n = 0; return ++n; }
cnt(); cnt(); printf("%d", cnt());      // static n persists
→ 3
```

The preprocessor

NOTES

Macros are **textual substitution** before compilation — no type checking, no evaluation of arguments. Always parenthesise arguments and the whole body.

WORKED EXAMPLE

Worked numerical — macro pitfalls

```
#define SQ(x) x*x
SQ(2+3)       →  2+3*2+3 = 11           (not 25)
100 / SQ(5)   →  100/5*5 = 20*5 = 100   (not 4)
#define SQ2(x) ((x)*(x))   →  SQ2(2+3) = 25   ✔
```

2 Pointers, Arrays, Strings & Memory

Pointer arithmetic

RULES & FACTS

- For `T *p`, `p + k` advances by **k · sizeof(T)** bytes. `p - q` (same array) gives the number of **elements** between them, not bytes.
- `a[i]` is defined as `*(a + i)`, so `i[a]` works too: for `int arr[] = {1,2,3,4}`, `2[arr] = 3`.
- The array name **decays** to a pointer to its first element in most expressions — but `sizeof(a)` gives the whole array (20 bytes for `int a[5]`), while for **a function parameter** `int a[]` is really `int *a`, so `sizeof(a)` there is 8.
- An array name is not an lvalue: `a = p;` is illegal, `a++` is illegal.

KEY POINT

Pre/postfix with pointers

```
*p++    value at p, then p moves  (postfix binds tighter than *)
(*p)++  increments the VALUE
*++p    moves p first, then reads
++*p    increments the value, yields new value
```

KEY POINT

Dereference and address

```
int x = 5, *p = &x, **pp = &p;
*p = 7;     // x is 7
**pp = 9;   // x is 9
*(&x) == x
```

WORKED EXAMPLE

Worked numerical — traversing with a pointer

```
int a[] = {10, 20, 30, 40, 50};  int *p = a;
int v1 = *p++;     // v1 = 10, p now points to a[1]
int v2 = *p;       // v2 = 20
int v3 = ++*p;     // a[1] becomes 21, v3 = 21
// p - a = 1, and a[1] is now 21         → v1, v2, v3 = 10, 20, 21
```

Two-dimensional arrays

NOTES

C stores 2-D arrays in **row-major order**. For `int m[R][C]`: `&m[i][j] = base + (i·C + j)·sizeof(int)`, and `m[i][j] == *(*(m + i) + j)`. Here `m + 1` advances a whole row (C elements), whereas `*m + 1` advances one element.

WORKED EXAMPLE

Worked numerical

`int m[3][4]`: `&m[2][1] − &m[0][0] = 2·4 + 1 = 9` elements. `sizeof(m) = 48`, and `sizeof(m)/sizeof(m[0]) = 3` rows.

Call by value and pointers

NOTES

C passes **everything by value**. A function receives a copy, so swapping two ints requires passing their addresses:

FORMULA / WORKING

```
void swapv(int a, int b) { int t = a; a = b; b = t; }      // swaps the copies only
void swapp(int *a, int *b) { int t = *a; *a = *b; *b = t; } // swaps the caller's variables
// u = 3, w = 4:  swapv(u, w) → still 3 4;   swapp(&u, &w) → now 4 3
```

Strings

RULES & FACTS

- A string is a `char` array ending with `'\0'`. `strlen` does not count the terminator, `sizeof` of an array does.
- `char s[] = "GATE";` → a writable array: **sizeof(s) = 5**, **strlen(s) = 4**. `char *t = "GATE";` → a pointer to a read-only literal: `sizeof(t) = 8`, and modifying `t[0]` is undefined behaviour.
- Comparing strings with `==` compares addresses; use `strcmp`. `strcpy` needs a destination of at least strlen+1 bytes.

Structures, unions and padding

NOTES

Each member is aligned to its own size; the whole struct is padded to a multiple of its **largest alignment**. Member order matters. A **union** shares one block of memory; its size is the largest member, rounded up to the alignment.

WORKED EXAMPLE

Worked numerical — sizeof with padding (verified with gcc)

```
struct A { char c; int i; char d; };        c(1) + pad(3) + i(4) + d(1) + pad(3)   = 12
struct B { int i; char c; char d; };        i(4) + c(1) + d(1) + pad(2)            = 8
struct C { char c; double d; int i; };      c(1) + pad(7) + d(8) + i(4) + pad(4)   = 24
union  U { int i; char c[6]; };             max(4, 6) = 6, rounded to alignment 4   = 8
```

Reordering members from largest to smallest alignment usually minimises padding.

Dynamic memory

REFERENCE TABLE

Function	Behaviour
malloc(n)	n uninitialised bytes from the **heap**; NULL on failure
calloc(k, n)	k·n bytes, **zero-initialised**
realloc(p, n)	resize; may move the block (old pointer becomes invalid)
free(p)	returns the block; `free(NULL)` is safe; freeing twice is undefined

RULES & FACTS

- **Memory leak**: allocated memory that is no longer reachable and never freed.
- **Dangling pointer**: pointer to memory that was freed, or to a local variable that has gone out of scope (returning `&local` from a function).
- **Wild/uninitialised pointer**: never assigned; dereferencing it is undefined. Set pointers to NULL after free.
- Memory layout: **code** (text), **static/global data**, **heap** (grows up), **stack** (grows down).

Function pointers

FORMULA / WORKING

```
int (*op)(int, int) = add;   // pointer to a function returning int taking two ints
int r = op(2, 3);            // calls add(2, 3);   used for callbacks, qsort's comparator
```

3 Functions & Recursion

How a call works

NOTES

Each call pushes an **activation record** (stack frame) holding the parameters, the return address, saved registers and local variables; return pops it. Recursion works because every invocation has its own frame. **Stack depth** = number of active (not yet returned) calls — infinite recursion ends in stack overflow.

RULES & FACTS

- Parameters are passed **by value** (copies). Locals are re-created on every call; **static** locals and globals are shared across all invocations.
- The order in which operands of `+`, function arguments, etc. are evaluated is **unspecified** in C — avoid questions that depend on it (for example, an expression that reads a global that a recursive call in the same expression modifies).
- **Tail recursion**: the recursive call is the very last action (`return f(n-1, acc*n);`); a compiler can reuse the frame, turning it into a loop with O(1) stack. `return n * f(n-1);` is **not** tail recursive (a multiplication follows the call).

Tracing technique

RULES & FACTS

1. Write the call tree. Mark what is printed **before** the recursive calls (pre-order) and **after** them (post-order).
2. Evaluate from the base case upward; keep local variables separate per call.
3. For counting problems write a recurrence for the number of calls and solve it.

WORKED EXAMPLE

Worked numerical — print between two recursive calls

```
void ruler(int n) { if (n == 0) return;  ruler(n-1);
printf("%d ", n);  ruler(n-1); }
ruler(3):   ruler(2) prints  1 2 1;  then 3;  then ruler(2)
prints 1 2 1
Output:  1 2 1 3 1 2 1

Calls: C(0) = 1,  C(n) = 2·C(n−1) + 1  →  C(n) = 2^(n+1) − 1  →
C(3) = 15 calls
Number of values printed = 2^n − 1 = 7
```

Classic recursive functions

REFERENCE TABLE

Function	Recurrence	Result
factorial	n! = n · (n−1)!	n calls deep → O(n) time, O(n) stack
gcd(a, b)	gcd(b, a mod b); gcd(a, 0) = a	gcd(48, 18) → gcd(18, 12) → gcd(12, 6) → gcd(6, 0) = **6**
Fibonacci (naive)	F(n) = F(n−1) + F(n−2)	exponential time Θ(φ^n); stack depth only n
Tower of Hanoi	H(n) = 2H(n−1) + 1	2^n − 1 moves: 3 disks → 7, 10 disks → **1023**
Binary search (recursive)	T(n) = T(n/2) + 1	O(log n) time and stack
McCarthy 91	f(n) = n−10 if n > 100, else f(f(n+11))	f(n) = 91 for every n ≤ 100: f(50) = f(99) = **91**

WORKED EXAMPLE

Worked numerical — number of calls in naive Fibonacci

Let C(n) = calls made by `fib(n)` : C(0) = C(1) = 1 and C(n) = C(n−1) + C(n−2) + 1.

```
C(2) = 3   C(3) = 5   C(4) = 9   C(5) = 15   C(6) = 25
In general C(n) = 2·F(n+1) − 1.  fib(5) returns 5 and makes
15 calls (counted by running the program).
```

Many of these calls recompute the same value — the reason dynamic programming and memoisation turn the time into O(n).

GATE FAVOURITE

GATE angle

Three things to check every time: (1) **what is printed before vs after** the recursive call; (2) **static / global** state shared across calls; (3) the **base case** — a missing or unreachable base case means infinite recursion. Also be ready to give the space complexity as the **maximum depth** of the recursion, not the number of calls.

4 Arrays, Stacks & Queues

Arrays

REFERENCE TABLE

Operation	Unsorted array	Sorted array
Access by index	O(1)	O(1)
Search	O(n)	O(log n) (binary search)
Insert / delete at a position	O(n) (shift elements); O(1) at the end	O(n)

NOTES

Address of element i of a 1-D array = `base + (i − lower)·w`; 2-D addressing is as in the pointer section. Sparse matrices are stored as triples (row, col, value) or in CSR form to save space.

Stack (LIFO)

NOTES

Operations **push, pop, peek, isEmpty** all take O(1). Uses: function calls, undo, expression evaluation, balanced-bracket checking, DFS, backtracking.

RULES & FACTS

- **Two stacks in one array** of size n: start one at index 0 growing up, the other at n−1 growing down; overflow only when they meet — the whole array is usable.
- **Stack permutations**: if 1, 2, …, n are pushed in order and popped at any time, the number of possible pop sequences is the Catalan number C(n) = (1/(n+1))·C(2n, n): n = 3 → **5** (only 3 1 2 is impossible), n = 4 → **14**.

Infix → postfix with a stack

NOTES

Scan left to right. Operands go straight to the output. For an operator, pop operators of **higher or equal precedence** (for left-associative operators; for the right-associative `^` only strictly higher) to the output, then push it. `(` is pushed; `)` pops until `(`. At the end pop everything.

WORKED EXAMPLE

Worked numerical — conversions

```
A + B * C - D / E        →  A B C * + D E / -          (max stack size 2)
A * (B + C) / D - E      →  A B C + * D / E -          (max stack size 3, counting the parenthesis)
(A + B) * (C - D)        →  A B + C D - *
```

Postfix evaluation

NOTES

Operands are pushed; an operator pops two values (right operand first), applies the operation and pushes the result. The stack depth needed is the maximum number of pending operands.

WORKED EXAMPLE

Worked numerical

`5 6 2 + * 12 4 / -`:

```
push 5, 6, 2     [5 6 2]
+                [5 8]
*                [40]
push 12, 4       [40 12 4]
/                [40 3]
-                [37]       → result 37   (maximum stack size 3)
```

Queue (FIFO)

NOTES

Operations **enqueue** at the rear, **dequeue** from the front — O(1) with a linked list or a circular array. Uses: BFS, scheduling, buffering, level-order traversal.

RULES & FACTS

- **Circular array queue** of size n (indices mod n), with `front` = index of the first element and `rear` = index of the next free slot, one slot kept empty:

```
empty : front == rear
full  : (rear + 1) % n == front        → capacity n − 1
count : (rear − front + n) % n
```

  Without the "wasted slot" trick, front == rear is ambiguous between empty and full, so a separate counter is needed.
- **Queue using two stacks**: push into stack 1; to dequeue, if stack 2 is empty move everything from stack 1 over (reversing order), then pop stack 2. Each element moves at most once → **amortised O(1)** per operation. A **stack using two queues** needs one operation to cost O(n).
- **Deque** allows insertion/deletion at both ends; a **priority queue** removes the highest-priority element (implemented with a heap).

WORKED EXAMPLE

Worked numerical

Circular queue with n = 8, front = 5, rear = 2: count = (2 − 5 + 8) mod 8 = **5** elements (at indices 5, 6, 7, 0, 1).

COMMON TRAP

Remember

BFS uses a **queue**; iterative DFS uses a **stack** (recursion uses the system call stack). Minimum number of stacks needed to simulate a queue = 2; minimum number of queues to simulate a stack = 2.

5 Linked Lists

Node structure and types

FORMULA / WORKING

```
struct node { int data;  struct node *next; };       // singly linked
struct dnode { int data; struct dnode *prev, *next; }; // doubly linked
```

RULES & FACTS

- **Singly**: traverse one way only. **Doubly**: both ways; can delete a given node in O(1). **Circular**: last node points back to the first (no NULL); the traversal condition is `p->next != head`.
- Use a **dummy (sentinel) head node** to avoid special cases for the first element.
- A list with n nodes holds n `next` pointers (singly) or 2n pointers (doubly), plus the head pointer.

Complexity

REFERENCE TABLE

Operation	Singly (head only)	Singly (head + tail)	Doubly
Insert at front	O(1)	O(1)	O(1)
Insert at end	O(n)	O(1)	O(1) with tail
Delete first	O(1)	O(1)	O(1)
Delete last	O(n) (need predecessor)	O(n)	O(1) with tail
Delete a node given a pointer to it	O(n) normally (O(1) trick below)	–	**O(1)**
Search / access k-th	O(n) — binary search is not possible on a linked list in O(log n)		

NOTES

O(1) deletion trick (node is not the last): copy the next node's data into the given node, then unlink and free the next node.

Essential algorithms

FORMULA / RULE

Reverse a list (iterative, O(n) time, O(1) space)

```
prev = NULL; cur = head;
while (cur) {
  nxt = cur->next;
  cur->next = prev;
  prev = cur;  cur = nxt;
}
head = prev;
```

Performs exactly n pointer reversals.

FORMULA / RULE

Middle / cycle (two pointers)

```
slow = fast = head;
while (fast && fast->next) {
  slow = slow->next;
  fast = fast->next->next;   // meets slow iff a cycle
exists
}
// no cycle: slow is the middle
```

Floyd's cycle detection: O(n) time, O(1) space.

RULES & FACTS

- **k-th node from the end**: advance one pointer k steps, then move both until the first hits NULL — one pass.
- **Merge two sorted lists**: O(m + n) time, O(1) extra space. **Merge sort** on a linked list: O(n log n) time (and it is the preferred sort for lists). Insertion sort works on a list in $O(n^2)$.
- Polynomial and large-integer arithmetic, adjacency lists and hash-chaining all rely on linked lists. An **XOR linked list** stores prev ⊕ next in one field to halve the pointer cost.

WORKED EXAMPLE

Worked numerical — recursion on a list

```
void fun(struct node *h) {
    if (!h) return;
    printf("%d ", h->data);
    if (h->next) fun(h->next->next);
    printf("%d ", h->data);
}
List: 1 → 2 → 3 → 4 → 5          fun(head) prints:  1 3 5 5
3 1
fun(1) prints 1, jumps to 3; fun(3) prints 3, jumps to 5;
fun(5) prints 5, next is NULL so no call;
then each frame prints its value again on the way back: 5,
3, 1.
```

WORKED EXAMPLE

Worked numerical — delete alternate nodes

Keep a node, unlink its successor, move to the node after: on 1→2→3→4→5 the loop leaves **1→3→5**; on 1→2→3→4→5→6 it also leaves **1→3→5**.

COMMON TRAP

Array vs linked list

Array: random access O(1), contiguous, cache-friendly, fixed or costly resize, insertion O(n). Linked list: insertion/deletion O(1) once the position is known, no resize cost, extra pointer memory per node, no random access.

6 Trees, BST & AVL

Binary tree facts

NOTES

Take the root at **level 0** and the height of a single node as 0 (height = number of edges on the longest root–leaf path). Be careful: some problems count levels from 1 — read the convention.

RULES & FACTS

- Maximum nodes at level i = 2^i. Maximum nodes in a tree of height h = $\mathbf{2^{h+1} - 1}$ (h = 3 → 15). Minimum height of n nodes = $\lfloor \log_2 n \rfloor$.
- A binary tree with n nodes has **n − 1** edges and **n + 1** NULL child pointers.
- In any binary tree: (leaves) = (nodes with 2 children) + 1.
- For a tree in which every internal node has exactly k children: with i internal nodes, n = k·i + 1 nodes and (k−1)·i + 1 leaves. With k = 3 and i = 10 → **21 leaves**. A full binary tree with i internal nodes has i + 1 leaves.
- Number of structurally different binary trees with n nodes = Catalan(n) = C(2n, n)/(n+1): 1, 2, 5, 14, 42 for n = 1…5. The same number of distinct BSTs can be built on n distinct keys: **4 keys → 14**.
- Complete tree in an array (0-based): children of i are 2i+1 and 2i+2, parent is $\lfloor (i-1)/2 \rfloor$ (1-based: 2i, 2i+1, $\lfloor i/2 \rfloor$).

Traversals

REFERENCE TABLE

Traversal	Order	Typical use
Inorder	Left, Node, Right	sorted output of a BST
Preorder	Node, Left, Right	copy a tree; prefix expression
Postorder	Left, Right, Node	delete a tree; postfix expression; evaluate expression trees
Level order	level by level (queue)	BFS, shortest depth

RULES & FACTS

- All recursive traversals take O(n) time and O(h) stack space. Morris traversal does inorder in O(1) space using threads.
- **Unique reconstruction**: inorder + preorder, or inorder + postorder, determine a binary tree uniquely. **Preorder + postorder do not** (unless every node has 0 or 2 children). The first preorder element (or last postorder element) is the root; split the inorder sequence around it.

Binary search tree (BST)

NOTES

Left subtree keys < node < right subtree keys. Search, insert and delete cost **O(h)**: O(log n) when balanced, **O(n)** when skewed (e.g. keys inserted in sorted order). The **inorder traversal of a BST is sorted**.

Deletion: (1) leaf → remove; (2) one child → replace by that child; (3) two children → replace the key by its **inorder successor** (minimum of the right subtree) or predecessor, then delete that node.

WORKED EXAMPLE

Worked numerical — build, traverse, delete

Insert `50, 30, 70, 20, 40, 60, 80, 35` into an empty BST.

```
        50
      /    \
    30      70
   /  \    /  \
  20  40  60  80
    /
   35
Inorder  : 20 30 35 40 50 60 70 80        (sorted)
Preorder : 50 30 20 40 35 70 60 80
Postorder: 20 35 40 30 60 80 70 50        height = 3

Delete 30 (two children): successor = min of right subtree
{40, 35} = 35 → 35 replaces 30.
   Preorder now: 50 35 20 40 70 60 80
Delete 50 (root, two children): successor = min of {70, 60,
80} = 60 → 60 becomes the root.
   Preorder now: 60 35 20 40 70 80
```

AVL tree (height-balanced)

NOTES

A BST in which the **balance factor** (height of left − height of right) of every node is −1, 0 or +1. After an insert or delete, rebalance with rotations at the lowest unbalanced node:

REFERENCE TABLE

Case	Cause	Fix
LL	insertion into left subtree of left child	one right rotation
RR	insertion into right subtree of right child	one left rotation
LR	insertion into right subtree of left child	left rotation on the child, then right rotation
RL	insertion into left subtree of right child	right rotation on the child, then left rotation

RULES & FACTS

- Height is always O(log n) ($\leq 1.44 \log_2 n$), so search/insert/delete are **O(log n)**. Insertion needs at most **one** (single or double) rotation; deletion may need O(log n) rotations.
- **Minimum nodes** of an AVL tree of height h: N(h) = N(h−1) + N(h−2) + 1 with N(0) = 1, N(1) = 2: **1, 2, 4, 7, 12, 20, 33, 54** for h = 0…7. So height 4 needs at least **12** nodes.

WORKED EXAMPLE

Worked numerical — AVL insertions

Insert `10, 20, 30, 40, 50, 25`:

```
10, 20, 30   → right-right imbalance at 10        → left
rotation          root 20 (10, 30)
40           → fits
50           → right-right imbalance at 30        → left
rotation          20 (10, 40(30, 50))
25           → under 30's left; imbalance at 20 is right-
left → double rotation
Final tree:      30
                /    \
              20      40
             /  \       \
           10   25       50          preorder 30 20 10 25 40
50, height 2
```

A plain (unbalanced) BST with the same insertions has height 4 and degenerates to a list-like shape.

7 Heaps, Hashing & Graph Representation

Binary heap

NOTES

A **complete binary tree** stored in an array. **Max-heap**: every parent ≥ its children (the maximum is at the root); **min-heap**: every parent ≤ its children. With 1-based indexing: left child 2i, right child 2i+1, parent ⌊i/2⌋. A heap with n nodes has height ⌊$\log_2$ n⌋ and **⌈n/2⌉ leaves** (the last ⌈n/2⌉ array positions).

REFERENCE TABLE

Operation	Time	How
Find max (min-heap: min)	O(1)	root
Insert	O(log n)	place at the end, **sift up** while larger than the parent
Delete max	O(log n)	move the last element to the root, **sift down** (swap with the larger child)
Build heap from n elements	**O(n)**	sift down every internal node from ⌊n/2⌋ down to 1 (inserting one by one costs O(n log n))
Heapsort	O(n log n)	build max-heap, then repeatedly swap the root to the end and sift down; in place, not stable
Search for an arbitrary key	O(n)	a heap is not ordered across subtrees
Find the minimum in a max-heap	O(n)	it is one of the leaves

WORKED EXAMPLE

Worked numerical — bottom-up build and one insertion

```
Build a max-heap from [4, 10, 3, 5, 1]   (n = 5, internal
nodes at positions 2 and 1)
 sift-down position 2 (10): children 5, 1 → fine
 sift-down position 1 (4):  swap with 10 → [10, 4, 3, 5, 1];
4 then swaps with 5 → [10, 5, 3, 4, 1]
 Result: [10, 5, 3, 4, 1]   (2 swaps)

Insert 14 into the max-heap [15, 10, 12, 8, 9, 4, 3]:
 append → [15, 10, 12, 8, 9, 4, 3, 14];  14 > parent 8 →
swap;  14 > parent 10 → swap;  14 < 15 stop
 Result: [15, 14, 12, 10, 9, 4, 3, 8]   (2 swaps)

Delete max from [20, 15, 12, 8, 9, 4, 3]: root ← last (3) →
sift down: 3 ↔ 15, then 3 ↔ 9 → [15, 9, 12, 8, 3, 4]
```

FORMULA / RULE

Worked numerical — inserting 1, 2, 3, 4, 5 one by one into a max-heap

Each new key is larger than everything before, so it climbs to the root: swaps = 0 + 1 + 1 + 2 + 2 = **6**, final array **[5, 4, 2, 1, 3]**. Building the same heap bottom-up from [1,2,3,4,5] needs fewer swaps — one reason build-heap is O(n) rather than O(n log n).

Hashing

NOTES

A hash function h(k) maps a key to a slot in a table of size m. With n keys the **load factor** is α = n / m. A **collision** happens when two keys hash to the same slot.

REFERENCE TABLE

Method	How collisions are handled	Notes
Chaining	each slot holds a linked list	α can exceed 1; average search ≈ 1 + α; worst O(n)
Linear probing	try h(k)+1, +2, … (mod m)	simple, cache friendly; suffers **primary clustering**
Quadratic probing	try h(k)+1², +2², …	avoids primary clustering; may not visit every slot
Double hashing	try $h_1(k) + i \cdot h_2(k)$	closest to uniform hashing; $h_2(k)$ must be non-zero and coprime to m

RULES & FACTS

- **Open addressing** stores everything in the table, so n ≤ m (α ≤ 1). Expected probes for an unsuccessful search ≈ 1/(1 − α). Deleting a key must leave a **tombstone** marker, otherwise later searches stop too early.
- Good hash functions spread keys uniformly; the **division method** h(k) = k mod m works best with m prime and not close to a power of 2.
- Average search/insert/delete is **O(1)**; the worst case is O(n). Hash tables do not support ordered operations (min, range queries) efficiently.

WORKED EXAMPLE

Worked numerical — collision resolution (m = 10, h(k) = k mod 10)

Insert `31, 12, 41, 22, 32, 13`.

```
Linear probing:    31→1   12→2   41→1✗2✗→3   22→2✗3✗→4
32→2✗3✗4✗→5   13→3✗4✗5✗→6
   table: [_, 31, 12, 41, 22, 32, 13, _, _, _]       probes:
1, 1, 3, 3, 4, 4  (total 16)
Quadratic (+i²):   table: [_, 31, 12, 22, 13, 41, 32, _, _,
_]      (41 lands at slot 1+4 = 5)
Chaining:          slot 1: 31 → 41     slot 2: 12 → 22 → 32
slot 3: 13
```

Note how linear probing builds one long cluster (slots 1–6) — new keys hashing anywhere into it must probe to its end.

WORKED EXAMPLE

Worked numerical — load factor

1000 keys in a chained table of 200 slots: α = 1000 / 200 = **5** — the average chain holds 5 keys.

Graph representations

REFERENCE TABLE

	Adjacency matrix	Adjacency list
Space	$O(V^2)$	O(V + E)
Is (u, v) an edge?	O(1)	O(deg u)
List all neighbours of u	O(V)	O(deg u)
BFS / DFS of the whole graph	$O(V^2)$	**O(V + E)**
Best for	dense graphs	sparse graphs

RULES & FACTS

- Undirected graph → symmetric matrix; the sum of all matrix entries = 2E. Matrix powers: entry (i, j) of A^k counts walks of length k.
- **BFS** uses a queue and visits by increasing distance; **DFS** uses a stack (or recursion).

WORKED EXAMPLE

Worked numerical — traversal order

Graph with edges 0–1, 0–2, 1–3, 1–4, 2–5, 4–5 (neighbours visited in increasing order), starting at 0:

```
BFS order: 0 1 2 3 4 5          DFS order: 0 1 3 4 5 2
Edges = 6; adjacency matrix has 36 cells; adjacency lists
hold 12 entries.
```

Complexity cheat sheet

REFERENCE TABLE

Structure	Search	Insert	Delete
Unsorted array	O(n)	O(1) at end	O(n)
Sorted array	O(log n)	O(n)	O(n)
Linked list	O(n)	O(1) at known position	O(1) at known position
BST (average / worst)	O(log n) / O(n)	O(log n) / O(n)	O(log n) / O(n)
AVL tree	O(log n)	O(log n)	O(log n)
Binary heap	O(n)	O(log n)	O(log n) (max / min)
Hash table (average / worst)	O(1) / O(n)	O(1) / O(n)	O(1) / O(n)

8 Problem → Standard Method Playbook

1. Pointer declarators: array of pointers vs pointer to array

METHOD

Method: start at the identifier and read the declaration outward, respecting parentheses. `int *p[5]` means an **array of 5 pointers to int**; `int (*p)[5]` means a **pointer to an array of 5 int**. For function pointers, parentheses around `*name` are essential.

WORKED EXAMPLE

Worked problem — declaration parsing

Q: What is `int (*p)[10]`?

METHOD NOTES

Answer: `p` is a pointer to an array of 10 integers. Thus `p+1` advances by `10*sizeof(int)` bytes, not by one integer.

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- For an array a with n elements, valid indices are 0 through n-1.
- A binary tree with height h has at most 2^{h+1}-1 nodes when root height is 0.

LAST-MINUTE · MUST REMEMBER

- In C, arrays are contiguous objects; pointer arithmetic is scaled by the pointed-to type.
- String literals are not modifiable; char arrays initialised from literals are distinct from pointers to string literals.
- Call-by-value passes a value; C does not have native call-by-reference, but pointers can be used to modify caller-visible objects.
- Recursion needs a correct base case and progress toward it; analyse recurrence/stack depth separately from total work.
- Stack follows LIFO; queue follows FIFO.
- A linked list provides dynamic node-based storage but loses direct O(1) random indexing.
- BST inorder traversal produces keys in sorted order when the BST invariant holds.
- AVL trees maintain a balance-factor constraint, keeping height O(log n).
- A binary heap is a complete binary tree satisfying heap order; build-heap can be done in O(n).
- Adjacency matrix uses $\Theta(V^2)$ space; adjacency lists use $\Theta(V+E)$ space for sparse graphs.
- Read C declarations from the identifier outward: int *p[5] is an array of pointers; int (*p)[5] is a pointer to an array.
- Integer promotions happen before the usual arithmetic conversions determine the common operand type.
- Morris inorder traversal temporarily threads predecessor links and uses O(1) auxiliary space.
- Bit-field layout is partly implementation-defined; follow the assumptions stated by the question.
- A heap is a complete tree with heap order; it is not a BST.

LAST-MINUTE · COMMON TRAPS

- Array names usually decay to pointers to the first element in expressions, but an array is not itself a pointer.
- **Heap is not a BST:** heap order does not imply globally sorted left/right subtrees.
- volatile does not provide atomicity or inter-thread synchronization.

Algorithms

GATE CSE 2027 · complete formula, rule, method & trap sheet

50 reference boxes · **9** chapters ·

Asymptotic Analysis & Recurrences | Searching & Sorting | Divide & Conquer | Greedy Algorithms | Dynamic Programming | Graph Algorithms | Amortized Analysis, Hashing & NP-Completeness | Problem → Standard Method Playbook | Last-minute one-liners

1 Asymptotic Analysis & Recurrences

Asymptotic notation

REFERENCE TABLE

Notation	Meaning	Read as
$f = O(g)$	$f(n) \le c \cdot g(n)$ for all $n \ge n_0$	g is an **upper** bound
$f = \Omega(g)$	$f(n) \ge c \cdot g(n)$ for all $n \ge n_0$	g is a **lower** bound
$f = \Theta(g)$	both $O(g)$ and $\Omega(g)$	**tight** bound (same growth rate)
$f = o(g)$ / $\omega(g)$	$f/g \to 0$ / $f/g \to \infty$	**strictly** smaller / larger

RULES & FACTS

- Constants and lower-order terms are dropped: $5n^2 + 3n \log n = \Theta(n^2)$. The base of a logarithm only changes a constant factor, so $\log_2 n = \Theta(\log_{10} n)$; but 2^n and 4^n are **not** Θ of each other (the base of an exponential matters).
- **Worst, average and best case** are different functions of n; O/Ω/Θ describe bounds on whichever function you chose. "Insertion sort is Ω(n)" (best case) and "$O(n^2)$" (worst case) are both true.
- Transitivity holds for O, Ω, Θ; Θ is symmetric; $f = O(g) \Leftrightarrow g = \Omega(f)$.

Growth-rate ladder

FORMULA / WORKING

$$1 \;<\; \log\log n \;<\; \log n \;<\; (\log n)^k \;<\; n^{\varepsilon} \;<\; n \;<\; n \log n \;<\; n^2 \;<\; n^3 \;<\; 2^n \;<\; n! \;<\; n^n$$

RULES & FACTS

- Any polynomial beats any polylog; any exponential beats any polynomial.
- $\mathbf{n^{\log n} = 2^{(\log n)^2}}$ grows faster than every polynomial but slower than 2^n, because $(\log n)^2 = o(n)$.
- **$\log(n!) = \Theta(n \log n)$** (Stirling). Also $\sqrt{n}$ is much bigger than $\log n$; $n^{0.01}$ beats $(\log n)^{100}$ eventually.
- Compare functions by taking logs or by limits: $\lim f/g = 0 \Rightarrow f = o(g)$.

Analysing loops

WORKED EXAMPLE

Worked numerical — nested loop with a doubling outer index

```
for (i = 1; i <= n; i *= 2)
    for (j = 1; j <= i; j++)
        count++;
i takes the values 1, 2, 4, …, 2^k  with k = ⌊log₂ n⌋
count = 1 + 2 + 4 + … + 2^k = 2^(k+1) − 1  <  2n
n = 16 → 31       n = 100 → 1 + 2 + 4 + 8 + 16 + 32 + 64 =
127       →  Θ(n), not Θ(n log n)
```

RULES & FACTS

- `for (i = 1; i < n; i *= 2)` runs $\lceil \log_2 n \rceil$ times. `for (i = 1; i*i <= n; i++)` runs $\lfloor \sqrt{n} \rfloor$ times ($n = 100 \to 10$).
- Nested independent loops multiply: $n \times (\log n) \times \ldots$; when the inner bound depends on the outer index, **sum** the series instead.
- For `for (i = n; i > 1; i = √i)` the count is $\Theta(\log \log n)$.
- Space complexity counts extra memory beyond the input, including the **recursion stack depth**.

Solving recurrences

Master theorem for $T(n) = a \cdot T(n/b) + f(n)$, $a \ge 1$, $b > 1$

NOTES

Let $n^c = n^{\log_b a}$ (the work done at the leaves). Compare f(n) with it:

REFERENCE TABLE

Case	Condition	Result
1	$f(n) = O(n^{c-\varepsilon})$ — leaves dominate	$T(n) = \Theta(n^c)$
2	$f(n) = \Theta(n^c \log^k n)$, $k \ge 0$ — balanced	$T(n) = \Theta(n^c \log^{k+1} n)$
3	$f(n) = \Omega(n^{c+\varepsilon})$ and $a \cdot f(n/b) \le \delta \cdot f(n)$, $\delta < 1$ — root dominates	$T(n) = \Theta(f(n))$

REFERENCE TABLE

Recurrence	$n^{\log_b a}$	Case	Solution
$T(n) = 2T(n/2) + n$ *(merge sort)*	n	2	**$\Theta(n \log n)$**
$T(n) = 4T(n/2) + n$	n^2	1	$\Theta(n^2)$
$T(n) = 4T(n/2) + n^2$	n^2	2	**$\Theta(n^2 \log n)$**
$T(n) = 3T(n/2) + n^2$	$n^{1.58}$	3	$\Theta(n^2)$
$T(n) = T(n/2) + 1$ *(binary search)*	1	2	$\Theta(\log n)$
$T(n) = 2T(n/2) + 1$	n	1	$\Theta(n)$
$T(n) = 2T(n/2) + n \log n$	n	2 ($k = 1$)	**$\Theta(n \log^2 n)$**
$T(n) = 7T(n/2) + n^2$ *(Strassen)*	$n^{2.81}$	1	$\Theta(n^{\log_2 7})$

COMMON TRAP

When the Master theorem does not apply

a or b not constant, f(n) not polynomially related (e.g. n/log n), or subproblems of unequal size such as T(n/3) + T(2n/3). Use the recursion tree, substitution, or the Akra–Bazzi idea instead.

Other forms

REFERENCE TABLE

Recurrence	Solution	How
$T(n) = T(n-1) + n$	$\Theta(n^2)$	unroll: $n + (n-1) + \dots + 1$
$T(n) = T(n-1) + 1$	$\Theta(n)$	unroll
$T(n) = 2T(n-1) + 1$	$\Theta(2^n)$	$T(n) + 1$ doubles each step (Tower of Hanoi: $2^n - 1$)
$T(n) = T(n-1) + T(n-2) + 1$	$\Theta(\varphi^n)$	Fibonacci-type, exponential
$T(n) = T(\sqrt{n}) + 1$	$\Theta(\log \log n)$	$n = 2^{(2^k)}$ takes k steps
$T(n) = T(n/3) + T(2n/3) + n$	$\Theta(n \log n)$	recursion tree: every level costs $\le n$, depth $\log_{3/2} n$ (checked numerically)
$T(n) = T(n/5) + T(7n/10) + n$	$\Theta(n)$	sizes sum to $9n/10 < n$: geometric decrease (median of medians)

WORKED EXAMPLE

Worked numerical — a Case 2 recurrence with a log factor

Q: Solve $T(n) = 2T(n/2) + n \log n$.

A: $n^{\log_2 2} = n$ and $f(n) = n \cdot \log^1 n$, so $k = 1$ and $T(n) = \Theta(n \log^2 n)$. A numerical evaluation of the recurrence confirms that $T(n)/(n \log^2 n)$ settles near a constant (≈ 0.53–0.56 for $n = 2^{10} \dots 2^{18}$).

2 Searching & Sorting

Searching

RULES & FACTS

- **Linear search**: $\Theta(n)$ worst case, $(n+1)/2$ comparisons on average for a successful search.
- **Binary search** on a sorted array: at most $\lfloor \log_2 n \rfloor + 1$ comparisons ($n = 15 \to 4$, $n = 1000 \to 10$ because $2^{10} = 1024$). Recurrence $T(n) = T(n/2) + 1$. Not possible in $O(\log n)$ on a linked list.
- Interpolation search: $O(\log \log n)$ on uniformly distributed data, $O(n)$ worst case. Hash search: $O(1)$ average.

Comparison sorts at a glance

REFERENCE TABLE

Algorithm	Best	Average	Worst	Extra space	Stable?
Bubble	n (with early exit)	n^2	n^2	1	Yes
Selection	n^2	n^2	n^2	1	No
Insertion	**n**	n^2	n^2	1	Yes
Merge	n log n	n log n	n log n	O(n)	Yes
Quick	n log n	n log n	**n^2**	O(log n) stack (O(n) worst)	No
Heap	n log n	n log n	n log n	1	No

NOTES

A sort is **stable** if equal keys keep their original relative order, and **in place** if it uses O(1) extra memory. Selection sort makes at most $n-1$ swaps (the fewest of the simple sorts); insertion sort is best for nearly sorted data and small arrays; merge sort is the choice for linked lists and external sorting.

FORMULA / RULE

Useful exact counts

```
merge two sorted lists (m, n): m + n − 1 worst
merge sort, n = 2^k:  n log₂n − n + 1  worst
selection sort comparisons: n(n−1)/2
max AND min together:  ⌈3n/2⌉ − 2
second largest:       n + ⌈log₂n⌉ − 2
```

Lower bound and non-comparison sorts

RULES & FACTS

- Any comparison sort corresponds to a binary **decision tree** with at least $n!$ leaves, so its height is $\ge \lceil \log_2 n! \rceil = \Omega(n \log n)$. Sorting 5 items needs at least $\lceil \log_2 120 \rceil = \mathbf{7}$ comparisons in the worst case.
- **Counting sort**: keys are integers in $0 \dots k$; $\Theta(n + k)$ time and space, stable. **Radix sort**: d passes of a stable sort, $\Theta(d(n + k))$. **Bucket sort**: $\Theta(n)$ expected for uniform input. These beat $n \log n$ only because they do not compare keys.

WORKED EXAMPLE

Worked numerical — inversions and shifts

```
[2, 4, 1, 3, 5]  : inversions (2,1) (4,1) (4,3) = 3  → insertion sort makes 3 shifts
[3, 8, 2, 5, 1, 4, 7, 6] : 12 inversions → 12 shifts, and bubble sort makes 12 swaps
[5, 4, 3, 2, 1] : 10 inversions = 5·4/2 (the maximum), bubble sort needs 10 swaps
```

Counting operations

FORMULA / RULE

Inversions

A pair $i < j$ with $a[i] > a[j]$. Maximum $n(n-1)/2$ (reverse sorted). Insertion sort performs **exactly one shift per inversion**; bubble sort performs exactly one swap per inversion.

WORKED EXAMPLE

Worked numerical — quicksort worst case

With the first (or last) element as the pivot, an already sorted array splits into 0 and $n-1$ elements every time, so comparisons $= (n-1) + (n-2) + \dots + 1 = n(n-1)/2$: $n = 6 \to \mathbf{15}$, $n = 8 \to 28$ (counted by running the algorithm). The recurrence is $T(n) = T(n-1) + n$, i.e. $\Theta(n^2)$. **Randomised pivot** or **median-of-three** avoids this on sorted input; expected comparisons $\approx 2n \ln n \approx 1.39\, n \log_2 n$.

WORKED EXAMPLE

Worked numerical — merge sort comparisons (n = 8)

Worst case = 8·3 − 8 + 1 = **17** comparisons, best case = 12 ($n/2 \cdot \log_2 n$). Both values were confirmed by sorting all 8! = 40 320 permutations and recording the minimum and maximum comparison counts.

Selection problems

RULES & FACTS

- **Quickselect** (k-th smallest): partition around a pivot and recurse into one side — O(n) average, $O(n^2)$ worst.
- **Median of medians** guarantees a good pivot (groups of 5), giving O(n) worst case: T(n) = T(n/5) + T(7n/10) + O(n).
- Finding the maximum needs exactly n − 1 comparisons. Finding max and min together takes $\lceil 3n/2 \rceil - 2$ (compare in pairs first): n = 8 → 10, n = 100 → **148** (verified by simulation).

GATE FAVOURITE

GATE angle

Know which sort is stable, in place, adaptive (insertion), and which has an $O(n^2)$ worst case (quick, insertion, bubble, selection). Expect questions on **number of swaps/comparisons for a given array**, **effect of the pivot rule** and "which sorting algorithm will take the least time on an already sorted input" (insertion sort: Θ(n)).

3 Divide & Conquer

The pattern

RULES & FACTS

1. **Divide** the input into a smaller instances of size n/b.
2. **Conquer** by solving them recursively (base case for tiny inputs).
3. **Combine** the answers in f(n) time.

REFERENCE TABLE

Algorithm	Recurrence	Time
Binary search	T(n) = T(n/2) + 1	Θ(log n)
Merge sort	T(n) = 2T(n/2) + n	Θ(n log n)
Quicksort (balanced split)	T(n) = 2T(n/2) + n	Θ(n log n); worst $\Theta(n^2)$
Max & min of an array	T(n) = 2T(n/2) + 2	3n/2 − 2 comparisons (n a power of 2)
Maximum-subarray (D&C)	T(n) = 2T(n/2) + n	Θ(n log n) (Kadane's DP does Θ(n))
Closest pair of points	T(n) = 2T(n/2) + n	Θ(n log n)
Karatsuba multiplication	T(n) = 3T(n/2) + n	$\Theta(n^{\log_2 3}) \approx n^{1.585}$
Strassen matrix multiplication	T(n) = 7T(n/2) + n^2	$\Theta(n^{\log_2 7}) \approx n^{2.81}$ (naive: n^3)
Median of medians selection	T(n) = T(n/5) + T(7n/10) + n	Θ(n)
Fast exponentiation	T(n) = T(n/2) + 1	Θ(log n) multiplications

Worked ideas

WORKED EXAMPLE

Worked numerical — Strassen

Naive 2×2 block multiplication needs 8 recursive products, giving n^3. Strassen uses 7 products plus extra additions ($\Theta(n^2)$ work).

```
fully recursing an 8×8 matrix down to 1×1 scalars:
   naive:    8³ = 512 scalar multiplications
   Strassen: 7³ = 343 scalar multiplications          (4×4: 49 vs 64)
exponent log₂7 ≈ 2.807  vs  3
```

WORKED EXAMPLE

Worked numerical — fast exponentiation (square and multiply)

x^{13}: $13 = 1101_2$. Squarings needed = $\lfloor \log_2 13 \rfloor = 3$ and extra multiplications = (number of 1 bits) − 1 = 2, so **5** multiplications instead of 12. x^{15} needs 6, x^{16} needs only 4.

WORKED EXAMPLE

Worked numerical — max and min with n = 8

Recurrence T(2) = 1, T(n) = 2T(n/2) + 2 → T(n) = 3n/2 − 2 = **10** for n = 8. Compare this with the straightforward 2(n − 1) = 14 comparisons.

RULES & FACTS

- **Merge step of merge sort** uses O(n) extra space; sorting an array of n elements has recursion depth $\log_2 n$.
- **Counting inversions** in O(n log n): piggy-back on merge sort — when an element from the right half is merged before elements of the left half, it forms inversions with all remaining left elements.
- Efficiency of D&C comes from the combine cost: if the subproblems overlap (e.g. Fibonacci), plain D&C is exponential and **dynamic programming** is needed instead.

4 Greedy Algorithms

When greedy works

RULES & FACTS

- **Greedy-choice property**: some optimal solution contains the greedy first choice (proved by an **exchange argument**: swap any optimal solution's first choice for the greedy one without making it worse).
- **Optimal substructure**: after the greedy choice, the rest is a smaller instance of the same problem.
- Greedy algorithms are usually simple and fast (often O(n log n) because of a sort or a heap) but can be **wrong**: always look for a counter-example.

Classic problems

REFERENCE TABLE

Problem	Greedy rule	Time
Activity selection (maximum number of non-overlapping activities)	sort by **earliest finish time**; take each activity that starts after the last chosen one ends	O(n log n)
Fractional knapsack	sort by value/weight ratio; take whole items, then a fraction of the next	O(n log n)
Huffman coding	repeatedly merge the two least-frequent nodes	O(n log n) with a min-heap
Optimal merge pattern	always merge the two smallest files (same as Huffman)	O(n log n)
Job sequencing with deadlines	sort by profit; place each job in the latest free slot ≤ its deadline	$O(n^2)$ (O(n log n) with union-find)
Minimising total waiting time	shortest job first	O(n log n)
MST (Kruskal, Prim)	cheapest safe edge	see graph section
Dijkstra	closest unvisited vertex	see graph section

WORKED EXAMPLE

Worked numerical — activity selection

Activities (start, finish): (1,4) (3,5) (0,6) (5,7) (3,9) (5,9) (6,10) (8,11) (8,12) (2,14) (12,16). Sorted by finish time, the greedy picks **(1,4), (5,7), (8,11), (12,16)** — 4 activities, which is the maximum. Picking the shortest or the earliest-starting activity first is **not** always optimal.

WORKED EXAMPLE

Worked numerical — Huffman coding

Q: Frequencies (per 100 characters): a 45, b 13, c 12, d 16, e 9, f 5. Find the total encoded length.

A: Merge the two smallest repeatedly: f+e = 14; c+b = 25; 14+d = 30; 25+30 = 55; a+55 = 100.

```
code lengths:  a = 1,  b = 3,  c = 3,  d = 3,  e = 4,  f = 4
total bits = 45·1 + 13·3 + 12·3 + 16·3 + 9·4 + 5·4 = 45 + 39
+ 36 + 48 + 36 + 20 = 224
fixed-length 3-bit code: 300 bits.   (also = sum of all
internal-node weights: 14 + 25 + 30 + 55 + 100 = 224)
```

A Huffman tree on n symbols has n − 1 internal nodes; Huffman codes are **prefix-free**, and the two least frequent symbols end up as siblings at the deepest level.

WORKED EXAMPLE

Worked numerical — knapsack: fractional vs 0/1

Items (value, weight): (60, 10), (100, 20), (120, 30); capacity 50. Ratios: 6, 5, 4.

```
Fractional (greedy by ratio): all of item 1 (60) + all of
item 2 (100) + 20/30 of item 3 (80) = 240
0/1 greedy by ratio: items 1 and 2 → weight 30, value 160
(item 3 does not fit whole)
0/1 optimum (dynamic programming): items 2 and 3 → weight
50, value 220
```

So greedy is optimal for the fractional version but **fails for 0/1 knapsack**, which needs DP.

WORKED EXAMPLE

Worked numerical — coin change where greedy fails

Coins {1, 3, 4}, amount 6: greedy takes 4 + 1 + 1 = **3** coins, but 3 + 3 = **2** coins is optimal. Greedy is correct for canonical systems such as {1, 5, 10, 25} and for {1, 2, 5, 10}.

WORKED EXAMPLE

Worked numerical — job sequencing with deadlines

Jobs (profit, deadline): A (100, 2), B (19, 1), C (27, 2), D (25, 1), E (15, 3). Process in decreasing profit: A → slot 2; C → slot 1 (latest free ≤ 2); D and B need slot 1 (taken) → rejected; E → slot 3. Schedule C, A, E with profit 27 + 100 + 15 = **142**.

WORKED EXAMPLE

Worked numerical — optimal merge pattern

Merging files of sizes 20, 30, 10, 5, 30 pairwise with minimum total cost: 5+10 = 15; 15+20 = 35; 30+30 = 60; 35+60 = 95 → total cost 15 + 35 + 60 + 95 = **205**.

5 Dynamic Programming

Method

RULES & FACTS

1. Define the subproblem (the state) and what the table entry means.
2. Write the recurrence that builds a state from smaller ones, plus the base cases.
3. Fill the table in an order that respects dependencies (**tabulation**, bottom-up) or recurse with a cache (**memoisation**, top-down).
4. Time = (number of states) × (work per state); space = table size (often reducible to one or two rows).

NOTES

DP vs divide & conquer: D&C subproblems are independent; DP subproblems overlap. **DP vs greedy**: DP examines all choices (guaranteed optimal when the structure holds); greedy commits to one. Naive recursion on overlapping subproblems is exponential — Fibonacci with memoisation drops from $\Theta(\varphi^n)$ to $\Theta(n)$.

Standard problems

REFERENCE TABLE

Problem	Recurrence	Time / space
Longest common subsequence (strings of length m, n)	L[i][j] = L[i−1][j−1] + 1 if $x_i = y_j$, else max(L[i−1][j], L[i][j−1])	Θ(mn) time; (m+1)(n+1) table
Edit distance	D[i][j] = min(D[i−1][j] + 1, D[i][j−1] + 1, D[i−1][j−1] + [$x_i \neq y_j$])	Θ(mn)
0/1 knapsack (n items, capacity W)	K[i][w] = max(K[i−1][w], v_i + K[i−1][w − w_i])	Θ(nW) — *pseudo-polynomial*
Matrix-chain multiplication	M[i][j] = $\min_{i \le k < j}$ (M[i][k] + M[k+1][j] + $p_{i-1}p_kp_j$)	Θ(n^3) time, Θ(n^2) space
Longest increasing subsequence	L[i] = 1 + max L[j] over j < i with a[j] < a[i]	Θ(n^2); Θ(n log n) with binary search
Coin change (minimum coins)	C[x] = 1 + min C[x − c]	Θ(amount · #coins)
Coin change (number of ways)	for each coin, W[x] += W[x − c]	Θ(amount · #coins)
Subset sum / partition	reachable[s] |= reachable[s − a_i]	Θ(n · sum)
Floyd–Warshall (all-pairs shortest path)	$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$	Θ(V^3)
Rod cutting, optimal BST, Catalan numbers, longest palindromic subsequence	same pattern: states over intervals or prefixes	

WORKED EXAMPLE

Worked numerical — LCS table

Q: Find the length of the LCS of `ABCB` and `BDCAB`.

```
      ε  B  D  C  A  B
ε     0  0  0  0  0  0
A     0  0  0  0  1  1
B     0  1  1  1  1  2
C     0  1  1  2  2  2
B     0  1  1  2  2  3
```

The bottom-right entry is **3** (the subsequence B C B). Strings `AGGTAB` and `GXTXAYB` have an LCS of length **4** (G T A B); the CLRS strings `ABCBDAB` and `BDCABA` also give 4. A recursive solution without memoisation is exponential in the worst case, while the table needs Θ(mn).

WORKED EXAMPLE

Worked numerical — edit distance

`kitten → sitting`: substitute k→s, substitute e→i, insert g = **3** operations (`sunday → saturday` also takes 3: insert a, insert t, substitute n→r).

WORKED EXAMPLE

Worked numerical — 0/1 knapsack

Weights {1, 3, 4, 5}, values {1, 4, 5, 7}, capacity 7. The best choice is the items of weight 3 and 4 (value 4 + 5 = **9**). Taking weights 1 + 5 gives only 8, and 1 + 3 gives 5. Runtime Θ(nW) = 4 · 7 table cells.

WORKED EXAMPLE

Worked numerical — matrix-chain multiplication

```
A1 (10×30), A2 (30×5), A3 (5×60):
(A1A2)A3 : 10·30·5 + 10·5·60 = 1500 + 3000  = 4500
A1(A2A3) : 30·5·60 + 10·30·60 = 9000 + 18000 = 27000
Dimensions p = (40, 20, 30, 10, 30) for four matrices →
minimum = 26000 scalar multiplications
```

The number of ways to fully parenthesise n matrices is the Catalan number C(n−1); the table has about $n^2/2$ entries and each takes O(n).

WORKED EXAMPLE

Worked numerical — LIS and coin change

LIS of `[10, 22, 9, 33, 21, 50, 41, 60]` has length **5** (10, 22, 33, 50, 60). Minimum coins for 11 with {1, 2, 5} = **3** (5 + 5 + 1). Number of ways to make 5 with {1, 2, 5} = **4** (5; 2+2+1; 2+1+1+1; 1+1+1+1+1).

GATE FAVOURITE

GATE angle

Typical asks: the **time or space** of a DP, the **table size**, filling a few cells by hand, whether a problem has overlapping subproblems, and "which of these is solved by DP vs greedy". Remember that 0/1 knapsack and subset sum are **pseudo-polynomial** (polynomial in the numeric value W, not in its bit length), so they do not contradict NP-completeness.

6 Graph Algorithms

Traversal: BFS and DFS

RULES & FACTS

- **BFS** (queue): visits vertices in increasing distance (number of edges) from the source; gives shortest paths in **unweighted** graphs; Θ(V + E) with adjacency lists, Θ(V^2) with a matrix. A graph is **bipartite** iff BFS can 2-colour it (no odd cycle).
- **DFS** (stack / recursion): records discovery and finish times; Θ(V + E). Edge types in a directed graph: **tree, back, forward, cross**. A directed graph has a cycle **iff DFS finds a back edge**; an undirected graph has a cycle iff DFS meets a visited vertex that is not the parent.
- A DFS forest of an undirected graph with c components has V − c tree edges. **Articulation points** and **bridges** are found with DFS low-link values in Θ(V + E).

Topological sort and SCC

RULES & FACTS

- A topological ordering (every edge u → v has u before v) exists **iff the graph is a DAG**. Two methods, both Θ(V + E): (1) **Kahn** — repeatedly remove a vertex of in-degree 0; (2) DFS — output vertices in **decreasing finish time**. The ordering is usually **not unique**.
- **Strongly connected components**: Kosaraju (DFS on G, then DFS on the transpose in decreasing finish time) or Tarjan (one DFS with low-links), both Θ(V + E). Contracting each SCC gives a DAG.

WORKED EXAMPLE

Worked numerical — counting topological orderings

DAG on vertices 0–5 with edges 5→2, 5→0, 4→0, 4→1, 2→3, 3→1. Kahn's algorithm with the smallest-index rule gives **4 5 0 2 3 1**. Counting all linear extensions (by testing every permutation of the 6 vertices) gives **13** valid topological orders.

WORKED EXAMPLE

Worked numerical — SCCs

Directed graph: 0→1, 1→2, 2→0, 2→3, 3→4, 4→5, 5→3, 6→5, 6→7, 7→6. SCCs: **{0,1,2}**, **{3,4,5}**, **{6,7}** — three components, and the condensation DAG is {6,7} → {3,4,5} ← {0,1,2}.

Minimum spanning tree

RULES & FACTS

- An MST of a connected weighted graph has V − 1 edges and minimum total weight. If all edge weights are **distinct**, the MST is **unique**.
- **Cut property**: for any cut, the lightest crossing edge belongs to some MST. **Cycle property**: the heaviest edge on a cycle (if unique) belongs to no MST.
- **Kruskal**: sort edges, add the next edge if it joins two different components (union-find). Θ(E log E) = Θ(E log V).
- **Prim**: grow one tree from a start vertex, always adding the cheapest edge leaving it. Θ(E log V) with a binary heap; Θ(V²) with an adjacency matrix and no heap; Θ(E + V log V) with a Fibonacci heap.
- Adding the same constant to every edge, or squaring all positive weights, does not change which spanning tree is minimum (only the ordering of weights matters).

WORKED EXAMPLE

Worked numerical — one graph, three algorithms

Undirected graph on A–F with edges: A–B 4, A–C 2, B–C 1, B–D 5, C–D 8, C–E 10, D–E 2, D–F 6, E–F 3.

```
Kruskal (sorted: 1, 2, 2, 3, 4, 5, 5…):
  B–C (1) ✔   A–C (2) ✔   D–E (2) ✔   E–F (3) ✔   A–B (4) ✘ cycle   B–D (5) ✔ joins {A,B,C} with {D,E,F}
  MST weight = 1 + 2 + 2 + 3 + 5 = 13
Prim from A: A–C (2), C–B (1), B–D (5), D–E (2), E–F (3)  → also 13

Dijkstra from A:   A 0,  C 2,  B 3 (via C),  D 8 (via B),  E 10 (via D),  F 13 (via E)
Shortest path to F: A → C → B → D → E → F   with length 13
```

Single-source shortest paths

REFERENCE TABLE

Algorithm	Handles	Time
BFS	unweighted graphs	Θ(V + E)
Dijkstra	non-negative weights only	Θ((V + E) log V) with a binary heap; Θ(V²) with an array
Bellman–Ford	negative edges; **detects negative cycles**	Θ(VE)
DAG relaxation in topological order	negative edges, DAG only	Θ(V + E)
Floyd–Warshall	all pairs; negative edges but no negative cycle	Θ(V³), Θ(V²) space
Johnson's algorithm	all pairs on sparse graphs with negative edges	Θ(VE + V² log V)

RULES & FACTS

- Relaxation: if d[u] + w(u, v) < d[v], set d[v] = d[u] + w(u, v). Bellman–Ford relaxes every edge V − 1 times; a further improving relaxation means a negative cycle.
- **Dijkstra fails with negative edges** because a vertex's distance is finalised when it is extracted, and a later negative edge could still shorten it.

WORKED EXAMPLE

Worked numerical — Bellman–Ford with negative edges

Directed graph, source s: s→a 6, s→b 7, a→b 8, a→c 5, a→d −4, b→c −3, b→d 9, c→a −2, d→s 2, d→c 7.

```
after V − 1 = 4 passes:  d(s) = 0,  d(a) = 2,  d(b) = 7,  d(c) = 4,  d(d) = −2
shortest path s → d : s → b → c → a → d, with length 7 + (−3) + (−2) + (−4) = −2 ✔
One more pass improves nothing → no negative cycle
```

WORKED EXAMPLE

Worked numerical — Floyd–Warshall on 4 vertices

Initial weights: 1→2 = 3, 1→4 = 7, 2→1 = 8, 2→3 = 2, 3→1 = 5, 3→4 = 1, 4→1 = 2 (others ∞). After the three nested loops (V³ = 64 updates):

	1	2	3	4
1	0	3	5	6
2	5	0	2	3
3	3	6	0	1
4	2	5	7	0

For example d(1,4) improves from 7 to 6 via 1→2→3→4 (3 + 2 + 1).

GATE FAVOURITE

GATE angle

Know the **time complexity** of each algorithm (and its dependence on the data structure), **when it fails**, the order in which Dijkstra extracts vertices, and what the first few Kruskal/Prim steps pick. Floyd–Warshall detects a negative cycle by a negative value on the diagonal.

7 Amortized Analysis, Hashing & NP-Completeness

Amortized analysis

NOTES

The amortized cost of an operation is the **average over a worst-case sequence** — no probability involved. Three methods: **aggregate** (total ÷ n), **accounting** (charge extra credit to cheap operations), **potential** (a potential function Φ ≥ 0, amortized cost = actual + ΔΦ).

WORKED EXAMPLE

Worked numerical — dynamic array doubling

Push onto an array whose capacity doubles when full. Copies happen at sizes 1, 2, 4, 8, … For n = 16 pushes: 16 writes + (1 + 2 + 4 + 8) = 15 copies = **31** total, i.e. under 2 per push. In general the total is < 3n, so the amortized cost per push is **O(1)** even though a single push can cost Θ(n).

WORKED EXAMPLE

Worked numerical — binary counter

Incrementing a k-bit counter flips many bits occasionally. Total flips over n increments from 0 equals 2n − (number of 1s in n): n = 8 → 15, n = 16 → **31**, n = 100 → 197. So the amortized cost per increment is **< 2** flips = O(1).

RULES & FACTS

- **Union–find** with union by rank and path compression: m operations cost O(m·α(n)), where α is the inverse Ackermann function (≤ 4 for any practical n).
- Stack with **multipop**, queue from two stacks, and splay trees are standard amortized examples.

Hashing — analysis

RULES & FACTS

- With chaining and simple uniform hashing, load factor α = n/m: expected unsuccessful search Θ(1 + α), successful ≈ 1 + α/2. With α = O(1), operations are O(1) expected.
- Open addressing, uniform probing: expected probes ≈ 1/(1 − α) for an unsuccessful search and (1/α)·ln(1/(1 − α)) for a successful one.
- A **universal family** of hash functions makes the expected number of collisions small regardless of the input. The **birthday paradox**: collisions are likely once about √m keys are inserted. Cuckoo hashing gives worst-case O(1) lookups.

P, NP and NP-completeness

REFERENCE TABLE

Class	Definition
P	decision problems solvable in polynomial time by a deterministic algorithm
NP	problems whose "yes" answers can be **verified in polynomial time** given a certificate (equivalently, solvable by a non-deterministic polynomial-time machine). P ⊆ NP.
NP-hard	at least as hard as every problem in NP (every NP problem reduces to it); need not be in NP
NP-complete	in NP **and** NP-hard

RULES & FACTS

- **Polynomial-time reduction** $A \leq_p B$: an easy translation of instances of A into instances of B. If B is easy then A is easy; if A is hard then B is hard. To prove B NP-hard, reduce a **known NP-complete** problem **to** B (direction matters!).
- **Cook–Levin theorem**: SAT is NP-complete (the first one). If any NP-complete problem is in P, then **P = NP**; if any is proved to need super-polynomial time, then P ≠ NP.

REFERENCE TABLE

NP-complete	In P (polynomial time)
SAT, **3-SAT**, Clique, Independent Set, Vertex Cover, Hamiltonian cycle/path, Travelling Salesman (decision), Subset Sum, Partition, 3-colouring, 0/1 knapsack (decision), Set cover	**2-SAT**, shortest paths, MST, matching, maximum flow, 2-colouring (bipartiteness), Euler circuit, linear programming, primality testing (AKS), sorting, searching

WORKED EXAMPLE

Worked numerical — vertex cover and independent set are complements

Fact: S is a vertex cover of G iff V − S is an independent set. So min vertex cover + max independent set = |V|. Check on the graph with edges 0–1, 1–2, 2–3, 3–4, 4–0, 0–5 (a 5-cycle with a pendant vertex): minimum vertex cover = 3 ({0, 2, 3} works), maximum independent set = 3 ({1, 3, 5}), and 3 + 3 = 6 = |V| (checked by brute force over all subsets).

RULES & FACTS

- **Pseudo-polynomial** algorithms run in time polynomial in the numeric *value* of the input (knapsack Θ(nW)); NP-complete problems that have them are only "weakly" NP-complete.
- **Approximation**: vertex cover has a simple 2-approximation (take both endpoints of a maximal matching); metric TSP has a 2-approximation (double the MST) and Christofides' 1.5.
- Relationship of classes believed true: P ⊂ NP ⊂ PSPACE ⊂ EXPTIME (P ≠ NP is an open question). Co-NP contains the complements of NP problems; P ⊆ NP ∩ co-NP.

GATE FAVOURITE

GATE angle

Typical questions: "which of these is not known to be NP-complete / is in P" (2-SAT, shortest path, MST are in P; 3-SAT, Clique, Hamiltonian cycle, Subset Sum are NPC), "what follows if X is in P", and which direction a reduction must go. The decision version of a problem is the one that is NP-complete; the optimisation version is NP-hard.

8 Problem → Standard Method Playbook

METHOD

Heap Sort — canonical procedure

1. Build a **max-heap** for ascending order (bottom-up heapify, O(n)).
2. Swap the root with the last element of the active heap.
3. Reduce heap size by one and max-heapify the root.
4. Repeat until one element remains.

Time: **O(n log n)** in best/average/worst cases. Auxiliary array space: **O(1)** for in-place heap sort. It is not stable.

METHOD

Open addressing — double hashing

For table size m, the standard probe sequence is:

`h(k,i) = (h₁(k) + i·h₂(k)) mod m`, i = 0,1,2,…

- Choose $h_2(k)$ relatively prime to m so the sequence can visit the entire table.
- Do not confuse double hashing with linear probing: linear probing adds a fixed step of 1.
- Deletion normally uses a tombstone/deleted marker; simply clearing a slot can break future searches.

METHOD

BFS / DFS — recognition and invariants

- **BFS:** queue; explores by nondecreasing distance from the source. In an unweighted graph, first discovery gives a shortest-path distance.
- **DFS:** recursion/explicit stack; timestamps classify edges and support cycle detection, topological ordering, SCCs and articulation/bridge algorithms.
- For an undirected graph, DFS produces tree/back edges; a cross-edge classification from directed DFS should not be imported blindly.

METHOD

Articulation point — Tarjan low-link test

During DFS, maintain `disc[u]` and `low[u]`. For a DFS tree edge u→v:

- If u is not the root: **u is an articulation point iff some child v has low[v] ≥ disc[u]**.
- If u is the DFS root: it is an articulation point iff it has at least **2 DFS children**.
- For an undirected graph, a bridge is detected by `low[v] > disc[u]`.

METHOD

Greedy vs DP — selection test

Before choosing an algorithm, ask:

1. Does a locally optimal choice provably extend to a global optimum? → seek a **greedy-choice/exchange proof**.
2. Do many subproblems overlap and can an optimal solution be decomposed into optimal subsolutions? → use **DP**.
3. If neither property is apparent, do not force greedy or DP; consider graph search, divide-and-conquer, backtracking, or another formulation.

WORKED EXAMPLE

Worked problem — double hashing

Table size m=11, $h_1(k)$=k mod 11, $h_2(k)$=7−(k mod 7). Insert 22, 33, 44 using open addressing.

```
22: h₁=0, h₂=6 → slot 0.
33: h₁=0, h₂=2 → slot 0 occupied, then (0+2) mod 11 = 2.
44: h₁=0, h₂=5 → slot 0 occupied, then 5 is free.
Final occupied slots: 0→22, 2→33, 5→44.
```

WORKED EXAMPLE

Worked problem — articulation point

DFS tree: 1→2, 2→3, 3→4, with a back edge 4→2. Is 2 an articulation point?

```
disc[2] < disc[3].
The subtree rooted at 3 has a back edge to 2, so low[3] = disc[2].
Articulation condition for non-root 2 is low[3] ≥ disc[2].
Equality holds → vertex 2 is an articulation point (removing it separates 1 from the {3,4} part).
```

WORKED EXAMPLE

Worked problem — heap sort

Build a max-heap from [4, 1, 3, 2].

```
Start: [4,1,3,2].
Heapify index 1: 1 has child 2 → swap → [4,2,3,1].
Index 0 already has largest child 4.
Max-heap = [4,2,3,1].
Then repeatedly swap root with the last active element and heapify the reduced heap.
```

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **Master theorem:** $T(n)=aT(n/b)+f(n)$; compare $f(n)$ with $n^{\log_b a}$ before choosing the case.
- **Merge sort:** O(n log n) time and O(n) auxiliary space in the usual array implementation.
- Dijkstra is valid with non-negative edge weights; it is not a general shortest-path algorithm for negative edges.
- Union-Find with path compression + union by rank/size has near-constant amortized cost: $O(\alpha(n))$.

LAST-MINUTE · MUST REMEMBER

- For nested loops, derive the exact iteration count when possible; do not assume every nested loop is $O(n^2)$.
- Binary search requires a sorted search space; its standard comparison complexity is O(log n).
- Greedy algorithms require a proof of correctness, typically via greedy-choice and/or exchange arguments.
- DP is appropriate when subproblems overlap and the problem has optimal substructure.
- A DAG has a topological ordering iff it is acyclic; DFS finishing times can produce one.
- Hash-table performance depends on load factor and collision handling; worst-case search can still be O(n).
- **Heap sort:** build heap in O(n), then perform n−1 extract-max operations; O(n log n) time and O(1) auxiliary array space.
- BFS gives shortest paths in unweighted graphs; DFS supplies timestamps/low-link structure for cycle, SCC, bridge and articulation problems.

LAST-MINUTE · COMMON TRAPS

- Quicksort has $O(n^2)$ worst-case time; average/expected time is O(n log n) under standard randomisation assumptions.

LAST-MINUTE · GATE FAVOURITES

- Know the difference between MST algorithms, shortest-path algorithms, and traversal algorithms; they solve different problems.
- **Articulation:** non-root u is critical when some DFS child v has low[v] ≥ disc[u]; DFS root needs at least two children.

Operating Systems

GATE CSE 2027 · complete formula, rule, method & trap sheet

135 reference boxes · **9** chapters ·

System Calls, Processes & Threads · CPU Scheduling · Synchronization & IPC · Deadlocks · Memory Management & Paging · Virtual Memory · File Systems, Disk Scheduling & I/O · Problem → Standard Method Playbook · 🎯 Representative GATE-Style Worked Problems · Last-minute one-liners

1 System Calls, Processes & Threads

What is an OS? High-yield

NOTES

Think of the OS as the **manager of a shared house**: the hardware (CPU, RAM, disk) is the house, programs are the tenants, and the OS decides who gets the kitchen (CPU) and when, keeps tenants out of each other's rooms (memory protection), and gives everyone simple keys instead of raw wiring (abstraction: files instead of disk sectors).

KEY POINT

User mode

Normal programs run here. Cannot touch hardware or other programs' memory directly.

KEY POINT

Kernel mode

The OS core runs here with full power. A mode bit in the CPU tracks which mode you are in.

System calls Core

NOTES

A **system call** is how a user program politely asks the kernel to do something privileged (open a file, create a process, allocate memory). It works via a **trap**: a special instruction that flips the CPU into kernel mode and jumps to a fixed kernel routine.

FLOW

User program calls read() → TRAP → mode bit = kernel → Kernel runs the service → Return → mode bit = user

NOTES

Common ones: `fork` (create process), `exec` (replace process image), `wait`, `exit`, `open/read/write/close`.

GATE FAVOURITE

GATE trap

Privileged instructions (I/O, halt, changing the mode bit, memory-management registers) work only in kernel mode. Trying them in user mode raises a trap. Also: a **trap/system call is a software (synchronous) event**, an **interrupt is a hardware (asynchronous) event**.

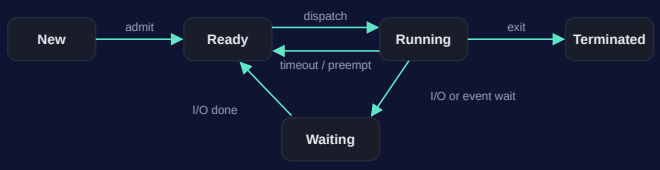
Process = a program in execution Core

NOTES

A program is a file sitting on disk. A **process** is that program running: it has its own address space with **text** (code), **data** (globals), **heap** (malloc) and **stack** (function calls), plus a record the OS keeps about it called the **PCB** (Process Control Block): PID, state, program counter, CPU registers, memory info, open files, scheduling info.

Process state diagram

DIAGRAM



Context switch

NOTES

When the CPU moves from process A to B, the OS **saves A's registers/PC into A's PCB and loads B's**. It's pure overhead (no user work happens), so the OS wants it cheap and rare.

fork() — the favourite GATE question Core

NOTES

`fork()` clones the calling process. Both continue from the next line. It returns **0 in the child**, the **child's PID in the parent**, and −1 on failure. Each fork **doubles** the number of processes.

WORKED EXAMPLE

Worked numerical

Q: How many processes exist (including the original) after `fork(); fork(); fork();` and how many times is `printf("hi")` printed if it comes after all three?

A: 1 → 2 → 4 → 8 processes, so total = $2^n = 2^3 =$ **8** (7 children). Every process runs the printf → **8 times**.

WORKED EXAMPLE

Worked numerical

Q: How many *child* processes are created by `if (fork() == 0) fork();` ?

A: Parent forks child C1 (in C1, fork returned 0). Only C1 enters the if-body and forks C2. The parent skips it. Total = 3 processes → **2 children** (C1, C2). Rule: **a fork inside a condition only doubles the processes that satisfy the condition**.

WORKED EXAMPLE

Worked numerical

Q: `for (i = 0; i < 4; i++) fork();` — how many processes in total?

A: Every process (including new ones) executes every remaining iteration, so it doubles each round: $2^4 =$ **16** total, **15** newly created.

Threads Core

NOTES

A **thread** is a lightweight unit of execution *inside* a process. Threads of one process **share** code, data, heap, open files; each has its **own** stack, registers and PC. Creating a thread and switching between threads is much cheaper than for processes.

REFERENCE TABLE

	User-level threads (ULT)	Kernel-level threads (KLT)
Managed by	Library in user space, kernel unaware	The kernel itself
Switch cost	Very low, no trap needed	Higher (needs kernel entry)
Blocking syscall	Blocks the **whole process**	Only that thread blocks
Multi-core parallelism	No (kernel sees 1 thread)	Yes

COMMON TRAP

Remember

Threads share the heap and globals, so they need synchronization (Part 2). Each thread keeps its **own stack and register set**, that is why local variables are not shared.

Types of schedulers & the dispatcher High-yield

REFERENCE TABLE

Scheduler	Job	How often it runs
Long-term (job scheduler)	Admits jobs from the job pool into memory; controls the **degree of multiprogramming** and the mix of CPU-bound and I/O-bound jobs	Rarely (seconds to minutes)
Short-term (CPU scheduler)	Picks which ready process gets the CPU next	Very often (every few ms) — must be fast
Medium-term (swapper)	Swaps processes out of and back into memory (the suspended states) to lower the degree of multiprogramming temporarily	Occasionally

MUST REMEMBER

Dispatcher = the module that actually hands the CPU to the process the short-term scheduler chose: it performs the **context switch**, switches to user mode and jumps to the right place in the user program. The time it takes is **dispatch latency**. Remember: the *scheduler decides*, the *dispatcher acts*.

Multithreading models Extension

REFERENCE TABLE

Model	Mapping	Consequences
Many-to-one	Many user threads → 1 kernel thread	Cheap, but one blocking system call blocks the whole process, and there is no multicore parallelism
One-to-one	Each user thread → its own kernel thread	True parallelism, no blocking of siblings; costs a kernel thread per user thread (Linux, Windows)
Many-to-many	n user threads multiplexed onto m ≤ n kernel threads	Flexible and parallel, but harder to implement (the two-level model also allows binding a user thread to one kernel thread)

Zombie and orphan processes High-yield

KEY POINT

Zombie (defunct)
The process has **terminated**, but its process-table entry (PID and exit status) stays because the parent has not yet called `wait()`. It uses no CPU or memory, only a table slot. Removed when the parent calls `wait()`, or when the parent dies and `init` reaps it.

KEY POINT

Orphan
The process is **still running** but its **parent has terminated first**. It is adopted by `init` (PID 1, or systemd), which later calls `wait()` for it, so orphans never become permanent zombies.

GATE FAVOURITE

GATE trap
Zombie = dead but not reaped. Orphan = alive but parentless. You cannot "kill" a zombie — it is already dead; the fix is to make its parent reap it.

Copy-on-write (COW) fork High-yield

KEY POINT

A naive `fork()` copies the whole address space, which is wasted if the child calls `exec()` immediately. With COW the child initially **shares the parent's physical frames**, all marked read-only. The first **write** to a shared page raises a protection fault; the OS then copies *only that page* and makes the copy private and writable. Pages that are never written are never copied.

WORKED EXAMPLE

Worked numerical
Q: A parent with 100 pages forks a child. The child writes to 10 pages and the parent writes to none. How many extra frames are used, with and without COW?
A: Without COW: 100 extra frames (full copy). With COW: only the 10 written pages are copied → **10 extra frames**, and the other 90 stay shared.

2 CPU Scheduling

Terms you must know Core

REFERENCE TABLE

Term	Meaning
Arrival Time (AT)	When the process enters the ready queue
Burst Time (BT)	CPU time it needs
Completion Time (CT)	When it finishes
Turnaround Time (TAT)	`CT − AT`
Waiting Time (WT)	`TAT − BT` (time spent waiting in ready queue)
Response Time (RT)	First time on CPU − AT

NOTES

Preemptive = the OS may take the CPU away from a running process (on timeout or when a better process arrives). **Non-preemptive** = process keeps the CPU until it finishes or blocks.

The algorithms Core

REFERENCE TABLE

Algorithm	Rule	Type	Watch out
FCFS	First come, first served	Non-preemptive	Convoy effect: one long job delays everyone
SJF	Shortest burst first	Non-preemptive	Minimum avg WT; long jobs may starve
SRTF	Shortest *remaining* time first	Preemptive SJF	Re-check at every arrival
Round Robin	Each gets a time quantum q, then goes to back of queue	Preemptive	Big q → FCFS, tiny q → all overhead
Priority	Highest priority first	Either	Starvation; fixed by **aging** (raise priority over time)

Worked numerical 1: FCFS vs SRTF Core

REFERENCE TABLE

Process	AT	BT
P1	0	8
P2	1	4
P3	2	9
P4	3	5

GANTT CHART

FCFS Gantt chart
P1 [0–8] · P2 [8–12] · P3 [12–21] · P4 [21–26]

FORMULA / WORKING

```
CT:  P1=8  P2=12  P3=21  P4=26
TAT = CT-AT: 8, 11, 19, 23   → avg 15.25
WT  = TAT-BT: 0, 7, 10, 18   → avg WT = 35/4 = 8.75
```

GANTT CHART

SRTF Gantt chart (P2 preempts P1 at t=1; at t=5 shortest remaining is P4=5, then P1=7, then P3=9)

P1 [0–1] · P2 [1–5] · P4 [5–10] · P1 [10–17] · P3 [17–26]

FORMULA / WORKING

```
CT:  P1=17  P2=5  P3=26  P4=10
TAT: 17, 4, 24, 7
WT : 9, 0, 15, 2   → avg WT = 26/4 = 6.5
```

NOTES

SRTF beat FCFS (6.5 vs 8.75). That is the whole point of favouring short jobs.

Worked numerical 2: Round Robin Core

NOTES

P1 BT=24, P2 BT=3, P3 BT=3, all arrive at 0, quantum q = 4.

GANTT CHART

P1 [0–4] · P2 [4–7] · P3 [7–10] · P1 [10–14] · P1 [14–18] · P1 [18–22] · P1 [22–26] · P1 [26–30]

FORMULA / WORKING

```
CT: P1=30  P2=7  P3=10
WT = CT - AT - BT:  P1=30-24=6,  P2=7-3=4,  P3=10-3=7
avg WT = 17/3 ≈ 5.67
```

GATE FAVOURITE

GATE convention

In Round Robin, if a new process arrives at the same instant a running process is preempted, the **new arrival goes into the ready queue first**, then the preempted process. Read the question: if it states a different rule, follow it.

Context-switch overhead High-yield

WORKED EXAMPLE

Worked numerical

Q: Round Robin with quantum q = 10 ms and context-switch time s = 2 ms. What fraction of time is useful work (assume every quantum is fully used)?
A: Each cycle = q + s = 12 ms, of which 10 ms is useful → efficiency = 10/12 = **83.3%** (overhead 16.7%). General: `efficiency = q / (q + s)`.

QUICK FACTS

Quick Facts for Exam

- Among non-preemptive algorithms, SJF gives the **minimum average waiting time**; SRTF is optimal among preemptive ones.
- FCFS has the convoy effect; RR gives the best (lowest) **response time** for interactive systems.
- I/O-bound processes use short CPU bursts; CPU-bound use long bursts. Good schedulers favour I/O-bound ones to keep devices busy.
- If all processes arrive at t=0, FCFS and non-preemptive SJF need no arrival-time juggling, just sort by order/burst.

Highest Response Ratio Next (HRRN) High-yield

KEY POINT

```
Response ratio   RR = (W + S) / S  =  1 + W/S
W = waiting time so far,   S = service (burst) time
```

Non-preemptive. At every scheduling point, run the ready process with the **highest RR**. For the same waiting time a short job has a higher RR (small S), while a long job's RR keeps rising the longer it waits — so HRRN favours short jobs **without starving long ones**. It needs burst-time estimates, like SJF.

REFERENCE TABLE

Algorithm	Starvation?	Weak point
FCFS	No	Convoy effect
SJF	Yes (long jobs)	Needs burst times; unfair to long jobs
HRRN	No	Needs burst times; RR must be recomputed at each decision

WORKED EXAMPLE

Worked numerical

Q: P1 (arrives 0, burst 3), P2 (1, 6), P3 (2, 2). Schedule with non-preemptive HRRN.
A:

```
t = 0 : only P1 is ready → runs 0–3.
t = 3 : P2 has W = 2, S = 6 → RR = (2+6)/6 = 1.33
        P3 has W = 1, S = 2 → RR = (1+2)/2 = 1.50   → P3 runs 3–5.
t = 5 : only P2 is left (RR = (4+6)/6 = 1.67) → runs 5–11.
Waiting times: P1 = 0, P3 = 1, P2 = 4 → average = 5/3 ≈ 1.67 (FCFS gives 3.0)
```

Multilevel queue vs multilevel feedback queue (MLFQ) High-yield

REFERENCE TABLE

	Multilevel queue	Multilevel feedback queue
Queue assignment	Fixed by process type (system, interactive, batch)	Dynamic — jobs **move between queues** by observed behaviour
Per-queue policy	Each queue may use its own algorithm (e.g. RR for interactive, FCFS for batch)	Same idea: top queues small quantum RR, bottom queue FCFS
Between queues	Fixed priority, or time slices shared among queues	Strict priority; lower queues run only when higher ones are empty
Starvation	Possible for low queues	Prevented by **aging** (promote jobs that wait too long)

KEY POINT

MLFQ rules: a new job enters the **top** queue (highest priority, smallest quantum). If it uses its **whole quantum** it is demoted one level (it looks CPU-bound); if it blocks for I/O before the quantum ends it stays high (it looks interactive). So MLFQ favours short and I/O-bound jobs **without knowing burst lengths in advance** — it approximates SRTF. Design parameters: number of queues, algorithm per queue, promotion rule, demotion rule, and the queue a new job enters.

WORKED EXAMPLE

Worked numerical

Q: Q0 = RR (quantum 4), Q1 = RR (quantum 8), Q2 = FCFS. P1 (burst 20), P2 (6) and P3 (3) all arrive at 0 in that order. Find the completion times.
A:

```
Q0 : P1 0–4 (16 left → Q1)   P2 4–8 (2 left → Q1)   P3 8–11 done at 11
Q1 : P1 11–19 (8 left → Q2)  P2 19–21  done at 21
Q2 : P1 21–29  done at 29
Completion: P3 = 11, P2 = 21, P1 = 29.   Waiting (= CT − BT): P1 9, P2 15, P3 8 → average 32/3 ≈ 10.67
```

The shortest job (P3) finishes first and the longest (P1) sinks to the bottom queue — SRTF-like behaviour with no advance knowledge.

3 Synchronization & IPC

Race condition & the critical section Core

NOTES

If two threads update shared data at the same time, the final answer depends on **who wins the race**. Even `x = x + 1` is really three steps: *load x → add 1 → store x*, and a context switch can land between them. The code touching shared data is the **critical section (CS)**.

FLOW

Entry section → Critical section → Exit section → Remainder

KEY POINT

A correct solution needs all three:

1. **Mutual exclusion**: only one process in the CS at a time.
2. **Progress**: if the CS is free, one of the waiting processes must be allowed in (no needless blocking by processes outside the CS).
3. **Bounded waiting**: a limit on how many times others can enter before a waiting process gets its turn (no starvation).

Peterson's solution (2 processes, pure software) Core

FORMULA / WORKING

```
// shared: flag[2] = {false,false};  int turn;
// process i (j = 1-i):
flag[i] = true;          // I want to enter
turn = j;                // but you go first if you also
want
while (flag[j] && turn == j);   // busy wait
   /* critical section */
flag[i] = false;         // exit
```

NOTES

It satisfies mutual exclusion, progress **and** bounded waiting. Real hardware uses atomic instructions like `test-and-set` / `compare-and-swap` instead. Note: a plain lock variable (`while(lock); lock=1;`) does **not** work, because checking and setting are two separate steps.

Semaphores Core

NOTES

A semaphore is an integer `S` touched only by two **atomic** operations:

FORMULA / WORKING

```
wait(S)   /* P */ :  S--;  if (S < 0) block this process
signal(S) /* V */ :  S++;  if (S <= 0) wake up one blocked
process
```

KEY POINT

Binary semaphore / mutex

Value 0 or 1. Used for mutual exclusion: init to 1, `wait` before the CS, `signal` after.

KEY POINT

Counting semaphore

Counts available units of a resource (e.g. 5 printers, or free buffer slots).

WORKED EXAMPLE

Worked numerical

Q: A counting semaphore starts at S = 3. After 5 wait() and 1 signal() operations, what is S and how many processes are blocked?
A: S = 3 − 5 + 1 = **−1**. A negative value means |S| processes are blocked → **1 process blocked**. (Rule: S ≥ 0 ⇒ S free units; S < 0 ⇒ |S| waiting processes.)

WORKED EXAMPLE

Worked numerical

Q: Semaphore S = 10 initially. 6 wait() and 4 signal() operations complete. What is S?
A: 10 − 6 + 4 = **8** (nobody blocked, since S never went below 0).

Classic problems Core

1. Producer–Consumer (bounded buffer of N slots)

FORMULA / WORKING

```
semaphore mutex = 1, empty = N, full = 0;

Producer:                Consumer:
 wait(empty);             wait(full);
 wait(mutex);             wait(mutex);
   add item                 remove item
 signal(mutex);           signal(mutex);
 signal(full);            signal(empty);
```

NOTES

`empty` counts free slots, `full` counts filled slots, `mutex` guards the buffer.

GATE FAVOURITE

GATE trap

Swap the order to `wait(mutex); wait(empty);` in the producer and you get a **deadlock**: a producer holding the mutex sleeps on a full buffer, and the consumer can never take the mutex to free a slot. The order of waits matters, the order of signals does not.

2. Readers–Writers

NOTES

Many readers may read together, but a writer needs exclusive access. The simple solution favours readers (writers can starve); a writer-priority version flips this. Uses a `read_count` protected by a mutex, and a `wrt` semaphore for writers.

3. Dining Philosophers

NOTES

5 philosophers, 5 forks, each needs both neighbouring forks. If everyone grabs their left fork at the same time, all wait forever → **deadlock**. Fixes: allow at most 4 to sit; pick up both forks atomically; or make one philosopher pick right-first (breaks circular wait).

NOTE

GATE classic: lost updates

Q: Two threads each run `for (i=0;i<N;i++) x = x + 1;` on a shared `x = 0`, with no synchronization and non-atomic load/add/store. What are the maximum and minimum possible final values of x (N ≥ 2)?
A: Max = 2N (no interleaving damage). **Min = 2**:
1. T1 loads x = 0, then is paused.
2. T2 runs N−1 full iterations → x = N−1.
3. T1 stores 1. T2 loads 1 for its last iteration and is paused.
4. T1 runs its remaining N−1 iterations → x = N.
5. T2 stores 1+1 = **2**. Final x = 2.

Hardware support: test-and-set, swap and compare-and-swap Core

NOTES

Pure-software locks (Peterson) are rarely used on real machines. The CPU instead gives **atomic read-modify-write** instructions that cannot be interrupted halfway:

FORMULA / WORKING

```
boolean TestAndSet(boolean *lock) {      // executes atomically
    boolean old = *lock;  *lock = true;  return old;
}
// shared: boolean lock = false;
do {
    while (TestAndSet(&lock)) ;        // spin until old value was false
        /* critical section */
    lock = false;
} while (true);

int CompareAndSwap(int *v, int expected, int new_value) {   // atomic
    int tmp = *v;
    if (tmp == expected) *v = new_value;
    return tmp;
}
```

REFERENCE TABLE

Property	Simple test-and-set lock
Mutual exclusion	**Yes** — only the process that saw `false` enters
Progress	**Yes**
Bounded waiting	**No** — a fast process can release and re-grab the lock every time, so another one can spin forever (starvation). A bounded-waiting version adds a `waiting[n]` array and hands the lock to the next waiting process in cyclic order
Cost	Busy waiting: the CPU burns cycles while spinning

MUST REMEMBER

Spinlock = a lock built on busy waiting. It wastes CPU on a single core (the lock holder cannot run while you spin), but on a **multiprocessor** it beats blocking when the critical section is shorter than a context switch. A **blocking mutex/semaphore** instead puts the waiter to sleep and wakes it on release. Rule of thumb: **short CS on many CPUs → spinlock; long CS → sleep.**

GATE FAVOURITE

GATE trap

Disabling interrupts also gives mutual exclusion, but only on a **single processor** and only for kernel code — on a multiprocessor the other CPUs keep running, so it does not work. And a plain `while(lock); lock=1;` fails because checking and setting are two separate steps; test-and-set fuses them into one atomic step.

Monitors & condition variables High-yield

NOTES

A **monitor** is a language-level construct: shared data plus the procedures that touch it, packaged so that **only one process can be active inside the monitor at a time**. The compiler/runtime supplies the mutual exclusion, so the programmer cannot forget a `wait(mutex)`. To wait for a *condition* (e.g. "buffer not empty") the monitor uses **condition variables** with two operations:

FORMULA / WORKING

```
x.wait()    // the caller releases the monitor lock and blocks on x
x.signal()  // wake ONE process waiting on x; if none is waiting, it does NOTHING

monitor BoundedBuffer {
    int count = 0;  condition notFull, notEmpty;
    void put(item) {
        while (count == N) notFull.wait();   // while, not if (Mesa-safe)
        add item; count++;
        notEmpty.signal();
    }
    item get() {
        while (count == 0) notEmpty.wait();
        remove item; count--;
        notFull.signal();
        return item;
    }
}
```

REFERENCE TABLE

Semaphore	Condition variable in a monitor
`signal(S)` is **remembered** (S is incremented even if nobody waits)	`x.signal()` is **lost** if no process is waiting on x
`wait(S)` blocks only if S would go negative	`x.wait()` **always** blocks
Programmer must pair every wait/signal correctly	Mutual exclusion is automatic; only the condition logic is manual

KEY POINT

Signal semantics. When P calls `x.signal()` and Q is waiting on x, both cannot run inside the monitor at once. **Hoare (signal-and-wait):** P hands the monitor to Q immediately and waits, so Q knows the condition is still true — an `if` test is enough. **Mesa (signal-and-continue):** P keeps running and Q only becomes ready; the condition may have changed before Q gets in, so Q must **re-check in a `while` loop**. Most real systems (Java, pthreads) use Mesa semantics, so always write `while (cond) x.wait();`.

Priority inversion & priority inheritance High-yield

NOTES

Priority inversion happens when a **high-priority** task is stuck waiting for a lock held by a **low-priority** task, and a **medium-priority** task that does not need the lock keeps preempting the low one — so the high-priority task is effectively delayed by the medium one.

FORMULA / WORKING

```
L (low) takes lock → H (high) arrives, blocks on the lock
→ M (medium) arrives and preempts L (M has higher priority than L)
→ L cannot run, so it cannot release the lock → H waits as long as M runs
```

KEY POINT

Fix — priority inheritance: while L holds a lock that H wants, L temporarily **inherits H's priority**. M can no longer preempt L, L finishes its critical section quickly, releases the lock, drops back to low priority, and H runs. (The 1997 Mars Pathfinder resets are the textbook real-world example.) A related fix is the **priority ceiling** protocol: a lock's ceiling is the highest priority of any task that may use it, and a task holding it runs at that ceiling.

WORKED EXAMPLE

Worked example

Q: Priorities H > M > L. L starts at t=0 and locks R; it needs 4 ms of CPU inside the lock in total. H arrives at t=2 and requests R (it blocks). M arrives at t=3 and needs 10 ms of CPU. When does H get R (a) without and (b) with priority inheritance?

A: By t=2 L has used 2 ms in the lock. **(a) Without inheritance:** L runs t=2→3 (1 ms more, 1 ms left). At t=3 M preempts L, since M outranks L, and runs t=3→13. L finishes its last 1 ms at t=13→14, so **H gets R at t=14** — held up by M's entire burst although H outranks M.

(b) With inheritance: from t=2 L runs at H's priority, so M cannot preempt it. L runs t=2→4, releases R, and **H gets R at t=4**.

Deadlock vs livelock vs starvation High-yield

REFERENCE TABLE

	Deadlock	Livelock	Starvation
State of processes	Blocked, waiting for each other	**Running** and changing state, but making no progress	Ready/waiting, but never chosen
Typical cause	Circular wait on resources	Both keep reacting to each other (two people sidestepping in a corridor; repeated retry/back-off in lock-step)	Unfair scheduling or lock policy (SJF, reader-priority)
Fix	Prevent, avoid, detect and recover	Randomised back-off, break the symmetry	Aging, fair queues

Inter-Process Communication (IPC) Core

REFERENCE TABLE

Method	Idea
Shared memory	Fastest: processes map the same region, but you must synchronize yourself
Message passing	send/receive through the kernel; easier and safer, slower
Pipes	One-way byte stream (parent-child); named pipes (FIFO) work between unrelated processes
Sockets	Two-way, works across machines

4 Deadlocks

The four necessary conditions (Coffman) Core

NOTES

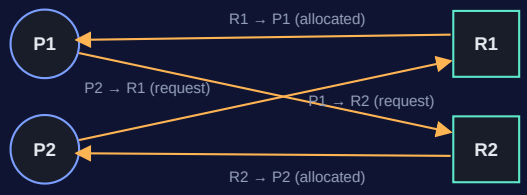
A deadlock can occur only if **all four** hold at once:

REFERENCE TABLE

Condition	Meaning	Break it by
Mutual exclusion	A resource is held by one process at a time	Make resources sharable (rarely possible)
Hold and wait	Holds some resources while waiting for more	Request everything at once
No preemption	Resources cannot be forcibly taken away	Allow the OS to preempt
Circular wait	P1 waits for P2 waits for ... waits for P1	Number resources, request in increasing order

Resource Allocation Graph (RAG) Core

DIAGRAM



NOTES

Edge **R → P** = allocated, **P → R** = requested. Here P1 holds R1 and wants R2, while P2 holds R2 and wants R1: a cycle.

COMMON TRAP

GATE rule

- Every resource has **a single instance**: cycle ⇔ deadlock.
- Resources have **multiple instances**: cycle is necessary but **not sufficient**. Check whether a free instance or a process outside the cycle can still finish.
- No cycle ⇒ no deadlock.

Handling deadlocks Core

KEY POINT

Prevention
Design so at least one Coffman condition can never hold.

KEY POINT

Avoidance
Grant a request only if the system stays in a **safe state** (Banker's algorithm).

KEY POINT

Detection & recovery
Let deadlock happen, detect via graph/algorithm, then kill processes or preempt resources.

KEY POINT

Ignore (ostrich)
What Linux/Windows mostly do: deadlocks are rare, so avoid the overhead.

NOTES

Safe state = there exists an order (safe sequence) in which every process can finish. Safe ⇒ no deadlock; unsafe ⇒ deadlock *possible* (not certain).

Banker's algorithm Core

NOTES

Keep `Need = Max − Allocation`. Repeatedly pick any process whose Need ≤ Available, let it finish, and add its Allocation back to Available. If all can finish, the state is safe.

WORKED EXAMPLE

Worked numerical

Q: 5 processes, resources A, B, C. Available = (3,3,2). Is the state safe?

Proc	Allocation	Max	Need = Max − Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

A: Work = Available (3,3,2).

```
P0 need 7 4 3 > 3 3 2  → cannot run yet
P1 need 1 2 2 ≤ 3 3 2  → run. Work = 3 3 2 + 2 0 0 = 5 3 2
P3 need 0 1 1 ≤ 5 3 2  → run. Work = 5 3 2 + 2 1 1 = 7 4 3
P4 need 4 3 1 ≤ 7 4 3  → run. Work = 7 4 3 + 0 0 2 = 7 4 5
P2 need 6 0 0 ≤ 7 4 5  → run. Work = 7 4 5 + 3 0 2 = 10 4 7
P0 need 7 4 3 ≤ 10 4 7 → run. Work = 10 5 7
```

Safe sequence: **<P1, P3, P4, P2, P0>** → the state is **safe**. (Other valid sequences may exist; GATE asks "is it safe?" or "which sequence is valid?".)

Minimum resources for deadlock freedom Core

FORMULA / RULE

```
n processes, each needs at most m units of one resource
type.
Deadlock-free if   R ≥ n(m − 1) + 1
```

Logic: worst case each process holds m−1 units and all wait for one more. One extra unit lets someone finish.

WORKED EXAMPLE

Worked numerical

Q: 3 processes each need up to 4 units of the same resource. Minimum units to guarantee no deadlock?
A: R = 3(4−1) + 1 = **10**. With 9 units, each could hold 3 and all be stuck.

WORKED EXAMPLE

Worked numerical

Q: 12 units of a resource, each process needs at most 3. Maximum number of processes that keeps the system deadlock-free?
A: Need 12 ≥ n(3−1)+1 = 2n+1 → n ≤ 5.5 → **5 processes**.

Wait-for graph High-yield

FORMULA / RULE

When every resource has a **single instance**, the resource-allocation graph can be collapsed into a **wait-for graph**: delete the resource nodes and draw **Pi → Pj** whenever Pi is waiting for a resource currently held by Pj. A **cycle in the wait-for graph ⇔ deadlock**. With multiple instances per resource, a cycle in the RAG is only *necessary* — use the detection algorithm instead (see Q12). The OS runs cycle detection periodically, or when CPU utilisation drops; a cycle search on n processes costs O(n²). (DBMS uses the same idea for transactions.)

WORKED EXAMPLE

Worked example

Q: P1 holds R1 and requests R2; P2 holds R2 and requests R1; P3 holds R3 and requests R1. Is there a deadlock?
A: Wait-for edges: P1 → P2 (R2 is held by P2), P2 → P1 (R1 is held by P1), P3 → P1 (R1 is held by P1).

```
P1 ⇄ P2   is a cycle → P1 and P2 are deadlocked.
P3 → P1   is only an edge INTO the cycle: P3 is blocked
behind the deadlock but is not on a cycle.
```

5 Memory Management & Paging

Logical vs physical addresses Core

NOTES

A program uses **logical (virtual) addresses**, as if it owns memory starting at 0. The hardware **MMU** translates them into **physical addresses** in real RAM. This gives protection (you cannot reach another process's memory) and lets the OS place programs anywhere.

Contiguous allocation & fragmentation Core

KEY POINT

Internal fragmentation
Wasted space *inside* an allocated block (fixed partitions, or the last page of a process).

KEY POINT

External fragmentation
Enough total free memory, but split into holes too small to use. Fixed by **compaction** or by paging.

REFERENCE TABLE

Strategy	Picks
First fit	First hole big enough (fast)
Best fit	Smallest hole big enough (leaves tiny leftovers)
Worst fit	Largest hole (leaves a big leftover)

WORKED EXAMPLE

Worked numerical

Q: Free holes (in order): 100K, 500K, 200K, 300K, 600K. Processes arrive: 212K, 417K, 112K, 426K. Place them with First, Best and Worst fit.
A:

```
First fit: 212→500 (rem 288) | 417→600 (rem 183) | 112→288
(rem 176) | 426 → NO hole ≥ 426 → waits
Best fit : 212→300 (rem 88)  | 417→500 (rem 83)  | 112→200
(rem 88)  | 426→600 (rem 174) → all placed
Worst fit: 212→600 (rem 388) | 417→500 (rem 83)  | 112→388
(rem 276) | 426 → NO hole → waits
```

Best fit is the only one that places all four here. No algorithm always wins.

Paging Core

NOTES

Split logical memory into fixed-size **pages** and physical memory into same-size **frames**. Any page can go into any free frame, so there is **no external fragmentation** (only a bit of internal fragmentation in the last page). A **page table** maps page number → frame number.

FLOW

Logical address = [page no | offset] → Page table[page no] = frame no → Physical address = [frame no | offset]

KEY POINT

```
offset bits      = log2(page size)
page-number bits = logical address bits − offset bits
frame-number bits= physical address bits − offset bits
page table size  = (number of pages) × (PTE size)
```

WORKED EXAMPLE

Worked numerical

Q: 32-bit logical address, 4 KB pages, 1 GB physical memory, 4-byte page table entry. Find offset bits, page bits, frame bits and the page table size.
A: Offset = $\log_2 4K$ = **12**. Page bits = 32−12 = **20** → 2^{20} pages. Physical address = 30 bits → frame bits = 30−12 = **18**. Page table = $2^{20} \times 4$ B = **4 MB** per process.

Multilevel paging

NOTES

A 4 MB page table per process is wasteful, so we page the page table itself. With 4 KB pages and 4-byte PTEs, one page holds 4096/4 = 1024 entries = 2^{10}. So the 20 page bits split into **10 (outer) + 10 (inner)**: address = `[10 | 10 | 12]` . Only the parts of the table actually in use need to exist. Cost: more memory accesses on a miss (2 table lookups + 1 data access for 2 levels).

TLB and effective access time

NOTES

The **TLB** is a tiny, fast cache of recent page-table entries. Hit → skip the page table lookup.

KEY POINT

```
EAT = h × (t_tlb + t_mem)  +  (1 − h) × (t_tlb + 2·t_mem)
(single-level page table)
```

For a k-level table, a TLB miss costs t_tlb + (k+1)·t_mem.

WORKED EXAMPLE

Worked numerical

Q: TLB access 20 ns, memory access 100 ns, TLB hit ratio 90%, single-level paging. Find EAT.
A: EAT = 0.9(20+100) + 0.1(20+200) = 108 + 22 = **130 ns**.

Segmentation (quick)

NOTES

Memory is divided into **variable-size logical units** (code, stack, heap). Address = `(segment no, offset)` . The segment table stores **base and limit**; if `offset ≥ limit` the hardware traps. Segmentation matches how programmers think but suffers external fragmentation; paging is invisible to the programmer.

Inverted page table Extension

KEY POINT

A normal page table has one entry per *virtual* page of *each* process. An **inverted page table** has **one entry per physical frame**: each entry records **(process id, page number)** of the page currently in that frame, and the index of the matching entry *is* the frame number. Translation searches the table for (pid, page) — made fast with a **hash table** plus a TLB.

REFERENCE TABLE

	Normal page table	Inverted page table
Entries	Virtual pages, per process	Physical frames, one table for the whole system
Size depends on	Virtual address-space size × number of processes	**RAM size only**
Lookup	Direct index by page number	Search / hash on (pid, page) — slower
Shared pages	Easy (two tables point at one frame)	Hard (one entry holds only one (pid, page))

WORKED EXAMPLE

Worked numerical

Q: 32-bit virtual addresses, 4 KB pages, 4-byte entries, 64 MB of RAM. Compare the page-table size per process with an inverted table using 8-byte entries.
A: Normal: $2^{32}/2^{12} = 2^{20}$ pages × 4 B = **4 MB per process**. Inverted: 64 MB / 4 KB = 2^{14} = 16384 frames × 8 B = **128 KB for the whole system**, however many processes exist.

Buddy system Extension

KEY POINT

Memory is allocated in blocks whose size is a **power of two**. A request is rounded **up** to the next power of two; if no block of that size is free, a larger block is repeatedly **split in half** — the two halves are **buddies**. On release, a block is **merged with its buddy** if that buddy is also free, and the merge repeats upward. A block's buddy address = **block address XOR block size**. Merging is fast, but the rounding causes **internal fragmentation** (up to nearly 50%). Linux uses it for physical page frames.

WORKED EXAMPLE

Worked example

Q: From 1024 KB, allocate a 70 KB request.
A:

```
1024 → split into 512 + 512      (use one 512)
 512 → split into 256 + 256
 256 → split into 128 + 128      128 ≥ 70 and 64 < 70  →
allocate a 128 KB block
Free blocks left: 512 (other half), 256, 128.   Internal
fragmentation = 128 − 70 = 58 KB.
```

6 Virtual Memory

Demand paging & page faults Core

NOTES

A page is loaded only when first used. If the page table says the page is not in memory (valid bit = 0), the CPU raises a **page fault**:

FLOW

Trap to OS → Find page on disk → Get a free frame (or evict one) → Read page in, update table → Restart the instruction

KEY POINT

```
EAT = (1 − p) × t_mem  +  p × t_fault        (p = page fault
rate)
```

WORKED EXAMPLE

Worked numerical

Q: Memory access 200 ns, page fault service time 8 ms, p = 1/1000. Find EAT, then the maximum p for at most 10% slowdown.
A: t_fault = 8,000,000 ns. EAT = 0.999×200 + 0.001×8,000,000 = 199.8 + 8000 ≈ **8200 ns** (a 40× slowdown!).
For ≤10% slowdown: EAT ≤ 220 → 200 + p(8,000,000 − 200) ≤ 220 → p ≤ 20 / 7,999,800 ≈ **2.5×10^{-6}** (about 1 fault per 400,000 accesses). Page faults must be extremely rare.

Page replacement algorithms Core

NOTES

When memory is full, pick a victim page to evict.

REFERENCE TABLE

Algorithm	Evicts	Notes
FIFO	Oldest loaded page	Simple; suffers **Belady's anomaly**
Optimal (OPT)	Page not needed for the longest time in future	Fewest faults, but needs the future (used as a benchmark)
LRU	Least recently used page	Good approximation of OPT; needs hardware support

WORKED EXAMPLE

Worked numerical: Belady's anomaly

Q: Reference string 1,2,3,4,1,2,5,1,2,3,4,5 with FIFO. Count faults with 3 frames and with 4 frames.
A:

```
3 frames:  1F 2F 3F  4F(evict 1) 1F(evict 2) 2F(evict 3)
5F(evict 4) 1H 2H 3F(evict 1) 4F(evict 2) 5H
           faults = 9
4 frames:  1F 2F 3F 4F  1H 2H  5F(evict 1) 1F(evict 2)
2F(evict 3) 3F(evict 4) 4F(evict 5) 5F(evict 1)
           faults = 10
```

More frames gave **more** faults: that is Belady's anomaly. It happens with FIFO but never with LRU or OPT (these are **stack algorithms**: the pages held with k frames are always a subset of those held with k+1).

COMMON TRAP

Classic result

For the textbook string 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1 with 3 frames: **FIFO = 15, LRU = 12, OPT = 9** faults. Always expect OPT ≤ LRU ≤ FIFO for such strings (not guaranteed for FIFO in every case, but typical).

Thrashing Core

NOTES

If a process does not have enough frames, it faults constantly and spends more time swapping than computing. CPU utilization **drops**, and a naive OS may then admit *more* processes, making it worse. Cure: reduce the degree of multiprogramming or use the **working-set model** (give each process enough frames for the pages it actively uses).

Second-chance (Clock) page replacement High-yield

KEY POINT

FIFO with a safety valve. Frames sit in a **circular list** with a pointer (the clock **hand**); each frame has a **reference bit** that is set to 1 when the page is used. On a fault the hand sweeps: if the bit is **1** it is cleared to 0 and the hand moves on (a *second chance*); if it is **0** that page is the victim. The new page is loaded with its bit = 1 and the hand moves past it. If every bit is 1 the hand clears them all and replaces the first frame — **the algorithm degenerates to FIFO**.

Enhanced clock uses the pair (reference, modify): (0,0) best victim → (0,1) → (1,0) → (1,1) worst, because clean, unused pages are cheapest to evict. A full trace is in solved problem Q16.

LFU, MFU and counting-based replacement Extension

REFERENCE TABLE

Policy	Victim	Reasoning and weakness
LFU — least frequently used	Page with the **smallest** reference count	Actively used pages have large counts. Weakness: a page used heavily early on keeps its big count forever (fix: periodically halve the counts)
MFU — most frequently used	Page with the **largest** count	The page with the smallest count was probably just brought in and has not been used yet. Rare in practice

NOTES

Neither is common; real systems use LRU approximations (reference bit, additional-reference-bits **aging**, second chance). Recall that FIFO suffers Belady's anomaly while LRU and OPT (stack algorithms) do not.

Frame allocation & global vs local replacement High-yield

REFERENCE TABLE

Scheme	Frames given to process i
Equal	m / n (m frames, n processes)
Proportional	$a_i = (s_i / S) \times m$, where s_i = size of process i and $S = \Sigma\, s_i$
Priority	Proportional to priority (or to a mix of priority and size)

NOTES

Every process needs a **minimum** number of frames, fixed by the instruction set (the most pages one instruction can touch, e.g. with indirect addressing).

KEY POINT

Global replacement

The victim may be a frame of *any* process. Higher throughput and flexible, but a process's fault rate depends on the others, and a high-priority process can steal frames — thrashing can spread.

KEY POINT

Local replacement

The victim must come from the process's *own* frames. Performance is predictable, but free frames may sit idle while a process thrashes within its fixed allocation.

WORKED EXAMPLE

Worked numerical

Q: 62 free frames are shared between processes of 10 pages and 127 pages using proportional allocation (round down).
A: S = 137. $a_1 = 62 \times 10/137 = 4.53 \rightarrow$ **4**; $a_2 = 62 \times 127/137 = 57.47 \rightarrow$ **57**. (One frame is left over after rounding down.)

7 File Systems, Disk Scheduling & I/O

File allocation methods Core

REFERENCE TABLE

Method	How	Pros / Cons
Contiguous	File occupies consecutive blocks	Fast sequential/direct access; external fragmentation, hard to grow
Linked	Each block points to the next	No fragmentation; slow random access, pointer loss breaks the chain (FAT keeps the links in a table)
Indexed	An index block lists all data blocks	Fast direct access; index block overhead

Unix inode

NOTES

The inode holds file attributes plus block pointers: some **direct**, then **single**, **double** and **triple indirect** pointers. Small files are reached fast, huge files are still possible.

WORKED EXAMPLE

Worked numerical

Q: Inode with 10 direct pointers and 1 single, 1 double, 1 triple indirect pointer. Block size 1 KB, block pointer 4 bytes. Maximum file size?
A: Pointers per block = 1024/4 = 256.

```
Direct          = 10 blocks
Single indirect = 256
Double indirect = 256² = 65,536
Triple indirect = 256³ = 16,777,216
Total blocks    = 16,843,018  →  ≈ 16,843,018 KB ≈ 16 GB
```

Almost all of the capacity comes from the triple-indirect level.

Disk access time Core

KEY POINT

```
Access time = seek time + rotational latency + transfer time
Average rotational latency = ½ × (time for one rotation)
```

WORKED EXAMPLE

Worked numerical

Q: 7200 RPM disk, average seek 9 ms, transfer rate 100 MB/s. Time to read a 4 KB block?
A: One rotation = 60/7200 s = 8.33 ms → average latency = **4.17 ms**. Transfer = 4 KB / 100 MB/s ≈ **0.04 ms**. Total ≈ 9 + 4.17 + 0.04 ≈ **13.2 ms**. Seek and rotation dominate, so scheduling to reduce head movement matters.

Disk scheduling Core

REFERENCE TABLE

Algorithm	Idea
FCFS	Serve in arrival order (fair, lots of movement)
SSTF	Nearest request first (can starve far requests)
SCAN (elevator)	Sweep to one end of the disk serving requests, then reverse
C-SCAN	Sweep one way only; at the end jump back to the start (more uniform wait)
LOOK / C-LOOK	Like SCAN / C-SCAN but turn around at the last request instead of the disk end

WORKED EXAMPLE

Worked numerical

Q: Disk cylinders 0–199, head at 53 moving toward 0. Request queue: 98, 183, 37, 122, 14, 124, 65, 67. Find total head movement for each algorithm.
A:

```
FCFS: 53→98→183→37→122→14→124→65→67
      45+85+146+85+108+110+59+2 = 640

SSTF: 53→65→67→37→14→98→122→124→183
      12+2+30+23+84+24+2+59 = 236

SCAN (going down first): 53→37→14→0, then reverse
0→65→67→98→122→124→183
      53 + 183 = 236

LOOK (going down first): 53→14 (39), then 14→183 (169)
      39 + 169 = 208
```

LOOK < SCAN because it does not travel to cylinder 0. When a question says "C-SCAN", check whether the return jump is counted in the total, and follow what the question states.

Free-space management Core

REFERENCE TABLE

Method	How it works	Trade-off
Bit vector (bitmap)	1 bit per block: 1 = free (or 0 = free, by convention)	Simple; finding contiguous runs is easy; the map takes **blocks / 8 bytes** of space
Linked list	Free blocks chained through a pointer in each block	No wasted map, but traversing the list is slow and contiguous runs are hard to find
Grouping	First free block holds the addresses of n free blocks; the last of those points to the next such block	Many free blocks found quickly
Counting	Store (start block, run length) pairs	Compact when free space is in long contiguous runs

WORKED EXAMPLE

Worked numerical

Q: Bit-vector size for a 512 GB disk with 4 KB blocks?
A: Blocks = 2^{39} / 2^{12} = 2^{27}. One bit each → 2^{27} bits = 2^{24} bytes = **16 MB**.

Directory structures Core

REFERENCE TABLE

Structure	Key property	Problem
Single-level	One directory for all files	Name clashes between users; no grouping
Two-level	A master directory with one directory per user	No sharing between users; no sub-grouping
Tree	Arbitrary depth; absolute and relative path names	Sharing is awkward; deleting a non-empty directory needs a policy
Acyclic graph	Shared files and subdirectories via links	Dangling pointers and several names per file; use reference counts
General graph	Cycles allowed	Traversal can loop forever; needs cycle detection or garbage collection

Hard links vs soft (symbolic) links High-yield

REFERENCE TABLE

	Hard link	Soft link
What it is	Another **directory entry** pointing at the **same inode**	A **separate file** (own inode) whose content is a path name
Link count	Stored in the inode; data are freed only when it reaches 0	Does not change the target's link count
Crosses file systems?	No (inode numbers are per file system)	Yes
Can link a directory?	Normally not (would create cycles)	Yes
If the original name is deleted	Data still reachable through the other name	Becomes **dangling** (broken)

WORKED EXAMPLE

Worked example

Q: File `a` has link count 1. After `ln a b` (hard) and `ln -s a c` (soft), then `rm a`, what happens?
A: `ln a b` raises the count to 2; the soft link `c` does not change it. `rm a` drops the count to 1, so the data stay reachable through `b`. But `c` stores the path "a", which no longer exists, so `c` is **dangling**.

FAT file system High-yield

KEY POINT

The **File Allocation Table** sits at the start of the volume and has **one entry per disk block (cluster)**. A directory entry stores only the **first block number** of a file; each FAT entry holds the **next block** of the chain, an **end-of-file marker**, or a free/bad marker. Entry widths give FAT-12, FAT-16 and FAT-32. Reaching the k-th block means following k − 1 links — cheap when the FAT is cached in memory, because no data block has to be read to find the chain. **FAT size = number of blocks × entry size.**

WORKED EXAMPLE

Worked numerical

Q: 512 GB disk, 4 KB blocks, 4-byte FAT entries. FAT size? How many FAT entries does a 100 KB file use?
A: Blocks = 2^{27}, so FAT = $2^{27} \times 4$ B = 2^{29} B = **512 MB**. The file occupies 100/4 = **25 blocks** → a chain of 25 FAT entries.

I/O subsystem: polling, interrupts and DMA High-yield

REFERENCE TABLE

Technique	How data move	CPU cost
Programmed I/O (polling)	CPU repeatedly reads the device status register (busy-waiting), then copies each word	Highest — the CPU is tied up for the whole transfer
Interrupt-driven I/O	Device interrupts when ready; the CPU copies each byte or word	One interrupt per byte or word
DMA	A DMA controller moves a whole block between device and memory without the CPU	Setup plus **one interrupt per block**

KEY POINT

DMA steps: (1) the CPU programs the controller with the memory address, byte count and direction; (2) the controller requests the bus; (3) it transfers the block directly; (4) it raises **one interrupt** when the count reaches zero.

Modes: burst (block) mode — the controller keeps the bus for the entire block, so the CPU is locked out of memory meanwhile; **cycle stealing** — it takes the bus one word at a time between CPU cycles, slowing the CPU slightly; **transparent mode** — it transfers only when the CPU is not using the bus.

WORKED EXAMPLE

Worked numerical

Q: A 1 GHz CPU serves a device moving 100 blocks of 16 KB per second. Interrupt-driven I/O costs 300 cycles per byte; DMA costs 1500 cycles per block (setup + completion interrupt). What fraction of CPU time goes to I/O in each case?
A: Interrupt-driven: 16384 × 300 = 4,915,200 cycles per block × 100 = 4.9152 × 10^8 cycles/s → **49.15%** of the CPU. DMA: 1500 × 100 = 1.5 × 10^5 cycles/s → **0.015%**.

Buffering & spooling High-yield

REFERENCE TABLE

Concept	Purpose
Buffering (single, double, circular)	Hides speed mismatch between producer and consumer, adapts different transfer sizes, and supports "copy semantics". With **double buffering** one buffer is filled while the other is drained, so the two overlap
Cache vs buffer	A cache holds a *copy* of data that also exists elsewhere (for speed); a buffer may hold the *only* copy of data in transit
Spooling	Output for a device that cannot interleave jobs (a printer) is first written to a disk file; a spooler daemon then sends the files to the device **one job at a time**. This stops jobs interleaving and lets the CPU carry on while the slow device works

RAID (redundant arrays of disks) Extension

NOTES

RAID combines several disks for **speed** (striping) and/or **fault tolerance** (mirroring or parity). It is a lighter GATE topic than disk scheduling, but the capacity and failure rules are quick marks.

REFERENCE TABLE

Level	Idea	Usable capacity (N disks of size D)	Failures survived
RAID 0	Striping, no redundancy	N × D	**0** — one failure loses everything
RAID 1	Mirroring (every block written to two disks)	N × D / 2	1 per mirrored pair (at least 1)
RAID 5	Block striping with **distributed parity** (parity = XOR of the data blocks of a stripe)	(N − 1) × D	**1**
RAID 6	Two independent parity blocks per stripe	(N − 2) × D	**2**
RAID 10	Stripe across mirrored pairs	N × D / 2	1 per pair

KEY POINT

Parity recovery. If a stripe holds blocks A, B, C and parity P = A ⊕ B ⊕ C, a lost block is rebuilt by XOR-ing all the survivors: e.g. A = B ⊕ C ⊕ P.

RAID 5 small-write penalty. Updating one data block needs **4 disk I/Os**: read old data, read old parity, write new data, write new parity (new P = old P ⊕ old data ⊕ new data). Because parity is **distributed** across all disks rather than kept on one dedicated disk (RAID 4), no single parity disk becomes the write bottleneck.

WORKED EXAMPLE

Worked numerical

Q: Six 4 TB disks are used as (a) RAID 0, (b) RAID 1, (c) RAID 5, (d) RAID 6. Find the usable capacity of each.
A: (a) 6 × 4 = **24 TB**; (b) 24/2 = **12 TB**; (c) (6 − 1) × 4 = **20 TB**; (d) (6 − 2) × 4 = **16 TB**.

8 Problem → Standard Method Playbook

METHOD

1. Scheduling

Build the execution timeline first. Mark arrivals and preemptions, then calculate completion time `C`, turnaround `C−A`, waiting `turnaround−burst`, and response `first start−arrival`. Add context-switch overhead only where the schedule actually switches.

METHOD

2. HRRN / MLFQ

HRRN is non-preemptive: at each completion choose the arrived job maximizing `(waiting+burst)/burst`. MLFQ is priority-queue based: determine the queue/quantum rule, then apply demotion, promotion/aging, and preemption exactly as specified.

METHOD

3. Deadlock / Banker

Separate *prevention*, *avoidance*, and *detection*. For Banker, compute `Need = Max−Allocation`, maintain `Work`, and repeatedly find a process whose Need ≤ Work; then release its Allocation. A complete safe sequence proves the state is safe.

METHOD

4. Paging / TLB

Split address into page number + offset. Offset bits = $\log_2(\text{page size})$. For EAT, enumerate TLB-hit and TLB-miss paths rather than memorising a formula blindly.

METHOD

5. Page replacement

Maintain the frame contents after every reference. FIFO → oldest arrival; LRU → least recently referenced; OPT → page whose next use is farthest in the future; Clock → reference-bit second chance. Count faults only after checking whether the referenced page is already resident.

METHOD

6. Disk scheduling

Draw the cylinder line and mark the initial head. For SCAN/C-SCAN, determine direction and whether the endpoint is physically visited under the question's convention. Total movement is the sum of absolute head-position differences.

METHOD

7. File systems / links

Distinguish inode metadata, directory entries, allocation method, and free-space management. Hard links share an inode; symbolic links store a pathname and can dangle. In FAT, follow the FAT chain from the first block.

METHOD

8. I/O

Polling = CPU repeatedly checks status; interrupt-driven I/O = device notifies CPU; DMA = controller transfers a block with minimal CPU intervention. Buffering handles rate mismatch; spooling queues whole jobs for a dedicated device.

9 🎯 Representative GATE-Style Worked Problems

NOTE

HRRN

Q: At time 0, P1 arrives with burst 8. At time 2, P2 arrives with burst 4; at time 4, P3 arrives with burst 2. Which process runs after P1 under HRRN?
A: P1 finishes at t=8. Waiting: P2=6, P3=4. Ratios: P2=(6+4)/4=**2.5**; P3=(4+2)/2=**3**. Therefore **P3** runs next.

NOTE

MLFQ

Q: A job uses its entire quantum repeatedly while another frequently blocks for I/O. Under the standard MLFQ rule that full-quantum jobs are demoted, what happens?
A: The CPU-bound job moves to a lower-priority queue; the I/O-bound job tends to remain in a higher queue. Aging/priority boost can later prevent starvation.

NOTE

Banker's algorithm

Q: A system has Available = (3,3,2). P1 Need=(1,2,2), P2 Need=(2,1,0), P3 Need=(4,0,1). Which process can safely finish first?
A: Compare Need with Available component-wise. P1 satisfies (1,2,2) ≤ (3,3,2), so P1 can finish and release its Allocation. P2 also satisfies the current Work vector. A complete safe sequence must then be constructed; one qualifying first choice alone is not sufficient to claim the entire state safe.

NOTE

Paging

Q: A 32-bit address uses 4 KB pages. How many bits are the page offset?
A: 4 KB = 2^{12} bytes, so the offset is **12 bits**. The remaining **20 bits** identify the virtual page.

NOTE

LRU

Q: With 3 frames and reference string 1,2,3,1,4,2,1,3,4, how is an LRU replacement decided at the reference to 4?
A: Before 4, frames contain 1,2,3. Their most recent uses are 1 at the immediately previous reference, 2 at the earlier reference, and 3 at the initial reference. Therefore **3** is least recently used and is replaced.

NOTE

Disk scheduling

Q: The disk head is at 50 and requests are 82, 170, 43, 140. Under FCFS, what total head movement occurs?
A: Movement = |50−82| + |82−170| + |170−43| + |43−140| = 32+88+127+97 = **344 cylinders**.

10 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **Banker:** Need = Max−Allocation; a safe sequence repeatedly selects Need ≤ Work and then adds Allocation to Work.
- Disk head movement is the sum of absolute differences between consecutive head positions.
- FCFS can suffer convoy effect; SJF/SRTF minimise average waiting time when burst information is known under standard assumptions.
- Page-table address translation splits a virtual address into page number + offset; offset bits depend on page size.
- HRRN response ratio = (waiting + service) / service; it is non-preemptive and picks the highest ratio, so long-waiting jobs cannot starve.
- **Buddy system:** round each request up to a power of two (internal fragmentation); buddy address = block address XOR block size.
- Proportional frame allocation $a_i = (s_i / S) \times m$; bit-vector size = blocks / 8 bytes; FAT size = blocks × entry size.
- **RAID usable capacity:** RAID 0 = N·D, RAID 1/10 = N·D/2, RAID 5 = (N−1)·D, RAID 6 = (N−2)·D; RAID 5 small write = 4 I/Os.

LAST-MINUTE · MUST REMEMBER

- MLFQ is defined by its queue/quantum/promotion rules; always simulate the specified policy rather than assuming one universal implementation.
- FIFO can show Belady's anomaly; LRU and OPT are stack algorithms and do not.
- A process has its own address space/resources; threads within a process share the process address space and many resources.
- Context switching changes execution context but does not itself perform useful application work.
- Race conditions arise when concurrent execution accesses shared state and correctness depends on timing/interleaving.
- A critical section requires mutual exclusion plus appropriate progress/bounded-waiting properties for the intended solution.
- **Deadlock requires all four:** mutual exclusion, hold-and-wait, no preemption, and circular wait.
- TLB is a cache of page-table translations; a TLB miss does not necessarily mean a page fault.
- Belady's anomaly can occur in FIFO page replacement; it does not occur in stack algorithms such as LRU and OPT.
- In a file system, distinguish logical file structure, allocation method, directory structure, and free-space management.
- MLFQ moves jobs between queues by observed behaviour: full-quantum users are demoted, I/O-bound jobs stay high, and aging promotes starving jobs.
- DMA interrupts the CPU once per block, not once per byte; burst mode holds the bus for the whole block, cycle stealing takes it one word at a time.
- Test-and-set / compare-and-swap give mutual exclusion and progress but not bounded waiting; they spin (busy-wait), so they suit short critical sections on multiprocessors.

LAST-MINUTE · COMMON TRAPS

- TLB miss ≠ page fault; a TLB miss can still find a valid page-table entry in memory.
- Preemptive and non-preemptive scheduling are different; always check whether a running process can be interrupted.
- Page fault and TLB miss are different events; a TLB miss can still be satisfied from memory-resident page-table entries.
- Zombie = terminated but not yet reaped by wait(); orphan = still running but its parent has died (adopted by init).
- An inverted page table has one entry per physical frame, so its size depends only on RAM, not on the number of processes; lookup needs hashing.
- Hard links share one inode and a link count; soft links are separate files holding a path and can dangle.
- A monitor condition-variable signal is lost if nobody is waiting, unlike a semaphore signal, which is remembered. Under Mesa semantics re-check the condition in a while loop.
- Deadlock = blocked forever in a cycle; livelock = running but making no progress; starvation = never chosen, fixed by aging.

LAST-MINUTE · GATE FAVOURITES

- HRRN chooses the maximum (waiting+burst)/burst at each scheduling decision; it is non-preemptive.
- Banker's algorithm is an avoidance algorithm; deadlock detection is different from prevention and avoidance.
- Disk scheduling questions require careful calculation of head movement; know FCFS, SSTF, SCAN, C-SCAN and their variants.
- **Priority inversion:** a high-priority task waits for a lock held by a low-priority task that a medium-priority task keeps preempting. Fix: priority inheritance.

Theory of Computation

GATE CSE 2027 · complete formula, rule, method & trap sheet

78 reference boxes · **8** chapters ·

Finite Automata (DFA & NFA) | Regular Expressions & Closure Properties | Context-Free Grammars | Pushdown Automata | Pumping Lemma & Language Properties | Turing Machines & Undecidability | Problem → Standard Method Playbook | Last-minute one-liners

1 Finite Automata (DFA & NFA)

What is a DFA?

NOTES

A **Deterministic Finite Automaton** is a 5-tuple `(Q, Σ, δ, q0, F)` : a finite set of states Q, an alphabet Σ, a transition function δ (exactly one arrow per symbol from every state), a start state q0, and accepting states F. It reads the input left to right, one symbol at a time, and accepts if it ends in an accepting state.

Example: strings over {0,1} ending in "01"

DIAGRAM



NOTES

q0 = "haven't seen anything useful yet", q1 = "just saw a 0", q2 (double circle = accepting) = "just completed ...01". Any further symbol from q2 loops back on itself only if it re-forms "01" — here simplified as a self-loop for both symbols for illustration; a fully correct machine would route q2 on '0' back to q1 and on '1' back to q0. **GATE tip:** always trace 2-3 test strings by hand ("101" → q0-1→q0-0→q1-1→q2 ✓ accepted) rather than trusting a diagram at a glance.

NFA — nondeterminism

NOTES

An **NFA** allows zero, one, or many outgoing arrows per symbol from a state (and optionally ε-moves that consume no input). It accepts a string if **at least one** path through the machine ends in an accepting state.

COMMON TRAP

Key theorem

Every NFA has an **equivalent DFA** (they accept exactly the same class of languages — regular languages). NFAs are just a convenient shorthand; DFAs are what actually run in constant memory per step.

NFA → DFA: the subset construction

NOTES

Each DFA state = a **set** of NFA states reachable so far. Start state = ε-closure of the NFA's start state. On each symbol, take the union of where every state in the current set can go.

WORKED EXAMPLE

Worked numerical

Q: NFA: states {A,B}, A is start, B is accepting. δ(A,0)={A,B}, δ(A,1)={A}, δ(B,0)=∅, δ(B,1)={B}. Convert to a DFA (list only reachable states).
A:

```
Start = {A}
{A} --0--> {A,B}          {A} --1--> {A}
{A,B} --0--> {A,B}∪∅ = {A,B}      {A,B} --1--> {A}∪{B} =
{A,B}
```

Reachable DFA states: `{A}` (non-accepting, start), `{A,B}` (accepting, since it contains B). Only **2 DFA states** needed — even though the "obvious" powerset has $2^2=4$ subsets, only the reachable ones matter.

DFA Minimization

NOTES

Different DFAs can accept the same language; the **minimal DFA** has the fewest states. Method: start by splitting states into {accepting} and {non-accepting}, then repeatedly split a group if two states in it go to *different* groups on some symbol. Stop when no group splits further — the final groups are the minimized states.

WORKED EXAMPLE

Worked numerical

Q: A DFA has states A(start),B,C,D,E(accepting) with: δ(A,0)=B, δ(A,1)=C, δ(B,0)=A, δ(B,1)=D, δ(C,0)=E, δ(C,1)=A, δ(D,0)=E, δ(D,1)=C, δ(E,0)=D, δ(E,1)=B. Minimize it.
A:

```
Initial partition: P0 = {A,B,C,D} (non-accept), {E} (accept)

Round 1 — check each state's behavior against groups
{A,B,C,D} / {E}:
  A: 0→B(grp1) 1→C(grp1)  → pattern (1,1)
  B: 0→A(grp1) 1→D(grp1)  → pattern (1,1)
  C: 0→E(grp2) 1→A(grp1)  → pattern (2,1)   ← different!
  D: 0→E(grp2) 1→C(grp1)  → pattern (2,1)   ← different!
So split: {A,B} stay together, {C,D} split off, {E} alone.
New partition: {A,B}, {C,D}, {E}

Round 2 — recheck with 3 groups:
  A: 0→B(same grp as A) 1→C(grp2)
  B: 0→A(same grp) 1→D(grp2)      → A,B still match, no
further split
  C: 0→E(grp3) 1→A(grp1)
  D: 0→E(grp3) 1→C(grp2)          ← C goes to grp1 on '1', D
goes to grp2 on '1' → SPLIT {C,D}!

Final partition: {A,B}, {C}, {D}, {E} → 4 states after
minimization
```

The 5-state DFA minimizes to **4 states** (only A and B are truly equivalent).

GATE FAVOURITE

GATE angle: minimum states for a language

Classic question type: "minimum number of states in a DFA accepting binary strings where the third-last symbol is 1" needs **2^k** states for "k-th last symbol" style languages (the DFA must remember the last k symbols seen, and there are 2^k possible windows). For k=3, that's **8 states**. Always ask: "how much history must this DFA remember?" — that count becomes your state count.

More DFA construction patterns — worked examples

NOTES

GATE tests a handful of recurring "language types". Here are the main families, each with the trick that builds the DFA fast.

NOTE

Type 1 — Counting mod k

Q: Construct a DFA over {0,1} accepting strings where the number of 1's is ≡ **2 (mod 3)**. How many states are needed?
A: Use one state per remainder: `S0, S1, S2` (count mod 3 so far), start = S0. On '0' stay in the same state (doesn't change the count); on '1' move to the next remainder, wrapping around: S0→S1→S2→S0. Accepting state = **S2** (remainder 2). **3 states** total — one per residue class. This "counting mod k" pattern always needs exactly **k states**.

NOTE

Type 2 — Forbidden substring

Q: Construct a DFA over {0,1} accepting strings that do **not** contain "101" as a substring. How many states?

A: States track "how much of the forbidden pattern have I just seen": `q0` =nothing useful, `q1` =just saw "1", `q2` =just saw "10", `q3` =dead (saw "101", trap forever). Transitions: q0 -1→ q1, q0 -0→ q0; q1 -0→ q2, q1 -1→ q1; q2 -1→ q3(dead), q2 -0→ q0; q3 loops on both symbols. Accepting = {q0,q1,q2} (all except the dead trap). **4 states** (3 useful + 1 dead/trap state) — this "prefix-tracking" technique works for any forbidden substring, needing (length of pattern + 1) states in the simplest case.

NOTE

Type 3 — Product construction (AND of two conditions)

Q: Construct a DFA for strings with an **even** number of 0's **and** an **odd** number of 1's.

A: Build two small DFAs separately (2 states each: even/odd count of 0's; even/odd count of 1's), then take their **Cartesian product**: each combined state = (0-parity, 1-parity), giving 2×2 = **4 states**: (even,even), (even,odd), (odd,even), (odd,odd). Accepting state = (even,odd). This "product construction" is the standard way to combine two separate regular conditions with AND — and it directly proves regular languages are closed under intersection.

ε-NFA and ε-closure

NOTES

An **ε-NFA** allows transitions that consume no input symbol at all (labeled ε). The **ε-closure** of a state is every state reachable using only ε-moves (including itself).

WORKED EXAMPLE

Worked numerical

Q: ε-NFA: states A,B,C. ε-moves: A→B, B→C. δ(C,0)={A}. Find ε-closure(A) and the DFA-equivalent start state.

A: From A you can reach B (ε), then C (ε from B) → **ε-closure(A) = {A,B,C}**. This whole set becomes the single start state of the equivalent DFA (same idea as subset construction, but every step first takes the ε-closure of wherever the input symbols land you).

Finite automata with output — Moore & Mealy machines

NOTES

Unlike a DFA (which only accepts/rejects), these machines **produce an output string**. They have no final states.

REFERENCE TABLE

	Moore machine	Mealy machine
Output depends on	the **current state** only	the **current state AND input** (output sits on the transition)
Output length for an input of length n	**Depends on the question's convention — not on Moore vs Mealy.** If the start state's output is printed before any input, a Moore machine gives n + 1; if the question says it is not printed, Moore also gives n. A Mealy machine prints one output per transition, so n. Always read how the question defines the output sequence.	
Reacts to input	one clock later	immediately

KEY POINT

Conversions. Moore → Mealy keeps the **same number of states** (move each state's output onto its incoming transitions). Mealy → Moore may need **more states** — a state must be split by the output it receives, so up to |Q| × |outputs| states. When the Moore machine's start-state output is printed, the two outputs match after dropping that first symbol.

WORKED EXAMPLE

Worked numerical — "11" detector

Q: Mealy machine, states q0 (last bit not 1) and q1 (last bit was 1): q0 −0/0→ q0, q0 −1/0→ q1, q1 −0/0→ q0, q1 −1/1→ q1. Output for input 0110111? Equivalent Moore machine size?

A:

```
Input : 0 1 1 0 1 1 1
Output: 0 0 1 0 0 1 1     (n = 7 input symbols → 7 output symbols)
Moore version needs 3 states: A (no trailing 1, out 0), B (one trailing 1, out 0), C (≥2 trailing 1s, out 1)
Under the convention that the start state's output is printed, its output on the same input has 8 symbols: 0 0010011 — the extra leading 0 is the start state's output.
```

2 Regular Expressions & Closure Properties

Regex operators

REFERENCE TABLE

Operator	Meaning	Example
Concatenation `ab`	a followed by b	`01` matches only "01"
Union `a+b` (or a|b)	a or b	`0+1` matches "0" or "1"
Kleene star `a*`	zero or more repetitions of a	`0*` matches "", "0", "00", ...
Plus `a+`	one or more repetitions (a·a*)	`1+` matches "1","11",...

MUST REMEMBER

Kleene's theorem — three ways to describe exactly the same class of languages (regular languages): **regular expressions** ⇔ **NFAs** ⇔ **DFAs**. GATE often asks you to convert between them or count states/symbols in one representation given another.

Reading tricky regexes

WORKED EXAMPLE

Worked numerical

Q: What language does `(0+1)*011(0+1)*` describe, and does it accept "10110"?

A: All binary strings that **contain "011" as a substring** somewhere. Check "10110": does it contain "011"? Positions: 1-0-1-1-0 → substring at index 1-3 is "011" → **yes, accepted**.

WORKED EXAMPLE

Worked numerical

Q: Simplify `(0*1*)*` to the smallest equivalent regex.

A: `0*1*` already generates every possible mix once you allow repeating the whole group any number of times — repeating (0*1*) just re-generates strings already reachable. So `(0*1*)* = (0+1)*` — **every string over {0,1}**. (This is a favorite GATE trick: a star-of-a-star-like pattern often collapses to "everything".)

Closure properties of regular languages

NOTES

Regular languages are closed (the result is still regular) under:

REFERENCE TABLE

Operation	Closed?
Union	✔ Yes
Intersection	✔ Yes
Complement	✔ Yes
Concatenation	✔ Yes
Kleene star	✔ Yes
Reversal	✔ Yes

GATE FAVOURITE

GATE trap

Regular languages are closed under **every** common operation — this is what makes them so "well-behaved". Context-free languages (Part 2) are **not** closed under intersection or complement — that's where GATE likes to test whether you mixed up the two classes.

More closure operations — homomorphism, quotient & traps

REFERENCE TABLE

Operation	Regular languages	How / example
Difference, symmetric difference	✔ Closed	$L_1 - L_2 = L_1 \cap L_2^c$
Prefix (Init), suffix	✔ Closed	make every state on a path to an accepting state accepting
Homomorphism h(L)	✔ Closed	h replaces each symbol by a string: h(a)=01 turns a regex a* into (01)*
Inverse homomorphism $h^{-1}(L)$	✔ Closed	{w : h(w) ∈ L}; the DFA reads h(a) in one step
Substitution by regular sets	✔ Closed	replace each symbol by a regular language
Quotient L_1 / L_2	✔ Closed for **ANY** L_2	{x : xy ∈ L_1 for some y ∈ L_2}

GATE FAVOURITE

GATE trap

Closure is only for **finitely many** operations. An **infinite union** of regular languages need not be regular (every language is the infinite union of its single-string sets). Also: a **subset** of a regular language need not be regular ($\{a^n b^n\} \subset a^*b^*$), while any **finite** language is regular, and adding/removing finitely many strings never changes regularity.

WORKED EXAMPLE

Worked numerical

Q: L = $\{a^n b^n : n \geq 0\}$. With h(a)=0 and h(b)=ε, what is h(L)? With h(a)=0 and h(b)=1?

A: h(a)=0, h(b)=ε gives $\{0^n : n \geq 0\}$ = **0*** (regular, even though L is not). h(a)=0, h(b)=1 gives $\{0^n 1^n\}$ (not regular). So the image of a *non-regular* language may or may not be regular.

Counting accepted strings — transfer matrix trick

NOTES

To count how many length-n strings a DFA accepts, build the adjacency matrix M where M[i][j] = number of symbols taking state i to state j, then **M^n**, and sum the (start, accepting) entries.

WORKED EXAMPLE

Worked numerical

Q: A 2-state DFA over {0,1}: state A (start, accepting) with δ(A,0)=B, δ(A,1)=A; state B with δ(B,0)=A, δ(B,1)=B. How many length-2 strings does it accept?

A: Adjacency matrix (rows/cols = A,B): `M = [[1,1],[1,1]]` (A→A on '1' =1 way, A→B on '0'=1 way; B→A on '0'=1, B→B on '1'=1).

```
M^2 = [[1,1],[1,1]] × [[1,1],[1,1]] = [[2,2],[2,2]]
```

Start=A, accepting={A} → answer = M^2[A][A] = **2**. (Direct check: strings "10" and "01" both end back at A — matches.)

Arden's Theorem — converting a DFA directly into a regex

NOTES

Write one equation per state: `state = Σ (incoming symbol · source state) [+ ε if it's the start state]`. Then solve using **Arden's rule**: if `R = Q + RP` (and ε ∉ P), then `R = QP*`.

WORKED EXAMPLE

Worked numerical

Q: DFA: states A (start, accepting), B. δ(A,0)=B, δ(A,1)=A, δ(B,0)=A, δ(B,1)=B. Find the regex for the accepted language using Arden's theorem.

A: Write the state equations (a state equals ε for the start state, plus one term per incoming edge):

```
A = ε + A·1 + B·0        (A is start: add ε; incoming edges: A--1-->A, B--0-->A)
B = A·0 + B·1             (incoming edges: A--0-->B, B--1-->B)

Solve B first using Arden's rule (B = A·0 + B·1  ⇒  R=B, Q=A·0, P=1):
  B = A·0·1*

Substitute into the A equation:
  A = ε + A·1 + (A·0·1*)·0
  A = ε + A·(1 + 0·1*·0)

Apply Arden's rule again (R=A, Q=ε, P = 1+0·1*·0):
  A = ε·(1 + 0·1*·0)* = (1 + 0·1*·0)*
```

Since A is the only accepting state, the final regex is `(1 + 0·1*·0)*` — this describes exactly the strings where every "0" is eventually followed (after any number of 1's) by another "0", i.e. an even number of 0's overall.

COMMON TRAP

GATE technique tip

Always solve the equation with the **most self-loops referring only to itself** first (apply Arden's rule to isolate it), then substitute backward into the remaining equations. Non-accepting states with no useful path can often be dropped early to simplify the algebra.

More regex ⇄ language identification — varied examples

NOTE

Type A — "exactly one" pattern

Q: What does `0*10*10*` describe?

A: Any string over {0,1} containing **exactly two 1's** (with any number of 0's before, between, and after). Extending the pattern (adding more "0*1" blocks) generalizes to "exactly n ones".

NOTE

Type B — union of prefixes

Q: What does `(0+1)*0(0+1)*1(0+1)* + (0+1)*1(0+1)*0(0+1)*` describe, more simply?

A: The first part = "contains a 0 somewhere before a 1 somewhere"; the second = "contains a 1 somewhere before a 0 somewhere". Together they cover every string containing **both a 0 and a 1** — simplified to just `(0+1)*0(0+1)*1(0+1)* + (0+1)*1(0+1)*0(0+1)*`, but really it's just "not all-0's and not all-1's", i.e. the complement of `0* + 1*`.

NOTE

Type C — length restriction

Q: Give a regex for binary strings of **length exactly divisible by 3** (length ≡ 0 mod 3, length can be 0).

A: Group the alphabet into blocks of 3 symbols: `((0+1)(0+1)(0+1))*`. Each repetition of the parenthesized group consumes exactly 3 symbols, so the star produces every string whose length is a multiple of 3 (including the empty string).

3 Context-Free Grammars

What is a CFG?

NOTES

A **Context-Free Grammar** is a 4-tuple `(V, Σ, R, S)` : variables V, terminals Σ, production rules R of the form `A → α` (one variable to a string of variables/terminals), and a start symbol S. "Context-free" means a variable can be replaced the same way no matter what surrounds it.

KEY POINT

Example — balanced parentheses: `S → (S)S | ε`
Leftmost derivation of "()()" :

```
S ⇒ (S)S ⇒ (ε)S ⇒ ()S ⇒ ()(S)S ⇒ ()(ε)S ⇒ ()()S ⇒ ()()ε ⇒ ()()
```

Each step expands the leftmost remaining variable — exactly the kind of derivation GATE asks you to complete or verify.

Ambiguity

NOTES

A grammar is **ambiguous** if some string has **two different parse trees** (or leftmost derivations).

WORKED EXAMPLE

Worked numerical

Q: Grammar `E → E+E | E*E | id` . Show that "id+id*id" is ambiguous.
A: Two different parse trees exist:

```
Tree 1 (groups as id+(id*id)):        Tree 2 (groups as (id+id)*id):

      E                                       E
    / | \                                   / | \
   E  +  E                                 E  *  E
  id   / | \                               / | \  id
      E  *  E                           E  +  E
     id     id                         id     id
```

Both are valid parse trees for the same string using the same grammar → **ambiguous**. Fix: rewrite with precedence levels: `E → E+T | T, T → T*F | F, F → id` . This forces multiplication to bind tighter than addition, removing the ambiguity.

GATE FAVOURITE

GATE angle

Some languages are **inherently ambiguous** — every possible grammar for them is ambiguous, no rewrite fixes it (e.g. $\{a^i b^j c^k : i=j \text{ or } j=k\}$). GATE sometimes asks you to identify such a language rather than just spot ambiguity in one specific grammar.

Chomsky Normal Form (CNF)

NOTES

A CFG is in CNF if every rule looks like `A → BC` (two variables) or `A → a` (one terminal), with at most `S → ε` as an exception. Any CFG can be converted to an equivalent CNF grammar in three steps:

RULES & FACTS

1. Replace every terminal appearing alongside other symbols with a new variable (`a → Ca`).
2. Eliminate ε-productions and unit productions (`A → B`).
3. Break any RHS longer than 2 symbols into a chain of new binary variables.

WORKED EXAMPLE

Worked numerical — full CNF conversion

Q: Convert to CNF: `E → E+T | T, T → T*F | F, F → (E) | id` .
A:

```
Step 1 — terminal variables:
  C+ → +,  C* → *,  C( → (,  C) → )
  E → E C+ T | T
  T → T C* F | F
  F → C( E C) | id

Step 2 — eliminate unit productions (E→T, T→F):
  E → E C+ T | T C* F | C( E C) | id
  T → T C* F | C( E C) | id
  F → C( E C) | id

Step 3 — binarize every RHS longer than 2 symbols:
  "E C+ T" (3 symbols) → introduce X1 → C+ T,  giving E → E X1
  "T C* F" (3 symbols) → introduce X2 → C* F,  giving T → T X2
  "C( E C)" (3 symbols) → introduce Y1 → E C),  giving F → C( Y1

Final CNF grammar:
  E  → E X1 | T X2 | C( Y1 | id
  T  → T X2 | C( Y1 | id
  F  → C( Y1 | id
  X1 → C+ T
  X2 → C* F
  Y1 → E C)
  C+ → +     C* → *     C( → (     C) → )
```

Every rule now has exactly 2 variables on the RHS, or exactly 1 terminal — this is CNF. GATE often asks "how many new variables are introduced" (here: C+, C*, C(, C), X1, X2, Y1 = **7 new variables**) or "how many total productions does the CNF grammar have".

Derivations, parse trees & ambiguity counting

NOTES

A **leftmost** derivation always expands the leftmost variable; a **rightmost** derivation the rightmost. Every parse tree corresponds to exactly **one** leftmost and **one** rightmost derivation, so a grammar is ambiguous ⇔ some string has **two leftmost derivations** ⇔ two parse trees. For S → SS | a, the string a^n has Catalan(n−1) parse trees (n = 4 gives 5).

Simplifying a CFG

KEY POINT

Remove, **in this order**: (1) ε-productions → (2) unit productions (A → B) → (3) useless symbols (first non-generating, then unreachable). Another order can reintroduce unwanted productions. After this, convert to CNF or GNF.

Greibach Normal Form (GNF) & CYK

REFERENCE TABLE

Form	Production shape	Handy fact
CNF	A → BC or A → a	A string of length n ≥ 1 needs exactly **2n − 1** derivation steps
GNF	A → aα, a a terminal, α a (possibly empty) string of variables	Every step emits one terminal, so a string of length n takes exactly **n** steps; no left recursion

FORMULA / RULE

CYK algorithm decides w ∈ L(G) for a grammar in **CNF** in **$O(n^3 \cdot |G|)$** time. Table cell (i, l) = set of variables deriving the substring of length l starting at position i; combine two shorter cells using each rule A → BC. w is accepted iff the start symbol is in the top cell.

WORKED EXAMPLE

Worked numerical — CYK

Q: G: S → AB | BC, A → BA | a, B → CC | b, C → AB | a. Is w = baaba in L(G)?

A:

```
length 1:  b:{B}   a:{A,C}   a:{A,C}   b:{B}   a:{A,C}
length 2:  ba:{A,S}   aa:{B}   ab:{C,S}   ba:{A,S}
length 3:  baa:{ }   aab:{B}   aba:{B}
length 4:  baab:{ }   aaba:{A,C,S}
length 5:  baaba:{A,C,S}   → contains S
```

S is in the top cell, so baaba ∈ L(G).

4 Pushdown Automata

What is a PDA?

NOTES

A PDA is a 7-tuple `(Q, Σ, Γ, δ, q0, Z0, F)` : states Q, input alphabet Σ, **stack alphabet Γ**, transition function δ, start state q0, initial stack symbol Z0, and final states F. Each move can read an input symbol (or ε), pop the top stack symbol, and push a string of stack symbols — all in one step.

KEY POINT

Two ways to define "accept":

1. **By final state**: the PDA ends in a state in F (stack contents don't matter).
2. **By empty stack**: the PDA empties its entire stack (regardless of state).

These two definitions are provably **equivalent** in power — any PDA of one kind can be converted to the other.

Worked example: PDA for $a^n b^n$ (n ≥ 1)

KEY POINT

```
States: q0 (pushing), q1 (popping), qf (accept)
Stack alphabet: {a, Z0}

δ(q0, a, Z0) = (q0, aZ0)     -- push first 'a' on top of
bottom marker
δ(q0, a, a)  = (q0, aa)      -- push another 'a' for each
'a' read
δ(q0, b, a)  = (q1, ε)       -- first 'b' seen: pop one 'a',
switch to popping mode
δ(q1, b, a)  = (q1, ε)       -- pop one 'a' per 'b'
δ(q1, ε, Z0) = (qf, Z0)      -- all a's matched, bottom
marker exposed → accept
```

WORKED EXAMPLE

Worked trace

Q: Trace this PDA on input "aabb" and show the stack at each step.

A:

```
Step  Input read  State  Stack (top on left)
0     -           q0     Z0
1     a           q0     aZ0
2     a           q0     aaZ0
3     b           q1     aZ0      (popped one a)
4     b           q1     Z0       (popped the other a)
5     ε           qf     Z0       (Z0 exposed, moves to
accept state)
```

Input fully consumed, ended in accepting state qf → **"aabb" is accepted**. If a 5th symbol appeared before the stack was reduced to Z0, or if a 'b' arrived while the stack top was Z0 (no 'a' left to pop), the PDA would have no valid move and reject.

CFG ⇔ PDA equivalence

COMMON TRAP

Key theorem

A language is context-free **if and only if** some PDA accepts it. Every CFG can be mechanically converted into an equivalent PDA (simulate derivations using the stack to hold the "not yet expanded" part of the sentential form), and vice versa.

Deterministic vs Non-deterministic PDA

REFERENCE TABLE

	DPDA	NPDA
Power	Recognizes only **DCFLs** (deterministic CFLs) — a strict subset	Recognizes **all** CFLs
Example that needs NPDA	—	$\{ww^R : w \in (0+1)^*\}$ (even palindromes) — the PDA must "guess" the midpoint
Closure	DCFLs are closed under complement	CFLs are **not** closed under complement or intersection

WORKED EXAMPLE

Worked numerical

Q: Is the language $\{a^n b^n : n \geq 1\}$ deterministic context-free?

A: Yes — the PDA above never had to guess anything; at every step, the next move was uniquely determined by the current input symbol and stack top. It is a **DCFL**. Contrast with even palindromes $\{ww^R\}$, where the PDA cannot know where "w" ends and "w^R" begins without guessing — that language is context-free but **not** deterministic context-free.

5 Pumping Lemma & Language Properties

Pumping Lemma for Regular Languages

KEY POINT

If L is regular, there exists a **pumping length p** such that every string w ∈ L with |w| ≥ p can be split as `w = xyz` where:

1. |xy| ≤ p 2. |y| ≥ 1 (y is non-empty) 3. $xy^iz \in L$ for every i ≥ 0 (including i=0, "un-pumping").

NOTES

It's used as a **proof by contradiction**: assume L is regular (so p exists), pick a clever string, show that *every* way of splitting it fails condition 3 — contradiction, so L isn't regular.

WORKED EXAMPLE

Worked numerical

Q: Prove L = {0^n1^n : n ≥ 0} is not regular.

A:

```
Assume L is regular. Let p = pumping length.
Choose w = 0^p 1^p  (length 2p ≥ p, and w ∈ L). ✓ valid
choice

By the lemma, w = xyz with |xy| ≤ p and |y| ≥ 1.
Since |xy| ≤ p, xy lies entirely within the first p symbols
→ entirely within the 0's block.
So y = 0^k for some k ≥ 1 (y is made only of 0's).

Pump with i = 0:  xy⁰z = x z = 0^(p-k) 1^p
This has (p-k) zeros but p ones → p-k ≠ p since k ≥ 1 → NOT
of the form 0ⁿ1ⁿ.
xy⁰z ∉ L, contradicting condition 3.

∴ L is not regular.
```

Pumping Lemma for Context-Free Languages

KEY POINT

If L is context-free, there exists p such that every w ∈ L with |w| ≥ p can be split as `w = uvxyz` where:
1. |vxy| ≤ p 2. |vy| ≥ 1 (at least one of v,y is non-empty) 3. `uvⁱxyⁱz ∈ L` for every i ≥ 0.

WORKED EXAMPLE

Worked numerical

Q: Prove L = {$a^nb^nc^n$: n ≥ 0} is not context-free.

A:

```
Assume L is context-free. Let p = pumping length.
Choose w = a^p b^p c^p  (length 3p ≥ p, w ∈ L). ✓

By the lemma, w = uvxyz with |vxy| ≤ p and |vy| ≥ 1.
Since block boundaries are p apart and |vxy| ≤ p, the
substring "vxy"
can touch AT MOST 2 of the 3 blocks (a's, b's, c's) — it's
too short to
reach into all three.

Case: vxy touches only a's (or only b's, or only c's, or
straddles two
adjacent blocks) — in every case, pumping with i=2 (uv²xy²z)
increases
the count of symbols in AT MOST 2 of the 3 letter-blocks,
while the
third block's count stays the same.
→ the counts of a's, b's, c's can no longer all be equal.
→ uv²xy²z ∉ L, contradicting condition 3 for every possible
split.

∴ L is not context-free.
```

COMMON TRAP

Alternative technique: closure properties

You can also prove $a^nb^nc^n$ is not context-free **without** the pumping lemma, using closure properties: L1={$a^nb^nc^m$: n,m≥0} and L2={$a^mb^nc^n$: n,m≥0} are both context-free (simple PDAs exist for each). If $a^nb^nc^n$ were context-free, then since `L1 ∩ L2 = {aⁿbⁿcⁿ}`, and CFLs are **not closed under intersection** in general — this alone doesn't immediately give a contradiction, but combined with the pumping lemma result above, it illustrates exactly *why* CFLs aren't closed under intersection: the intersection of two CFLs can escape the class entirely.

CFL closure properties — the full picture

REFERENCE TABLE

Operation	Closed?
Union, concatenation, Kleene star, reversal	✔ Yes
Intersection with a **regular** language	✔ Yes
Intersection of two CFLs	✘ No
Complement	✘ No

Decision properties — quick facts

REFERENCE TABLE

Question	Decidable?
Is a given CFG's language empty?	✔ Decidable
Is a given CFG ambiguous?	✘ Undecidable
Do two CFGs generate the same language?	✘ Undecidable
Do two DFAs generate the same language?	✔ Decidable

COMMON TRAP

Pattern to notice

Almost everything about **finite automata / regular languages** is decidable (they're simple enough to fully analyze). The moment you ask an equivalence/ambiguity question about **grammars or Turing machines**, it usually becomes undecidable — Section 6 explains exactly why.

Master closure table — all language classes

REFERENCE TABLE

Operation	Regular	DCFL	CFL	Recursive	RE
Union	✔	✘	✔	✔	✔
Intersection	✔	✘	✘	✔	✔
Complement	✔	✔	✘	✔	✘
Concatenation	✔	✘	✔	✔	✔
Kleene star	✔	✘	✔	✔	✔
Reversal	✔	✘	✔	✔	✔
Difference $L_1 - L_2$	✔	✘	✘	✔	✘
Intersection with a regular language	✔	✔	✔	✔	✔
Homomorphism	✔	✘	✔	✘ (✔ if ε-free)	✔
Inverse homomorphism	✔	✔	✔	✔	✔

GATE FAVOURITE

GATE angle

Two reliable anchors: **DCFL** is closed under **complement** but not union/intersection; **RE** is closed under union and intersection but **not complement** (if both L and L^c are RE, L is recursive). Context-sensitive languages are closed under complement (Immerman–Szelepcsényi).

6 Turing Machines & Undecidability

What is a Turing Machine?

NOTES

A TM is a 7-tuple `(Q, Σ, Γ, δ, q0, B, F)` : states Q, input alphabet Σ, **tape alphabet Γ** (Σ ⊆ Γ, plus a blank symbol B), transition function `δ: Q×Γ → Q×Γ×{L,R}` , start state, blank symbol, final states. Unlike a PDA's stack, the TM has an **infinite tape** it can move left or right on and overwrite freely — this extra freedom makes it the most powerful model in the hierarchy.

KEY POINT

```
Tape:   ... [B][a][a][b][b][c][c][B] ...
                  ↑
             head here, current state q0
```

WORKED EXAMPLE

Worked example: deciding $a^n b^n c^n$

Q: Describe (informally) a TM algorithm that decides L = {$a^n b^n c^n$: n≥0}.
A:

```
Repeat:
  1. Scan right, find leftmost unmarked 'a', mark it (e.g.
replace with X). If no unmarked 'a' remains and no unmarked
b/c remains either → ACCEPT.
     If no unmarked 'a' but unmarked b or c still remain →
REJECT.
  2. Continue scanning right, find leftmost unmarked 'b',
mark it. If none exists → REJECT.
  3. Continue scanning right, find leftmost unmarked 'c',
mark it. If none exists → REJECT.
  4. Return head to the start, repeat from step 1.
```

This "cross off one of each, repeat, check all crossed off together" technique is the standard TM idea for matching counts across multiple blocks — exactly why a TM can decide this language even though no PDA (Part 4's pumping lemma proof) can.

TM variants & the Church-Turing Thesis

NOTES

Multi-tape TMs, nondeterministic TMs (NTMs), and TMs with a 2-D tape are all **equally powerful** as the standard single-tape TM — they can simulate each other (only with a difference in speed/efficiency, never in what's computable). The **Church-Turing Thesis** states: anything that is "intuitively/mechanically computable" is computable by some Turing Machine. It's a thesis (a belief backed by decades of evidence), not a provable theorem.

Recursive vs Recursively Enumerable (RE)

REFERENCE TABLE

Class	Definition
Recursive (decidable)	Some TM halts on **every** input, accepting strings in L and rejecting all others
Recursively enumerable (RE / recognizable)	Some TM halts and accepts every string in L, but may **loop forever** on strings not in L

COMMON TRAP

The theorem that ties them together

A language L is **recursive** if and only if **both L and its complement are RE**. So: recursive ⊂ RE (strictly) — every decidable language is recognizable, but some recognizable languages have a complement that isn't recognizable (they're "semi-decidable" — you can confirm membership but never confirm non-membership).

The Halting Problem

NOTES

HALT = { (M, w) : Turing machine M halts on input w }. This is the most famous undecidable problem — **no algorithm can decide it in general.**

NOTE

Proof sketch (diagonalization)

```
Suppose a TM H decides HALT: H(M,w) = accept if M halts on
w, reject otherwise.

Build a new TM D using H as a subroutine:
  D(M):  run H(M, M)
         if H says "accept" (M halts on itself) → D loops
forever
         if H says "reject" (M loops on itself)  → D halts
and accepts

Now ask: does D halt on input D itself?
  If D halts on D → H(D,D) must have said "reject" (that's
the only
    way D halts) → meaning D does NOT halt on D.
Contradiction.
  If D loops on D → H(D,D) must have said "accept" → meaning
D DOES
    halt on D. Contradiction.

Either way, contradiction → no such H can exist → HALT is
undecidable.
```

This is the same self-reference trick as Cantor's diagonalization / Russell's paradox — a recurring idea across CS theory.

Rice's Theorem — the fast way to prove undecidability

MUST REMEMBER

Any **non-trivial property of the language** recognized by a TM is undecidable. ("Non-trivial" = the property is true for some RE languages and false for others — not universally true or universally false.)

WORKED EXAMPLE

Worked numerical

Q: Classify each as decidable or undecidable: (a) Is L(M) empty? (b) Is L(M) = Σ*? (c) Does M have at least 50 states? (d) Does M halt on the empty string?
A:

```
(a) Is L(M) empty?          → UNDECIDABLE (non-trivial
property of L(M), by Rice's theorem)
(b) Is L(M) = Σ*?           → UNDECIDABLE (non-trivial
property of L(M), by Rice's theorem)
(c) Does M have ≥50 states?  → DECIDABLE (this is a property
of M's own description/structure,
                                NOT a property of the
language it recognizes — just count the
                                states listed. Rice's
theorem does not apply here.)
(d) Does M halt on ε?        → UNDECIDABLE (a specific
instance of the Halting Problem)
```

GATE FAVOURITE

GATE trap

Rice's theorem only applies to properties **of the language L(M)** — never to properties of the machine's own textual description (number of states, number of transitions, whether it uses a particular symbol). Those syntactic questions are usually trivially decidable — just read the machine's description.

Proving undecidability by reduction

NOTES

To prove a new problem P is undecidable: assume a decider for P exists, then use it to build a decider for a **known** undecidable problem (usually HALT) — a contradiction.

WORKED EXAMPLE

Worked numerical

Q: Prove `EMPTY = {M : L(M) = ∅}` is undecidable, by reduction from HALT.

A:

```
Suppose a decider R for EMPTY exists.
Given any instance (M, w) of HALT, construct M':
  M'(x): ignore x, simulate M on w.
         If M halts on w → M' accepts x. Else M' loops
forever.

L(M') = Σ* if M halts on w, or L(M') = ∅ if M never halts on
w.
So: M halts on w  ⇔  L(M') ≠ ∅  ⇔  R(M') says "not empty".

This means R could be used to decide HALT — but HALT is
undecidable.
Contradiction → no such R exists → EMPTY is undecidable.
(This is just Rice's theorem applied directly, worked out by
hand.)
```

Designing a TM — worked construction

KEY POINT

L = {$a^n b^n$: n ≥ 1}. Idea: mark the leftmost unmarked a with X, run right to the first unmarked b and mark it Y, return to the X, repeat; accept when only Y's remain.

```
δ(q0,a)=(q1,X,R)    δ(q0,Y)=(q3,Y,R)                    q0 =
find next a (or finish)
δ(q1,a)=(q1,a,R)    δ(q1,Y)=(q1,Y,R)    δ(q1,b)=(q2,Y,L)
q1 = scan right for a b
δ(q2,a)=(q2,a,L)    δ(q2,Y)=(q2,Y,L)    δ(q2,X)=(q0,X,R)
q2 = return to the X
δ(q3,Y)=(q3,Y,R)    δ(q3,□)=(qacc,□,R)
q3 = verify only Y's remain
(every missing transition = reject)
```

Running time is Θ(n^2): the machine accepts $a^n b^n$ in exactly **$2n^2 + 2n + 1$** moves (ab → 5, aabb → 13, aaabbb → 25).

Universal TM, enumerators & PCP

KEY POINT

Universal TM (UTM): takes ⟨M, w⟩ as input and simulates M on w; its language A_{TM} is **RE but not recursive**, and its complement is not RE.
Enumerator: a TM that prints strings; a language is RE ⇔ some enumerator prints exactly its strings.
Post Correspondence Problem (PCP): given dominoes (x_i / y_i), is there a sequence $i_1 \ldots i_k$ with $x_{i1} \ldots x_{ik} = y_{i1} \ldots y_{ik}$? **Undecidable** (for alphabets of size ≥ 2). It is the standard tool for showing that CFG ambiguity and "$L(G_1) \cap L(G_2) = \emptyset$" are undecidable.

```
Dominoes: 1 = a/ab, 2 = b/ca, 3 = ca/a, 4 = abc/c.
Sequence 1,2,3,1,4:
top    = a · b · ca · a · abc = abcaaabc
bottom = ab · ca · a · ab · c = abcaaabc   ✓ match
```

KEY POINT

Linear Bounded Automata / CSLs: every CSL is **decidable** (recursive), so CSL ⊊ Recursive ⊊ RE. For an LBA, *"does M accept w?"* is decidable, but *"is L(M) empty?"* is undecidable.

The Chomsky Hierarchy — the full picture

REFERENCE TABLE

Type	Grammar	Recognized by	Language class
Type 3	Regular grammar	DFA / NFA	Regular languages
Type 2	Context-free grammar	PDA	Context-free languages
Type 1	Context-sensitive grammar	Linear Bounded Automaton (LBA)	Context-sensitive languages
Type 0	Unrestricted grammar	Turing Machine	Recursively enumerable

NOTES

Each class is a **strict subset** of the next: `Regular ⊊ Context-Free ⊊ Context-Sensitive ⊊ Recursively-Enumerable`. This single hierarchy is the backbone connecting every part of this site — worth memorizing cold.

7 Problem → Standard Method Playbook

METHOD

Moore vs Mealy — output questions

Recognition: the question gives a finite-state machine and asks about output timing, number of states, or conversion.

- **Moore:** output depends only on the current state; output changes after entering a state. For n states, at most n distinct state outputs.
- **Mealy:** output depends on current state and input; output is attached to transitions. A Mealy machine can often implement the same behaviour with fewer states.
- For Mealy → Moore, split a state when different incoming transitions require different output labels; this can increase the state count.

METHOD

CFG ambiguity — standard test

Recognition: the question asks whether a grammar is ambiguous or asks for two derivations of a string.

1. Find a candidate string that admits two distinct parse trees.
2. Prefer short strings generated through two competing productions, such as expression grammars without precedence rules.
3. Show two distinct **leftmost derivations** or two parse trees. Two different rightmost derivations alone are not sufficient unless they correspond to different parse trees.

GATE trap

Ambiguity is a property of a grammar, not merely of a language. A language can be inherently ambiguous, but a particular ambiguous grammar may have an equivalent unambiguous grammar.

METHOD

CYK parsing — recognition method

CYK requires the grammar in **Chomsky Normal Form** (plus the usual handling of ε). For a string of length n, build a triangular table. Cell T[i,j] stores variables that derive substring w[i…j].

1. Length 1: put variables generating the terminal.
2. For each longer span, try every split k.
3. If B ∈ T[i,k] and C ∈ T[k+1,j], add A whenever A → BC.
4. Accept iff the start variable is in the full-span cell.

Standard time complexity: **O(n³\|G\|)** for a fixed grammar, commonly stated as O(n³).

METHOD

Greibach Normal Form — conversion anchor

GNF productions have the form `A → aα`, where `a` is a terminal and α is a (possibly empty) sequence of variables. Consequently, a derivation of a non-empty string of length n uses exactly **n production steps** in GNF.

Do not confuse this with CNF: CNF uses `A → BC` or `A → a` and is convenient for CYK; GNF starts every production with a terminal and is useful for PDA/derivation reasoning.

METHOD

TM construction — marking / matching template

For languages such as `{aⁿbⁿ}`, `{aⁿbⁿcⁿ}`, or equal-count constraints:

1. Mark one unmatched symbol from the left block.
2. Scan right to find and mark its matching symbol in the next block.
3. Return to the left and repeat.
4. When the source block is exhausted, make a final pass to verify that no unmatched symbols remain.
5. Reject immediately if a required matching symbol is missing or appears in the wrong order.

WORKED EXAMPLE

Worked problem — CYK

Grammar: S → AB | BC, A → BA | a, B → CC | b, C → AB | a. Determine whether `ba` is generated.

```
Length 1:
T[1,1] for 'b' = {B}
T[2,2] for 'a' = {A,C}

Length 2, split after position 1:
(B, A) gives no S because neither S→BA nor S→BC with
second=A.
(B, C) gives S→BC, so S ∈ T[1,2].
Therefore 'ba' ∈ L(G).
```

WORKED EXAMPLE

Worked problem — Moore/Mealy state reasoning

Question: A Mealy machine has 4 states and is converted to a Moore machine by splitting states according to distinct output labels on incoming transitions. If the four states have respectively 1, 2, 1, and 3 distinct incoming output labels, what is the resulting Moore state count before unreachable-state removal?

```
Each Mealy state becomes one Moore state per distinct
incoming output label:
1 + 2 + 1 + 3 = 7 states.
Answer: 7 before removing unreachable states.
```

8 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **CNF:** |w| = n takes 2n−1 steps. GNF: n steps. CYK needs CNF and runs in O(n³).

LAST-MINUTE · MUST REMEMBER

- **Moore:** output is associated with the STATE; Mealy: output is associated with the TRANSITION (state + input). Mealy→Moore can add states. Never fix an output count from the machine type alone — it depends on the question's output convention (is the start state's output printed?) and the input length.
- PCP is undecidable; it proves CFG ambiguity and CFG-intersection emptiness undecidable. LBA acceptance is decidable, LBA emptiness is not.
- DFA and NFA recognise exactly the regular languages; nondeterminism does not increase expressive power here.
- Every DFA is an NFA, but an NFA can be exponentially more compact than an equivalent DFA.
- Regular languages are closed under union, intersection, complement, concatenation, Kleene star, and difference.
- CFGs generate context-free languages; parse trees and derivations describe how strings are generated.
- PDA recognises exactly the context-free languages under the standard acceptance model.
- Regular languages are a proper subset of context-free languages; e.g. $\{a^n b^n \mid n \geq 0\}$ is CFL but not regular.
- For CFL pumping lemma, the pumped portions are constrained to lie within a bounded window around a decomposition of the string.
- A Turing machine is more powerful than DFA/PDA and can model general algorithmic computation.
- GNF productions begin with a terminal; a length-n nonempty string has an n-step leftmost derivation in GNF.
- **TM matching template:** mark one symbol, scan for its mate, return, repeat, then verify no leftovers.

LAST-MINUTE · COMMON TRAPS

- Regular closure covers FINITE operations only — infinite unions and arbitrary subsets of regular languages need not be regular.
- Closure under an operation does not mean a particular construction is identical for every representation; use the appropriate automaton/product/complement construction.
- Pumping lemma is mainly a tool to prove non-membership/non-regularity; satisfying the pumping lemma does not by itself prove regularity.
- **Reduction direction matters:** to prove B hard/undecidable using known-hard A, reduce A to B.
- Ambiguous grammar ≠ inherently ambiguous language; prove grammar ambiguity by two distinct parse trees/leftmost derivations for one string.

LAST-MINUTE · SHORTCUTS

- **CFG simplification order:** ε-productions → unit productions → useless symbols.

LAST-MINUTE · GATE FAVOURITES

- **DCFL:** closed under complement and inverse homomorphism; NOT union, intersection, concatenation, star, reversal.
- Ambiguous grammar means some string has more than one parse tree/leftmost derivation/rightmost derivation.
- Decidable means some TM halts on every input and answers correctly; recognizable/RE only requires halting and accepting members.
- **CYK:** convert/assume CNF, fill substrings bottom-up, and accept iff start variable appears in the full-span cell.

Computer Networks

GATE CSE 2027 · complete formula, rule, method & trap sheet

101 reference boxes · **9** chapters ·

Principles of Layering | Switching & Performance Metrics | Data Link Layer & MAC | Routing Algorithms | IPv4: Fragmentation, CIDR & NAT | TCP: Flow & Congestion Control | DNS & HTTP | Problem → Standard Method Playbook | Last-minute one-liners

1 Principles of Layering

Why layer at all? Core

NOTES

A network has to solve many different problems at once: turning bits into electrical signals, getting a frame across one wire, routing across the globe, making delivery reliable, and letting an app request a web page. Layering splits this into independent jobs — each layer offers a clean **service** to the layer above and uses services from the layer below, without needing to know how those services are implemented internally.

The TCP/IP stack (5 layers) Core

FLOW

Application → Transport → Network → Data Link → Physical

REFERENCE TABLE

Layer	Job	Data unit (PDU)	Example
Application	What the user's program actually wants to say	Message	HTTP request, DNS query
Transport	End-to-end delivery between processes (ports)	Segment (TCP) / Datagram (UDP)	TCP, UDP
Network	Routing across many networks, host-to-host	Packet (IP datagram)	IP, routers
Data Link	Reliable transfer across ONE link, framing, MAC addressing	Frame	Ethernet, Wi-Fi
Physical	Raw bits over a physical medium	Bits/symbols	Cables, radio signals

GATE FAVOURITE

GATE note

The classic 7-layer **OSI model** (with separate Session and Presentation layers) is still useful background, but the current syllabus focuses on the practical **TCP/IP 5-layer model** actually used on the real internet. Know that OSI's Session+Presentation+Application roughly collapse into TCP/IP's single Application layer.

Encapsulation & decapsulation Core

NOTES

As data goes DOWN the stack at the sender, each layer wraps the data from above inside its own header (and sometimes trailer) — this is **encapsulation**. At the receiver, data goes UP the stack and each layer strips off its own header — **decapsulation**.

KEY POINT

```
App data
  → [TCP header | App data]                    = Segment
    → [IP header | TCP header | App data]       = Packet
      → [Ethernet header | IP header | ... | Ethernet
trailer] = Frame
        → 1011010010110...                     = Bits on
the wire
```

Each layer only reads/writes its OWN header — it treats everything from the layer above as opaque data (this is exactly what "layering" buys you: a router only looks at the IP header, never at the TCP or application data inside).

WORKED EXAMPLE

Worked numerical

Q: An application sends 1000 bytes of data. TCP adds a 20-byte header, IP adds a 20-byte header, and Ethernet adds an 18-byte header+trailer. What is the total frame size placed on the wire, and what is the header overhead as a percentage?

A: Total = 1000 + 20 + 20 + 18 = **1058 bytes**. Overhead = 58 bytes → 58/1058 ≈ **5.5%** of the frame is pure header/trailer overhead.

Header sizes, MTU & MSS — the overhead arithmetic Core

REFERENCE TABLE

Layer	Header (typical)	Notes
TCP	20 B (up to 60 B with options)	UDP header is a fixed 8 B
IP (IPv4)	20 B (up to 60 B)	IHL counts 4-byte words; min 5
Ethernet	14 B header + 4 B CRC = 18 B	Payload 46–1500 B; **MTU = 1500 B**

KEY POINT

```
MSS = MTU − IP header − TCP header = 1500 − 20 − 20 = 1460
bytes of application data per segment
Efficiency = useful data / total bytes on the wire
Switches work up to layer 2 · routers up to layer 3 · hubs /
repeaters at layer 1
```

2 Switching & Performance Metrics

Three switching techniques Core

REFERENCE TABLE

Type	How it works	Pros	Cons
Circuit switching	A dedicated end-to-end path is reserved before data flows (e.g. old telephone network)	Guaranteed bandwidth, predictable delay once set up	Setup/teardown time; wastes capacity if the link sits idle
Packet switching	Data is split into packets, each routed independently ("store-and-forward"), no reservation	Efficient — links shared via statistical multiplexing	Variable, unpredictable delay (congestion, queuing)
Virtual circuit switching	A logical path is set up once (like circuit switching), but data still travels as separate packets along it	Combines fast, ordered forwarding with statistical sharing	Still needs setup signaling; used in ATM, MPLS, Frame Relay

Delay components — memorize these four Core

KEY POINT

```
Transmission delay = L / B      (L = packet size in bits, B = link bandwidth in bits/sec)
Propagation delay  = d / v      (d = distance, v = signal propagation speed in the medium)
Queuing delay      = time waiting in a router's buffer before transmission
Processing delay   = time a router/switch takes to examine the packet header
```

Total end-to-end delay for ONE hop = Transmission + Propagation + Queuing + Processing.

WORKED EXAMPLE

Worked numerical

Q: A 10 Mbps link carries a 5000-bit packet over a 2000 km cable where the signal travels at 2×10^8 m/s. Find the transmission delay, propagation delay, and total delay (ignore queuing/processing).
A:

```
Transmission delay = 5000 bits / (10×10⁶ bps) = 0.5 ms
Propagation delay  = (2000×1000 m) / (2×10⁸ m/s) = 10 ms
Total delay = 0.5 + 10 = 10.5 ms
```

Notice propagation dominates over long distances even at decent bandwidth — a classic GATE observation.

Store-and-forward delay across multiple hops (packet switching) Core

NOTES

In pure packet switching, a router must receive the **entire packet** before it can start forwarding it to the next hop (it can't forward partial packets).

FORMULA / RULE

Worked numerical

Q: A packet of L=6400 bits travels through 3 links (2 intermediate routers) to reach its destination, each link having bandwidth B=1 Mbps. Ignoring propagation delay, find the total time for the packet to reach the destination.
A: Each hop needs a full transmission time of L/B = 6400/(1×10⁶) = 6.4 ms before the NEXT hop can even begin (store-and-forward).

```
Total time (ignoring propagation) = number_of_links × (L/B) = 3 × 6.4 ms = 19.2 ms
```

General formula: for n links (n−1 intermediate routers), **Total ≈ n × (L/B) + Σ(propagation delays)**.

WORKED EXAMPLE

Worked numerical — circuit switching comparison

Q: The same data (6400 bits) is sent using circuit switching over the same 3-link path (B=1 Mbps per link), with a call setup time of 15 ms and negligible propagation delay. Compare total time to the packet-switched case above.
A: Once the circuit is set up, data flows straight through at the bottleneck rate without per-hop store-and-forward waiting:

```
Circuit switching total = setup time + transmission time = 15 + 6.4 = 21.4 ms
Packet switching total (from above) = 19.2 ms
```

Here packet switching is actually *faster* for a single message, because circuit switching pays a big upfront setup cost — this trade-off (setup overhead vs. per-hop store-and-forward overhead) is exactly what GATE likes to compare.

Bandwidth-Delay Product Core

KEY POINT

```
Bandwidth-Delay Product (BDP) = Bandwidth × RTT
```

This is the amount of data that can be "in flight" on the link at any instant — the size of the "pipe". It tells you the minimum window size (Part 3) needed to keep a link 100% utilized.

WORKED EXAMPLE

Worked numerical

Q: A link has bandwidth 100 Mbps and RTT 40 ms. Find the bandwidth-delay product, and the minimum window size (in bytes) needed to keep the link fully utilized.
A:

```
BDP = 100×10⁶ bps × 40×10⁻³ s = 4×10⁶ bits = 4,000,000 bits
    = 4,000,000 / 8 = 500,000 bytes = 500 KB
```

The sender needs a window of at least **500 KB** "in flight" at all times, or the link will sit idle waiting for acknowledgments — this is exactly why TCP's window size matters so much on high-bandwidth, high-latency ("long fat") links.

Multiplexing — sharing one link High-yield

REFERENCE TABLE

Technique	How the link is shared	Note
FDM (frequency-division)	Each channel gets its own frequency band	Guard bands waste bandwidth; analogue-style sharing
WDM	FDM for light: each channel uses its own wavelength on one fibre	Number of channels = fibre bandwidth / channel spacing
Synchronous TDM	Fixed time slots, assigned in rotation to each input whether or not it has data	An idle owner wastes its slot; used in circuit switching
Statistical TDM	Slots go only to inputs that have data (packet switching)	Better utilisation, but needs addressing and gives variable delay

WORKED EXAMPLE

Worked numerical

Q: A T1 line multiplexes 24 voice channels of 64 kbps. What is its line rate?
A: Each frame carries 24 × 8 = 192 data bits plus 1 framing bit = **193 bits**, sent 8000 times per second: 193 × 8000 = **1.544 Mbps**. The payload alone is 24 × 64 kbps = 1.536 Mbps; the extra 8 kbps is framing overhead.

3 Data Link Layer & MAC

Error detection Core

REFERENCE TABLE

Method	Idea	Catches
Parity bit	One extra bit makes the total number of 1's even (or odd)	Any single-bit error; misses even numbers of bit-flips
Checksum	Sum the data in fixed-size blocks, send the complement	Cheap, used in IP/UDP/TCP headers; weaker than CRC
CRC (Cyclic Redundancy Check)	Treat data as a binary polynomial, divide by a fixed generator polynomial (mod-2 division), send the remainder	Very strong — catches all burst errors up to the length of the remainder

WORKED EXAMPLE

Worked numerical — CRC

Q: Data = 1101011011, Generator = 10011 (a 4-bit CRC, so append 4 zeros). Find the CRC remainder and the transmitted frame.
A:

```
Message with 4 zeros appended: 11010110110000  (14 bits)
Divide by 10011 using mod-2 (XOR) long division — XOR with the generator
whenever the leading bit of the current window is 1, otherwise just shift:

 11010 (leading 1, XOR)         01001
  10011                   →
  -----
  01001

 → bring down next bit, window=10011 (leading 1, XOR)   → 00000
 → bring down next bit, window=00001 (leading 0, shift)  → 00001
 → bring down next bit, window=00010 (leading 0, shift)  → 00010
 → bring down next bit, window=00101 (leading 0, shift)  → 00101
 → bring down next bit, window=01011 (leading 0, shift)  → 01011
 → bring down next bit, window=10110 (leading 1, XOR)    → 00101
 → bring down next bit, window=01010 (leading 0, shift)  → 01010
 → bring down next bit, window=10100 (leading 1, XOR)    → 00111
 → bring down last bit, window=01110 (leading 0, shift)  → 01110

All 14 bits consumed. Remainder = last 4 bits of the final window = 1110
```

Transmitted frame = original data + remainder = `1101011011 1110` The receiver divides the received frame by the same generator; a **zero remainder** means no error was detected. (GATE typically gives you the full division to perform by hand — practice mod-2 long division carefully, it's just repeated XOR, no carries.)

Framing — marking where a frame starts and ends High-yield

REFERENCE TABLE

Method	How it works
Character (byte) count	A length field starts the frame — one corrupted count desynchronises everything after it
Byte stuffing	Frames delimited by FLAG (0x7E). If FLAG or ESC (0x7D) appears in the data, the sender inserts an ESC byte before it
Bit stuffing (HDLC)	Flag = `01111110`. After **five consecutive 1s** in the data, the sender inserts a **0**; the receiver deletes a 0 that follows five 1s

WORKED EXAMPLE

Worked numerical

Q: Bit-stuff the data `0111110111111011111`. Byte-stuff the data 7E 41 7D 42 (FLAG = 7E, ESC = 7D).
A:

```
Bit stuffing — insert 0 after every run of five 1s:
  0 11111 [0] 0 11111 [0] 1 0 11111 [0]   →
01111100111110101111110   (19 bits become 22 bits)
Byte stuffing — ESC before every FLAG/ESC byte in the data:
  7E 41 7D 42  →  7D 7E 41 7D 7D 42      (4 bytes become 6 bytes)
```

Stuffing overhead depends on the data: all-ones data gets one extra bit per five.

Medium Access Control (MAC) — who transmits when? Core

ALOHA

KEY POINT

Pure ALOHA
Transmit whenever you have data — no coordination at all. Throughput: `S = G·e^(-2G)`. Maximum at G=0.5: **S_max ≈ 18.4%**.

KEY POINT

Slotted ALOHA
Time is divided into slots; transmit only at slot boundaries. Throughput: `S = G·e^(-G)`. Maximum at G=1: **S_max ≈ 36.8%**.

WORKED EXAMPLE

Worked numerical

Q: In a slotted ALOHA system, the offered load G = 2. Find the throughput S.
A: S = G·e^(-G) = 2·e^(-2) = 2 × 0.1353 ≈ **0.271** (27.1% — notice this is LOWER than the peak at G=1, since too many stations are now colliding).

CSMA / CSMA-CD / CSMA-CA

REFERENCE TABLE

Protocol	Rule	Used in
CSMA (Carrier Sense)	Listen before transmitting ("listen before talk")	Basic wired LANs
CSMA/CD	Listen while transmitting; if a collision is detected, abort immediately and back off	Classic (wired) Ethernet
CSMA/CA	Can't reliably detect collisions (wireless), so try to AVOID them — random backoff before every transmission, optional RTS/CTS handshake	Wi-Fi (802.11)

COMMON TRAP

Why CSMA/CD needs a MINIMUM frame size
A transmitting station must still be sending when a collision's signal comes back, or it will never know the collision happened. This forces: `Transmission time ≥ 2 × Propagation time`.

WORKED EXAMPLE

Worked numerical — minimum Ethernet frame size

Q: A CSMA/CD LAN has bandwidth 10 Mbps, and the maximum distance between two stations is 2500 m, with signal speed 2×10^8 m/s. Find the minimum frame size needed.
A:

```
Propagation delay (one way) = 2500 / (2×10⁸) = 12.5 µs
Worst case: collision signal must travel there AND back → 2 × Tprop = 25 µs

Minimum transmission time = 2 × Tprop = 25 µs
Minimum frame size = Bandwidth × Minimum transmission time
                   = 10×10⁶ bps × 25×10⁻⁶ s = 250 bits ≈ 32 bytes
```

(Real Ethernet uses extra safety margin, giving the well-known 64-byte minimum frame size — but the derivation logic is exactly this "2 × Tprop" rule.)

Ethernet framing & addressing Core

KEY POINT

```
| Preamble | Dest MAC (6B) | Src MAC (6B) | Type (2B) | Data (46-1500B) | CRC (4B) |
```

Each device has a globally unique **48-bit MAC address** burned into its NIC. A **switch** forwards frames only out the port leading to the destination MAC (learned by watching source addresses), unlike a **hub** which blindly repeats to every port — this is why switches eliminate collisions entirely (each switch port is its own collision domain).

Bridges & learning switches High-yield

NOTES

A **transparent (learning) bridge** or switch builds a table **MAC address → port** automatically by looking at the **source** address of every arriving frame. For each frame it then decides based on the **destination**:

REFERENCE TABLE

Destination	Action
Known, on a **different** port	**Forward** only out that port
Known, on the **same** port the frame arrived on	**Filter** (discard) — the destination already saw it
Unknown unicast, or broadcast / multicast	**Flood** out every port except the arrival port

KEY POINT

Table entries age out (typically ~5 min) so moved hosts are relearned. Redundant links in a bridged LAN create loops and broadcast storms; the **Spanning Tree Protocol (STP)** blocks some ports so the active topology is a tree rooted at the bridge with the lowest ID.

Device	Collision domains	Broadcast domains
Hub	1 (all ports)	1
Bridge / switch with n ports	**n** (one per port)	1 (unless VLANs are used)
Router with n interfaces	n	**n**

Flow control at the link layer — sliding window protocols High-yield

NOTES

Let T_t = frame transmission time (L/B) and T_p = propagation delay, and define **$a = T_p / T_t$**. A sender that waits for each ACK idles for a whole round trip ($2T_p$), so keeping the link busy needs several frames in flight.

REFERENCE TABLE

Protocol	Sender / receiver window	Efficiency (utilisation)	Sequence numbers (k bits)	After a lost frame
Stop-and-wait	1 / 1	$1 / (1 + 2a)$	1 bit (0, 1)	Resend that frame after the timeout
Go-Back-N	N / 1	$N / (1 + 2a)$, capped at 1	$N \le 2^k - 1$	Resend the lost frame **and every later one**; receiver discards out-of-order frames
Selective Repeat	N / N	$N / (1 + 2a)$, capped at 1	$N \le 2^{k-1}$ (sequence space ≥ 2N)	Resend **only** the lost frame; receiver buffers out-of-order frames

KEY POINT

The window that keeps the pipe exactly full is **1 + 2a** frames. Efficiency can never exceed 1, so for $N \ge 1 + 2a$ the utilisation is 100%. If a question gives the round-trip time directly: efficiency = $N \cdot T_t / (T_t + RTT)$. ACK transmission time is ignored unless the question says otherwise.

WORKED EXAMPLE

Worked numerical

Q: A 1 Mbps link, 200 km long (signal speed 2×10^8 m/s), carries 1000-bit frames. Find the stop-and-wait efficiency and throughput, the window needed for full utilisation, and the sequence-number bits needed for Go-Back-N and for Selective Repeat at that window.
A: T_t = 1000 / 10^6 = 1 ms; T_p = 200 000 / (2×10^8) = 1 ms, so a = 1.

```
Stop-and-wait: η = 1/(1+2a) = 1/3 = 33.3%  →  throughput = 1 Mbps × 1/3 ≈ 333 kbps
Window for 100% = 1 + 2a = 3 frames
Go-Back-N : N + 1 = 4 sequence numbers  →  2 bits
Selective Repeat: 2N = 6 sequence numbers → 3 bits
```

WORKED EXAMPLE

Worked numerical

Q: T_t = 1 ms and T_p = 5 ms. What is the efficiency of Go-Back-N with N = 7?
A: a = 5, so 1 + 2a = 11. Efficiency = 7 / 11 ≈ **63.6%**. With N = 11 or more it would reach 100%.

GATE FAVOURITE

GATE trap

With **3-bit** sequence numbers Go-Back-N can use a window of **7**, but Selective Repeat only **4**. If the Selective Repeat window equals the whole sequence space, a retransmitted old frame cannot be told apart from a new one.

Error correction — Hamming code Extension

NOTES

For m data bits, r check bits are needed so that $2^r \ge m + r + 1$: m = 4 → r = 3 (the (7,4) code); m = 7 or 8 → r = 4. Check bits sit at the **power-of-two positions** 1, 2, 4, 8, … and each one gives even parity over the positions whose binary index contains that bit. After a single-bit error, the **syndrome** (the failing checks read as a binary number) equals the **position of the flipped bit**. In terms of **minimum Hamming distance d**: a code detects up to d − 1 errors and corrects up to ⌊(d − 1)/2⌋ (the (7,4) code has d = 3).

WORKED EXAMPLE

Worked numerical

Q: Encode data bits 1 0 1 1 with the (7,4) Hamming code (even parity), then locate the error if the received word is 0110111.
A: Positions 1…7 are p1 p2 d3 p4 d5 d6 d7, with d3 d5 d6 d7 = 1 0 1 1.

```
p1 covers 1,3,5,7: 1⊕0⊕1 = 0      p2 covers 2,3,6,7: 1⊕1⊕1 = 1      p4 covers 4,5,6,7: 0⊕1⊕1 = 0
Codeword = 0 1 1 0 0 1 1  (0110011)
Received 0110111: checks p1 → 1 (fails), p2 → 0 (ok), p4 → 1 (fails)
Syndrome = (p4 p2 p1) = 101 = 5  →  bit 5 flipped; correct it to get 0110011.
```

Channel capacity and line coding Extension

REFERENCE TABLE

Law	Formula	Use when
Nyquist	$C = 2\,B\,\log_2 L$ bps	Noiseless channel, L signal levels, bandwidth B Hz
Shannon	$C = B\,\log_2(1 + S/N)$ bps	Noisy channel; this is the **upper limit** whatever the signal levels

NOTES

Convert decibels first: $S/N = 10^{(SNR_{dB}/10)}$, so 30 dB means S/N = 1000. When a problem gives both limits, the achievable rate is the **smaller** of the two.

WORKED EXAMPLE

Worked numerical

Q: A 3 kHz telephone line has a 30 dB signal-to-noise ratio. What is its maximum data rate? What does Nyquist give for a noiseless 4 kHz channel with 8 levels?
A: S/N = 10^3 = 1000, so Shannon: C = 3000 × $\log_2(1001)$ ≈ 3000 × 9.97 ≈ **29.9 kbps**. Nyquist: C = 2 × 4000 × $\log_2$ 8 = 2 × 4000 × 3 = **24 kbps**.

KEY POINT

Line coding. Manchester encoding puts a transition in the middle of **every** bit and the **direction** of that transition carries the bit value (textbooks differ on which direction means 1, so follow the convention the question states), so the receiver can recover the clock, but the **signalling rate is twice the bit rate** — 10 Mbps Ethernet used a 20 Mbaud signal. NRZ uses one signal level per bit (baud rate = bit rate) but long runs of equal bits lose synchronisation.

4 Routing Algorithms

Modeling the network as a graph Core

NOTES

Routers = nodes, links = edges, link costs = edge weights (could be based on delay, hop count, or bandwidth). Both algorithms below try to find the **shortest-cost path** from every router to every other router.

Distance Vector Routing (Bellman-Ford based) Core

NOTES

Each router only knows the cost to its **direct neighbors**. It periodically shares its entire distance table with its neighbors, and each router updates its own table using the **Bellman-Ford equation**:

KEY POINT

```
D_x(y) = min over all neighbors v of { c(x,v) + D_v(y) }
```

"My best cost to Y = my link cost to neighbor V, plus V's own best-known cost to Y — minimized over every neighbor V."

WORKED EXAMPLE

Worked numerical

Q: 3 routers A, B, C. Direct link costs: A-B=2, B-C=1, A-C=7. Using distance vector routing, what does A's table converge to for reaching C?
A:

```
Initial: A's table — A-B=2, A-C=7 (direct link only)
B shares its table with A: B says "my cost to C = 1"
A recomputes: D_A(C) = min( c(A,C)=7 ,  c(A,B)+D_B(C) =
2+1=3 ) = min(7,3) = 3

A updates its table: cost to C via B = 3 (better than the
direct 7-cost link)
Next hop for C, from A's table: via B
```

This is exactly how distance vector routing "discovers" that going through B is cheaper than A's own direct link to C — information propagates hop by hop until everyone converges.

COMMON TRAP

Count-to-infinity problem

If a link fails, bad news (increased cost) travels much slower than good news, because routers can keep telling each other stale, now-wrong "shortcuts" back and forth, incrementing the cost by tiny amounts each round instead of realizing the path is broken. Fixes: **split horizon** (don't advertise a route back to the neighbor you learned it from) and **poison reverse**.

Link State Routing (Dijkstra based) Core

NOTES

Every router **floods** its direct link costs to the ENTIRE network (not just neighbors), so every router ends up with a full map of the topology. Each router then independently runs **Dijkstra's algorithm** on this map to compute shortest paths to everyone.

WORKED EXAMPLE

Worked numerical — Dijkstra's trace

Q: Graph: A-B=4, A-C=1, C-B=2, C-D=5, B-D=1. Find shortest paths from A to every other node.
A:

```
Initialize: dist[A]=0, dist[B]=∞, dist[C]=∞, dist[D]=∞.
Visited={}

Round 1: pick unvisited min-dist node = A (dist 0). Visit A.
  Update neighbors: dist[B]=min(∞,0+4)=4,
dist[C]=min(∞,0+1)=1

Round 2: pick unvisited min-dist = C (dist 1). Visit C.
  Update neighbors: dist[B]=min(4, 1+2=3)=3 (better via C!)
                    dist[D]=min(∞, 1+5=6)=6

Round 3: pick unvisited min-dist = B (dist 3). Visit B.
  Update neighbors: dist[D]=min(6, 3+1=4)=4 (better via B!)

Round 4: pick unvisited min-dist = D (dist 4). Visit D.
Done.

Final shortest distances from A:  B=3 (path A-C-B),  C=1
(direct),  D=4 (path A-C-B-D)
```

Distance Vector vs Link State

REFERENCE TABLE

	Distance Vector	Link State
Info shared	Entire distance table, only to neighbors	Only own link costs, flooded to everyone
Each router knows	Only costs, not the full topology	The full network topology
Algorithm used locally	Bellman-Ford (distributed)	Dijkstra (run locally by each router)
Convergence issue	Count-to-infinity	Faster, more robust convergence, but more messaging/flooding overhead

Real routing protocols — RIP, OSPF, BGP Extension

REFERENCE TABLE

	RIP	OSPF	BGP
Scope	Interior (within an AS)	Interior (within an AS)	**Exterior** (between ASes)
Algorithm	Distance vector (Bellman-Ford)	Link state (Dijkstra)	**Path vector**
Metric	Hop count; **16 = infinity** (max usable 15)	Link cost (e.g. based on bandwidth)	Policy-based; AS-path length is one factor
Transport	UDP, port 520	Directly over IP (protocol 89)	**TCP**, port 179
Updates	Whole table to neighbours every ~30 s	LSAs flooded on change; hello packets	Incremental updates over a persistent session

KEY POINT

• RIP limits the network diameter to 15 hops precisely so that count-to-infinity terminates quickly (and uses split horizon / poison reverse).
• OSPF splits a big AS into **areas** joined by the backbone area 0, reducing flooding; it converges faster than RIP.
• BGP carries the full **AS path**, so a router rejects any route whose path already contains its own AS — this prevents loops without count-to-infinity. Routing policy (business relationships) matters more than shortest path.

5 IPv4: Fragmentation, CIDR & NAT

Fragmentation Core

NOTES

Every link has a **Maximum Transmission Unit (MTU)** — the largest frame it can carry. If a datagram is bigger than the next link's MTU, a router splits it into **fragments**. Each fragment gets its own IP header, with three key fields: **Identification** (same for all fragments of one datagram), **Fragment Offset** (position of this fragment's data, in units of **8 bytes**), and the **MF (More Fragments) flag** (1 for all fragments except the last).

WORKED EXAMPLE

Worked numerical

Q: A 4000-byte IP datagram (20-byte header + 3980-byte data) must cross a link with MTU = 1500 bytes. How many fragments are needed, and what is each fragment's offset?

A:

```
Data per fragment (excluding the 20-byte header each
fragment needs) must be
a multiple of 8 bytes, and at most 1500-20=1480 bytes → use
1480 bytes/fragment.

Total data to carry = 3980 bytes
Fragment 1: data bytes 0-1479    (1480 bytes) → offset = 0/8
= 0,      MF=1
Fragment 2: data bytes 1480-2959 (1480 bytes) → offset =
1480/8 = 185, MF=1
Fragment 3: data bytes 2960-3979 (1020 bytes) → offset =
2960/8 = 370, MF=0 (last)

Total fragments = 3
```

Each fragment is now (20-byte header + its data) ≤ 1500 bytes, satisfying the MTU.

CIDR (Classless Inter-Domain Routing) Core

NOTES

Instead of rigid class A/B/C blocks, CIDR writes an address as `a.b.c.d/n`, where **n** is the number of leading bits that form the network portion (the "subnet mask" length).

KEY POINT

```
Number of host addresses in a /n block = 2^(32-n)
Usable hosts (excluding network & broadcast addresses) =
2^(32-n) - 2
```

WORKED EXAMPLE

Worked numerical

Q: Given the block 192.168.10.0/26, find the network address, broadcast address, and number of usable hosts.

A:

```
/26 means 26 network bits, 32-26 = 6 host bits → 2^6 = 64
total addresses
Network address = 192.168.10.0     (all host bits = 0)
Broadcast address = 192.168.10.63  (all host bits = 1, since
64-1=63)
Usable hosts = 64 - 2 = 62
```

WORKED EXAMPLE

Worked numerical — subnetting

Q: You are given 192.168.1.0/24 and must split it into 4 equal-sized subnets. Find each subnet's CIDR block.

A:

```
4 subnets needs 2 extra bits (2² = 4) → new prefix = /24+2 =
/26 (64 addresses each)
Subnet 1: 192.168.1.0/26     (.0  to  .63)
Subnet 2: 192.168.1.64/26    (.64 to  .127)
Subnet 3: 192.168.1.128/26   (.128 to .191)
Subnet 4: 192.168.1.192/26   (.192 to .255)
```

NAT (Network Address Translation) Core

NOTES

Private networks use reserved address ranges (like `192.168.0.0/16`, `10.0.0.0/8`) that are never routed on the public internet. A NAT router at the network's edge rewrites the **(private IP, private port)** of outgoing packets to a single **(public IP, chosen port)**, keeping a translation table so replies can be routed back correctly.

KEY POINT

```
NAT Translation Table
Private side              ⇄   Public side
192.168.1.5 : 5000        ⇄   203.0.113.9 : 40001
192.168.1.6 : 5000        ⇄   203.0.113.9 : 40002
```

Two internal hosts using the SAME private port (5000) are distinguished on the public side purely by the different port NAT assigns them (40001 vs 40002) — this is exactly why NAT lets one public IP serve an entire private network.

IP support protocols — ARP, DHCP, ICMP Extension

ARP (Address Resolution Protocol)

NOTES

Maps a next-hop **IP address** → **MAC address** on the same LAN. The sender **broadcasts** an ARP request (destination MAC FF:FF:FF:FF:FF:FF: "who has IP x?"); the owner replies by **unicast**. Answers are kept in an **ARP cache**.

COMMON TRAP

GATE classic

Going from host H1 to a host on **another** network, H1 ARPs for the **default router's** interface, not for the final destination. As a packet crosses each router the **MAC addresses change at every hop**, but the **source/destination IP addresses stay the same** (unless NAT rewrites them).

DHCP (Dynamic Host Configuration Protocol)

KEY POINT

Gives a booting host its IP address, subnet mask, default gateway and DNS server, as a time-limited **lease**. Runs over **UDP** (server port 67, client port 68). The four steps — **DORA**:

```
1. DISCOVER  client → broadcast (src 0.0.0.0, dst
255.255.255.255)
2. OFFER     server → proposes an address
3. REQUEST   client → broadcast, accepting one offer
4. ACK       server → confirms the lease
```

A **relay agent** forwards broadcasts to a DHCP server on another network.

ICMP (Internet Control Message Protocol)

KEY POINT

Carried inside IP datagrams (**protocol number 1**); reports errors and supports diagnostics: **Echo request/reply** (ping), **Destination unreachable**, **Time exceeded** (TTL reached 0), **Redirect**. **traceroute** sends probes with TTL = 1, 2, 3 … ; each router that drops one returns "Time exceeded", revealing the path hop by hop. ICMP never generates errors about ICMP error messages.

Classful addressing and special addresses Extension

REFERENCE TABLE

Class	First octet	Default mask	Usable hosts per network
A	0 – 127 (1 – 126 usable; 127 is loopback)	/8	$2^{24} - 2 = 16{,}777{,}214$
B	128 – 191	/16	$2^{16} - 2 = 65{,}534$
C	192 – 223	/24	$2^{8} - 2 = 254$
D	224 – 239	— (multicast)	—
E	240 – 255	— (reserved)	—

KEY POINT

Special addresses. `127.0.0.0/8` loopback · `0.0.0.0` "this host" (e.g. a DHCP client before it has an address) · `255.255.255.255` **limited broadcast** (never forwarded by routers) · host bits all 1 = **directed broadcast** for that network · host bits all 0 = the network address · `224.0.0.0/4` multicast · private ranges `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16` · `169.254.0.0/16` link-local (self-assigned when DHCP fails).

Forwarding: longest-prefix match, and VLSM Core

NOTES

A router compares the destination with **every** table entry and, among the entries that match, forwards by the one with the **longest prefix** (the most specific route). Subnetting with **variable-length masks (VLSM)** gives each subnet only as many addresses as it needs: allocate the **largest** subnet first and keep every block aligned to its own size.

WORKED EXAMPLE

Worked numerical

Q: A router has entries 192.168.0.0/16, 192.168.5.0/24 and 192.168.5.128/25. Which entry is used for 192.168.5.130, 192.168.5.20 and 192.168.9.9?
A: 192.168.5.130 matches all three (it lies in .128–.255) → the **/25** wins. 192.168.5.20 is outside the /25 but inside the /24 → the **/24**. 192.168.9.9 matches only the /16 → the **/16**.

WORKED EXAMPLE

Worked numerical — VLSM

Q: Split 192.168.1.0/24 for three LANs needing 100, 50 and 20 hosts.
A: Add 2 for the network and broadcast addresses and round up to a power of two:

```
100 hosts → 102 → 128 addresses → /25 : 192.168.1.0/25
(.0   – .127, 126 usable)
 50 hosts →  52 →  64 addresses → /26 : 192.168.1.128/26
(.128 – .191,  62 usable)
 20 hosts →  22 →  32 addresses → /27 : 192.168.1.192/27
(.192 – .223,  30 usable)
```

Addresses .224 – .255 stay free for later.

IPv4 header — fields worth knowing Core

REFERENCE TABLE

Field	Size / meaning
Version, IHL	4 bits each. IHL counts 4-byte words, so the header is 20 – 60 bytes
Total length	16 bits → a datagram is at most **65,535 bytes**
Identification, flags (DF, MF), fragment offset	Reassembly; the offset is 13 bits in units of 8 bytes
TTL	Decremented by each router; at 0 the packet is dropped and ICMP "time exceeded" is sent
Protocol	Names the payload: **1** = ICMP, **6** = TCP, **17** = UDP
Header checksum	Covers the **header only**, and is recomputed at every router because TTL changes

IPv6 in brief Extension

KEY POINT

128-bit addresses written as 8 groups of 4 hex digits; leading zeros may be dropped and **one** run of all-zero groups may be replaced by `::` (2001:0db8:0000:0000:0000:ff00:0042:8329 → `2001:db8::ff00:42:8329`). The header is a fixed **40 bytes** with **no header checksum** and **no fragmentation by routers** (only the source fragments, after path-MTU discovery). There is **no broadcast**: multicast and anycast replace it, and Neighbour Discovery (inside ICMPv6) replaces ARP. Migration uses **dual stack** or **tunnelling** (IPv6 packets carried inside IPv4).

6 TCP: Flow & Congestion Control

Flow control — sliding window Core

NOTES

The receiver advertises a **receive window (rwnd)**: how many unacknowledged bytes it can currently buffer. The sender must never have more than `min(cwnd, rwnd)` bytes "in flight" (sent but not yet acknowledged).

WORKED EXAMPLE

Worked numerical

Q: Sender's window size is 4000 bytes, MSS (max segment size) is 1000 bytes, RTT is 100 ms. What is the maximum achievable throughput?
A: The sender can have 4000 bytes in flight per RTT (send, wait for ack, then send more): `Throughput = Window size / RTT = 4000 bytes / 0.1 s = 40,000 bytes/s = 320 kbps`.

Congestion control — slow start & AIMD Core

KEY POINT

Slow Start

`cwnd` starts at 1 MSS and **doubles every RTT** (exponential growth) until it hits `ssthresh`, or a loss occurs.

KEY POINT

Congestion Avoidance (AIMD)

Once past ssthresh, `cwnd` grows by just **+1 MSS per RTT** (linear, additive increase). On a loss, `ssthresh = cwnd/2` and `cwnd` drops (multiplicative decrease).

WORKED EXAMPLE

Worked numerical — cwnd trace

Q: ssthresh = 16 MSS. Starting from cwnd=1 MSS, trace cwnd for the first several RTTs assuming no losses.

A:

```
Slow start (doubles each RTT, until reaching ssthresh=16):
RTT1: 1   RTT2: 2   RTT3: 4   RTT4: 8   RTT5: 16  (=
ssthresh, switch phase here)

Congestion avoidance (adds +1 MSS each RTT from here on):
RTT6: 17   RTT7: 18   RTT8: 19 ...
```

If a timeout (loss) occurred at, say, cwnd=19: new ssthresh = 19/2 ≈ 9, and cwnd resets to 1, restarting slow start up to the new ssthresh=9 before switching back to linear growth.

Socket API — the basic flow Core

KEY POINT

Server

`socket()` → `bind()` → `listen()` → `accept()` (blocks until a client connects) → `recv()/send()` → `close()`

KEY POINT

Client

`socket()` → `connect()` (initiates the 3-way handshake) → `send()/recv()` → `close()`

GATE FAVOURITE

GATE note

The 3-way handshake itself (SYN → SYN-ACK → ACK) is what `connect()` triggers on the client and what `accept()` completes on the server — remember it costs **1.5 RTT** before any actual data can be sent.

TCP segment header & UDP High-yield

KEY POINT

```
| Source port 16 | Dest port 16 |
| Sequence number 32 |
| Acknowledgment number 32 |
| HLEN 4 | Reserved | URG ACK PSH RST SYN FIN | Window 16 |
| Checksum 16 | Urgent pointer 16 | Options (0–40 B) |
```

Header = 20 B minimum, 60 B maximum (HLEN counts 4-byte words, range 5–15). **UDP** is connectionless with an 8-byte header: source port, destination port, length, checksum — no sequencing, retransmission or flow control.

Sequence & acknowledgment numbers, connection setup and teardown High-yield

KEY POINT

Sequence number = byte-stream number of the **first** data byte in the segment. **ACK number** = the **next byte expected** (cumulative). **SYN and FIN each consume one sequence number**; a pure ACK consumes none.

```
Setup (3-way):   Client → SYN, seq = x
                 Server → SYN+ACK, seq = y, ack = x + 1
                 Client → ACK, seq = x + 1, ack = y + 1
(data may ride with this segment)
Teardown (4-way): FIN → ACK → FIN → ACK  (each direction is
closed separately — "half-close")
The side that closes first enters TIME_WAIT for 2 × MSL
before releasing the connection.
```

COMMON TRAP

Remember

Dropped, reordered or lost data makes the receiver repeat the **same ACK number** (**duplicate ACK**). **Three duplicate ACKs** trigger **fast retransmit** of the missing segment without waiting for the timer. Tahoe then restarts at cwnd = 1; Reno (fast recovery) sets ssthresh = cwnd/2 and cwnd = ssthresh (+3). A **timeout** always drops cwnd to 1 MSS.

RTT estimation and the retransmission timeout High-yield

KEY POINT

```
SRTT   = (1 − α)·SRTT   + α·SampleRTT          α = 1/8
RTTVAR = (1 − β)·RTTVAR + β·|SRTT − SampleRTT|   β = 1/4
(computed with the OLD SRTT, as in RFC 6298)
RTO    = SRTT + 4·RTTVAR
```

Karn's algorithm: never take an RTT sample from a **retransmitted** segment (the ACK is ambiguous), and **double the RTO** on every timeout (exponential back-off).

Tahoe vs Reno — reacting to loss Core

REFERENCE TABLE

Event	TCP Tahoe	TCP Reno
Timeout	ssthresh = cwnd/2, cwnd = 1, slow start	Same
3 duplicate ACKs	Fast retransmit, then ssthresh = cwnd/2 and **cwnd = 1**	Fast retransmit, ssthresh = cwnd/2, **fast recovery**: cwnd = ssthresh + 3, +1 per further duplicate ACK, and when the new ACK arrives cwnd = ssthresh (back to congestion avoidance)

WORKED EXAMPLE

Worked numerical

Q: cwnd = 20 MSS when three duplicate ACKs arrive. Give ssthresh and cwnd for Tahoe and Reno. What if it had been a timeout?

A: ssthresh = 20/2 = **10** in both. Tahoe restarts at **cwnd = 1**. Reno sets **cwnd = 10 + 3 = 13**, inflates it during recovery, then drops to **10** when the new ACK arrives. On a **timeout** both set ssthresh = 10 and cwnd = 1.

Small segments: Nagle, silly window syndrome, delayed ACK Extension

KEY POINT

The problem: sending 1 byte of data costs 20 + 20 header bytes, so only 1/41 ≈ **2.4%** of the packet is useful.

Nagle's algorithm (sender): send a small segment immediately only if there is **no unacknowledged data** in flight; otherwise buffer small writes until an ACK arrives or a full MSS has accumulated.

Silly window syndrome (receiver): a slow application reads a few bytes at a time and the receiver keeps advertising tiny windows. **Clark's fix:** do not advertise a new window until it can hold a full MSS (or half the buffer).

Delayed ACK: the receiver waits (at most about **500 ms**) hoping to piggyback the ACK on data, and ACKs at least **every second full-sized segment**.

Window limits and window scaling Core

NOTES

The TCP window field is 16 bits, so the largest window is **65,535 bytes**. At RTT = 100 ms this caps throughput at 65 535 × 8 / 0.1 ≈ **5.24 Mbps** however fast the link is. The **window-scale option** (negotiated in the SYN, shift up to 14) multiplies the window by 2^{shift}, giving up to 65 535 × 2^{14} ≈ **1 GB**. This is the bandwidth-delay product again: the window must be at least BDP = bandwidth × RTT to keep the pipe full.

Traffic shaping — leaky bucket and token bucket Extension

REFERENCE TABLE

	Leaky bucket	Token bucket
Output	Constant rate ρ, however bursty the input	Bursts allowed, up to the tokens saved
Mechanism	A queue (buffer) drained at a fixed rate; overflow is dropped	Tokens arrive at rate r into a bucket of size b; sending a bit uses a token
Idle periods	Capacity is lost	Credit accumulates, up to b

WORKED EXAMPLE

Worked numerical — token bucket

Q: Token rate r = 2 Mbps, bucket size b = 1 Mb (full), line rate M = 10 Mbps. How long can the sender burst at full speed, and how much does it send?
A: During a burst of length S the bucket gives b tokens and r·S more arrive: M·S = b + r·S, so `S = b / (M − r)` = 1 / (10 − 2) = **0.125 s**. Data sent = 10 Mbps × 0.125 s = **1.25 Mb**, which equals 1 + 2 × 0.125 tokens ✓.

WORKED EXAMPLE

Worked numerical — leaky bucket

Q: Output rate 2 Mb/s, buffer 3 Mb, input a steady 6 Mb/s for 1 s. How much is dropped?
A: The queue grows at 6 − 2 = 4 Mb/s and fills after 3/4 = 0.75 s. For the last 0.25 s the excess (6 − 2 = 4 Mb/s) cannot be stored: 4 × 0.25 = **1 Mb dropped**.

Port numbers and common services High-yield

REFERENCE TABLE

Range	Name
0 – 1023	Well-known (needs privilege to bind)
1024 – 49151	Registered
49152 – 65535	Dynamic / ephemeral (client side)

NOTES

Common ports: FTP 20/21 · SSH 22 · Telnet 23 · SMTP 25 · DNS 53 (UDP for queries, TCP for zone transfers) · DHCP 67/68 (UDP) · HTTP 80 · POP3 110 · IMAP 143 · SNMP 161 (UDP) · HTTPS 443. Both TCP and UDP checksums also cover a **pseudo-header** (source and destination IP, protocol, length) in addition to the segment itself.

7 DNS & HTTP

DNS — Domain Name System Core

NOTES

DNS is a distributed, hierarchical database mapping domain names to IP addresses, organized in levels: **Root servers** → **TLD servers (.com, .org)** → **Authoritative servers** (for the specific domain).

FLOW

Client asks local resolver → Root server → TLD server (.com) → Authoritative server → IP address returned

REFERENCE TABLE

Query type	Behavior
Iterative	Each server gives the resolver the address of the NEXT server to ask — resolver does all the legwork
Recursive	The resolver asks one server, which itself goes and asks the rest, returning only the final answer

WORKED EXAMPLE

Worked numerical

Q: A DNS lookup requires visiting the local resolver, then a root server, then a TLD server, then the authoritative server (fully iterative, no caching). If each round-trip takes 20 ms, find the total resolution time.
A: 3 iterative round trips (resolver↔root, resolver↔TLD, resolver↔authoritative) = 3 × 20 = **60 ms**, plus the client's own request to the resolver and its reply (another 20 ms round trip) = **80 ms total**. (Caching at the resolver — thanks to each record's **TTL** — is exactly what avoids repeating this full chain on every single lookup.)

HTTP Core

REFERENCE TABLE

Method	Purpose
GET	Retrieve a resource
POST	Submit data to be processed (e.g. a form)
PUT	Replace a resource entirely
DELETE	Remove a resource

NOTES

Status codes: **2xx** success (200 OK), **3xx** redirection (301 Moved), **4xx** client error (404 Not Found), **5xx** server error (500 Internal Error).

KEY POINT

Non-persistent HTTP
A brand-new TCP connection (with its own 3-way handshake) for EVERY object on a page.

KEY POINT

Persistent HTTP
One TCP connection stays open and is reused for multiple objects/requests — avoids repeated handshake overhead.

WORKED EXAMPLE

Worked numerical

Q: A webpage has 1 HTML file plus 5 embedded images, all on the same server. RTT = 50 ms (assume negligible transmission time). Compare total time using non-persistent HTTP (no parallel connections) vs. persistent HTTP without pipelining (one connection, one request at a time).
A:

```
Non-persistent (serial, new connection each time):
  Each object = 1 RTT for TCP handshake + 1 RTT for HTTP
request/response = 2 RTT
  6 objects (1 HTML + 5 images) × 2 RTT = 12 RTT = 12 × 50 =
600 ms

Persistent, no pipelining (1 handshake total, then one
request/response at a time):
  1 RTT for the single handshake + 6 RTT for the 6
request/response exchanges
  = 7 RTT = 7 × 50 = 350 ms
```

With **pipelining** (all requests sent without waiting for each response) the six objects need only 1 RTT (handshake) + 1 RTT (HTML) + 1 RTT (all 5 images together) = 3 RTT = 150 ms. Persistent HTTP saves a huge amount of time by paying the handshake cost only **once** — this exact "count the RTTs" calculation is a favorite GATE question type.

Email and file transfer — SMTP, POP3/IMAP, FTP Extension

REFERENCE TABLE

Protocol	Port	Role
SMTP	TCP 25	**Push**: user agent → sender's mail server, and server → recipient's mail server (commands HELO, MAIL FROM, RCPT TO, DATA)
POP3	TCP 110	**Pull**: recipient downloads mail (usually deleting it from the server)
IMAP	TCP 143	**Pull**: mail stays on the server, folders managed remotely
FTP	TCP 21 (control) + 20 (data)	Separate **control** and **data** connections ("out-of-band" control); stateful

KEY POINT

Mail path: sender's UA —SMTP→ sender's server —SMTP→ recipient's server —POP3/IMAP→ recipient's UA. SMTP is **never** used to retrieve mail. HTTP is stateless and uses port 80 (443 for HTTPS); DNS uses port 53 (UDP for ordinary queries, TCP for zone transfers).

DNS resource records Core

REFERENCE TABLE

Type	Meaning
A / AAAA	Name → IPv4 / IPv6 address
NS	Name of the authoritative name server for a domain
CNAME	Alias → canonical name
MX	Mail server for a domain (with a preference value)
PTR	Reverse lookup: IP address → name

HTTP time with pipelining Core

KEY POINT

Base page + n embedded objects on the same server (RTT = R, transmission time negligible):

```
Non-persistent, serial :  (n + 1) × 2R          (handshake + request for each object)
Persistent, no pipeline:  R + (n + 1) × R        (one handshake, then one RTT per object)
Persistent, pipelined  :  R + R + R = 3R         (handshake, base page, then ALL objects together)
```

Cookies, conditional GET and web caching Core

KEY POINT

HTTP is stateless — the server remembers nothing between requests. **Cookies** add state: the server sends `Set-Cookie: id=…` in a response, and the browser returns `Cookie: id=…` on every later request to that site, which is how logins and shopping carts persist.

Conditional GET: a cache that already holds an object asks `If-Modified-Since: <date>`. If it has not changed, the server answers **304 Not Modified** with **no body**, saving a full transfer. A **web proxy cache** serves repeat requests locally, cutting both response time and the load on the access link.

WORKED EXAMPLE

Worked numerical

Q: A cache answers 40% of requests in 10 ms; the rest go to the origin server and take 200 ms. What is the average response time?
A: 0.4 × 10 + 0.6 × 200 = 4 + 120 = **124 ms** (versus 200 ms with no cache).

GATE FAVOURITE

GATE note

Status codes: **200** OK, **301** moved permanently, **304** not modified, **404** not found, **500** server error. HTTP/1.0 is non-persistent by default; HTTP/1.1 is **persistent** by default and supports pipelining.

8 Problem → Standard Method Playbook

METHOD

Framing / bit stuffing

```
Bit stuffing: after five consecutive 1s in the data, insert a 0.
At the receiver, remove the stuffed 0 after five 1s.
Byte stuffing: reserve an ESC byte and escape occurrences of the delimiter/ESC inside payload.
```

Trap: bit stuffing is bit-oriented; byte stuffing is byte/character-oriented.

METHOD

IP fragmentation / CIDR

```
Prefix /p → host bits = 32−p.
Addresses in block = 2^(32−p).
First address = network; last = broadcast (IPv4 subnet).
For fragmentation, fragment payload sizes must respect the outgoing MTU and, except the final fragment, be multiples of 8 bytes. MF=1 means more fragments follow.
```

METHOD

Routing: DV vs LS

```
Distance Vector → Bellman-Ford recurrence:
D_x(y) = min_v { c(x,v) + D_v(y) }
Link State → build a complete topology view, then run Dijkstra from the source.
RIP → distance-vector, hop count metric, max usable hop count 15.
OSPF → link-state, Dijkstra, cost metric.
BGP → inter-domain path-vector protocol.
```

METHOD

TCP sequence / ACK

```
TCP sequence number identifies the first byte in a segment's payload.
ACK number = next byte expected (cumulative ACK).
SYN and FIN each consume one sequence number even though they carry no application data.
If segment starts at seq S and carries L bytes, next expected byte is S+L.
```

METHOD

TCP timeout / RTO

```
Smoothed RTT: SRTT = (1−α)SRTT + α·SampleRTT.
RTT variation: RTTVAR = (1−β)RTTVAR + β|SRTT−SampleRTT|.
Typical RTO form: SRTT + 4·RTTVAR.
Do not use a retransmitted sample as an ordinary RTT sample (Karn's rule).
```

METHOD

DNS / HTTP timing

```
DNS maps names to records; ordinary DNS queries commonly use UDP 53, while TCP 53 is used when required (e.g. zone transfer).
HTTP is application-layer request/response.
For timing questions, count each required RTT separately and add transmission/processing time only when the question gives it.
Persistent connections avoid repeated TCP handshakes; pipelining can overlap requests.
```

METHOD

Delay / throughput

```
Transmission = packet bits / link rate.
Propagation = distance / propagation speed.
Store-and-forward across H links: H·(transmission + propagation) when all links have the same values and processing/queueing are ignored.
BDP = bandwidth × RTT.
```

1. Bit stuffing

METHOD NOTES

Suppose the payload contains **1111101**. What bit-oriented stuffing occurs?

After the first five consecutive 1s, the sender inserts a 0: **11111001**. The receiver removes that inserted 0 after detecting five consecutive 1s.

Trap: the inserted 0 is not application data.

2. CIDR block size

METHOD NOTES

How many IPv4 addresses are in **192.168.10.0/26**?

Host bits = 32−26 = 6. Therefore addresses = 2^6 = **64**. The block spans 192.168.10.0 through 192.168.10.63; in the usual subnet interpretation, .0 is the network address and .63 the broadcast address.

3. TCP sequence and ACK

METHOD NOTES

A TCP segment has sequence number 1000 and carries 500 bytes of data. What cumulative ACK should the receiver send if all bytes through the segment arrived correctly?

The bytes are 1000 through 1499, so the next expected byte is **1500**. ACK = **1500**.

If the segment instead had SYN set and carried no data, the SYN would still consume one sequence number.

4. RTT/RTO update

METHOD NOTES

Let previous SRTT = 100 ms, α = 0.125, and a new SampleRTT = 140 ms. Compute the new SRTT.

SRTT = 0.875×100 + 0.125×140 = 87.5 + 17.5 = **105 ms**. In a full RTO calculation, RTTVAR must also be maintained; do not replace RTO with SRTT alone.

5. HTTP persistent timing

METHOD NOTES

A page and 3 embedded objects are on the same server. RTT = R and transmission time is negligible. Under persistent HTTP with pipelining, after the TCP connection is established, requests can be sent together.

Using the standard simplified GATE model: TCP handshake = R, base-object request/response = R, pipelined embedded-object requests/responses = R. Total ≈ **3R**.

Trap: if the problem explicitly includes DNS lookup or TLS handshake, add those RTTs according to the stated model.

6. Distance Vector update

METHOD NOTES

Router X has neighbor A with link cost 2. A advertises distance 5 to destination Y. What candidate distance does X obtain through A?

Bellman-Ford candidate = c(X,A) + D_A(Y) = 2 + 5 = **7**. X compares this with all other neighbors and chooses the minimum candidate.

⚡ CN Last-Minute Recall

METHOD

- Bit stuffing: insert **0 after five consecutive 1s**.
- /p IPv4 block size = **2^(32−p)** addresses.
- Fragment offset is measured in **8-byte units**; MF=1 means more fragments follow.
- DV = Bellman-Ford; LS = Dijkstra. RIP = DV, OSPF = LS, BGP = path vector.
- TCP ACK = **next byte expected**; SYN/FIN each consume one sequence number.
- BDP = bandwidth × RTT.
- SMTP sends mail; POP3/IMAP retrieve it.
- DNS A/AAAA = address, CNAME = alias, MX = mail server, PTR = reverse lookup.

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- MSS = MTU − IP − TCP headers = 1460 B for a 1500-B MTU. TCP header 20–60 B; UDP header 8 B.
- RTO = SRTT + 4·RTTVAR; SRTT uses α = 1/8, RTTVAR uses β = 1/4. Karn: skip RTT samples from retransmissions, back off exponentially.
- Transmission delay = packet size / link rate; propagation delay depends on distance and propagation speed.
- Throughput is limited by the bottleneck link/rate along the path, not simply by the fastest link.
- **Subnet IPv4 address:** network prefix + host part. Increasing prefix length reduces hosts per subnet.
- TCP flow control is receiver-window based; congestion control is network-condition based. Do not conflate the two.
- **Sliding window efficiency (a = Tp/Tt):** stop-and-wait 1/(1+2a); Go-Back-N and Selective Repeat N/(1+2a), capped at 1. Window for 100% = 1+2a; GBN needs sequence space N+1, SR needs 2N.
- Shannon $C = B \cdot \log_2(1 + S/N)$ with $S/N = 10^{dB/10}$; Nyquist $C = 2B \cdot \log_2 L$. Use the smaller when both apply.
- Token bucket burst time $S = b/(M - r)$; a leaky bucket outputs at a constant rate and drops overflow.

LAST-MINUTE · MUST REMEMBER

- **ARP:** IP→MAC on the same LAN (broadcast request, unicast reply). Across routers: MAC changes every hop, IP stays the same.
- DHCP = DORA over UDP (server 67, client 68). ICMP = errors/diagnostics, IP protocol 1; traceroute uses TTL expiry.
- SYN and FIN each consume one sequence number; ACK number = next byte expected; 3 duplicate ACKs trigger fast retransmit.
- OSI is a reference model; TCP/IP is the practical protocol suite used on the Internet.
- CRC detects errors using polynomial division; the receiver checks whether the received polynomial is divisible by the generator.
- CSMA/CD belongs to traditional shared Ethernet; modern switched full-duplex Ethernet does not use collision detection.
- CIDR uses prefix notation and enables route aggregation; longest-prefix matching selects the most specific matching route.
- TCP is connection-oriented and reliable; UDP is connectionless and does not provide TCP-style reliability.
- DNS maps names to records such as A/AAAA, MX, NS, and CNAME; DNS commonly uses UDP, with TCP also used when required.
- HTTP is application-layer; HTTPS is HTTP carried over TLS.
- Router forwarding uses the longest matching prefix, not the first match. Usable hosts in a /n block = $2^{32-n} - 2$; VLSM allocates the biggest subnet first.

LAST-MINUTE · COMMON TRAPS

- **Bit stuffing:** insert 0 after FIVE consecutive 1s (flag 01111110). A switch has one collision domain per port but ONE broadcast domain.
- Fragmentation can occur in IPv4 when a packet exceeds an outgoing link MTU; fragmentation fields include Identification, Flags, and Fragment Offset.
- **Tahoe vs Reno:** three duplicate ACKs → Tahoe cwnd = 1, Reno cwnd = ssthresh + 3 (fast recovery). A timeout gives cwnd = 1 in both.
- **HTTP is stateless:** cookies add state. Conditional GET answers 304 Not Modified with no body. IPv4 header checksum covers the header only and changes at each hop because of TTL.

LAST-MINUTE · GATE FAVOURITES

- **HTTP RTT counting:** non-persistent serial = 2R per object; persistent = R + one R per object; persistent + pipelined = 3R.
- Know slow start, congestion avoidance, fast retransmit, and fast recovery at the level needed to trace cwnd evolution.

DBMS

GATE CSE 2027 · complete formula, rule, method & trap sheet

103 reference boxes · **8** chapters ·

ER Model | Relational Model, Algebra & Keys | SQL | Integrity Constraints & Normal Forms | File Organization & Indexing (B/B+ Trees) | Transactions & Concurrency Control | Problem → Standard Method Playbook | Last-minute one-liners

1 ER Model

The building blocks

REFERENCE TABLE

Concept	Meaning	Example
Entity	A distinguishable "thing" you want to store data about	Student, Course
Attribute	A property of an entity	Student's name, roll number
Relationship	An association between two or more entities	Student ENROLLS_IN Course

Types of attributes

REFERENCE TABLE

Type	Meaning
Simple	Cannot be divided further (e.g. Age)
Composite	Made of sub-parts (e.g. Address = Street+City+Pincode)
Single-valued	One value per entity (e.g. DOB)
Multi-valued	Can have many values (e.g. Phone Numbers) — shown with a double oval
Derived	Computed from another attribute (e.g. Age derived from DOB) — shown with a dashed oval

Cardinality ratios & participation constraints

KEY POINT

Cardinality ratio — how many instances of one entity can relate to instances of another: **1:1**, **1:N**, or **M:N**.

KEY POINT

Participation constraint — **total** (every entity instance MUST participate, drawn as a double line) vs **partial** (participation is optional, single line).

KEY POINT

Weak entity: an entity with no primary key of its own — it's identified only via a relationship to an "owner" (identifying) entity plus a **partial key**. Drawn with a double-bordered rectangle, connected via a double-bordered diamond (identifying relationship).

Mapping an ER diagram to relational tables — the GATE trick

REFERENCE TABLE

ER construct	How many tables needed
Strong entity	1 table (attributes become columns, key becomes primary key)
1:1 relationship	Can often be merged into ONE of the two entity tables (foreign key on either side, or the side with total participation)
1:N relationship	Add the "1" side's primary key as a foreign key in the "N" side's table — no separate table needed
M:N relationship	ALWAYS needs its own separate table, holding both entities' primary keys (as a composite key) plus any relationship attributes
Weak entity	1 table, with the owner's primary key + the weak entity's partial key together forming its primary key

FORMULA / RULE

Worked numerical

Q: An ER diagram has: Entity A (strong), Entity B (strong), Entity C (weak, owned by B). Relationships: A–B is 1:N, B–C is the identifying relationship (1:N, total participation of C). Find the MINIMUM number of tables needed.

A:

```
Entity A → 1 table
Entity B → 1 table (also absorbs the 1:N relationship from A, via a foreign key A_id in B)
Entity C (weak) → 1 table (primary key = B_id + C's partial key; no separate relationship table needed since it's identifying)

Total = 3 tables
```

The golden rule: **only M:N relationships force a genuinely new, separate table** — every 1:1 and 1:N relationship can be absorbed as a foreign key into an existing entity's table.

Extended ER (EER) — generalization, specialization, aggregation

KEY POINT

Specialization is the **top-down** process of defining sub-classes of an entity set (Person → Student, Employee); **generalization** is the reverse **bottom-up** process of merging similar entity sets into one super-class. Sub-classes **inherit** all attributes and relationships of the super-class (the "IS-A" relationship). **Aggregation** treats a whole relationship (plus its participants) as a single higher-level entity so it can itself take part in another relationship.

REFERENCE TABLE

Constraint on a specialization	Meaning
Disjoint (d)	An entity can belong to **at most one** sub-class
Overlapping (o)	An entity may belong to **several** sub-classes at once
Total (double line)	Every super-class entity must belong to **at least one** sub-class
Partial (single line)	A super-class entity may belong to no sub-class at all

KEY POINT

Mapping EER to tables (4 standard options)

```
1. One table per class:   Super(K, a…)  +  Sub_i(K, own attrs…)   — works for every constraint combination
2. One table per sub-class only (each holds inherited + own attrs) — needs total + disjoint
3. Single table with a type flag (all attrs, NULLs for unused) — best for disjoint
4. Single table with one boolean flag per sub-class — handles overlapping
```

2 Relational Model, Algebra & Keys

Basic terminology

REFERENCE TABLE

Term	Meaning
Relation	A table
Tuple	A row
Attribute	A column
Degree	Number of attributes (columns)
Cardinality	Number of tuples (rows)

Keys

REFERENCE TABLE

Key type	Meaning
Super key	Any set of attributes that uniquely identifies a tuple (may include extra, unnecessary attributes)
Candidate key	A MINIMAL super key — remove any attribute and it stops being unique
Primary key	The candidate key chosen by the designer to be THE identifier
Foreign key	An attribute in one table that references the primary key of another table

GATE FAVOURITE

GATE angle: counting keys via Functional Dependencies

Given a relation R and a set of functional dependencies (FDs), a **candidate key** is a minimal attribute set whose **closure** (under those FDs) equals ALL attributes of R.

Attribute closure algorithm: start with the given set, repeatedly add the RHS of any FD whose LHS is already fully contained in your current set, until nothing new is added.

FORMULA / RULE

Worked numerical

Q: R(A,B,C,D) with FDs: A→B, B→C, C→D. Find a candidate key.

A:

```
Try {A}: closure({A}) → A,B (from A→B) → A,B,C (from B→C) → A,B,C,D (from C→D)
closure({A}) = {A,B,C,D} = all attributes → {A} is a candidate key (and it's minimal, being just 1 attribute)
```

Relational Algebra operators

REFERENCE TABLE

Operator	Symbol	Meaning
Selection	$\sigma_{condition}(R)$	Filter ROWS matching a condition
Projection	$\pi_{attributes}(R)$	Keep only specific COLUMNS (duplicates removed)
Union	$R \cup S$	Rows in R or S (same schema required)
Set difference	$R - S$	Rows in R but not in S
Cartesian product	$R \times S$	Every row of R paired with every row of S
Rename	ρ	Rename a relation or its attributes
Natural Join	$R \bowtie S$	Join on all common attribute names, keep one copy of each

WORKED EXAMPLE

Worked numerical

Q: R has 5 tuples over attributes (A,B); S has 3 tuples over attributes (B,C). If B has 2 common values between R and S, and matching creates exactly 6 combined tuples, how many tuples are in R ⋈ S (natural join), vs R × S (Cartesian product)?

A:

```
R × S (Cartesian product) = 5 × 3 = 15 tuples (every R row paired with every S row, ignoring B values)
R ⋈ S (natural join) = 6 tuples (only pairs where R.B = S.B actually match, as given)
```

Natural join is always ≤ the Cartesian product size — it's effectively "Cartesian product, then filter where the shared attribute matches, then drop the duplicate column".

WORKED EXAMPLE

Worked numerical — combining operators

Q: Given Employee(eid, name, dept, salary), write a relational algebra expression for "names of employees in the 'Sales' department earning more than 50000", and identify which operators are used.

A:

```
π_name ( σ_(dept='Sales' AND salary>50000) (Employee) )
```

Uses **Selection** (σ) to filter rows matching the department+salary condition, then **Projection** (π) to keep only the name column. This "select-then-project" pattern is the relational-algebra equivalent of a SQL `SELECT name FROM Employee WHERE dept='Sales' AND salary>50000`.

Join variants

REFERENCE TABLE

Join	Meaning
Theta join $R \bowtie_\theta S$	$\sigma_\theta(R \times S)$ for any condition θ (<, >, =, ≠…)
Equi-join	Theta join where θ uses only `=` — both copies of the join column are kept
Natural join	Equi-join on ALL common attribute names, keeping one copy of each
Left / Right / Full outer join	Natural (or θ) join PLUS the unmatched tuples of the left / right / both sides, padded with NULLs

Division (÷) — the "for ALL" operator

MUST REMEMBER

Definition. For R(A, B) and S(B), `R ÷ S` returns every value **a** (over the attributes of R that are NOT in S) such that the pair **(a, b) appears in R for EVERY b in S**.

```
R ÷ S  =  π_A(R)  −  π_A( (π_A(R) × S) − R )
          ──────     ──────────────────────
          all candidates     candidates that are MISSING at least one required b → disqualified
```

Result schema = attrs(R) − attrs(S). If S is empty, R ÷ S = π_A(R). Division is the relational-algebra answer to every "…*all* / *every*…" query (students who took *all* required courses, suppliers who supply *all* parts).

WORKED EXAMPLE

Worked numerical — division

Q: Enrolled(student, course) = {(Amy,DB),(Amy,OS),(Amy,CN),(Ben,DB),(Ben,OS),(Cara,DB),(Cara,CN)}; Required(course) = {DB, OS}. Compute Enrolled ÷ Required.

A:

```
Candidates π_student(Enrolled) = {Amy, Ben, Cara}
Candidates × Required = {Amy,Ben,Cara} × {DB,OS} = 6 pairs
Missing pairs = (those 6) − Enrolled = {(Cara, OS)}   → Cara is disqualified
Result = {Amy, Ben, Cara} − {Cara} = {Amy, Ben}
```

Same query in SQL needs a double negation: `SELECT DISTINCT e.student FROM Enrolled e WHERE NOT EXISTS (SELECT * FROM Required r WHERE NOT EXISTS (SELECT * FROM Enrolled x WHERE x.student=e.student AND x.course=r.course))` — "there is no required course that this student has NOT taken".

GATE FAVOURITE

GATE trap

Division is **not** "Cartesian product in reverse". Extra courses (Amy's CN) don't matter — only whether ALL of S is covered. Also |R ÷ S| ≤ |π_A(R)| always.

Tuple Relational Calculus (TRC) — a quick contrast

NOTES

Where relational algebra is **procedural** (specifies HOW to compute the result, step by step), Tuple Relational Calculus is **declarative** (specifies WHAT you want, using predicate logic over tuple variables) — e.g. `{ t | Employee(t) ∧ t.dept='Sales' ∧ t.salary>50000 }` describes the same query as above, using logical implication/conjunction instead of chained operators. Both are proven to be equally expressive (relationally complete).

3 SQL

Basic query anatomy

KEY POINT

```
SELECT [columns]
FROM [table(s)]
WHERE [row filter — applied BEFORE grouping]
GROUP BY [columns]
HAVING [group filter — applied AFTER grouping]
ORDER BY [columns]
```

GATE FAVOURITE

GATE trap

`WHERE` filters individual rows before any grouping happens; `HAVING` filters entire GROUPS after aggregation. You cannot use an aggregate function (like `COUNT(*)`) inside a `WHERE` clause — it must go in `HAVING`.

Joins — with example tables

KEY POINT

```
Student            Enroll
sid | name          sid | course
1   | Amy           1   | CS101
2   | Ravi          1   | CS102
3   | Zara          4   | CS101   (sid=4 doesn't exist in Student!)
```

REFERENCE TABLE

Join type	Result on the tables above
INNER JOIN (Student ⋈ Enroll on sid)	3 rows: (Amy,CS101), (Amy,CS102), and nothing for Ravi/Zara (no match), sid=4 row dropped (no matching student)
LEFT OUTER JOIN (Student left, Enroll right)	Amy's 2 rows + Ravi(NULL) + Zara(NULL) = 4 rows — every Student row kept even without a match
RIGHT OUTER JOIN	Amy's 2 rows + (NULL, CS101) for sid=4 = 3 rows — every Enroll row kept even without a match
FULL OUTER JOIN	All of the above combined = 5 rows (Amy×2, Ravi-NULL, Zara-NULL, NULL-sid4)

Aggregate functions

FORMULA / RULE

Worked numerical

Q: Employee(eid, dept, salary): (1,'CS',40000), (2,'CS',60000), (3,'EE',50000), (4,'EE', NULL). What does `SELECT dept, COUNT(*), COUNT(salary), AVG(salary) FROM Employee GROUP BY dept;` return?

A:

```
dept | COUNT(*) | COUNT(salary) | AVG(salary)
CS   |    2     |      2        |   50000        (avg of 40000,60000)
EE   |    2     |      1        |   50000        (avg ignores the NULL, uses only 50000)
```

Key trap: `COUNT(*)` counts ALL rows including NULLs; `COUNT(column)` and every other aggregate (`SUM` , `AVG` , `MIN` , `MAX`) silently **ignore NULL values** in that column.

Nested queries

KEY POINT

IN / NOT IN

`WHERE dept IN (SELECT dept FROM ... )` — checks membership in a subquery's result set.

KEY POINT

EXISTS / NOT EXISTS

`WHERE EXISTS (SELECT * FROM ... WHERE ...)` — checks if a correlated subquery returns ANY row at all.

COMMON TRAP

The classic NULL + NOT IN trap

If the subquery's result set for a `NOT IN` contains even ONE `NULL` , the ENTIRE outer query returns **zero rows** — because comparing anything to NULL yields UNKNOWN, not TRUE/FALSE, and `NOT IN` needs every comparison to definitively be "not equal". GATE loves testing this exact gotcha.

Set operations

REFERENCE TABLE

Operator	Meaning
UNION	Rows in either query (duplicates removed)
UNION ALL	Rows in either query (duplicates KEPT)
INTERSECT	Rows in BOTH queries
EXCEPT / MINUS	Rows in the first query but NOT in the second

DDL — defining and changing schema

REFERENCE TABLE

Command	Effect
`CREATE TABLE`	Define columns, types, `PRIMARY KEY`, `UNIQUE`, `NOT NULL`, `CHECK`, `FOREIGN KEY`
`ALTER TABLE`	Add / drop / modify a column or constraint
`DROP TABLE`	Removes the table **structure and data** completely
`TRUNCATE TABLE`	Removes **all rows**, keeps the structure; no WHERE clause
`DELETE FROM … WHERE`	(DML) removes the selected rows, structure stays

KEY POINT

Referential actions on a foreign key: `ON DELETE / ON UPDATE` + `NO ACTION / RESTRICT` (reject the change — the default), `CASCADE` (propagate the delete/update to child rows), `SET NULL`, `SET DEFAULT`.

```
CREATE TABLE Emp (
  eid INT PRIMARY KEY,
  did INT REFERENCES Dept(did) ON DELETE CASCADE,
  salary INT CHECK (salary > 0)
);
```

Views

NOTES

A **view** is a stored query — a *virtual table* whose rows are computed from the base tables each time it is used: `CREATE VIEW HighPaid AS SELECT eid, salary FROM Emp WHERE salary > 50000;` Views give security (expose only some columns/rows) and simplify queries.

KEY POINT

Updatable views (INSERT/UPDATE/DELETE pass through to the base table) generally need: a **single base table**, no `DISTINCT`, no aggregates, no `GROUP BY` / `HAVING`, and the view must include the base table's key / NOT NULL columns. `WITH CHECK OPTION` rejects any update through the view that would make the row disappear from the view. A **materialized view** (unlike a plain view) physically stores its result and must be refreshed.

Other SQL features worth recognising

REFERENCE TABLE

Feature	What it does
Trigger	Event–Condition–Action rule that runs automatically on INSERT / UPDATE / DELETE, e.g. `CREATE TRIGGER t AFTER UPDATE ON emp FOR EACH ROW …`; used for audit logs and derived data
Cursor / embedded SQL	Lets a host-language program fetch a result set one row at a time (DECLARE, OPEN, FETCH, CLOSE)
Stored procedure	Named SQL program kept and run inside the server, which cuts network round trips
GRANT / REVOKE	Access control: `GRANT SELECT ON emp TO alice`; `REVOKE` withdraws it; `WITH GRANT OPTION` lets the user pass it on
COMMIT / ROLLBACK / SAVEPOINT	Transaction control: make changes permanent, undo all of them, or undo back to a marked point

4 Integrity Constraints & Normal Forms

Integrity constraints

REFERENCE TABLE

Constraint	Meaning
Domain constraint	A column's value must come from its declared data type/domain
Entity integrity	Primary key columns can NEVER be NULL
Referential integrity	A foreign key must either be NULL or match an existing primary key value in the referenced table
NOT NULL / UNIQUE / CHECK	Column-level rules enforced on every insert/update

Functional Dependencies & Armstrong's Axioms

NOTES

An FD `X → Y` means: whenever two tuples agree on X, they must also agree on Y. It's **trivial** if $Y \subseteq X$ (always true, tells you nothing new); otherwise **non-trivial**.

KEY POINT

Armstrong's Axioms (sound and complete — derive every valid FD from these):

```
Reflexivity:    if Y ⊆ X, then X → Y              (trivial FDs)
Augmentation:   if X → Y, then XZ → YZ             (add same attrs to both sides)
Transitivity:   if X → Y and Y → Z, then X → Z
```

Derived rules (provable from the three above, but useful shortcuts): **Union** ($X \to Y, X \to Z \Longrightarrow X \to YZ$), **Decomposition** ($X \to YZ \Longrightarrow X \to Y$ and $X \to Z$), **Pseudotransitivity** ($X \to Y, WY \to Z \Longrightarrow WX \to Z$).

Minimal (canonical) cover of an FD set

NOTES

A **minimal cover** F_c of F is an equivalent FD set (same closure F^+) in which (1) every RHS is a **single attribute**, (2) no LHS has an **extraneous attribute**, and (3) no FD is **redundant**. It is the starting point of the **3NF synthesis** algorithm (one relation per FD, then add a candidate key if none is present, then drop subsumed relations).

KEY POINT

Algorithm

```
Step 1  Split RHS:                X→YZ  becomes  X→Y, X→Z
Step 2  Remove extraneous LHS:    in XA→B, A is extraneous if
B ∈ closure(X) under F
Step 3  Remove redundant FDs:     drop X→Y if Y ∈ closure(X)
using the OTHER FDs
```

Do the steps **in this order**. The minimal cover is **not unique** — different orders can give different (but equivalent) covers.

WORKED EXAMPLE

Worked numerical

Q: F = { A→BC, B→C, A→B, AB→C }. Find a minimal cover.

A:

```
Step 1: A→B, A→C, B→C, A→B (duplicate), AB→C
Step 2: AB→C — is A extraneous? closure({B}) = {B,C}
contains C → yes, A is extraneous → AB→C becomes B→C
(duplicate)
        F = { A→B, A→C, B→C }
Step 3: A→C — drop it and compute closure({A}) with {A→B,
B→C} = {A,B,C} ∋ C → redundant, remove
Minimal cover = { A→B, B→C }
```

Normal forms — the ladder

REFERENCE TABLE

Form	Requirement
1NF	Every attribute holds only atomic (indivisible), single values — no repeating groups or multi-valued columns
2NF	1NF + no **partial dependency** (no non-key attribute depends on only PART of a composite candidate key)
3NF	2NF + no **transitive dependency** (no non-key attribute depends on another non-key attribute)
BCNF	For every non-trivial FD X→Y, X must be a **super key** (stricter than 3NF — no exceptions allowed)

FORMULA / RULE

Worked numerical — finding the highest normal form

Q: R(A,B,C,D) with candidate key {A,B} and FDs: AB→CD, C→D. What is the highest normal form R satisfies?

A:

```
Check 1NF: assume all atomic values → ✓ 1NF holds

Check 2NF: is there a partial dependency? Candidate key is
{A,B} (composite).
  Does C or D depend on just A, or just B alone (not both)?
  We're only given AB→CD and C→D — no dependency on A alone
or B alone → no partial dependency → ✓ 2NF holds

Check 3NF: is there a transitive dependency (non-key → non-
key)?
  C→D: here C is a NON-KEY attribute, and D is also non-key
→ this IS a transitive
  dependency (key → C → D, where C itself isn't part of the
key) → ✗ 3NF VIOLATED

Highest normal form satisfied = 2NF
```

FORMULA / RULE

Worked numerical — BCNF check

Q: R(A,B,C) with FDs: A→B, B→C. Candidate key = {A} (since A→B→C by transitivity, A alone determines everything). Is R in BCNF? What about 3NF?

A:

```
BCNF check — every FD's left side must be a super key:
  A→B: A is the candidate key → OK
  B→C: is B a super key? No — B alone doesn't determine A →
this FD VIOLATES BCNF

3NF check — for every FD X→Y, X must be a super key OR Y
must be a prime attribute:
  A→B: A is a super key → OK
  B→C: B is not a super key, AND C is not part of any
candidate key (not prime)
       → this FD VIOLATES 3NF too (C transitively depends
on the key A, through B)

R is NOT in BCNF and NOT in 3NF. Since the key {A} is a
single attribute, no partial
dependency is possible → 2NF is satisfied automatically.
Highest normal form satisfied = 2NF.
```

Decomposition — lossless join test

NOTES

Splitting R into R1, R2 is **lossless** if and only if: $(R1 \cap R2) \to R1$ OR $(R1 \cap R2) \to R2$ (the common attributes must functionally determine at least one of the two pieces).

WORKED EXAMPLE

Worked numerical

Q: R(A,B,C,D), FDs: A→B, A→C, C→D. Decomposed into R1(A,B,C) and R2(C,D). Is this lossless?

A:

```
Common attribute: R1 ∩ R2 = {C}
Check: does C → R1 (i.e. C → ABC)? We only know C→D, not C→A
or C→B → NO
Check: does C → R2 (i.e. C → CD)? C→D is given, and C→C is
trivial → C→CD holds → YES

Since C → R2 holds, the decomposition IS lossless.
```

Finding candidate keys quickly

RULES & FACTS

1. Attributes that **never appear on the right-hand side** of any FD (or in no FD at all) **must be in every candidate key**.
2. Compute the closure of that core. If it contains every attribute, the core is the **only** candidate key.
3. Otherwise add one attribute at a time (then pairs, …) until the closure is everything; keep only minimal sets. Every attribute in some candidate key is **prime**; the others are **non-prime**.

FORMULA / RULE

Worked numerical

Q: R(A, B, C, D, E) with F = { A→B, BC→D, D→E }. Find the candidate keys and the highest normal form.

A: A and C never appear on a right-hand side, so both are in every key. closure(AC) = {A, C, B, D, E} = everything, so **AC is the only candidate key**; prime = {A, C}, non-prime = {B, D, E}.

```
2NF: A→B has a part of the key (A) determining non-prime B →
partial dependency → 2NF violated
1NF holds, so the highest normal form is 1NF.
```

Multiple keys: for R(A, B, C) with AB→C, C→A, C→B we get C→AB, so both **C** and **AB** are candidate keys. Every left-hand side is a superkey, so R is in BCNF.

Dependency preservation vs lossless join

NOTES

A decomposition of R into $R_1 \dots R_n$ is **dependency preserving** if the FDs that hold in the pieces (the projections of F onto each R_i) together still imply every FD of F, so each FD can be checked *within a single table* without a join. It is **lossless** if joining the pieces gives back exactly R. For a binary split into R_1 and R_2: lossless iff $(R_1 \cap R_2) \to R_1$ or $(R_1 \cap R_2) \to R_2$ (the common attributes are a key of one side).

REFERENCE TABLE

Decomposition method	Lossless?	Dependency preserving?
3NF synthesis from a minimal cover	**Always**	**Always**
BCNF decomposition	**Always**	**Not guaranteed**

FORMULA / RULE

Worked example

Q: R(C, S, Z) (city, street, zip) with F = { CS→Z, Z→C }. Candidate keys: CS and SZ. Is Z→C a BCNF violation, and what happens if we decompose on it?

A: Z is not a superkey, so Z→C violates BCNF (but R is in 3NF because C is prime). Decompose into R_1(Z, C) and R_2(S, Z). The common attribute Z is a key of R_1, so the split is **lossless**. But C and S now live in different tables, so **CS→Z cannot be enforced** without a join: the decomposition is **not dependency preserving**.

Contrast: R(A, B, C) with F = {A→B, B→C} split into (A, B) and (B, C) is both lossless (B is a key of (B, C)) and dependency preserving. Split into (A, B) and (A, C) is lossless (A is a key) but loses B→C.

Multivalued dependencies and 4NF (brief)

NOTES

X ↠ Y holds when, for each X value, the set of Y values is independent of the remaining attributes: if (x, y_1, z_1) and (x, y_2, z_2) are in the relation, so are (x, y_1, z_2) and (x, y_2, z_1). Every FD is an MVD, but not conversely. **4NF** = BCNF and, for every non-trivial MVD X ↠ Y, X is a superkey. Classic case: Course ↠ Teacher and Course ↠ Book, independently; storing both lists in one table multiplies the rows, and the fix is two tables (Course, Teacher) and (Course, Book). **5NF** goes one step further and removes join dependencies not implied by keys.

5 File Organization & Indexing (B/B+ Trees)

File organization methods

REFERENCE TABLE

Method	How records are stored
Heap (unordered)	Records inserted wherever there's space; fast inserts, slow search (must scan)
Sequential	Records sorted by a key; fast range queries, but inserts/deletes are expensive (must maintain order)
Hashed	A hash function maps a key directly to a storage location/bucket; very fast exact-match lookup, poor for range queries

Index basics

KEY POINT

Clustered index
The table's actual rows are physically stored in the SAME order as the index (only one clustered index possible per table).

KEY POINT

Non-clustered (secondary) index
A separate structure pointing to rows stored elsewhere in some other order (a table can have many of these).

KEY POINT

Dense index
One index entry for EVERY search key value in the file.

KEY POINT

Sparse index
One index entry only for SOME search key values (e.g. one per block) — smaller, but needs the data to be sorted.

B-Tree vs B+-Tree

REFERENCE TABLE

	B-Tree	B+-Tree
Where data pointers live	BOTH internal nodes and leaves hold actual record pointers	ONLY leaf nodes hold record pointers; internal nodes hold just "signpost" copies of keys to guide the search
Leaf linking	Leaves not linked together	Leaves are linked in a chain — enables fast sequential/range scans
Used in practice	Less common	The standard choice for almost every real DBMS index

B+-Tree order formulas — memorize these

KEY POINT

For a B+-tree of **order p** (maximum p pointers/children per internal node):

```
Max keys per internal node        = p - 1
Max children per internal node    = p
Min children per internal node (except root) = ⌈p/2⌉
Min keys per internal node (except root)     = ⌈p/2⌉ - 1
Max keys per leaf node             = p - 1
Min keys per leaf node (except root-as-leaf) = ⌈p/2⌉ - 1
```

(The root is special: it only needs a minimum of 2 children if internal, or 1 key if it's a leaf.)

WORKED EXAMPLE

Worked numerical — min/max number of leaf nodes

Q: A B+-tree index is built on a relation with n = 100,000 records. Each node (leaf or internal) has room for at most p = 8 pointers (so max 7 keys per leaf). Find the minimum and maximum possible number of leaf nodes.

A:

```
Max keys per leaf = p - 1 = 7          → gives the MINIMUM
number of leaves (tree as full as possible)
Min keys per leaf = ⌈p/2⌉ - 1 = 4-1 = 3 → gives the MAXIMUM
number of leaves (tree as sparse as possible)

Minimum leaves = ⌈100000 / 7⌉ = 14,286
Maximum leaves = ⌈100000 / 3⌉ = 33,334
```

This "min fill → max count, max fill → min count" logic is the standard GATE pattern for these questions — always double-check which extreme (fullest vs sparsest) the question is asking about.

FORMULA / RULE

Worked numerical — tree height

Q: Using order p = 101 (max 100 keys per node) and N = 10,000 unique search key records, find the MINIMUM height of the B+-tree (best case: every node packed as full as possible).

A:

```
Leaf level: minimum leaves needed = ⌈10000 / 100⌉ = 100
leaves

One level above the leaves: a single internal node (the
root) can point to up to
p = 101 children — since we only have 100 leaves, ALL of
them fit under ONE root node.

So the tree needs just 2 levels: [Root] → [100 leaves]
Minimum height = 2
```

Hash indexing

REFERENCE TABLE

Scheme	How it works	Weakness
Static hashing	Fixed number of buckets; bucket = h(key) mod M; overflow handled by chaining overflow blocks	Long overflow chains as the file grows; reorganising means rehashing everything
Extendible hashing	A **directory** of 2^d pointers (d = **global depth**) indexed by the first d bits of the hash; each bucket has a **local depth** d′ ≤ d	Directory can double; extra indirection
Linear hashing	Buckets split one at a time in round-robin order (no directory), regardless of which bucket overflowed	Overflow chains exist temporarily

KEY POINT

Extendible hashing rules

```
• Directory has 2^d entries; a bucket with local depth d' is pointed to by exactly 2^(d − d') entries
• Bucket overflows and d' < d  →  split just that bucket (d' + 1), NO directory doubling
• Bucket overflows and d' = d  →  DOUBLE the directory (d + 1), then split the bucket
• Great for equality lookups (about 1–2 I/Os); useless for range queries (use a B+-tree)
```

GATE FAVOURITE

GATE angle

Hash index → exact-match only. If a question mentions a **range** or **ORDER BY** on the key, the B+-tree is the right answer.

Block arithmetic — blocking factor and access cost

REFERENCE TABLE

Quantity	Formula
Blocking factor (records per block)	`bf = ⌊B / R⌋` (B = block size, R = record size, unspanned)
Data blocks for r records	`b = ⌈r / bf⌉`
Heap file search	average `b/2` blocks, worst case `b`
Sorted file, search on the sort key	`⌈log₂ b⌉` (binary search)
Index entries per block	`⌊B / (key + pointer)⌋`
Primary index (sparse, one entry per data block)	index blocks = `⌈b / entries-per-block⌉`; access = `⌈log₂ index blocks⌉ + 1`
Secondary index (dense, one entry per record)	index blocks = `⌈r / entries-per-block⌉`; access = `⌈log₂ index blocks⌉ + 1`
Multilevel index	Index the index until one block remains; access = **number of levels + 1**

FORMULA / RULE

Worked numerical

Q: r = 100,000 records of 128 bytes in 4096-byte blocks, sorted on a key. An index entry is 16 bytes (12-byte key + 4-byte pointer). Find the block accesses for (a) a heap file, (b) binary search on the file, (c) a primary index, (d) a secondary index on a non-ordering field, (e) a multilevel primary index.
A: bf = 4096/128 = 32, so b = ⌈100000/32⌉ = **3125** blocks. Entries per index block = 4096/16 = 256.

```
(a) heap, average        = 3125/2 → 1563 block accesses
(b) binary search        = ⌈log₂ 3125⌉ = 12
(c) primary index        : ⌈3125/256⌉ = 13 index blocks → ⌈log₂ 13⌉ + 1 = 4 + 1 = 5
(d) secondary index      : ⌈100000/256⌉ = 391 blocks → ⌈log₂ 391⌉ + 1 = 9 + 1 = 10
(e) multilevel primary   : 13 blocks → 1 block (2 levels) → 2 + 1 = 3
```

FORMULA / RULE

Worked numerical — B+ tree order from the block size

Q: Block = 4096 B, key = 12 B, tree pointer = 8 B, record pointer = 8 B. Find the order of an internal node and the keys per leaf.
A: Internal node holding p pointers and p − 1 keys: `p·8 + (p − 1)·12 ≤ 4096` → 20p ≤ 4108 → **p = 205** (check: 205·8 + 204·12 = 4088 ≤ 4096). A leaf with k (key, record-pointer) pairs plus one next-leaf pointer: `k·(12 + 8) + 8 ≤ 4096` → k ≤ 204.4, so **204 keys per leaf**.

6 Transactions & Concurrency Control

ACID properties

REFERENCE TABLE

Property	Guarantee
Atomicity	A transaction happens completely, or not at all — no partial updates survive a crash
Consistency	A transaction moves the database from one valid state to another valid state
Isolation	Concurrent transactions don't see each other's uncommitted, intermediate changes
Durability	Once committed, changes survive any subsequent crash

Serializability — the gold standard for correctness

NOTES

A **serial schedule** runs transactions one at a time, start to finish, with zero interleaving — always correct, but slow. An interleaved (concurrent) schedule is **conflict serializable** if it's equivalent to SOME serial schedule, checked using a **precedence graph**:

KEY POINT

Draw an edge **Ti → Tj** whenever an operation of Ti **conflicts** with, and comes before, an operation of Tj on the same data item. Two operations conflict if they're from different transactions, touch the same data item, and **at least one is a WRITE** (R-W, W-R, or W-W all conflict; R-R never conflicts).

```
If the precedence graph has a CYCLE  → NOT conflict serializable
If the precedence graph is ACYCLIC   → conflict serializable
(a topological sort gives an equivalent serial order)
```

WORKED EXAMPLE

Worked numerical

Q: Schedule S: R1(A), R2(A), W1(A), R3(B), W2(B), W3(B). Build the precedence graph and determine if S is conflict serializable.
A:

```
Conflicts on A: R2(A) occurs before W1(A) → different transactions, one is a write → edge T2 → T1

Conflicts on B: R3(B) occurs before W2(B) → edge T3 → T2
                W2(B) occurs before W3(B) → edge T2 → T3

Precedence graph edges: T2→T1,  T3→T2,  T2→T3

Check for a cycle: T2→T3→T2 forms a 2-node cycle (T2 and T3 point to each other)

Result: the graph HAS a cycle → S is NOT conflict serializable.
```

View serializability

NOTES

Two schedules are **view equivalent** if (1) each transaction reads the **initial value** of an item in both or in neither, (2) every **read-from** relation (Ti reads the value written by Tj) is the same in both, and (3) the **final write** on every item is by the same transaction. A schedule is **view serializable** if it is view equivalent to some serial schedule.

KEY POINT

Key facts

```
Conflict serializable ⊂ View serializable ⊂ All schedules
A schedule that is view- but NOT conflict-serializable must contain BLIND WRITES
(a write on an item by a transaction that did not read it first)
Testing view serializability is NP-complete (no simple precedence-graph test)
```

WORKED EXAMPLE

Worked numerical

Q: S: R1(A) W2(A) W1(A) W3(A). Is it conflict serializable? View serializable?
A:

```
Precedence edges: R1(A)→W2(A): T1→T2;  W2(A)→W1(A): T2→T1  → cycle → NOT conflict serializable
View test against serial order T1, T2, T3:
  initial read of A by T1 in both ✓ · no read-from relations in either ✓ · final write on A by T3 in both ✓
→ view equivalent to T1 T2 T3 → VIEW SERIALIZABLE (W2 and W3 are blind writes)
```

Recoverability of schedules

NOTES

Serializability says nothing about what happens when a transaction **aborts**. Recoverability classes handle that:

REFERENCE TABLE

Class	Rule
Recoverable	If Tj **reads from** Ti, then Ti must **commit before** Tj commits
Cascadeless (ACA)	Tj may read an item only **after** the transaction that last wrote it has committed — no dirty reads, so no cascading aborts
Strict	No transaction may **read or overwrite** an item written by Ti until Ti commits or aborts

MUST REMEMBER

```
Strict ⊂ Cascadeless ⊂ Recoverable ⊂ All schedules
Serial schedules are strict · Strict 2PL schedules are strict + conflict serializable
Serializability and recoverability are INDEPENDENT properties (a schedule can have either, both or neither)
```

WORKED EXAMPLE

Worked numerical — classify

```
S1: W1(A) R2(A) C2 C1   T2 read from T1 but committed first → NOT recoverable
S2: W1(A) R2(A) C1 C2   dirty read, but commit order is right       → recoverable, NOT cascadeless
S3: W1(A) W2(A) C1 C2   no reads; T2 overwrites uncommitted A       → cascadeless, NOT strict
S4: W1(A) C1 R2(A) C2   reads/writes only after commit → strict
```

Two-Phase Locking (2PL)

KEY POINT

Every transaction has a **growing phase** (can only ACQUIRE locks, never release) followed by a **shrinking phase** (can only RELEASE locks, never acquire any new ones). The moment a transaction releases its first lock, it may never request another. 2PL guarantees conflict serializability.

REFERENCE TABLE

Variant	Extra rule	Benefit
Basic 2PL	Just the growing/shrinking rule above	Serializable, but can still allow cascading rollbacks
Strict 2PL	ALL locks (especially write locks) held until the transaction commits or aborts	Prevents cascading rollbacks — the industry-standard choice
Rigorous 2PL	Every lock (read AND write) held until commit/abort	Simplifies recovery further; transactions serialize in commit order

Deadlocks — detection and prevention

KEY POINT

Detection: keep a **wait-for graph** (node per transaction; edge Ti → Tj when Ti waits for a lock held by Tj). A **cycle** ⇔ **deadlock**; abort one victim on the cycle. **Prevention** uses transaction timestamps (smaller TS = **older**) when a lock conflict occurs:

REFERENCE TABLE

Scheme	Older requests a lock held by younger	Younger requests a lock held by older
Wait-Die (non-preemptive)	Older **waits**	Younger **dies** (rolled back)
Wound-Wait (preemptive)	Older **wounds** the younger (younger rolled back)	Younger **waits**

COMMON TRAP

Memory trick

In **Wait-Die the OLD wait**; in **Wound-Wait the YOUNG wait**. A restarted transaction keeps its **original timestamp**, so it eventually becomes the oldest — no starvation. Both schemes are deadlock-free (all waits go in one direction of age).

Timestamp Ordering Protocol

NOTES

Every transaction gets a unique timestamp TS(T) when it starts. Each data item tracks the timestamp of its most recent successful read (R-timestamp) and write (W-timestamp). A read or write is **rejected (and the transaction rolled back)** if it would violate the timestamp order — e.g. a transaction tries to write a value that a "younger" (later-timestamped) transaction has already read, which would make that earlier read see a "future" value it shouldn't have.

GATE FAVOURITE

GATE angle

Unlike 2PL (which can cause transactions to WAIT for locks), timestamp ordering never makes a transaction wait — it either proceeds immediately or is rolled back and restarted with a NEW timestamp. This trades lock-waiting for a different cost: rollback/restart overhead.

KEY POINT

Basic TO rules (item Q has R-TS(Q) and W-TS(Q)):

```
Read(Q) by T:   if TS(T) < W-TS(Q)  → reject, roll back T
                else execute; R-TS(Q) = max(R-TS(Q), TS(T))
Write(Q) by T:  if TS(T) < R-TS(Q)  → reject, roll back T (a younger txn already read the old value)
                if TS(T) < W-TS(Q)  → reject, roll back T (basic TO)  /  IGNORE the write (Thomas)
                else execute; W-TS(Q) = TS(T)
```

Thomas write rule: in the case TS(T) < W-TS(Q) the write is *obsolete* (a later write has already overwritten it), so instead of aborting T we simply **skip it**. This admits schedules that are **view serializable but not conflict serializable**. Basic TO guarantees conflict serializability and freedom from deadlock, but not recoverability/cascadelessness on its own.

Multiple-granularity locking & intention locks

NOTES

Data items form a hierarchy (database → file → page → record). Locking a coarse node implicitly locks everything below it; **intention locks** on ancestors announce that finer locks are held below.

REFERENCE TABLE

Held ↓ / Requested →	IS	IX	S	SIX	X
IS	✔	✔	✔	✔	✘
IX	✔	✔	✘	✘	✘
S	✔	✘	✔	✘	✘
SIX	✔	✘	✘	✘	✘
X	✘	✘	✘	✘	✘

KEY POINT

```
To lock a node in S or IS → must hold IS or IX on its parent
To lock a node in X, IX or SIX → must hold IX or SIX on its parent
Lock from the ROOT downward; release from the LEAF upward
SIX = S on the whole subtree + IX (read everything, update a few records)
```

Isolation levels and the anomalies they allow

REFERENCE TABLE

Level (SQL standard)	Dirty read	Non-repeatable read	Phantom
Read Uncommitted	possible	possible	possible
Read Committed	**prevented**	possible	possible
Repeatable Read	prevented	**prevented**	possible
Serializable	prevented	prevented	**prevented**

NOTES

Dirty read: reading another transaction's uncommitted write. **Non-repeatable read:** reading the same row twice and getting different values because another transaction committed an update in between. **Phantom:** re-running a range query and seeing new rows that another transaction inserted.

Lock modes and compatibility

REFERENCE TABLE

Held ↓ / Requested →	Shared (S)	Exclusive (X)
Shared (S)	granted	must wait
Exclusive (X)	must wait	must wait

NOTES

Many readers can share an item, but a writer needs it alone. A transaction may **upgrade** S→X (only if it is the sole holder) and, in some schemes, downgrade X→S. The **lock point** is the moment a 2PL transaction holds all its locks, and ordering transactions by their lock points gives an equivalent serial order. Basic 2PL guarantees conflict serializability but **not** freedom from deadlock and **not** recoverability (a transaction may release a lock on dirty data); strict 2PL, which holds X locks until commit, fixes the second.

Recovery — write-ahead logging

RULES & FACTS

- **Log record** `<T, X, old, new>` is written to stable storage **before** the changed page reaches disk (**write-ahead logging**). A transaction counts as committed once its `<T commit>` record is on disk.
- **Deferred modification** writes to the database only after commit, so recovery needs only **REDO**. **Immediate modification** may write before commit, so recovery needs **UNDO** (and REDO).
- **Checkpoint:** flush logs and dirty pages and note the active transactions, so recovery never scans the log before it.
- **Recovery after a crash:** REDO transactions that committed after the last checkpoint; UNDO transactions that were active (started, no commit) at the crash.

WORKED EXAMPLE

Worked example

Q: Log: <T1 start>, <T1, A, 10, 20>, <T1 commit>, <checkpoint>, <T2 start>, <T2, B, 5, 8>, <T3 start>, <T3, C, 1, 2>, <T3 commit>, then a crash. What does recovery do?

A: T1 committed **before** the checkpoint, so its updates are already on disk: nothing to do. T3 committed after the checkpoint: **REDO** it (C = 2). T2 never committed: **UNDO** it (B back to 5, its old value).

FORMULA / RULE

Shadow paging keeps a current page table and a shadow page table; updates go to new pages (copy-on-write), commit atomically swaps the table pointer, and abort just discards the new pages, so no log is needed but data gets fragmented. **ARIES** uses write-ahead logging with log sequence numbers (LSNs) and recovers in three passes: **analysis** (find dirty pages and active transactions), **redo** (repeat history from the log, even for transactions that will be undone) and **undo** (roll back the losers).

7 Problem → Standard Method Playbook

METHOD

“for every / all” query

Recognize: an entity must be related to *every* member of another relation.

```
1. Project the required entity-key pairs.
2. Use relational division R ÷ S, or
3. rewrite as NOT EXISTS (missing required match).
4. Check that duplicates cannot change the logical result.
```

Trap: ordinary join + GROUP BY needs a count equal to the number of required values; forgetting this turns “all” into “at least one”.

METHOD

Minimal / canonical cover

```
1. Split every RHS: X → YZ becomes X → Y, X → Z.
2. Minimize each LHS: remove an attribute if the reduced
determinant still implies the RHS.
3. Remove redundant FDs: temporarily delete one FD and test
whether it follows from the rest using attribute closure.
4. Recombine equivalent RHSs only at the end.
```

Trap: do not delete an FD merely because its RHS appears elsewhere; test implication.

METHOD

Schedule → conflict serializability

```
1. Keep only conflicting pairs: same item, different
transactions, at least one WRITE.
2. Draw Ti → Tj when Ti's operation occurs first.
3. Acyclic precedence graph ⇒ conflict serializable.
4. A cycle ⇒ not conflict serializable.
5. If needed, topologically order the acyclic graph to
obtain the equivalent serial order.
```

METHOD

View serializability / Thomas write rule

```
Track: initial reads, reads-from relationships, and final
writer of each item.
Thomas write rule: if TS(T) < W-TS(X), T's write is obsolete
→ ignore it rather than aborting T.
Use this when the schedule is not conflict-serializable but
may still be view-serializable.
```

METHOD

Locking / deadlock prevention

2PL: growing phase acquires locks; shrinking phase releases them. Strict 2PL keeps write locks until commit/abort.

Wait-die: older waits; younger aborts. **Wound-wait:** older preempts/aborts younger; younger waits for older.

Intention locks: use IS/IX/SIX on ancestors before finer-grained locks below.

METHOD

Index / access-path choice

```
Equality on a hashable key → hash index is attractive.
Range / ORDER BY / predecessor-successor → B+ tree.
B+ tree search: follow separator keys to leaf; linked leaves
support ordered/range scans.
Always distinguish logical comparison count from disk
I/O/page accesses.
```

1. Relational algebra division

METHOD NOTES

R(Student, Course) = {(A,DBMS),(A,CN),(B,DBMS),(C,DBMS),(C,CN)}. Required(Course) = {DBMS,CN}. Find the students who took **all** required courses.

Student A has both required pairs; B lacks CN; C has both. Therefore the result is **{A,C}**.

SQL equivalent: select students for which `NOT EXISTS` a required course missing from R.

2. Minimal cover

METHOD NOTES

Given F = {A→BC, B→C, A→B, AB→C}. Find a minimal cover.

Step 1: split A→BC into A→B and A→C.

Step 2: AB→C has extraneous B because A→C is already implied by the other dependencies.

Step 3: A→C is redundant because A→B and B→C imply A→C.

One minimal cover is **{A→B, B→C}**.

3. Precedence graph

METHOD NOTES

Schedule: $r_1(X)$, $w_1(X)$, $r_2(X)$, $w_2(X)$, $r_1(Y)$, $w_2(Y)$. What does the conflict graph imply?

On X, T_1's write precedes T_2's read/write ⇒ $T_1 \to T_2$. On Y, T_1 reads before T_2 writes ⇒ $T_1 \to T_2$. The graph is acyclic, so the schedule is **conflict serializable**, equivalent to serial order **T_1 then T_2**.

Trap: r-r pairs do not create edges.

4. Wait-die vs wound-wait

METHOD NOTES

T_1 is older than T_2. T_1 requests a lock held by T_2. What happens?

Wait-die: older T_1 may **wait** for younger T_2. **Wound-wait:** older T_1 **wounds** T_2, so T_2 is rolled back and T_1 obtains the lock.

Memory rule: wait-die = old waits; wound-wait = old wounds.

5. B+ tree vs hash

METHOD NOTES

A query asks for all employee IDs in the range 5000–7000. Which index is structurally suited?

B+ tree. After reaching the first qualifying leaf, linked leaf pages can be scanned in key order. A hash index is optimized for equality lookup and does not preserve key order for efficient range scans.

⚡ DBMS Last-Minute Recall

METHOD

- **Division = ALL**; join alone normally means “at least one”.
- Minimal cover: **split RHS → minimize LHS → remove redundant FDs**.
- Precedence graph: edge for same item + different transactions + at least one write.
- Acyclic precedence graph ⇔ conflict serializable.
- Thomas write rule can ignore obsolete writes; it can admit view-serializable schedules that are not conflict-serializable.
- **Wait-die:** older waits. **Wound-wait:** older aborts younger.
- B+ tree → ordered/range access; hash → equality access.

8 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- **Intention locks:** IS/IX on ancestors; SIX = S + IX; X conflicts with everything; IX compatible only with IS and IX.
- Selection filters rows; projection selects columns. Projection can remove duplicate tuples in the relational model.
- 2NF: 1NF + no non-prime attribute partially dependent on a candidate key. 3NF: for every non-trivial FD $X \to A$, X is a superkey or A is prime.
- B+ tree height and fan-out depend on node capacity, key/pointer sizes, and root/leaf structure; use the exact convention given in a question.
- Blocks $b = \lceil r/bf \rceil$ with $bf = \lfloor B/R \rfloor$; sorted-file search $\lceil \log_2 b \rceil$; primary index $\lceil \log_2 \text{index blocks} \rceil + 1$; multilevel index = levels + 1. B+ internal order p: $p \cdot P + (p-1) \cdot K \le B$.

LAST-MINUTE · MUST REMEMBER

- A candidate key is a minimal superkey; a primary key is one selected candidate key.
- Natural join matches attributes with the same names; be precise about join attributes and duplicate-column handling.
- WHERE filters rows before grouping; HAVING filters groups after GROUP BY.
- Functional dependency $X \to Y$ means equal X values imply equal Y values; it is a constraint on relation instances.
- B+ tree data pointers/records are stored at leaves; internal nodes primarily guide search. Leaf nodes are linked for ordered/range access.
- Conflict serializability can be tested using a precedence graph: acyclic iff the schedule is conflict-serializable.
- Two-phase locking has growing and shrinking phases; strict 2PL holds exclusive locks until commit/abort.
- **Candidate keys:** attributes never on any right-hand side are in every key; take the closure of that core. Prime attributes belong to some candidate key.
- **Isolation levels:** Read Committed stops dirty reads; Repeatable Read also stops non-repeatable reads; only Serializable stops phantoms.

LAST-MINUTE · COMMON TRAPS

- Serializability and recoverability are independent — a schedule can be one without the other.
- Hash index = equality only; range queries need a B+-tree. Extendible hashing: entries to a bucket = $2^{\text{global} - \text{local}}$.
- SQL NULL is not equal to anything, including another NULL; comparisons involving NULL use UNKNOWN.
- Recoverability, cascadelessness, strictness, and serializability are different properties—do not treat them as synonyms.
- BCNF decomposition is always lossless but may lose dependencies; 3NF synthesis from a minimal cover is always lossless and dependency preserving. Binary split lossless iff the common attributes are a key of one side.
- Basic 2PL gives conflict serializability but not deadlock freedom or recoverability; strict 2PL holds X locks to commit. 4NF removes non-trivial multivalued dependencies whose left side is not a superkey.

LAST-MINUTE · GATE FAVOURITES

- **Containment:** Strict $\subset$ Cascadeless $\subset$ Recoverable; Conflict-serializable $\subset$ View-serializable.
- BCNF requires every non-trivial FD determinant to be a superkey; BCNF is stricter than 3NF.

Compiler Design

GATE CSE 2027 · complete formula, rule, method & trap sheet

75 reference boxes · **9** chapters ·

Compiler Phases & Lexical Analysis · Top-Down Parsing (LL) · Bottom-Up Parsing (LR family) · Syntax-Directed Translation · Runtime Environments · Intermediate Code Generation · Code Optimisation & Code Generation · Problem → Standard Method Playbook · Last-minute one-liners

1 Compiler Phases & Lexical Analysis

The phases of a compiler — Core

NOTES

A compiler translates a source program into target code in a sequence of phases. The first three phases **analyse** the source (the **front end**); the rest **synthesise** the target program (the **back end**). The **symbol table** and **error handler** are used by every phase.

FLOW

Source code → Lexical analysis → Syntax analysis → Semantic analysis → Intermediate code gen → Code optimisation → Code generation → Target code

REFERENCE TABLE

Phase	Input → Output	Typical errors caught
Lexical analysis	characters → **tokens**	illegal character, unterminated string/comment
Syntax analysis	tokens → **parse tree**	missing `;` or `)`, malformed statement
Semantic analysis	parse tree → annotated tree	type mismatch, undeclared variable, wrong argument count
Intermediate code gen	annotated tree → three-address code	—
Code optimisation	IR → better IR	—
Code generation	IR → machine / assembly code	—

GATE FAVOURITE

GATE angle

"Which phase detects this error?" is a favourite. Remember: **lexer** = bad characters/tokens, **parser** = bad grammar structure, **semantic analyser** = bad meaning (types, declarations). Also: a **pass** is one traversal over the whole program and may bundle several phases; a **phase** is a logical step.

Tokens, patterns, lexemes — Core

REFERENCE TABLE

Term	Meaning	Example
Lexeme	actual character sequence in the source	`count`, `3.14`, `>=`
Token	(token-name, attribute) pair given to the parser	(id, ptr to symbol table), (num, 3.14), (relop, GE)
Pattern	rule describing the lexemes of a token, usually a regular expression	`letter (letter|digit)*` for identifiers

NOTES

Token patterns are **regular languages**, so a lexical analyser is built as a finite automaton: regular expressions → NFA → DFA → minimised DFA. Tools like **Lex/Flex** automate this.

Two disambiguation rules

RULES & FACTS

1. **Longest match (maximal munch):** the lexer always consumes the longest prefix that matches some pattern.
2. **Priority:** if two patterns match the same longest lexeme (e.g. `if` as keyword vs identifier), the pattern listed **first** wins — so keywords are listed before the identifier rule.

WORKED EXAMPLE

Worked numerical — counting tokens

Q: How many tokens are in `if (a >= b) x = a-1;` ?
A: Split by maximal munch:

```
if  (  a  >=  b  )  x  =  a  -  1  ;
 1  2  3  4   5  6  7  8  9  10 11 12   → 12 tokens
```

Note `>=` is **one** token (longest match), not `>` and `=`.

WORKED EXAMPLE

Worked numerical — maximal munch trap

Q: Tokenise `a+++b` in C.
A: `a`, then the longest match from `+++b` is `++`, then `+`, then `b`:

```
a   ++   +   b   → 4 tokens   (parsed as (a++) + b)
```

A string literal such as `"i=%d"` is always a **single** token, however many characters it holds.

Input buffering & the symbol table — High-yield

NOTES

Reading character by character is slow, so lexers use a **two-buffer scheme** with a **sentinel** (eof) at the end of each half: one comparison per character normally, and an extra check only when the sentinel is hit. The lexer also enters each new identifier into the **symbol table**; the token's attribute is a pointer to that entry. Whitespace and comments are stripped here and never reach the parser.

COMMON TRAP

What the lexer cannot do

It sees one token at a time with no notion of nesting, so it cannot check balanced brackets, a missing semicolon, or types. That is why syntax needs a grammar (context-free), not just regular expressions.

2 Top-Down Parsing (LL)

Preparing a grammar for top-down parsing — Core

KEY POINT

Remove left recursion (a top-down parser would loop forever)

```
A → Aα | β
⇓
A  → β A'
A' → α A' | ε
```

KEY POINT

Left-factor (so one lookahead can choose)

```
A → αβ1 | αβ2
⇓
A  → α A'
A' → β1 | β2
```

FIRST and FOLLOW Core

RULES & FACTS

- **FIRST(X)**: set of terminals that can begin a string derived from X (include ε if X ⇒* ε).
- **FOLLOW(A)**: set of terminals that can appear immediately to the right of A in some sentential form. FOLLOW(start) always contains `$`. ε is never in FOLLOW.

NOTES

FOLLOW rules for each production `B → α A β`: add FIRST(β) \{ε} to FOLLOW(A); if β is empty or β ⇒* ε, add FOLLOW(B) to FOLLOW(A).

WORKED EXAMPLE

Worked numerical — FIRST, FOLLOW and the LL(1) table

Q: Grammar (the left-recursion-free expression grammar):

```
E  → T E'
E' → + T E' | ε
T  → F T'
T' → * F T' | ε
F  → ( E ) | id
```

A:

```
FIRST(E) = FIRST(T) = FIRST(F) = { (, id }
FIRST(E') = { +, ε }        FIRST(T') = { *, ε }

FOLLOW(E)  = { $, ) }
FOLLOW(E') = FOLLOW(E)                = { $, ) }
FOLLOW(T)  = FIRST(E')\ε ∪ FOLLOW(E') = { +, $, ) }     (E' is nullable)
FOLLOW(T') = FOLLOW(T)                = { +, $, ) }
FOLLOW(F)  = FIRST(T')\ε ∪ FOLLOW(T)  = { *, +, $, ) }
```

Place `A → α` in cell M[A, a] for every a ∈ FIRST(α); if α ⇒* ε also for every a ∈ FOLLOW(A):

	id	+	*	(	)	$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id			F→(E)		

No cell has two entries → the grammar is **LL(1)**.

The LL(1) condition Core

NOTES

A grammar is LL(1) iff for every pair of productions `A → α | β`:

RULES & FACTS

1. FIRST(α) ∩ FIRST(β) = ∅.
2. At most one of α, β can derive ε.
3. If β ⇒* ε, then FIRST(α) ∩ FOLLOW(A) = ∅.

NOTES

Equivalently: **every cell of the parsing table has at most one entry**.

COMMON TRAP

Must-know facts

• Left-recursive, ambiguous, or non-left-factored grammars are **never** LL(1).
• Every LL(1) grammar is unambiguous and is also LR(1) — but not the other way round.
• LL(1) parsers need no backtracking; they use an explicit **stack** (start with `$ S`) and the table.

WORKED EXAMPLE

Worked numerical — a grammar that is NOT LL(1)

Q: The dangling-else grammar `S → i E t S S' | a, S' → e S | ε, E → b`. Is it LL(1)?
A: FOLLOW(S') = FOLLOW(S) = { e, $ } (because S → iEtS**S'** and S appears before S' giving e). The production S' → ε goes under FOLLOW(S') which contains `e`, but S' → eS also sits in M[S', e]. Two entries in one cell → **not LL(1)**.

Parse trace (table-driven) Core

NOTES

Input `id + id $` with the table above (stack top on the left):

FORMULA / WORKING

```
Stack        Input      Action
E $          id+id $    E → T E'
T E' $       id+id $    T → F T'
F T' E' $    id+id $    F → id
id T' E' $   id+id $    match id
T' E' $      +id $      T' → ε
E' $         +id $      E' → + T E'
+ T E' $     +id $      match +
T E' $       id $       T → F T'
F T' E' $    id $       F → id
id T' E' $   id $       match id
T' E' $      $          T' → ε
E' $         $          E' → ε
$            $          ACCEPT
```

3 Bottom-Up Parsing (LR family)

Shift-reduce parsing Core

NOTES

A stack holds grammar symbols; the parser repeatedly does one of four things: **shift** (push next input token), **reduce** (replace a **handle** on top of the stack by the LHS of its production), **accept**, or **error**.

RULES & FACTS

- **Handle**: the substring that matches the RHS of a production and whose reduction is one step of the reverse rightmost derivation.
- **Viable prefix**: a prefix of a right-sentential form that can appear on the stack (never extends past the handle).
- **Conflicts**: **shift-reduce** (both legal) or **reduce-reduce** (two reductions legal). Ambiguous grammars always produce conflicts.

WORKED EXAMPLE

Worked numerical — finding the handle

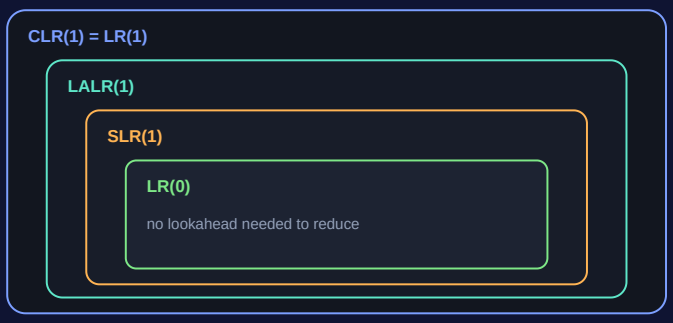
Q: Grammar `E → E+T | T, T → T*F | F, F → id`. Find the handle of the right-sentential form `E + T * id`.
A: Reverse the rightmost derivation: the rightmost symbol `id` was produced by `F → id`, so reduce it first.

```
E + T * id   ⇐ (reduce F→id)  E + T * F
E + T * F    ⇐ (reduce T→T*F) E + T
E + T        ⇐ (reduce E→E+T) E
```

Handle of `E + T * id` = **id** (reducing by F → id).

Power of the LR parsers Core

DIAGRAM



REFERENCE TABLE

Parser	Items	Reduce A→α when lookahead ∈	No. of states
LR(0)	LR(0) items	every symbol (no lookahead)	n
SLR(1)	LR(0) items	FOLLOW(A)	n (same as LR(0))
LALR(1)	LR(1) items, states with same core merged	merged lookahead set	n (same as LR(0))
CLR(1)	LR(1) items	exact lookahead set of the item	≥ n (largest)

GATE FAVOURITE

GATE angle

- LR(0), SLR(1) and LALR(1) tables have the **same number of states**; CLR(1) has the same or more.
- Merging LR(1) states into LALR can create **reduce-reduce** conflicts, but never a new **shift-reduce** conflict.
- No ambiguous grammar is LR(k) for any k. LL(1) ⊂ LR(1), but an LL(1) grammar need not be SLR(1).
- YACC/Bison generate **LALR(1)** parsers; they resolve S/R by shifting and R/R by the earlier production.

Building the LR(0) automaton Core

NOTES

An **item** is a production with a dot marking how much has been seen (`A → α·β`). **Closure** adds `B → ·γ` for every nonterminal B right after a dot; **goto(I, X)** moves the dot over X. Always **augment** the grammar with `S' → S` first.

WORKED EXAMPLE

Worked numerical — canonical LR(0) states

Q: Build the LR(0) automaton for `(1) S → AA, (2) A → aA, (3) A → b`.
A:

```
I0: S'→·S   S→·AA   A→·aA   A→·b
I1 = goto(I0,S): S'→S·                          (accept on $)
I2 = goto(I0,A): S→A·A   A→·aA   A→·b
I3 = goto(I0,a): A→a·A   A→·aA   A→·b
I4 = goto(I0,b): A→b·
I5 = goto(I2,A): S→AA·
I6 = goto(I3,A): A→aA·
goto(I2,a)=I3, goto(I2,b)=I4, goto(I3,a)=I3, goto(I3,b)=I4
```

7 states (I0–I6). The same grammar has **10 states** as a CLR(1) automaton, because states I3, I4, I6 each split into two copies with different lookaheads ({a,b} vs {$}).

NOTES

SLR(1) table — FOLLOW(S)={$}, FOLLOW(A)={a,b,$}:

REFERENCE TABLE

State	ACTION			GOTO	
	a	b	$	S	A
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5			r1		
6	r2	r2	r2		

NOTES

Trace on `abb$` (stack holds alternating symbol/state):

FORMULA / WORKING

```
Stack          Input   Action
0              abb$    shift 3
0 a 3          bb$     shift 4
0 a 3 b 4      b$      reduce A→b   (pop 2, goto(3,A)=6)
0 a 3 A 6      b$      reduce A→aA  (pop 4, goto(0,A)=2)
0 A 2          b$      shift 4
0 A 2 b 4      $       reduce A→b   (pop 2, goto(2,A)=5)
0 A 2 A 5      $       reduce S→AA  (pop 4, goto(0,S)=1)
0 S 1          $       ACCEPT
```

NOTES

A reduction by `A → α` pops **2·|α|** stack entries (symbol + state each), or just |α| states if only states are stored.

When SLR(1) is not enough Core

NOTE

Classic example

Grammar: `S → L = R | R, L → *R | id, R → L`

```
FOLLOW(S) = {$}      FOLLOW(L) = {=, $}      FOLLOW(R) = {=, $}

State after seeing L at the start:   S → L· = R      R → L·
 • shift on '='                      (from S → L·=R)
 • reduce R→L if '=' ∈ FOLLOW(R)  ✔  → shift-reduce conflict on '='
```

Not SLR(1) — FOLLOW is too coarse. But the true lookahead for `R → L·` in this state is only `$`, so the grammar is **LALR(1) and CLR(1)** (CLR: 14 states, LALR: 10).

COMMON TRAP

Another gotcha

`S → AaAb | BbBa, A → ε, B → ε` is **LL(1)** but **not SLR(1)**: after the empty prefix, both `A→ε` and `B→ε` reduce on {a,b} (FOLLOW sets), a reduce-reduce conflict. With exact LR(1) lookaheads (A on `a`, B on `b`) it is conflict-free.

Operator-precedence parsing (quick note) Extension

NOTES

A simpler bottom-up method for **operator grammars** (no ε-productions, no two adjacent nonterminals on any RHS). It uses precedence relations `⋖ ≐ ⋗` between terminals only, which makes it fast but less general than LR.

4 Syntax-Directed Translation

SDD vs SDT Core

RULES & FACTS

- **Syntax-Directed Definition (SDD)**: a CFG with semantic rules attached to productions — it says *what* to compute, not the order.
- **Syntax-Directed Translation scheme (SDT)**: a CFG with **program fragments (actions) embedded** in the RHS — the position of an action fixes *when* it runs.

Synthesized vs inherited attributes Core

KEY POINT

Synthesized

Value of a node is computed from its **children** (and the node's own attributes). Flows **bottom-up**.

```
E → E1 + T   E.val = E1.val + T.val
```

KEY POINT

Inherited

Value of a node comes from its **parent and/or siblings**. Flows **top-down / sideways**.

```
D → T L   L.type = T.type
```

REFERENCE TABLE

Class	Rule	Evaluation
S-attributed	only synthesized attributes	bottom-up; fits LR parsing (evaluate at reduce time)
L-attributed	inherited attribute of a RHS symbol may depend only on the parent's inherited attributes and symbols **to its left** (never to the right, never its own synthesized attributes)	single left-to-right depth-first traversal; fits LL parsing

GATE FAVOURITE

GATE angle

Every **S-attributed** SDD is also **L-attributed** (it has no inherited attributes at all), but not vice versa. The **dependency graph** among attributes must be **acyclic** for any valid evaluation order to exist.

WORKED EXAMPLE

Worked numerical — evaluating a synthesized attribute

Q: SDD: `E → E1 + T {E.val = E1.val + T.val} | T {E.val = T.val}`, `T → T1 * F {T.val = T1.val × F.val} | F`, `F → digit {F.val = digit.lexval}`. Compute E.val for `3 + 4 * 5`.

A: The grammar gives `*` higher precedence, so the tree is E → E + T with the right T → T * F:

```
F.val = 3   → T.val = 3   → E1.val = 3
F.val = 4, F.val = 5 → T.val = 4 × 5 = 20
E.val = E1.val + T.val = 3 + 20 = 23
```

SDT for translation — infix to postfix Core

NOTES

Place the `print` action at the **end** of the RHS (a **postfix SDT**); it then runs exactly when the production is reduced, so it works with bottom-up parsers:

FORMULA / WORKING

```
E → E + T   { print('+') }
E → T
T → T * F   { print('*') }
T → F
F → num     { print(num.val) }
```

WORKED EXAMPLE

Worked numerical

Q: What is printed for input `2 + 3 * 4 + 5`?

A: Parse as `(2 + (3*4)) + 5`. Actions fire in reduction order:

```
2   3   4   *   +   5   +      → output:  2 3 4 * + 5 +
```

COMMON TRAP

Rule of thumb

Actions at the **end** of every production behave like a **post-order** walk of the parse tree. An action placed in the middle of a RHS runs after the symbols to its left have been parsed — in an LR parser this is handled by introducing an ε-**marker** nonterminal.

Inherited attribute example — declarations Core

FORMULA / WORKING

```
D → T L          { L.type = T.type }
T → int          { T.type = integer }
L → L1 , id      { L1.type = L.type;  addType(id.entry,
L.type) }
L → id           { addType(id.entry, L.type) }
```

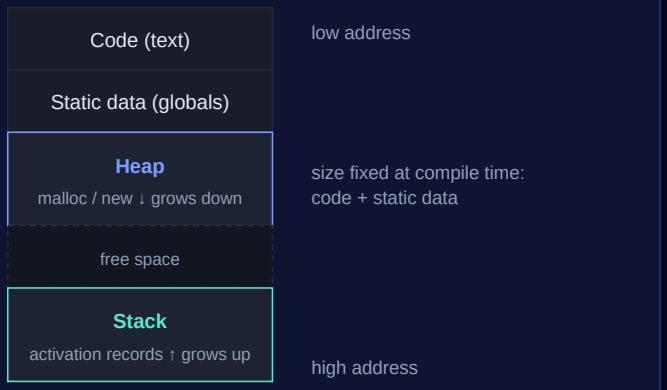
NOTES

For `int a, b, c` : T.type = integer is passed **down** into L, then copied left-to-right along the list, so every identifier is entered in the symbol table with type integer. L.type is an **inherited** attribute — this SDD is L-attributed but **not** S-attributed.

5 Runtime Environments

Storage organisation Core

DIAGRAM



NOTES

Code and static data have sizes known at compile time. The **stack** (procedure calls) and **heap** (dynamic allocation) grow toward each other into the free space. The textbook figure usually draws heap growing up and stack growing down; what matters is that they grow **towards each other**.

Activation record (stack frame) Core

NOTES

Each procedure call pushes one **activation record**; it is popped on return. Typical fields, top to bottom:

REFERENCE TABLE

Field	Purpose
Actual parameters	arguments passed by the caller
Returned value	space for the function's result
Control link (dynamic link)	points to the **caller's** activation record
Access link (static link)	points to the activation record of the **lexically enclosing** procedure — used to reach non-local variables
Saved machine status	return address, saved registers
Local data & temporaries	local variables and compiler temporaries

NOTES

Because every call gets its own record, **recursion** works: each invocation has independent locals. Stack depth equals the number of live (not yet returned) calls.

WORKED EXAMPLE

Worked numerical — access links

Q: In a language with nested procedures, a procedure at nesting depth 4 refers to a variable declared in an enclosing procedure at depth 2. How many access links must be followed?

A: Follow `4 − 2 = 2` access links (each hop climbs one lexical level).

Static vs dynamic scoping Core

RULES & FACTS

- **Static (lexical) scoping**: a non-local name is resolved by the **textual nesting** of the source. Used by C, Java, Python, Pascal.
- **Dynamic scoping**: a non-local name is resolved by the **chain of callers** at run time (follow control links).

WORKED EXAMPLE

Worked numerical — what is printed?

```
int x = 10;
void f()  { print(x); }
void g()  { int x = 20; f(); }
main()    { g(); }
```

Static scoping: f's `x` is the global (f is written outside g) → prints **10**.
Dynamic scoping: f was called by g, so the nearest `x` in the call chain is g's → prints **20**.

Parameter passing Core

REFERENCE TABLE

Mechanism	What is passed	Effect
Call by value	copy of the value	callee cannot change caller's variable
Call by reference	address of the actual	formal is an alias; changes visible to caller immediately
Copy-restore (value-result)	copy in, then copy back on return	changes visible only at return
Call by name	the **expression** itself, re-evaluated at every use	like textual substitution (macro)

WORKED EXAMPLE

Worked numerical — four mechanisms, one program

```
int i = 1;
void P(int x, int y) { x = x + 1;  y = y * 2; }
main: P(i, i);   print(i);
```

```
Value          : x, y are copies; i stays
→ 1
Reference      : x, y both alias i: i=1+1=2, then i=2*2=4
→ 4
Copy-restore   : x=1,y=1 → x=2, y=2; restore x then y: i=2,
i=2  → 2
Name           : x and y both mean i, same as reference
→ 4
```

Copy-restore depends on the **order** of copying back (assumed left to right here).

COMMON TRAP

Reference vs name

They differ when the actual is an expression whose variables change inside the callee — e.g. `A[i]` . Reference fixes the address **at call time**; name re-evaluates `A[i]` with the **current** i at every use (see solved problem 6).

6 Intermediate Code Generation

Three-address code (TAC) Core

NOTES

Each instruction has **at most one operator on the right-hand side** and at most three addresses: `x = y op z` . Complex expressions are broken up using compiler-generated **temporaries**.

WORKED EXAMPLE

Worked numerical — expression to TAC

Q: Write TAC for `a = b + c * d - e` .

A:

```
t1 = c * d
t2 = b + t1
a  = t2 - e          → 3 instructions, 2 temporaries (t1,
t2)
```

Representations

REFERENCE TABLE

Form	Fields	Notes
Quadruples	(op, arg1, arg2, result)	result is an explicit name → easy to move instructions when optimising
Triples	(op, arg1, arg2)	no result field; a result is referred to by the **position** of its triple → saves space but reordering is hard
Indirect triples	triples + a separate list of pointers	reorder by changing the pointer list, not the triples

WORKED EXAMPLE

Worked numerical — the three forms

Q: Represent `a = b * -c + b * -c`.

A:

```
Quadruples
   op      arg1  arg2  result
0  uminus  c           t1
1  *       b     t1    t2
2  uminus  c           t3
3  *       b     t3    t4
4  +       t2    t4    t5
5  =       t5          a          → 6 quadruples

Triples (positions are the names)
0  uminus  c
1  *       b     (0)
2  uminus  c
3  *       b     (2)
4  +       (1)   (3)
5  =       a     (4)
```

Quadruples need 4 fields × 6 rows, triples 3 fields × 6 rows. A DAG would share `b * -c` and need only 4 operations.

Array address calculation Core

NOTES

For an array with element width *w* and base address *base* (indices from 0):

KEY POINT

1-D

```
addr(A[i]) = base + i·w
```

KEY POINT

2-D, row-major, A[R][C]

```
addr(A[i][j]) = base + (i·C + j)·w
```

KEY POINT

2-D, column-major

```
addr(A[i][j]) = base + (j·R + i)·w
```

KEY POINT

Lower bounds l1, l2 (1-based etc.)

```
row-major: base + ((i−l1)·C + (j−l2))·w
```

WORKED EXAMPLE

Worked numerical

Q: `int A[10][20]`, base = 1000, w = 4. Find the address of `A[3][5]` if (a) 0-indexed row-major, (b) 0-indexed column-major, (c) 1-indexed row-major.

A:

```
(a) 1000 + (3·20 + 5)·4 = 1000 + 260 = 1260
(b) 1000 + (5·10 + 3)·4 = 1000 + 212 = 1212
(c) 1000 + ((3−1)·20 + (5−1))·4 = 1000 + 176 = 1176
```

Basic blocks and the control-flow graph Core

NOTES

A **basic block** is a maximal straight-line sequence with a single entry (the first instruction) and a single exit (the last). Find the blocks with the **leader** rules:

RULES & FACTS

1. The first instruction of the program is a leader.
2. Any instruction that is the **target** of a jump is a leader.
3. Any instruction **immediately after** a (conditional or unconditional) jump is a leader.

NOTES

Each block runs from a leader up to (not including) the next leader. The **flow graph** has an edge B→C if C can follow B: C's leader is the jump target of B's last instruction, or C directly follows B and B does not end in an unconditional jump.

WORKED EXAMPLE

Worked numerical — leaders, blocks, edges

```
1: i = 0
2: s = 0
3: if i >= n goto 9
4: t = i % 2
5: if t == 0 goto 7
6: s = s + i
7: i = i + 1
8: goto 3
9: return s
```

Targets: 9, 7, 3. After jumps: 4 (after 3), 6 (after 5), 9 (after 8). Leaders = {1, 3, 4, 6, 7, 9}.

```
B1 = {1,2}   B2 = {3}   B3 = {4,5}   B4 = {6}   B5 = {7,8}
B6 = {9}
Edges: B1→B2, B2→B3, B2→B6, B3→B4, B3→B5, B4→B5, B5→B2
```

6 basic blocks, 7 edges. The loop is B2 → B3 → (B4) → B5 → B2.

Backpatching Core

NOTES

In one pass, jump targets of **forward** jumps (boolean expressions, `if`, `while`) are not yet known. The compiler emits the jump with an empty target, records it in a **list** (`truelist` / `falselist`), and later `backpatch(list, label)` fills in the address once it is known.

7 Code Optimisation & Code Generation

Machine-independent optimisations Core

REFERENCE TABLE

Technique	Before	After
Constant folding	`x = 3 * 4`	`x = 12`
Constant propagation	`x = 5; y = x + 1`	`y = 6`
Common subexpression elimination	`a = b+c; d = b+c`	`a = b+c; d = a`
Copy propagation	`x = y; z = x + 1`	`z = y + 1`
Dead-code elimination	assignment whose value is never used	removed
Strength reduction	`x * 2`, `x * 8`	`x << 1`, `x << 3`
Loop-invariant code motion	`while(..){ t = a*b; ...}`	`t = a*b; while(..){...}`
Induction-variable elimination	`j = 4*i` recomputed every iteration	`j = j + 4`
Loop unrolling / jamming	many small iterations	fewer jumps, less overhead

NOTES

Peephole optimisation slides a small window (a few instructions) over the **target** code, replacing patterns with cheaper ones: redundant load/store removal, unreachable-code removal, algebraic simplification (`x+0` , `x*1`), flow-of-control tweaks (jump to a jump).

DAG of a basic block Core

NOTES

A DAG for a basic block has one node per distinct value; identical computations share a node, which exposes **common subexpressions** and **dead code** automatically.

WORKED EXAMPLE

Worked numerical — DAG construction

Q: Build the DAG for `a = b + c; b = a − d; c = b + c; d = a − d` and count its nodes.

A:

```
Leaves (initial values): b0, c0, d0                    → 3 nodes
a = b0 + c0              → node N1 (+)  labelled a
b = a − d0 = N1 − d0     → node N2 (−)  labelled b
c = b + c0 = N2 + c0     → node N3 (+)  labelled c   (b changed, so NOT the same as N1)
d = a − d0 = N1 − d0     → same as N2 → label d is added to N2, no new node

Total nodes = 3 leaves + 3 internal = 6
Optimised code:  a = b + c;  d = a − d;  c = d + c   (b = d)
→ 3 operations, not 4
```

Data-flow analysis Core

NOTES

For each basic block B, compute `IN[B]` and `OUT[B]` by iterating the equations until nothing changes (a **fixed point**). Three classic problems:

REFERENCE TABLE

Problem	Direction	Meet	Equations
Reaching definitions	forward	∪ (may)	OUT[B] = gen[B] ∪ (IN[B] − kill[B]); IN[B] = ∪ OUT[pred]
Live variables	backward	∪ (may)	IN[B] = use[B] ∪ (OUT[B] − def[B]); OUT[B] = ∪ IN[succ]
Available expressions	forward	∩ (must)	OUT[B] = e_gen[B] ∪ (IN[B] − e_kill[B]); IN[B] = ∩ OUT[pred]

COMMON TRAP

Memory hook

May analyses (reaching defs, liveness) use **union** and start with ∅. **Must** analyses (available expressions) use **intersection** and start all non-entry blocks with the universal set. Reaching defs & available exprs go **forward**; liveness goes **backward**.

WORKED EXAMPLE

Worked numerical — reaching definitions

```
B1:  d1: a = 1       d2: b = 2
B2:  d3: a = a + b          (B2 loops back to itself, then falls to B3)
B3:  d4: c = a + b
Edges: B1→B2, B2→B2, B2→B3
```

gen: B1={d1,d2}, B2={d3}, B3={d4}. kill: B1={d3}, B2={d1}, B3=∅ (d1 and d3 both define a).

```
OUT[B1] = {d1,d2}
Iteration:  IN[B2] = OUT[B1] ∪ OUT[B2] = {d1,d2} ∪ OUT[B2]
            OUT[B2] = {d3} ∪ (IN[B2] − {d1})
  pass 1:   IN[B2] = {d1,d2}            OUT[B2] = {d3, d2}
  pass 2:   IN[B2] = {d1,d2,d3}         OUT[B2] = {d3, d2}  (stable)
IN[B3]  = OUT[B2] = {d2, d3}            OUT[B3] = {d2, d3, d4}
```

So at B3, the definition **d1 does not reach** (killed by d3); only d3's value of `a` reaches. That justifies using d3 for constant/copy propagation decisions.

WORKED EXAMPLE

Worked numerical — live variables

```
B1: a = ..; b = ..      →  B2 and B3
B2: c = a + b           →  B4
B3: c = a * b           →  B4
B4: print(c)
```

```
B4: use={c}  def=∅          IN[B4]={c}   OUT[B4]=∅
B2: use={a,b} def={c}       OUT[B2]=IN[B4]={c}        IN[B2]={a,b}
B3: use={a,b} def={c}       OUT[B3]={c}               IN[B3]={a,b}
B1: use=∅   def={a,b}       OUT[B1]=IN[B2]∪IN[B3]={a,b}
IN[B1]=∅
```

IN[B1] = ∅ means no variable is used before being defined — a good sign. A variable assigned but not in OUT is **dead** and its assignment can be eliminated.

Code generation & register allocation Extension

RULES & FACTS

- Code generation chooses instructions, **allocates registers**, and orders the code. Variables in registers are much faster than memory.
- **Graph colouring**: one node per variable (live range); an edge between two variables that are **live at the same time**; with k registers, colour the graph with k colours. If it cannot be k-coloured, some variable is **spilled** to memory.
- Minimum registers needed (no spills) = **chromatic number** of the interference graph ≥ maximum number of simultaneously live variables.

WORKED EXAMPLE

Worked numerical — registers needed

```
1: a = 1
2: b = 2
3: c = a + b
4: d = a * c
5: e = b + d
6: print(e)
```

```
Live after each instruction:  1:{a}  2:{a,b}  3:{a,b,c}  4:
{b,d}  5:{e}  6:ø
Interference edges: a–b, a–c, b–c, b–d
```

a, b, c form a triangle, so at least 3 colours are needed, and 3 suffice (d can reuse a's or c's register). **Minimum registers = 3**. Note that `d` and `a` do not interfere because `a` 's last use is in instruction 4 itself, the one that defines `d` .

8 Problem → Standard Method Playbook

1. Type checking & semantic analysis High-yield

METHOD

Recognition: the question asks whether an expression, assignment, function call, array access, or operator application is type-correct.
Method: (1) annotate each identifier with its declared type, (2) propagate expression types bottom-up, (3) apply operator compatibility and implicit-conversion rules, (4) check assignment/return/argument compatibility, (5) distinguish compile-time semantic errors from runtime errors.
Trap: syntax can be valid while semantics are invalid; parsing alone does not establish type correctness.

WORKED EXAMPLE

Worked problem — type checking

Q: Suppose `f` has type `int (float, int)` , `x` is `int` , and `y` is `float` . Is `f(x, y)` type-correct if implicit widening `int→float` is allowed but `float→int` is not?

METHOD NOTES

Solution: The first argument `x:int` widens to `float` . The second argument `y:float` cannot narrow to the required `int` . Therefore the call is **not type-correct**.

9 Last-minute one-liners

LAST-MINUTE · FORMULAS & RULES

- FIRST(α) is the set of terminals that can begin strings derived from α; include ε when α can derive ε.
- Dom(entry)={entry}; for other nodes iterate Dom(n)={n}∪⋂Dom(pred) to a fixed point.

LAST-MINUTE · MUST REMEMBER

- Lexical analysis converts characters into tokens; lexical errors are detected before parsing.
- Longest-match (maximal munch) is the standard lexer rule; keyword recognition may be handled after identifier matching.
- An LL(1) grammar must avoid FIRST/FIRST conflicts and FIRST/FOLLOW conflicts for ε-productions.
- LR parsing is bottom-up and constructs a rightmost derivation in reverse.
- Shift-reduce conflicts and reduce-reduce conflicts are the two classic LR parsing conflicts.
- Syntax-directed definitions attach semantic rules/actions to grammar productions.
- Three-address code typically represents expressions using simple instructions with at most three addresses.
- Common optimisations include constant folding, common-subexpression elimination, dead-code elimination, and strength reduction.
- **Type checking is semantic analysis:** parse trees can be syntactically valid but type-invalid.
- A back edge u→v exists when v dominates u; the natural-loop header is v.
- Reference counting cannot reclaim cycles; tracing collectors use reachability from roots.
- Assembler makes object code; linker resolves cross-object symbols; loader places the executable in memory.
- Panic-mode recovery discards input to a synchronizing token; phrase-level recovery performs a local correction.

LAST-MINUTE · COMMON TRAPS

- SLR, canonical LR(1), and LALR differ mainly in the information used to resolve reductions and the number of states.
- Optimisation must preserve program semantics; a locally faster transformation is not valid if observable behaviour changes.
- Unreachable memory is a leak; a dangling pointer points to storage whose lifetime has ended.

LAST-MINUTE · GATE FAVOURITES

- Know activation records, static/dynamic links, parameter passing, and storage allocation for runtime-environment questions.